3PL Implementation

Paul Dunn

30th July 2021

Open report - CSIRO Mineral Resources

Contents

Chapter 1 - Overview

3PL parses source code and produces a code tree, the root of which is **ThreePL/main**. Leader, trailer and post-processing modules encountered during parsing are accumulated in lists **leader_modules**, **trailer_modules** and **post_modules**. All libraries are in source code form and are inserted into the source code via **source** or **module** statements.

After parsing -

- The leader module list is traversed and each module is executed.
- The code tree is then executed. Any target code generated during immediate execution is added to the intermediate code list **ThreePL/tdelist**.
- The trailer module list is traversed and each module is executed.
- The intermediate code list is optimised and then traversed to generate netlist code to the EDIF file.
- The post-processing module list is traversed and each module is executed to run vendor place-and-route tools.

1.1 Operation

The main program is in source file ThreePL.java. The parser is in source file parser/Parser.jj which is processed by the Java compiler compiler Javacc to produce file Parser.java which is then compiled by the Java compiler.

The main program reads and processes the command line arguments. If 3pl is called with no source file name, an interactive file chooser is then started from which the continuation of the main program can be called repeatedly. If there is a source file name, the continuation of the main program is called once only.

```
1     if (source_file == null) {
2         // Start up a file chooser GUI which can repeatedly call
3         // mainContinue().
4         .
5         .
6         .
7         .
8         .
9         file_chooser_used = true;
10        fc = new FileChooser();
11        FileChooser.display();
12    } else {
13        // Just continue execution - mainContinue().
14        file_chooser_used = false;
15        int ret = mainContinue();
16        if (ret != 0)
17            System.exit(ret);
18    }
```

After a great deal of initialisation the parser is called -

```
    try {
        input_stream = new SrcReader(source_file);
    } catch (FileNotFoundException e) {
        msg("unable to open source file '" + source_file + "'");
        return(1);
    }
    try {
        if (parser == null)
            parser = new Parser(input_stream); // first use
        else
            Parser.ReInit(input_stream);        // subsequent uses
        main = Parser.Filemodule();
    } catch (ParseException e) {
        msg("");
        msg("Parse error: file '" + file_name + "' line " + line_no);
        msg(e.getMessage());
        return(1);
    } catch (TokenMgrError e) {
        msg("");
        msg("Parse error: file '" + file_name + "' line " + line_no);
        msg(e.getMessage());
        return(1);
    } catch (ExEx e) {
        msg("");
        msg("Compilation error");
        msg(e.getMessage());
        return(1);
    }
    input_stream.close();
```

The parser builds a code tree rooted at local variable **main**. The root of the tree will be a module call (**ProcModCallNode**) to a file module associated with the initial source file. During parsing any leader or trailer modules encountered will be added to the leader or trailer module list. Any parsing errors result in immediate termination with an appropriate message.

Following parsing, any modules in the leader module list are executed by the interpreter. The file module in **main** is then passed to the intereter for execution. Finally any modules in the trailer module list are executed by the interpreter. Execution of the parsed code by the interpreter is refered to as "immediate" execution to distinguish it from later execution of target code in the FPGA. Most errors encountered during immediate execution result in termination with an error message.

As immediate execution proceeds, any statements which produce target code will add elements to the intermediate code list. These elements have been called Topological Description Elements, henceforth referred to as TDEs[1]. The list is in class variable

```
    public static  TDEList          tdelist = new TDEList(2000);
```

Class **TDEList** is defined in **codegen/TDEList.java**.

If requested by command line options the TDE list may be printed to the report file before, during and after the following optimisation passes.

The TDE list is now optimised. The optimisation not only eliminates simple logic redundancies, it also simplifies the control structure where possible.

```
    // Perform generic optimisation of the signaling logic in the TDE list.
    // Continue passes until no changes occur.
    //
    boolean changes = false;
    int     opasses = 0;
    do {
        // tdelist.linkCheck(); use if optimisation problems
        changes = tdelist.optimiseTDEList(continuous);
        opasses++;
    } while (changes);
    if (verbose)
        rpt("\toptimisation passes: " + opasses);
```

[1] In retrospect this terminology is not ideal, but it is now too deeply embedded in the code to be worthwhile changing.

The next step is to perform TDE optimisations particular to the FPGA family, done by calling -

```
1        family.optimiseTDEs();
```

Following this family-specific optimisation the TDE list is again optimised to remove any redundancies introduced by that previous step.

```
1    do {
2        changes = tdelist.optimiseTDEList(continuous);
3        opasses++;
4    } while (changes);
5    if (verbose)
6        rpt("\tcleanup optimisation passes: " + opasses);
```

Normally if the simulation command line option has been used the simulator is now invoked on the TDE list, however the simulator code has been disabled as it has atrophied as 3PL has evolved. The source code is still present should it be required again. Because FPGAs have increasingly included special hardware such as DSP blocks, serialisers/de-serialisers etc. and support for these has been moved from Java code within 3PL to 3PL libraries, it has become infeasible to support these in the simulator. In addition the simulator has received little use and the author of the simulator code has moved away from the project.

The final operation is to generate EDIF netlist output from the TDE list. This is done by -

```
1    tdelist.ncode(outdir, design_name, cell_name);
```

where **outdir** is a string specifying the directory into which the netlist and constraint files are to be written, **design_name** is the design name string, usually derived from the main source file name, and **cell_name** is the same as **design_name** except when a core netlist is being produced in which case **cell_name** is taken from the first argument to inbuilt procedure **export()**.

1.1.1 Main class - ThreePL

Class ThreePL contains the following static variables -

long nanotime0 start of 3PL
long nanotime1 start of immediate execution
long nanotime2 start of TDE list optimisation
long nanotime3 start of netlist generation
long nanotime4 start of netlist optimisation
long nanotime5 start of netlist output
long nanotime6 finish
These are execution times for different phases of 3PL execution.

Runtime main_rt - The time of 3PL execution start.

FileChooser fc - file chooser class for GUI
String source_file - 3PL input file
String current_directory -
String parent_directory - parent directory of source file
String netlist_file - EDIF netlist file
String report_file - report file
String icl_file - intermediate code list file
String net_file - list of equivalent nets file
StringBuffer msgsb - string buffer for GUI messages

StringBuffer cloptions - command line options

SrcReader input_stream - input stream from input source code reader. This reader inserts included files in the stream.

Parser parser - the parser class instance

TreeMap<String, Module> modules - a map of global modules.

TreeMap<String, Procedure> procedures - a map of global procedures.

TreeMap<String, Function> functions - a map of global functions.

InbuiltMods inbuilt_mods - a map of inbuilt modules. Class **InbuiltMods** contains the map where the keys are the inbuilt module identifiers and the values are class **InbuiltMod**. Each inbuilt module extends class **InbuiltMod**.

InbuiltProcs inbuilt_procs - a map of inbuilt procedures. Class **InbuiltProcs** contains the map where the keys are the inbuilt procedure identifiers and the values are class **InbuiltProc**. Each inbuilt procedure extends class **InbuiltProc**.

InbuiltFuncs inbuilt_funcs - a map of inbuilt functions. Class **InbuiltFuncs** contains the map where the keys are the inbuilt function identifiers and the values are class **InbuiltFunc**. Each inbuilt function extends class **InbuiltFunc**.

HashSet<Clock> clocks - a set of all clocks. This is only used by the simulator (currently not implemented by makefile).

ArrayList<Clock> clockvarstack - a stack of clock variables used by inbuilt procedure **useclock()**.

Clock currentclockvar - the current clock variable, maintained by inbuilt procedure **useclock()**.

ArrayList<Module> leader_modules - a list of leader modules.

ProcModCallNode main - the code tree root node.

ArrayList<Module> trailer_modules - a list of trailer modules.

ArrayList<Module> post_modules - a list of post-processing modules.

LinkedList<Body> body_stack - the stack of **Body** class instances for nested module/procedure/function calls. The **Body** class is the super class of **Module**, **Procedure** and **Function** classes.

LinkedList<Scope> scope_stack -

the stack of **Scope** class instances. The **Scope** class contains maps of variables. Each new invokation of a module, procedure or function creates a new scope (which is pushed on the stack) but new Scopes are also created for code blocks occurring in **if**, **while** and **for** statements.

LinkedList<SrcLoc> call_loc_stack - the stack of source code locations of current nested module, procedure and function calls.

ArrayList<ThreePL.FuncUsedPair> func_used - a list of functions for which the **used()** inbuilt function has been called.

String file_path - current input file path

String file_name - current input file name during compilation. This changes as new files are read by **source** and **module** statements.

String file_dir - current input file directory.

String abbrev_path - file path with leading string replaced by a short string for convenience.

int dir_index = -1 - a integer used to in geberating leading srings for shortened file names

ArrayList<String> file_paths - list of file paths.

ArrayList<String> file_dirs - list of file directories.

ArrayList<String> files - list of file names.

ArrayList<Integer> file_depths - list of depth of file inclusions.

ArrayList<Integer> file_indices - list of module file indices or -1.

int file_depth - depth of file inclusion.

int line_no - current input file line number during compilation.

String prev_file_name - previous input file name when reading from an file inserted by a **module** or **source** statement.

String prev_dir - directory of previous source file.

int prev_line_no - previous input line number when reading from an file inserted by a **module** or **source** statement.

int warnings - a count of warning messages.

FileDesc stdinOpenFile - file descriptor for standard input.

FileDesc stdoutOpenFile - file descriptor for standard output.

FileDesc stderrOpenFile - file descriptor for standard error.

PrintStream rptps - a **PrintStream** class instance used for report file output if the -r option has been specified.

PrintStream iclps - intermediate code list file.

LinkedList<Var> var_queue - a queue of all target mode variables. This queue is traversed at completion of immediate execution to create logic for target variables.

Procedure attributeProc - a user-provided 3PL procedure for handling attributes passed to inbuilt procedures clock(), input() and output(). This is set by the user by calling inbuilt procedure **attributeprocedure()**.

TreeMap<String, Val> directives - a map of directives.

int directive_max_key_length - the maximum key length of directives; used to space directive listing columns.

Scope global_scope - the **Scope** class for global variables and directives.

int errors - a count of errors encountered during compilation or immediate execution. Most errors cause termination of compilation or immediate execution but a few produce an error message and increment the error count rather than exiting 3PL. If this count is non-zero at the end of compilation then immediate execution will not occur. If the count is non-zero at the end of immediate execution then netlist output will not be produced.

TDEList tdelist - the list of intermediate target code elements.

boolean tdelist_optimisation_started is true when optimisation of the TDE list has started.

boolean tdelist_optimisation_finished is true when optimisation of the TDE list has been completed.

TDEVar startval_tdev - the TDEVar for the start target boolean expression.

SrcLoc errloc - the source file location of an error. This is assigned when the **ExEx** exception is thrown. It is used when printing a stack trace.

int icount - unique integer for element identifiers

boolean creating_variables - true when target variables are to be created following termination of immediate execution.

ArrayList<String> include_dirs - list of **source** and **module** statement inclusion directories.

boolean readonly - a debugging flag which limits a run to parsing-only with diagnostic output of tokens.

boolean isparsing - true during parsing

boolean postProcessing - true when postprocessing code is being executed.

boolean top_scope_skip - a flag used in specific circumstances to skip the top frame on the stack when searching for a variable.

final boolean sim_implemented - simulator implemented; currently false.

boolean sim - if true, run the simulator (disabled).

boolean valuelabel - if true, generate an additional simple value label for each value variable to aid variable identification in the simulator. Currently false as the simulator is disabled.

boolean develop - a flag set by the 'X' option to constrict operation of the interpreter during system development. Not for normal use.

boolean clockdiag - a currently unused flag which can be used to suppress some clock domain checking during system development.

boolean codeopt - if false, skip optimisation. Used during system development and currently unused.

boolean varerrors - if true errors in variable attributes are fatal, rather than just emitting warnings.

boolean continuous - shadows the 'continuous' directive.

boolean ALU - shadows the 'ALU' directive.

boolean oack_use_lut - this and the following booleans select whether LUTs or gates are generated in the EDIF netlist in various cases. The defaults are true, but the gates_not_luts boolean below, if set by command line option or directive, can set all these to false.

boolean as_fifo_read_use_lut - see comment above.

boolean as_fifo_empty_use_lut - see comment above.

boolean add_sub_use_lut - see comment above.

boolean ge_use_lut - see comment above.

boolean mux_use_lut - see comment above.

boolean incrdecr_use_lut - see comment above.

boolean cpld_use_lut - see comment above.

boolean gates_not_luts_option - set from command line option and copied to gates_not_luts below.

boolean gates_not_luts - if true generate gates rather than LUTs in netlist code.

boolean fifos_ok - use BRAM FIFOs for queues

boolean queuereg_ok - use an extra output register on a BRAM queue

boolean file_chooser_used - true if the GUI is used, i.e. 3PL called with no arguments.

boolean fc_skip_post_processing - when true any post-procerssing modules are ignored.

boolean fc_force_execution - if true 3PL parsing and execution occurs regardless of file ages.

boolean fc_list_tdes - if true the TDE list is printed to a .icl file.

boolean fc_list_nets - if true the list of equivalent nets is printed to a .net file.

boolean fc_report - if true a detailed report is printed to a .rpt file.

long dsp_threshold - static variable width at which the P register of a DSP block will be used instead of DFFs. This has proved to be of no use as the clock-to-output time is mcuh longer than that of a CLB flip-flop and the saving in flip-flops is no worth the penalty.

String ce_name - current element name

String outdir_opt - output directory option

String outdir - output directory

String design_name - design name

String cldn - command line design name

String cell_name - EDIF cell name

String partstring - part specification

TDECode family - device family class

String libref - EDIF library reference string

long currentTime - current millisecond time.

SimpleDateFormat dateFormat - format for most dates.

String currentDate - current date string.

long compilerTime - build millisecond time.

String fullVersion - long version of version.

long sourceTime - maximum of millisecond time since last modification of all source files.

long netlistTime - millisecond time since last modification of of netlist file.

long reportTime - millisecond time since last modification of of report file.

long netTime - millisecond time since last modification of of net file.

ArrayList¡String¿ shells - shells available (paths). This list is searched to find a shell with which to execute a command in the host operating system running 3PL. The first entry is that contained in the SHELL environment variable if not null. The following entries are "/usr/bin/bash" and "/bin/bash".

Map<String,String> environment - environment variables

String os_name - operating system

String arch_name - host computer architecture.

String version - release version number.

String revision - subversion revision number.

String lastChangedAuthor - subversion author.

String lastChangedRev - subversion rev.

String lastChangedDate - subversion date.

String buildDate - make run date/time.

String builtBy - user that ran make.

The last seven variables above are loaded as a resource bundle from file classes/threepl/ThreePL.properties. This might appear as -

```
version = 11.5.1
revision = 9931
lastChangedAuthor = jim123
lastChangedRev = 9862
lastChangedDate = 2021-01-12 09:50:00 +1100
buildDate = 2021-07-23 10:52:04 +1000
builtBy = jim123
```

This file is created by sources/threepl/Makefile using a template file sources/threepl/ThreePL.properties.template which contains -

```
version = @VERSION@
revision = @SVNVERSION@
lastChangedAuthor = @SVNAUTHOR@
lastChangedRev = @SVNREV@
lastChangedDate = @SVNDATE@
buildDate = @BUILDDATE@
builtBy = @BUILTBY@
```

The code to load variables from the resource bundle is -

```
1    ResourceBundle rb = ResourceBundle.getBundle("threepl.ThreePL");
2    for (Enumeration<String> keys = rb.getKeys(); keys.hasMoreElements(); ) {
3        final String key = keys.nextElement();
4        final String value = rb.getString(key);
5        ThreePL.class.getDeclaredField(key).set(null, value);
6    }
```

1.2 Source File Structure

The compiler source code is contained in the following directories -

- threepl - the compiler main program
- threepl/parser/ - the parser
- threepl/nodes/ - nodes for the compiled code tree
- threepl/exec/ - main classes for variables, types, references, values etc.
- threepl/mods/ - inbuilt modules
- threepl/procs/ - inbuilt procedures
- threepl/funcs/ - inbuilt functions
- threepl/codegen/ - intermediate code generator
- threepl/netlist/ - netlist generic classes
- threepl/netlist/xilinx - Xilinx-specific netlist code generator
- threepl/simulator/ - target simulator
- threepl/exceptions/ - exception handling

The parser processes 3PL source code and produces a code tree. That code tree is constructed using classes in the *nodes* directory. The compiler then executes (interprets) the code tree starting at the root. This is refered to as *immediate execution*. During immediate execution variables are constructed, expressions are evaluated and assignments to immediate variables are made. These operations construct a data structure using classes in directory *exec*. Inbuilt modules, procedures and functions may be called and the code for these is in directories *mods*, *procs* and *funcs*. Immediate execution usually also generates target intermediate code. The classes for this are contained in directory *codegen*. Following immediate execution the intermediate code is optimised and then converted to an internal netlist structure using classes in *netlist*. The netlist structure is then optimised at the gate level and checked for validity before finally being written out to the final netlist file. The intermediate code structure also contains some constraint information which is inserted into the netlist structure and finally written to constraint files.

1.2.1 inbuilt modules

Inbuilt modules are contained in directory **mods**.

Class **inbuiltMod()** is the superclass of all inbuilt module classes. The **execute()** method is overridden in all subclasses. It has a single argument which is a list of two argument lists, one a list of input arguments, the other a list of output arguments.

Class **inbuiltMods()** contains a map of all inbuilt modules. The list is initialised by the class contructor.

The other classes implement the inbuilt modules. Each is a subclass of **inbuiltMod()** and each has an **execute()** method.

1.2.2 inbuilt procedures

Inbuilt procedures are contained in directory **procs**.

Class **inbuiltProc()** is the superclass of all inbuilt procedure classes. The **execute()** method is overridden in all subclasses. The **execute()** method has 3 different signatures. The first is for procedures which create variables (and also module, procedure and function parameters), the second is for procedures which do not generate in-line target code and the third is for procedures which do generate in-line target code. All three share the same first argument which is a list of two argument lists, one a list of input arguments, the other a list of output arguments.

Class **inbuiltProc()** has two constructors. The first constructor is for procedures which create variables (the first **execute()** method). This constructor sets class boolean **allowed_as_param** true which signals that these procedures can be called within the parameter section of a module, procedure or function definition. The class boolean **target_inline** is set false. The two arguments to the constructor are booleans indicating if the input arguments or the output arguments should prohibited from being null (missing). The other constructor is for all other inbuilt procedures and has an additional boolean argument which indicates if they generate in-line target code or not.

Class **inbuiltProcs()** contains a map of all inbuilt procedures. The list is initialised by the class contructor.

The other classes implement the inbuilt procedures. Each is a subclass of **inbuiltProc()** and each has an **execute()** method.

1.2.3 inbuilt functions

Inbuilt functions are contained in directory **funcs**.

Class **inbuiltFunc()** is the superclass of all inbuilt function classes. The **getVal()** method is overridden in all subclasses. It has a single argument which is a list containing one argument list; a list of input arguments.

Class **inbuiltFuncs()** contains a map of all inbuilt functions. The list is initialised by the class contructor.

The other classes implement the inbuilt functions. Each is a subclass of **inbuiltFunc()** and each has a **getVal()** method.

1.3 Notes on parsing

The following static variables in class ThreePL in ThreePL.java

```
1    protected static ProcModCallNode    main;
2    private   static SrcReader          input_stream = null;
3    private   static Parser             parser;
```

are required by the parser. **main** is the root of the resulting immediate code tree following parsing. **input_stream** provides the token stream for the parser. Class **SrcReader** is used to read a character stream via method **read()** called by the token manager. SrcReader is used to insert included files into the input source code stream. It also inserts dummy tokens to keep track of source file names and line numbers when entering and exiting included files.

The parser is invoked by -

```
1    input_stream = new SrcReader(source_file);
2    parser = new Parser(input_stream);
3    main = Parser.Filemodule();
```

The parser is in **Parser.jj** which is processed by javacc to produce Parser.java plus several support files -

- ParserTokenManager.java
- SimpleCharStream.java
- Token.java
- TokenMgrError.java

Class Token, produced by javacc, has been modified to include member variables **fileName** and **lineNo** to provide a file name and line number for each token. The source file location of language constructs is stored in many classes and passed to many methods and class **SrcLoc** provides this information.

As a minor note the pre/post increment/decrement operators `++` and `--` are treated as flags on variables, not true operators.

1.3.1 general node - nodes/Node.java

The parser constructs a code tree using the *node* classes. The tree is then executed starting at the root node using methods **execute()** or **getVal()** as appropriate.

Class **node** is the super class of parser tree nodes. Each node has a list of nodes below it in the tree. This list is class **NodeList**, which extends class **ArrayList**. Each node also has a pointer to the node above it. A source location (file name, line number and column number) is also included in a **SrcLoc** instance.

In the case of a call to a module or procedure (ProcModCallNode) or a function (FuncCallNode) there are two subnodes, the first for input arguments and the second for output arguments. Each subnode is itself a list and this is implemented by class ArgListNode where its subnodes are the arguments.

Nodes have either a **getVal()** or an **execute()** method. Nodes for variables (class **VarNode**), expressions (class **ExprNode**) and function calls (class **FuncCallNode**) use the **getVal()** method to return a value. Expressions return a single primitive type value but variable instances or function calls may also return compound values. All other tree nodes, such as assignment statements, module and procedure calls or control structures, use the **execute()** method since they do not return a value.

The **getVal()** method is passed an execute signal. This is a single bit **TDEVar** control signal that is asserted when the value is used during execution in the FPGA. It is used to generate the logic for queue pop signals.

The **execute()** method is passed empty lists in which both start execution signals and finish execution signals can be returned from enclosed nodes. This scheme allows for nested immediate loops which generate multiple target statements whose serial/parallel execution is determined back up the tree in an enclosing seq or par block. The lists allow the decision as to how to link the execution of these target statements to be postponed until the return to the enclosing seq or par block. This method returns an enumerated type which indicates if a break, continue or return statement has been encountered (see parser/Constant.java).

Note that immediate constructs must nevertheless adhere to this start/finish signal scheme as they may contain embedded target code. Immediate constructs which simply return the start signal as the finish signal are recognised as having generated no target execution code and the execution signals have no inserted delays and simply pass through unchanged.

1.3.2 explicit nodes

It is not intended to describe the remaining node classes used by the parser to build the code tree. In most cases these can be understood from the parser code and the classes themselves. It is also probably unlikely that the parser or interpreter will need to be changed in the future compared with the netlist generation code which may need substantial modification in the future to track FPGA family evolution, Xilinx software tool changes or the need to port 3PL to the FPGAs of another vendor.

1.4 interpreter

The code tree uses the following classes for variables -

- Var - the super class of all variables.
- ClockedVar - the super class of target mode variables.
- Immediate - an immediate mode variable.
- Clock - a clock mode variable.
- Value - a value mode variable.
- SelectValue - a selectvalue mode variable.
- Static - a static mode variable.
- Queue - a queue mode variable.
- Priority a priority mode variable.
- Input - an input mode variable.
- Output - an output mode variable.
- Memory - a cmemory or rmemory mode variable.

Other important classes are -

- SubField - an array subscript, subscript range or a struct field.
- SubFieldList - a list of array subscripts, subscript ranges or struct fields acting on a variable.

1.4.1 Execution stacks

There are two execution stacks -

- a scope stack
- a code body stack

The scope stack contains all active scopes except global scope. Global scope and all file module scopes are static. File module scopes may appear on the scope stack but may also be accessed as a static scope via the **filescope** field in the current Body class (module, procedure or function code body).

The body stack contains all active module, procedure or function code bodies.

Active scopes and bodies are those currently involved in immediate execution. The top scope and body are for the current execution point and those deeper in the stacks apply to blocks within which the current one is nested.

The scope stack usually has more entries than the body stack because it has an entry for each body plus entries for braced code blocks in other control structures such as **if**, **for**, **when** etc.

1.4.2 Var class and its sub-classes - Exec/Var

The **Var** class is the super class of all variable modes. Classes **ClockedVar**, **Immediate**, **Value**, **Memory** and **Clock** are subclasses of class **Var**. Classes **SelectValue**, **Static**, **Queue**, **Priority**, **Input** and **Output** are subclasses of **ClockedVar** because these classes represent target variables which require one or two clocks. Member variables for class **Var** are -

String name - the variable identifier.

Context context - the context of the variable. **Context** is an enum defined in interface Parser/Constant.java and can have the values -

- GLOBAL - static global scope.
- FILE - the current static file module scope.
- CALLING - the scope in which the current module, procedure or function was called.
- LOCAL - the current module, procedure or function scope.
- ILMOD - an inline (un-named) module
- DEFAULT - none supplied. This is used when making a request regarding a variable and the scope is not specified, in which case the scope stacks will be searched.

String ename - an unambiguous 'extended' variable identifier for target mode variables. This identifier is used in intermediate code, in the final netlist and in constraint files.

Mode mode - the variable mode. This is an enum defined in interface Parser/Constant.java. Values can be -

- IMMEDIATE
- VALUE
- STATIC
- QUEUE
- SELECTVALUE
- PRIORITY
- CMEMORY
- RMEMORY
- INPUT
- OUTPUT
- CLOCK

Type type - the variable type if any (CMEMORY, RMEMORY and CLOCK mode variables do not have a type).

SrcLoc decloc - the source file location of the 'declaration' (actually 'creation', since it is done during immediate execution, not during parsing). **SrcLoc** is a class in directory **Parser** and it contains the source file name and the line number.

boolean isInPar - true if the variable is an input parameter to a module, procedure or function.

boolean isOutPar - true if the variable is an output parameter to a module, procedure or function.

boolean matched - true if the variable is an input or output parameter to a module, procedure or function and has been supplied a matching argument in a call.

boolean readonly - true if the variable is read-only.

boolean indass - true if the variable is an indirect reference to the actual variable. This is used where an argument is a variable and the parameter variable points to the argument variable.

SubFieldList ind_sfl - if this variable is a parameter this is a list of subscripts and fields supplied with a matching variable argument subsection 1.4.12 p26.

WordSpec wordspec - a word specifier for this variable subsection 1.4.6 p16.

Object[] val - an array of values for an IMMEDIATE or VALUE mode variable.

Type[] val_type - the variable type subsection 1.4.5 p15.

boolean assigned - true if a variable has been assigned (not used by INPUT, CMEMORY or RMEMORY mode variables).

TreeMap attributes - a map of variable attributes whose values have been explicitly set.

Var indirect - if **indass** is true then this points to the variable.

boolean used - true if the value of this variable has been used as an input to one or more devices.

The constructor mostly just assigns a few member variables from arguments passed to it from subclass constructors, but it also -

- generates member variable **ename** from the identifier string and context.
- sets any undimensioned arrays in the type to zero size since the dimensions have been previously set to -1 as markers.
- generates a **WordSpec** from the type.

The identifier extended name **ename** must be unambiguous. The compiler generates many signals with identifiers of the form "S123" with which variable extended identifiers must not clash. This is ensured by prepending strings to the variable identifiers to make them unique. The code line

```
1    ename = scopeHierarchy(context, loc) + name;
```

does this, where **scopeHierarchy()** is a method in ThreePL.jj.

In the case of GLOBAL context **scopeHierarchy()** simply returns the string "glob_". For other contexts the scope stack is searched backwards from the current scope until the specified scope is found. The scope stack is then traversed from the root to the specified scope. From each scope on the stack a unique string is derived. This string is either the module, procedure or function identifier with an underscore and a unique integer appended, or is a string indicating the block type such as "IF", "WHEN" etc again with a unique integer appended. These scope strings are concatenated along with an underscore and the variable identifier string. An example is - `kandinski.encoder_counter_0.cpu_write_var_3.if_3164.blocked` "kandinski" is the identifier for the file module (it does not have an underscore and integer appended as it is unique already and it cannot clash with generated netlist identifiers of the form "S123" as it has "." appended). "encoder_counter" is a module in the source code and as this is the first call of that module "_0" is appended. "cpu_write_var" is a procedure in a library source file and this is the fourth call so it has "_3" appended. A variable "blocked" is created within an **if** statement braced code block so "if_3164" is appended (there have been quite a lot of if statements processed prior to this one). Finally the variable identifier "blocked" is appended. It needs no trailing integer as it must be unique in its scope.

1.4.3 Body class - Exec/Body

The **Body** class is the super class for classes **Module**, **Procedure** and **Function**.

The **body** class has the following member variables -

String name - the module, procedure or function identifier. For a file module this is the file name. For an inline module this is the string "ilm" with a unique integer appended.

Body enclosing - the body from which this one is called.

Scope scope - the scope for the body.

Scope filescope - the file scope for the body. In the case of a file module this is the same as the body scope (above).

TreeMap modules - modules defined within this module, procedure or function. The module identifier is the key to the map.

TreeMap procedures - procedures defined within this module, procedure or function. The procedure identifier is the key to the map.

TreeMap functions - functions defined within this module, procedure or function. The function identifier is the key to the map.

NodeList input_nodes - a list of immediate code tree nodes representing calls to input parameter construction procedures.

NodeList output_nodes - as above but for output parameters.

ArrayList in_args - a list of input call arguments of class Val.

TreeMap in_keys - a map of input arguments passed by key=value rather than by position.

ArrayList out_args - as above but for output arguments

TreeMap out_keys - as above but for output keys.

int num_input_args - the number of input arguments.

int num_output_args - the number of output arguments.

BlockNode tree - the code tree for the body.

SrcLoc loc - the source file location of the definition.

Method **getArgs** processes call arguments to a module, procedure or function.

Method **params** creates parameter variables and assigns the passed arguments to these.

1.4.4 Scope class - Exec/Scope

This class contains scope information. Every code block ("if", "while", "for" etc) and every module, procedure or function invokation has an associated scope block created. There is also a scope block for global variables and a scope block for every file module (associated with each source file). The global scope block is not on the scope stack. One or more file scope blocks (source files can be nested) may be on the stack and are used for variable creation however variable evaluation avoids these and instead uses the file scope stack. Following these on the scope stack there may be module, procedure or function scopes each possible trailing one or more code scope blocks.

Scope contexts for variable creation are -

- GLOBAL global scope
- FILE scope associated with the current source file
- CALLING the scope of the caller to the current module, procedure or function
- LOCAL the current module, procedure or function scope
- ILMOD the current inline module scope
- DEFAULT the current block scope

Scope contexts for variable evaluation are -

- GLOBAL as above
- FILE as above
- CALLING not allowed
- LOCAL as above
- DEFAULT all accessible scopes -
  - block scopes nested back to the LOCAL scope,
  - LOCAL scope
  - FILE scope
  - GLOBAL scope

  searched in that order

The **Scope** class has the following member variables -

TreeMap dirs - a map of directives for this scope, the map key being the directive key.

TreeMap vars - a map of variables for this scope, the variable identifier being the map key.

Context context - the scope Context.

Btype btype - the type of code body. This is an enum with values

- UNDEF - the block type has not yet been assigned
- IF - an 'if' true or false code block in braces
- WHILE - a 'while' code block in braces
- DOWHILE - a 'for' code block
- FOR - a 'for' code block in braces
- FORVAR - 'for' loop variable scope
- FSRC - a code block, for a file inserted using a 'source' statement
- FMOD - a file module code block, for code contained in the command line source file or a file inserted using a 'module' statement.
- NMOD - a normal called module body code block
- ILMOD - an inline module code block
- PROC - a procedure body code block
- FUNC - a function body code block
- CASE - a code block for a 'case' within a switch statement
- WHEN - a 'when' true or false code block in braces
- SEQ - a 'seq' code block
- PAR - a 'par' code block
- SYNC - a 'sync' code block
- SYNCWHEN - a 'sync when' code block
- WAITPRI - a 'waitpri' code block
- EXCEP - an 'unless' code block
- PETRINET - a petrinet code block

Body body - the code tree for the body.

int callcount - a unique integer for this execution of the associated code body.

String fmodname - if this scope is for a file module then this member variable is the module name,

static TreeMap callcounts - values of the callcount integer indexed by the body name. This is used to keep separate sequences of integers for each unique module, procedure or function identifier and thus restrict the integer lengths compared with using a single increasing sequence for all identifiers.

static int sn - counter for generating unique number for each scope instance.

1.4.5 Type class - Exec/Type

The type of a variable, whether immediate mode or one of the target modes, is decribed by class **Type**.

A Type instance can be one of -

- a 1-dimensional array
- a struct
- a primitive

A Type instance which is a 1-dimensional array specifies the dimension and the type of the array. Multi-dimensional arrays are nested type where the array type is itself an array type.

A Type instance which is a struct contains a map whose keys are field identifiers, each associated value being the Type for that field. There is also a list of field identifiers.

A Type instance which is a primitive type contains a primitive type which is an enum defined in interface Parser/Constant. If the type is used for target variables it will usually also have a width (number of bits) except for type LOG which is always 1 bit.

fields

prim_type - this is the primitive type, an enumerated type Ptype defined in interface parser/Constant. For an array or a struct the primitive type is Ptype.NONE.

width - the primitive type width in bits.

fixoffest - the fraction size in bits for a fixed point type.

mant - the mantissa size in bits for a floating point type.

bexp - the biased exponent size in bits for a floating point type.

dim - the number of dimensions for an array type. This field avoids the need to scan down the nested array types to determine an array dimension.

field_map - a map of struct fields, the key being the field identifier and the value the nested type for that field.

field_index - a list of struct field identifiers.

array_type - the type of an array.

enum_ids - a list of the identifiers for an enumerated type.

enum_ords - a list of the ordinals for an enumerated type.

has_i_type - true if has only immediate type(s), primitive or nested.

has_t_type - true if has only target type(s), primitive or nested.

has_p_type - true if contains a pointer type, primitive or nested.

has_e_type - true if contains an enumerated type, primitive or nested

methods

The private method **makeType** is called by two of the five constructors. It returns a Type class instance which it builds from a type string argument. For nested types it calls itself recursively.

The method **getTypeString** does the reverse process, returning a string representing the type.

The method **getWordSpec** has three signatures, two more specific ones calling the more general third. They return instances of class WordSpec derived from the type, either as a whole or as modified by array subscripts or struct fields.

1.4.6 WordSpec class - Exec/WordSpec

The **WordSpec** class provides information about the mapping of data types into variables. Data types are mapped as a 1-dimensional array of words. Each word is a primitive.

A WordSpec may be generated from a Type by the method Type.getWordSpec(). There are two signatures, one for the entire variable and another for components of the variable given subscripts or fields.

The WordSpec class is intended to specify words or bit fields for an immediate or target mode variable. These words or bitfields should match the components of the type for the variable.

The member variable **word** is an ArrayList of word indices and is used for both immediate mode and target mode variables. When the WordSpec applies to an entire 3PL variable this list is a contiguous sequence starting at 0. When the WordSpec applies to only a part of a variable this list is ascending but not necessarily contiguous.

If the WordSpec applies to a target variable there is also a matching ArrayList of bit pairs, **bitpairs**, each entry giving the lower and upper bit indices of the associated word. For an immediate mode variable **bitpairs** is not used. The type of **bitpairs** is class **Pair**. If the type is "log" it is not clear if the mode is immediate or not so a bitpair 0..0 is included.

Generally a WordSpec describes an entire variable or one or more components of a variable. It should not specify a part of a variable which does not match the structure of the declared variable. The boolean field **complete** is true if a WordSpec specifies a complete variable.

A wordSpec can also be used to describe the structure of an expression generated by the compiler, which are usually primitive, i.e. have a single word and bitpair. These internal signals usually have identifiers of the form "E1234" which cannot clash with the identifiers of program variables.

Where a portion of a target variable component is required this is not obtained by a WordSpec specifying the bits directly as this would not match the declaration WordSpec. Instead the entire variable component is evaluated and a part is then extracted by a suitable operator and connected (assigned) to an internally generated signal as described in the previous paragraph.The CONNECT TDE can perform some pruning and shifting of bits, otherwise the OPERATOR TDE might be used. These methods are used for example to extract the sign of a variable or the biased exponent or mantissa of a floating point target variable.

The member variable **types** is an ArrayList of the primitive types of the words within the 3PL variable. Each entry in this list corresponds to the **word** entry at the same list index.

Class **Pair** contains four variables -

- **lower** - int - the lowest bit offset for this word
- **upper** - int - the highest bit offset for this word
- **indices** - ArrayList - a list of subscripts and field names for this word
- **select** - TDEVar - a select signal used in the case of a target variable subscript

For example the declarations

```
1    static(s1, "[3][4]int:4");
```

would create for **s1** a WordSpec containing the lists -

word	types	bitpairs		indices
		lower	upper	
0	typeINT	0	3	0, 0
1	typeINT	4	7	0, 1
2	typeINT	8	11	0, 2
3	typeINT	12	15	0, 3
4	typeINT	16	19	1, 0
5	typeINT	20	23	1, 1
6	typeINT	24	27	1, 2
7	typeINT	28	31	1, 3
8	typeINT	32	35	2, 0
9	typeINT	36	39	2, 1
10	typeINT	40	43	2, 2
11	typeINT	44	47	2, 3

which describes the layout of the entire variable. An instance of the variable

```
s1[1][1..2]
```

would create a WordSpec

word	types	bitpairs		indices
		lower	upper	
5	typeINT	20	23	1, 1
6	typeINT	24	27	1, 2

which locates two words within the variable.

The entries in **word** must be ascending but do not need to be contiguous. An example is a two-dimensional array where the second subscript or subscript range does not specifiy the entire dimension. If the previous exmaple were changed to -

```
s1[0..1][2..3]
```

the WordSpec would contain -

word	types	bitpairs		indices
		lower	upper	
1	typeINT	4	7	0, 1
2	typeINT	8	11	0, 2
5	typeINT	20	23	1, 1
6	typeINT	24	27	1, 2

A further example using both arrays and structs -

```
    struct(s,
        f1, "[4]int:4",
        f2, "log"
    );
    static(s2, "[3]s");
```

The WordSpec for the entire variable **s2** would be -

word	types	bitpairs lower	bitpairs upper	indices
0	typeINT	0	3	0, "f1", 0
1	typeINT	4	7	0, "f1", 1
2	typeINT	8	11	0, "f1", 2
3	typeINT	12	15	0, "f1", 3
4	typeLOG	16	16	0, "f2"
5	typeINT	17	20	1, "f1", 0
6	typeINT	21	24	1, "f1", 1
7	typeINT	25	28	1, "f1", 2
8	typeINT	29	32	1, "f1", 3
9	typeLOG	33	33	1, "f2"
10	typeINT	34	37	2, "f1", 0
11	typeINT	38	41	2, "f1", 1
12	typeINT	42	45	2, "f1", 2
13	typeINT	46	49	2, "f1", 3
14	typeLOG	50	50	2, "f2"

An instance

```
s2[1]
```

would create a WordSpec

word	types	bitpairs lower	bitpairs upper	indices
5	typeINT	17	20	1, "f1", 0
6	typeINT	21	24	1, "f1", 1
7	typeINT	25	28	1, "f1", 2
8	typeINT	29	32	1, "f1", 3
9	typeLOG	33	33	1, "f2"

```
s2[1].f1
```

would create a WordSpec

word	types	bitpairs lower	bitpairs upper	indices
5	typeINT	17	20	1, "f1", 0
6	typeINT	21	24	1, "f1", 1
7	typeINT	25	28	1, "f1", 2
8	typeINT	29	32	1, "f1", 3

and

```
s2[1].f2
```

would create a WordSpec

word	types	bitpairs lower	bitpairs upper	indices
9	typeLOG	33	33	1, "f2"

Where variable subscripts are used the **select** variable in class **Pair** is used. It is only used in a WordSpec describing a variable reference or value, not in a Wordspec describing a created variable. For example for a variable created by -

```
static(s3, "[3][4]int:4");
```

and evaluated by -

```
1    static(ss1, "uint:2");
2    value(ss2, "uint:2");
3    s3[ss1][ss2]
```

the WordSpec for this evaluation of **s3** might be -

word	types	bitpairs		indices	select
		lower	upper		
0	typeINT	0	3	0, 0	S147
1	typeINT	4	7	0, 1	S148
2	typeINT	8	11	0, 2	S149
3	typeINT	12	15	0, 3	S150
4	typeINT	16	19	1, 0	S151
5	typeINT	20	23	1, 1	S152
6	typeINT	24	27	1, 2	S153
7	typeINT	28	31	1, 3	S154
8	typeINT	32	35	2, 0	S155
9	typeINT	36	39	2, 1	S156
10	typeINT	40	43	2, 2	S157
11	typeINT	44	47	2, 3	S158

where the select signals are outputs of decoders for all pairs of values of the two variable subscripts.

If the first subscript was an immediate value -

```
1    s3[1][ss2]
```

the WordSpec for this evaluation of **s3** might be -

word	types	bitpairs		indices	select
		lower	upper		
4	typeINT	16	19	1, 0	S203
5	typeINT	20	23	1, 1	S204
6	typeINT	24	27	1, 2	S205
7	typeINT	28	31	1, 3	S206

where the select signals decode the single variable subscript.

If the second subscript was an immediate value -

```
1    s3[ss1][1]
```

the WordSpec for this evaluation of **s3** might be -

word	types	bitpairs		indices	select
		lower	upper		
1	typeINT	4	7	0, 1	S341
5	typeINT	20	23	1, 1	S342
9	typeINT	36	39	2, 1	S343

The select signals are mutually exclusive, each being high when the value of the subscript variables matches the indices of the associated word (array member). For a variable evaluation these signals are used with a SELECT TDE element to multiplex the output of a variable. In the case of a variable assignment using variable subscripts these signals are used to select the word within the variable which is to be written.

The following methods for class WordSpec provide access to information for the member variables discussed above -

- numWords() - the number of words
- numBits() - the total number of bits
- getWord(i) - the ith word index
- getPrimType(i) - the ith word primitive type

- getLower(i) - the ith word lower bit index
- getUpper(i) - the ith word upper bit index
- getWidth(i) - the ith word width (bits)
- getSelect(i) - the ith word select signal

Other member variables for WordSpec are -

ArrayList subvals

If this word specification applies to an expression containing target variable subscripts then this list contains the **Val**s for those subscripts in left-to-right order. The **select** entries in the **Pair** list are filled in by a single call to method **processVarSubs()** which uses the entries in member variable **subvals**.

int[] dim_des

A dimensional description - see below.

Type check_type

The type to check for assignment - see below.

Type type

The type - see below.

QueueRefs squeues

If this word specification applies to an expression containing queue reads or to a queue variable being written then this member variable specifies the queue dependencies.

Var var

If this word specification applies to a variable then this class member is the variable.

the dim_des, check_type and type fields

When a variable is assigned a value the type of the variable must be checked against the type of the value. For primitive types this is straightforward. For array types the process is more complicated. For example the 3PL code

```
1    var(a, "[7][3][4]int");
2    var(b, "[3][4]int");
3    a[2..4][1] = b;
```

is correct in that the dimensionality of **a[2..4][1]** is **[3][4]**. This is because the first subscript is a subrange of dimension 3, the second subscript eliminates that dimension and the third subscript is missing and so defaults to the dimension **[4]**.

When this class was first implemented the view was taken that the problem of dimensionality could best be solved by dividing a reference into leading subscripts followed by a type. The only complicating issue is the occurence of structs nested among arrays. To access an array within a struct however requires that the struct field be explicitly given, which immediately eliminates the rest of the struct. On the other hand any struct which is referenced intact cannot have the dimensionality of nested arrays changed (subrange or single subscript). Thus all references no matter how complicated can be reduced for the purposes of dimensionality and type checking into a leading number of array dimensions followed by a type. The type may itself contain arrays, however the dimensions of these will be as created, i.e. have not been changed by subscripts or subscript ranges. The dimensional description is the field **dim_des** which is a one-dimensional array whose size gives the number of dimensions of the reference and each entry gives the dimension. The type is the **check_type** field which can be checked for equality. The dimensional description describes the reference through the leading subscripts, be they missing dimensions or sub arrays including struct fields of that type. Single subscripts do not appear as they eliminate that dimension, similarly struct field references disappear as they eliminate the struct as a whole. When the description finally encounters a primitive of a complete struct (regardless of what the struct contains, including nested arrays) the rest of the type becomes **check_type**. For examples see appendix A.

It was later found necessary to add a proper type field representing the actual type of the reference. This is easily generated by iterating through the **dim_des** array creating nested class **Type** of type array and then finally nesting **check_type** at the end.

In retrospect it would have been easier and simpler to generate the type field directly and check types against each other for equality rather than bother with the dimensional description and then have to generate **Type** from it anyway. The code could be changed, however the tentacles of the current approach spread widely into surrounding code and eliminating bugs introduced by such a change would take some time. It may be cleaned up in the future but for the moment this obscure mechanism remains in place.

1.4.7 RefOrVal class - Exec/RefOrVal

The appearance of a variable on the LHS of an assignment statement is described by the **Ref** class (reference). The value of a variable or expression (for example on the RHS of an assignment statement) is described by the **Val** class (value). These classes are described in following sections, but as might be expected they require many of the same member variables. To avoid duplication these common member variables are contained in a super class, **RefOrVal** to which **Ref** and **Val** are subclasses.

The member variables are -

Mode mode; - the mode of the reference or value. For a target expression it will be VALUE mode.

WordSpec wordspec; - the word specifier.

SubFieldList subfields; - a list of subscripts or fields following a variable.

SubFieldList indsubfields; - a list of subscripts or fields following an indirect operator.

SrcLoc loc; - the source file location.

Var var; - the variable if the reference or value is for a single variable (not an expression).

TDEVar tdevar; - the TDE variable if a target reference or value.

String pkey; - a key used for matching an argument to a parameter in a module, procedure or function call when this form of argument passing is used.

HashSet<Object> ovars; - a set of source target variables (or VALUE mode/WordSpec pairs) collectively contributing to the output clock domain of a value, including target subscripts. This set is checked to determine the output clock domains and ensure that that clock domain of all variables in the set is the same. In the case of value mode variables a WordSpec is enclosed with the value mode variable in class ValWordSpecPair as the associated WordSpec is used to determine which components of the variable are being accessed and to label each as 'used'.

HashSet<Var> ivars; - a set of destination variables to be changed by assignment. This set is checked to determine the input clock domains and ensure that that clock domain of all variables in the set is the same.

boolean clks_resolved; - this is true if both the input and output clock domains have been checked and match (see method resolveClocks()).

Clock clkvar; - the resolved clock domain for this reference or value, determined by method resolveClocks().

QueueRefs queues; - information on any queues read or written.

ArrayList¡HashSet¡TDEVar¿¿ execs; - a list containing a number of sets. Each inner set is associated with a combinatorial memory read or a used() function. When a Val is evaluated for a target assignment the execution signal for that assignment is added to all inner sets in the 'execs' list of that Val. The field is also carried by a Ref so it can be transferred to a Val derived from that Ref. The inner set propagated from a used() function will only have a single execute signal added. Where a combinatorial output memory read is used multiple times via a Value variable the inner set propagated from that read will have multiple execute signals. Where an expression contains both a used() function call and multiple combinatorial output memory reads the 'execs' list will have several sets.

1.4.8 Ref class - Exec/Ref

This class extends class **RefOrVal**. It represents a variable reference. It is used when a variable is to be assigned and is also used to pass an argument if it is passed by reference.

The member variables are -

Flag flag; - a flag indicating if this variable reference has a pre or post increment or decrement operation associated with it.

TDEVar select; - a temporary variable holding a signal representing a decoded target subscript value for a partial reference (see subclass TargSubIterator and method TargSubIterator()).

Ident id; - an identifier used for arguments passed to modules, procedures and functions. It allows procedures to be written which can create variables using the identifier of the argument.

1.4.9 Val class - Exec/Val

This class extends class **RefOrVal**. It represents the value of a variable or an expression.

For immediate modes the value contains an array of objects holding the actual values of primitive or compound types. Expressions will have single primitive values but variables or functions may have an array of values from a compound type.

The value also contains an array of primitive type codes matching the value array. This type array is present for target values also, however target values do not use a value array.

Target values contain a TDEVar which is a signal representing the value computed in the FPGA (this is in the super class RefOrVal).

The member variables are -

Object[] val; - an array containing the immediate or target values. The 'wordspec' field inherited from class RefOrVal describes the entries in the 'val' array; each word in 'wordspec' matches the corresponding entry in 'val'.

Type[] val_type; - the type of each word. The type will always be primitive.

Note that arrays **val** and **val_type** and the associated **WordSpec** are contiguous. They may represent elements from a complex type which may have gaps, for example a slice through a multi-dimensional array which will probably not consist of spatially contiguous elements, but the selected elements in **val** and **val_type** will contain the values of selected elements with no gaps.

PtrType mpfcode; - indicates the type if the value is a pointer to a module, procedure or function.

1.4.10 QueueRefs class - Exec/QueueRefs

This class represents a queue reference. It is used in the generation of queue pop signals. Queue availability may be conditional on a signal; the test operand for a `?:` operator or a selection input to a **selecta()** function.

```
1     private HashSet<Queue>          write_queues = new HashSet<Queue>();
2     private HashSet<Queue>          read_queues = new HashSet<Queue>();
3     private HashSet<Queue>          write_av_queues = new HashSet<Queue>();
4     private HashSet<Queue>          read_av_queues = new HashSet<Queue>();
5     private ArrayList<SetPair>      read_ands = new ArrayList<SetPair>();
6     private HashMap<Scope,Val>      bqavails = new HashMap<Scope,Val>();
7     private TDEVar                  ubqwavails;
8     private TDEVar                  ubqravails;
9     private HashMap<Scope,Val>      syncavails = new HashMap<Scope,Val>();
10    private HashMap<Scope,Val>      ravails = new HashMap<Scope,Val>();
11    private int                     bwqueues;   // number of buffered queues being written
12    private int                     brqueues;   // number of buffered queues being read
13    private int                     ubwqueues;  // number of unbuffered queues being written
14    private int                     ubrqueues;  // number of unbuffered queues being read
```

write_queues is a (possibly empty) set containing the queue variables whose write availability is required. This is used by **sync**.

read_queues is a (possibly empty) set containing the queue variables whose read availability is required. This is used by **sync**.

write_av_queues is a set of queues for which write availability has already been checked and hence which can be excluded from a write availability logical expression.

read_av_queues is a set of queues for which read availability has already been checked and hence which can be excluded from a read availability logical expression.

bqavails is a map of availability signals for buffered queues. The key for each entry is the scope and the value is a combined read and write availability, including conditional execution signals. Entries are added when method **getBQAvail()** is called. If the call is made again for the same scope the value from the map is returned avoiding recomputing the logic expression.

ubqwavails is a signal giving write availability for unbuffered. The signal is encoded when method **getUBQWAvail()** is called. If the call is made again the same signal is returned avoiding recomputing it.

ubqravails is a signal giving read availability for unbuffered queues. The signal is encoded when method **getUBQWAvail()** is called. If the call is made again the same signal is returned avoiding recomputing it.

syncavails is a map of **sync** availability signals for both buffered and unbuffered queues. The key for each entry is the scope and the value is a combined read and write availability signal suitable for the control of entry to a **sync** block. read availability does not include conditional execution. Entries are added when evaluation of a **sync** block entry availability signal is computed on calling method **getSyncAvail()**. If the call is made again for the same scope the value from the map is returned avoiding recomputing the logic expression.

ravails is a map of read availability signals. The key for each entry is the scope and the value is a read availability signal for a < operator. Entries are added when evaluation of a < operator is performed and **getRavail()** is called. If the same read availability signal is required again for the same scope the value from the map is returned avoiding recomputing the logic expression.

read_ands is a set of queues and conditional signals which are all to be used to form availability and pop expressions. This field is a set of set pairs, the pair being a queue variable set and a conditional signal set. For each set entry the queue and signal sets are all ANDed together to form one term. The terms for each position in the lists are ORed together. Sets are used for the queues and signals to be ANDed to avoid duplicate entries.

When a write queue is ANDed to this queue reference by method **and_set()** it is added to the **write_queues** set.

When a read queue is ANDed to this queue reference by method **and_set()** it is added to the **read_queues** set. If set **read_ands** is empty the queue is added as a new pair otherwise it is added to every queue set in list **read_ands**.

When a conditional signal is ANDed to this queue reference by method **and()** it is added to every signal set in list **read_ands**.

When another queue reference is ANDed to this queue reference by method **and_set()**, if this queue reference **read_ands** set is not empty the product of the two queue references is generated by taking each combination of list indices from each queue reference and taking a union of the sets. If this queue reference **read_ands** set is empty the queue reference is copied.

When another queue reference is ORed to this queue reference by method **or()** the lists are appended.

This process of multiplying through each time means that we maintain the queue dependencies as a sum of products, no deeper.

example 1 -

```
1    p1 + (l1 ? p1 : p2) - (l2 ? p2 : p1)
```

The 1st term produces the reference (sets are shown in braces [..]) -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1]	[p1]	[-]	[]

The 2nd term -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2]	[p1] [p2]	[l1] [!l1]	[]

These two terms are now ANDed -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2]	[p1] [p1 p2]	[l1] [!l1]	[]

The last term is -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2]	[p2] [p1]	[l2] [!l2]	[]

which is ANDed with the accumulated reference -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2]	[p1 p2] [p1 p2] [p1] [p1 p2]	[l1 l2] [!l1 !l2] [l1 !l2] [!l1 l2]	[]

giving the availability expression

```
1    (p1.NE & p2.NE & l1 & l2) | (p1.NE & p2.NE & !l1 & !l2) | (p1.NE & l1 & !l2) |
2                                                            (p1.NE & p2.NE & !l1 & l2)
```

This looks messy but is just a function of 4 variables so only takes 1 LUT.

We get the pop signals by taking each queue in turn (from the **read_queues** set) and ORing together each conditional signal entry whose corresponding queue entry contains the queue. The set of signals is ANDed. The result is ANDed with the execution signal.

p1.POP

```
1    ((l1 & l2) | (!l1 & !l2) | (l1 & !l2) | (!l1 & l2)) & exec
```

p2.POP

```
1    ((l1 & l2) | (!l1 & !l2) | (!l1 & l2)) & exec
```

this will be optimised by the Xilinx software to -

p1.POP

```
1  exec
```

p2.POP

```
1    (!l1 | l2) & exec
```

and each will only take 1 LUT

example 2 -

```
p1 + l1 ? (l2 ? p1 : p2) : p3 - l3 ? p1 : c
```

The 1st term produces the reference -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1]	[p1]	[]	[]

The 2nd term nested conditional -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2]	[p1] [p2]	[l2] [!l2]	[]

The complete 2nd term -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2 p3]	[p1] [p2] [p3]	[l2 l1] [!l2 l1] [!l2 !l1]	[]

AND with the 1st term -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2 p3]	[p1] [p2 p1] [p3 p1]	[l2 l1] [!l2 l1] [!l2 !l1]	[]

The last term -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1]	[p1] []	[l3] [!l3]	[]

Which is ANDed to the accumulated reference -

read_queues	read_ands.queues	read_ands.sigs	write_queues
[p1 p2 p3]	[p1] [p2 p1] [p3 p1] [p1] [p2 p1] [p3 p1]	[l2 l1 l3] [!l2 l1 l3] [!l2 !l1 l3] [l2 l1 !l3] [!l2 l1 !l3] [!l2 !l1 !l3]	[]

giving the availability expression

```
p1 & l2 & l1 & l3 | p2 & p1 & !l2 & l1 & l3 | p3 & p1 & !l2 & !l1 & l3 | p1 & l2 & l1 & !l3 |
                        p2 & p1 & !l2 & l1 & !l3 | p3 & p1 & !l2 & !l1 & !l3
```

a function of 6 variables, and queue pop expressions

p1

```
(l2 & l1 & l3 | !l2 & l1 & l3 | !l2 & !l1 & l3 | l2 & l1 & !l3 | !l2 & l1 & !l3 | !l2 & !l1 & !l3) & exec
```

p2

```
(!l2 & l1 & l3 | !l2 & l1 & !l3) & exec
```

p3

```
(!l2 & !l1 & l3 | !l2 & !l1 & !l3) & exec
```

To OR a new term -

- Add the **write_queues** set (actually should never get a write queue OR).
- Add the **read_queues** set.
- Append the **read_ands** list.

To AND a new term -

- Add the 'write_queues' set.
- Add the 'read_queues' set.
- For queue/signal pairs in the class and queue/signal pairs in the argument take all combinations of one pair from each list and form new lists of the union of the sets (the product of the lists). For example

p1	p2
s1,s2	s3

by

p3	p5,p6
s4	

gives the products

p1,p3	p2,p3	p1,p5,p6	p2,p5,p6
s1,s2,s4	s3,s4	s1,s2,s4	s3

Obviously the product of any queue or signal with itself is itself, and the use of sets automatically brings this about.

Now eliminate any resulting superset terms - these are redundant product terms in the sum that have resulted from an AND, e.g.

```
(a | b) & (a | c)
```

gives

```
a | b & a | a & c | b & c
```

equals

```
a & (1 | b | c) | b & c
```

which is

```
a | b & c
```

The 2nd and 3rd terms are supersets of the 1st so can be eliminated.

1.4.11 SubField class - Exec/SubField

This class represents an array subscript or subscript range or a struct field. Class variables are -

SFType type - descriptor type -

- SFType.NULL - a missing subscript
- SFType.IMSUB - a single immediate subscript
- SFType.IMSUBS - an immediate subscript range
- SFType.TARSUB - a target subscript expression
- SFType.FIELD - a field name

int lower - immediate constant lower or only subscript

int upper - immediate constant upper subscript

String field - field or map key identifier

Val subval - target subscript value

SrcLoc loc - source file location

1.4.12 SubFieldList class - Exec/SubFieldList

This class creates a list of subscript and field identifiers as they occur with a variable value or reference.

Chapter 2 - Intermediate Target Code

The intermediate code is produced during immediate execution and respresents the FPGA logic. The code is contained in class **TDEList** which extends class **ArrayList**. The acronym **TDE** appears through this description and stands for *Topological Description Element*, i.e. elements describing the logic layout. The main constituents are class **TDE** which is a basic logic element and **TDEVar** which is a signal between elements.

2.1 TDEVar class - CodeGen/TDEVar.java

The intermediate code list (**TDE list**)is not a linked netlist in the usual sense. In a conventional netlist each net is a single signal linked to one or more logic element inputs and a single element output (unless a 3-state output). The intermediate code list employs TDEVar class instances to represent signals.

The TDEVar class represents a signal or signals in the intermediate code. A TDEVar may have an associated WordSpec which describes the signals in an array or struct, the type(s) these represent and optionally the variable for which the TDEVar was created.

In the simplest case a TDEVar may represent a logical signal controlling the flow of execution. For example an execution thread is just a logical high passed along a chain of delay or control elements. Conditional control signals also enable static variable assignments, memory reads and writes and queue pushes and pops. These simple single signals in most cases do not have an associated WordSpec, or if they do it has primitive type LOG. These TDEVars usually have compiler generated identifiers such as "S123" (these cannot clash with identifiers from 3PL source code variables).

TDEVars also represent inputs or outputs of variables or expressions (including constants). In that case they have an associated WordSpec which indicates the primitive types and bits of a variable and its fields or array members.

A TDEVar representing a value, for example the output of a static variable, is created when the need arises in generating logic during immediate execution. If multiple distinct instances of the same value are created this does not affect the conversion of intermediate code into the EDIF netlist as netlist names are generated from the identifier and the specified bits of the variable. Thus separate insances of the same TDEVar come together as connected when converted to the netlist since they share the same identifier and bit specification, i.e. they are imlicitly rather than explicitly connected. Even though this scheme is logically correct it does pose problems for optimisation as it is not possible within the intermediate code to recognise that identical TDEVar instances represent the same signals and will indeed lead ultimately to shared netlists. As a means of obviating this problem a map of created TDEVars is kept. Each TDEVar is entered into the map (catalog) with the identifier as key. The entries under each key are lists of instances of TDEVars. When a TDEVar instance is to be created the list from the map entry for that identifier is searched for an identical bit specification (provided by the WordSpec). If one is found then that TDEVar is returned, otherwise a new one is created and entered into the map. This avoids duplication of TDEVars and means that separate references to the same value are via a common TDEVar which implies that they are connected in the intermediate code list. Thus optimisation code can locate all sources and sinks for a value and effect any useful optimisation.

The TDEVar catalog is not applied universally. In many cases the TDEVars carry additional information in the associated WordSpec which varies from instance to instance, for example variable subscripts. But since the logic is still correct even when TDEVar duplication cannot be avoided these exceptions do not invalidate the process.

TDEVars may be explicitly connected in the TDE list by two means -

- a CONNECT TDE
- method TDEVar.merge()

In both cases the identifiers will be different as otherwise TDEVars are implicitly connected as explained above and do not need explicit connection.

A CONNECT TDE explicitly connects two TDEVars. The CONNECT TDE is also able to expand or truncate connections between input and output where the signals are of different size.

The merge() method is only called in method TDEList.optimise() and is restricted to the following specific situations -

- merging of TDEVars which specify complete variables, i.e. the associated WordSpecs will have the **complete** field true, and which have the same number of bits.
- merging of single control signals
- merging of clock signals

2.1.1 TDEVar fields

String id

The variable identifier if **TDEVtype.VAR** (actually if it is not type **TDEVtype.VAR** it will have an identifier similar to "CONST123" but this is solely for diagnostic labelling purposes and is not used by the compiler).

TDEVtype type

This distinguishes between a normal TDEVar, **TDEVtype.VAR**, and a constant. When a TDEVar is a constant it is an implicit signal array source. The type of constant is given by this field with values **TDEVtype.BITS**, **TDEVtype.UINT**, **TDEVtype.INT** and **TDEVtype.BOOL**. The actual value of the constant is stored in field **Object val**. A constant TDEVar is converted to a signal array, sourced with a pattern of connections to VCC and GND representing the value, during netlist structure generation.

WordSpec wordspec

If this TDEVar is a value array then this field describes the word boundaries and the word types of this reference.

If this TDEVar is an execution signal then this field is null.

Object val

Value if a constant - see **TDEVtype type** above. Otherwise this field is null.

SrcLoc loc

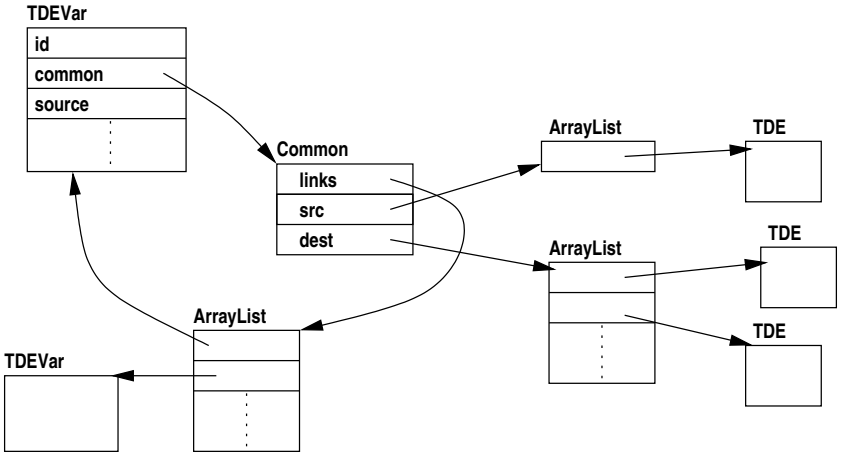
Source file location where this TDEVar was generated.

Common common

The **Common** subclass is described in the following section. It is used to link connected TDEVars which are simple control signals, clock signals or represent complete variables (not array members or struct fields).

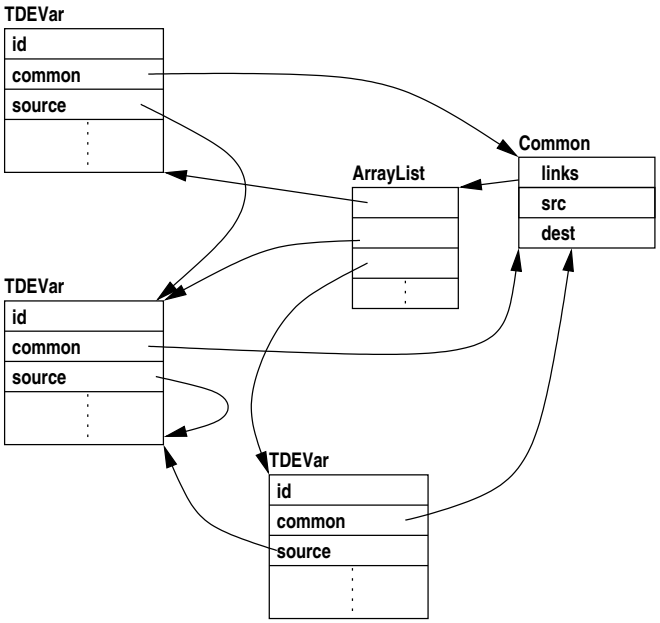
private static HashMap<String,ArrayList<TDEVar>> catalog

This static variable maps signal identifiers to all TDEVars which have that identifier.



2.1.2 Common class

Every TDEVar contains a **common** field. Where two TDEVar instances represent the same signal or signal array they can be merged by combining their common fields into a single shared object.



For example in the following 3PL code -

```
1    static(global.s1, "uint:4");
2    static(global.s2, "uint:4");
3    .
4    .
5    s1 <- s2;
6    .
7    .
```

The LHS reference in line 5 will generate a TDEVar which will appear in the TDE list as **glob.s1.D[0..3]**, i.e. the D or data input to register s1. The RHS expression in line 5 will contain a TDEVar listed as **glob.s2[0..3]**, the output of register s2.

The assignment statement will connect the s1 data input to the s2 output, the s1 CE input (clock enable) to a control signal that goes high when the statement is executed and the s1 RES (reset) to GND.

```
1 .
2 .
```

```
FMOD1/glob.s1.D[0..3] = FMOD1/glob.s2[0..3]
FMOD1/glob.s1.CE[0..3] = S123
FMOD1/glob.s1.RES[0..3] = GND
.
.
REG
    OUT  FMOD1/glob.s1[0..3]
    <-
    CLK  glob.c166
    D    FMOD1/glob.s1.D[0..3]
    CE   FMOD1/glob.s1.CE[0..3]
    R    FMOD1/glob.s1.RES[0..3]
    {0x0}
```

After optimisation the CONNECT TDEs will be removed and the merged TDEVars will result in -

```
.
.
REG
    OUT  FMOD1/glob.s1[0..3]
                <-
    CLK  glob.c166
    D    FMOD1/glob.s2[0..3]
    CE   s123
    R    GND
    {0x0}
```

The funtionality has not changed and the merges are not logically necessary, but the TDE list is shorter and easier to read.

The fields in common are -

- **Clock clock_var**
  This is the clock variable for the clock domain of the TDEVar(s). Constant TDEVars do not have a clock domain in which case this is null.
- **ArrayList<TDE> dest**
  A list of TDEs which are destinations (sinks) for this signal.
- **ArrayList<TDE> src**
  A list of TDEs which are sources for this signal.
- **ArrayList<TDEVar> links**
  TDEVars whose sources point here

2.1.3 the merge method

As immediate execution proceeds connection of signals is done by method TDEList.connect(). This generates a **CONNECT** TDE with one input TDEVar (the source) and one output TDEVar (the destination). If the input and output are different sizes then this represents a cast() which will pad or truncate the input array to match the output array. The input may even be a single signal to be connected to each bit of an output array, often the case with input GND or VCC.

These CONNECT TDEs are resolved when processed by netlist.Xilinx.XTDECode(), however the presence of these CONNECT TDEs makes the intermediate netlist code difficult to read. As a dump of the intermediate netlist is a useful resource for checking the generated code for validity or determining problems with combinatorial expressions failing timing constraints, some effort has gone into resolving some of these **CONNECT** TDEs during optimisation of the intermediate netlist, thus simplifying the presentation substantially. Note that this simplification is not required for generation of the final EDIF file from the intermediate netlist, it is simply to improve readability.

The process is carried out during optimisation by examining all **CONNECT** TDEs (there is a list of them in class TDEList) and where appropriate merging the set of destination TDEVars with the source TDEVar. This is done by method TDEVar.merge() using the common class.

Method **merge()** is called to merge two TDEVars. Before calling this method it must be determined that the two TDEVars meet one of the following criteria -

- the TDEVars are for control signals (WordSpec is null).
- the TDEVars are for clocks signals.
- the TDEVars are for complete variables.

This method does not perform these checks itself.

The TDEVar to which merge() is applied should be the source and the argument to merge() should be a destination (sink). The source ID is used as the preferred ID.

```
1     public void merge (TDEVar dtdev) {
2         Common  dc = dtdev.common;   // destination common
3         Common  sc = common;         // source common
4         String  old_dest_id = dtdev.id;
5         String  sid = id;
6
7         sc.src.addAll(dc.src);                  // add source TDEs
8         sc.dest.addAll(dc.dest);                // add destination TDEs
9         sc.links.addAll(dc.links);              // link destination TDEVar links to the source of this TDEVar
10
11        for (TDEVar t : dc.links) {
12            t.id = sid;
13            t.type = type;
14            t.wordspec = wordspec;
15            t.common = sc;
16            t.val = val;
17        }
18
19        // Update all occurrences of destination TDEVars with the same id
20        // with the new source id.
21        ArrayList<TDEVar>   al = catalog.get(old_dest_id);
22        for (TDEVar tdev : al) {
23            if (tdev == dtdev)
24                continue;
25            tdev.id = sid;
26        }
27    }
```

Both the source TDEVar (this) and destination TDEVar (dtdev) may already have other TDEVars linked via their respective Commons as a result of previous merges. As a minimum every TDEVar has a link to itself in its Common.

The destination common src, dest and links lists are appended to the corresponding lists in the source common.

The source TDEVar will already have the same ID, type, WordSpec, Common class instance and val field in all linked TDEVars as result of previous merges. All linked destination TDEVars now have the source ID, type, WordSpec, Common and val field written into them. The merged destination TDEVar will now have the same ID, type, WordSpec, Common and val fields as the source TDEVar.

Now that the destination TDEVars have had their IDs changed, any other instances of a TDEVar with the same ID, regardless of linkages or WordSpec, must be have their IDs changed as well. This is done using the catalog map.

2.1.4 target variables

Program target variables may be global, in which case the associated signal names begin with "glob.". Thus the output of a static global variable *a* would be a signal named *glob.a*.

All other target variables are prefixed by a string constructed from all the hierachical scopes within which they are created. Each scope has an associated identifier which is either a module, procedure or function identifier or else a control structure indicator such as "if" or "when". Unique integers are appended to these identifiers to avoid duplication that might result from repeated entry to a scope. A target variable *b* created within a source file prog.3pl would have the identifier *prog.b*. If it was created after calling procedure proc then it would be *prog.proc_1.b*. If the procedure were called again the resulting variable would be *prog.proc_2.b*. A variable *c* created within a 'for' loop would have identifiers *prog.proc_1.for_1.c*, *prog.proc_1.for_2.c* etc.

Hierarchical signal names must include file paths of source files. To avoid very long identifiers file paths from **source** or **module** statements are replaced by arbitrary strings SRCn or FMODn in identifiers, **n** being an integer. The optional intermediate code report file (prog.icl) lists these for reference purposes.

Signal array references have a subscript range appended, even if the entire variable is accessed, e.g. "glob.a[0..7]". For multiple ranges comma-separated ranges are shown, e.g. "glob.a[0..7,16..23]"

static variables

A global static variable *s* of size 8 bits would have the following associated signal arrays -

- "glob.s[8]" - the data output signal array
- "glob.s.D[8]" - the data input signal array
- "glob.s.CE[8]" - the clock enable signal array
- "glob.s.RES[8]" - the reset signal array

The value of the variable is available via the data output signal. Assignments are made by driving the data input signal and asserting the clock enable signals to the selected bits. The reset signal array is used to return some portion of the variable to its initial state.

queue variables

A global queue variable *p* of size 8 bits would have the following associated signal arrays -

- "glob.p[8]" - the data output signal array
- "glob.p.D[8]" - the data input signal array
- "glob.p.NE" - output availability (not empty)
- "glob.p.POP" - pop a datum
- "glob.p.PUSH" - push a datum
- "glob.p.NF" - input availability (not full)
- "glob.p.CNT[4]" - buffer occupancy count
- "glob.p.SPC[4]" - buffer space count
- "glob.p.RSTAT[5]" - buffer status in read clock domain
- "glob.p.WSTAT[5]" - buffer status in write clock domain

These signals are further described in the section on queue buffers below.

value variables

Value variables are effectively pointers and can be assigned multiple times. To ensure that each assigned value is distinguished a unique integer is appended. A global value variable *v* of 8 bits might thus appear variously as "glob.v_1", "glob.v_2" etc. At the completion of immediate program execution the signal array "glob.v" would be equated with the last value the variable was assigned. This is an attempt to give the unappended variable identifier a sensible value during simulation.

priority variables

A global priority variable *p* of size 8 bits would have the following associated signal arrays -

- "glob.p[8]" - the data output signal array
- "glob.p.D[8]" - the data input signal array
- "glob.p.PE" - the 'else' signal

memory variables

Memory variables have separate signal array identifiers for each port, e.g. global combinatorial-output memory *m* with two ports would have port output signal arrays "glob.m_0" and "glob.m_1".

2.1.5 expressions and execution control signals

The values resulting from expressions are given signal array identifiers of the form "E9".

"startex" is a level signal which is asserted to begin execution. It is resynchronised to every clock domain, for example for a global clock *clk* the resynchronised start signal would be "glob.clk.start".

Various other control signals are generated with prefices "A", "AND", "CD",, "CONST" "EO", "F", "FB", "FD", "FDCP", "FDRS", "FF", "FS", "I", "IN",, "INT" "INV", "LOG", "MADDR", "MDATA", "P", "PE", "PNTBUSY", "Q", "RADDR_", "ROPAV", "S", "SAMPLEOUT", "SB", "SC", "SD", "SS", "start", "TF", "TS", "UINT" and "WADDR_", integers being appended in each case to ensure identifiers are unique. there is no particular significance in these identifiers, the initial strings having been chosen to reflect the context in an attempt to improve readability of the intermediate code.

2.2 TDE class - CodeGen/TDE.java

The topological description is in terms of basic hardware units such as registers, selectors, control structures etc. linked by signals or signal arrays.

The topological description class includes the following fields -

type

This indicates the type of element. Types are (see class **TDEConstants** -

- TDEType.SIG - declare a signal or signal array
- TDEType.CONNECT - connect a signal or signal array to another or to a constant
- TDEType.SELECT - select one of several inputs
- TDEType.TSELECT - a 3-state driver
- TDEType.REG - a register
- TDEType.WAIT - wait for completion of parallel execution streams
- TDEType.DIVERGE - handle queue divergence
- TDEType.INV - invert a signal
- TDEType.AND - AND 2 or more signals
- TDEType.OR - OR 2 or more signals
- TDEType.XOR - XOR 2 or more signals
- TDEType.OPERATOR - apply an expression operator to operands
- TDEType.START - start an execution stream
- TDEType.EXECP - delay execution on queue availability
- TDEType.ILOOP - control an infinite loop
- TDEType.DEL - delay a signal one or more clock cycles
- TDEType.WHEN - a 'when' statement
- TDEType.WHILE - a 'while' statement
- TDEType.DOWHILE - a 'do while' statement
- TDEType.PRIORITY - a priority encoder
- TDEType.RESYNC - resynchronise a pulse or edge to another clock
- TDEType.SAMPLE - sample a value in another clock domain
- TDEType.QUEUEBUFFER - a queue variable (buffer)
- TDEType.IBUF - an FPGA pad input buffer
- TDEType.OBUF - an FPGA pad output buffer
- TDEType.DFF - a D-type flip-flop
- TDEType.CRAM - a combinatorial-output RAM
- TDEType.RRAM - a registered-output RAM
- TDEType.CLOCK - create a clock signal source
- TDEType.CLOCKINFO - a wrapper for information about a clock signal
- TDEType.ELEMENT - create a specific FPGA vendor library element
- TDEType.PORT - an external signal connection to/from another netlist
- TDEType.SPECIAL - special functions for specific FPGA families

ArrayList params is a parameter section which includes such things as initial values and flags. Items in the parameter list may be type **Integer**, **String** or **Boolean**.

ArrayList inputs and **ArrayList outputs** are lists of TDEVars which are inputs and outputs to the TDE.

location

This field gives the source file location. Since these structures represent the hardware rather than the source code the source file location does not provide a very good cross reference to the source code. For example the structure for a variable declaration includes all the logic required to assign values to that variable, including input selectors, however the selector logic is derived from the various assignment statements which target that variable. The line numbers of these statements are not available in the variable structure. We could partition the logic into smaller units to improve this situation, but that might impose restrictions on the implementation and might also preclude some simple low-level optimisations.

active

This field indicates if the TDE is still in use. During optimisation some TDEs are eliminated and rather than delete them from the list of elements they are instead tagged as inactive via this field.

no_delete

If this field is true the element should not be deleted. I think the only use for this is for the SELECT TDE where it is required when used for queue variable assignment that a single input SELECT not be eliminated as redundant.

2.2.1 constructors

There are a number of constructors all of which require the TDE type but which may omit parameter, input or output lists or source file location. Where omitted the lists are added later.

2.2.2 methods

add2i()
Add a signal to the input list.

add2o()
Add a signal to the output list.

add2p()
These add an item to the parameter list. There are multiple signatures for different argument types.

addPin2Element()
This method adds a pin and a connected signal to a TDE instance of type ELEMENT.

addProperty2Element()
This method adds a property to a TDE instance of type ELEMENT. The property name and value are added to the parameter list of the TDE instance.

setElementPin() This method adds a pin to a TDE instance of type ELEMENT. The port name, type (input, output, 3-state output or clock), associated TDEVar, port width and EDIF port array format are passed as arguments. The array size is used later (in XTDECode) to pad input nets formed from the TDEvar so that the full port width is connected even though the signal array may be narrower. Padding is GND unless the TDEVar is for a signed type in which case the MSB is propagated.

redundant()
This method examines a TDE to see if it contains redundancies or is entirely redundant. For example a duplicated selector data input is removed and the select input is ORed with the duplicate select. Logic

gates with a single input are effectively removed (see below) and the single input is connected to the output directly.

If the TDE is completely redundant it has all inputs, outputs and parameters removed, the active flag is made false and the method returns true. If the TDE is already in the TDE list it will be recognised as inactive during any traversals of the list and will be ignored.

The method is called in only two places, TDEList.addTDE() and TDEList.optimise(). The first is called to add a TDE to the TDE list but does not do so if the TDE is wholly redundant (redeundant() returns true). The second is a TDE list optimiser which calls redundant() only for SELECT TDEs as a part of its optimisation of the list. Since the TDE is already in the list a true return from redundant() simply means the TDE has been made inactive and will be ignored when encountered again in any context.

ncode()
This method generates netlist code from the TDE. A switch on the TDE type selects a matching method in abstract class **TDECode**, these methods being overridden in class XTDECode in the case of Xilinx.

deactivate()
Make a TDE inactive by unlinking all inputs and outputs from the associated TDEVars (removing the reverse link), clearing the parameter list and setting boolean **active** to false.

removeInput()
Remove a control signal input to a TDE. This is not used for signals which are arrays. The TDEVar is removed from the list of inputs and this TDE is removed from the destination list of the associated TYDEVar. This method assumes that there is only one occurrence of the TDEVar in the input list.

replaceInput()
Replace a TDEVar in the input list with another. The source link in the associated TDEVar is not changed by this method; it is assumed that that is done elsewhere.

replaceOutput()
Replace a TDEVar in the output list with another. The destination link in the associated TDEVar is not changed by this method; it is assumed that that is done elsewhere.

dump()
print a description of the TDE to the .icl file.

2.2.3 signal linking

Input and output signals to a TDE are representad by class TDEVar. TDE fields **input** and **output** are lists of TDEVars (class **TDEVarList** which extends class **ArrayList**). The function of each input and output is dependant on the specific TDE type. A TDEVar in turn has a list of TDEs to which it is linked (within subclass Common).

Optimisation of TDEs must handle this double linking where TDEVars are to be removed or relocated.

2.3 TDEVarList class - CodeGen/TDEVarList.java

The TDEVarList class extends class ArrayList. As well as providing a list class for TDEVars it provides methods for generating Nets from TDEVars.

The TDEVar class does not implement proper netlist but just provides a description of signals or signal arrays connected to TDEs. The TDEVar identifiers and WordSpecs imply that WHen the complete TDE list is processed to generate the final netlist each instance of a TDEVar must be converted to a Net. Methods in TDEVarList do this conversion.

2.4 TDEList class - CodeGen/TDEList.java

This class extends ArrayList. A TDE list is the intermediate form of generated code. The list has variable declarations at the front followed by logic using those variables. An index, 'variables', is used

to maintain the insertion point for variables at the head of the list. Method addTDE() appends code TDEs to the tail of the list. Method addVTDE inserts TDEs at the end of the leading variable section of the list. Field 'index' provides a unique integer to append to the end of generated signal names. It is incremented after each use.

There are a number of additional static lists of particular TDEs that are used for optimisation. These are -

- connects - CONNECT TDE
- dels - DEL TDE
- regs - REG TDE
- ops - OPERATOR TDE
- whens - WHEN TDE
- iloops - ILOOP TDE
- waits - WAIT TDE
- gates - AND, OR, XOR TDE
- sels - SELECT TDE
- execps - EXECP TDE
- dffs - DFF TDE

These lists avoid the need to scan the entire TDE list when optimising particular TDE types.

Another static list, **branches**, contains details of branch statements for later optimisation.

The static list **start_dffs** is built up as ILOOPs are optimised. It is later scanned to eliminate duplicate DFF elements with common start signal.

static HashMap<String,TDE> uelements is a map of elements created using the ELEMENT TDE. Static **current_uelement** is the most recent ELEMENT TDE to which connections can be added without explicitly using the element identifier.

2.4.1 method optimise

This method is called from the Main method (ThreePL.jj) after immediate execution. Optimisation, including that done in this method, is described in a later chapter.

2.4.2 method ncode

This method generates the EDIF netlist. It does this in the following steps -

The TDE list is traversed (skipping the initial signal declaration section) calling TDE.ncode() on each to generate netlist elements and nets via XTDECode methods.

```
Iterator<TDE>   it   = iterator();
TDE             tde  = null;
while (it.hasNext()) {
tde = it.next();
if (tde.isActive())
    tde.ncode(family);
}
```

A list of external ports is built from data previously extracted when processing IBUF and OBUF TDEs in XTDECode.

```
family.makePorts();
```

Any flip-flops in output pads whose outputs are used internally as well are duplicated by a CLB flip-flop (XNetListOptimise.optimiseOBUF() and XNetListOptimise.optimiseOBUFT()). Gates with output not connected, gates with one input and gates with duplicated inputs are removed or modified as appropriate. Family-specific devices such as XORCY, MUXF etc. are similarly optimised.

```
passes = family.optimiseNetlist();
```

Generic elements are converted to family-specific ones where necessary. Up to this point gates are as wide as required by the context. Here they are converted to combinatorial logic matching the CLBs in the FPGA family.

```
family.convertGenericToSpecific();
```

The netlist file header is written out.

The netlist cells are then written out to the netlist file.

```
Element.outputCells(pw, view);
```

The list of external ports is now written to the netlist file.

```
TDECode.outputExterns(pw);
```

Element instances are written.

```
Element.outputInstances(pw, view);
```

The nets are now written.

```
Net.outputInstances(pw);
```

The netlist file is now completed by writing out the part specification and the final curly bracket.

Finally constraints are written to the .ncf (ISE) or .xdc (Vivado) file.

```
family.outputConstraints(author, netlistcomments);
```

2.5 Intermediate code optimisation

The intermediate code is optimised prior to conversion to a netlist structure. The optimisation is described in a later chapter.

2.6 Conversion of intermediate code to netlist structure

The intermediate code consists largely of elements which can be expected to be available in any FPGA. This set of elements is enough to implement any 3PL program. Signals and signal arrays are arguments to these elements.

The intermediate code elements are -

- AND - AND 2 or more signals
- CONNECT - connect a signal or signal array to another or to a constant
- CRAM - a combinatorial-output RAM
- DEL - delay a signal one or more clock cycles
- DFF - a D-type flip-flop
- DIVERGE - handle queue divergence
- DOWHILE - a 'do while' statement
- ELEMENT - create a specific FPGA vendor library element
- EXECP - delay execution on queue availability
- IBUF - an FPGA pad input buffer
- ILOOP - control an infinite loop
- INV - invert a signal
- OBUF - an FPGA pad output buffer
- OPERATOR - apply an expression operator to operands
- OR - OR 2 or more signals
- PORT - an external signal connection to/from another netlist
- PRIORITY - a priority encoder

- QUEUEBUFFER - a queue variable (buffer)
- REG - a register
- RESYNC - resynchronise a pulse or edge to another clock
- RRAM - a registered-output RAM
- SAMPLE - sample a value in another clock domain
- SELECT - select one of several inputs
- SIG - declare a signal or signal array
- SPECIAL - special functions for specific FPGA families
- START - start an execution stream
- TSELECT - a 3-state driver
- WAIT - wait for completion of parallel execution streams
- WHEN - a 'when' statement
- WHILE - a 'while' statement
- XOR - XOR 2 or more signals

2.7 Intermediate code elements

2.7.1 AND

purpose

An AND gate for control signals.

arguments

Parameters	none
Inputs	input signal input signal . .
Outputs	AND signal

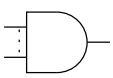
description

AND two or more input signals, the result appearing on the output.

dump format

```
1    S11 = A12 & A14 & S8
```

logic description



Xilinx implementation -

AND gates with 1 to 4 inputs will be implemented in a single LUT (single input gates will be eliminated during low-level code generation). AND gates with more than 4 inputs use the carry chain to AND together the outputs of 1 to 4 input LUT AND gates.

2.7.2 CONNECT

purpose

Connect signals or signal arrays.

arguments

Parameters	optional boolean for sign extension optional long right shift count optional boolean for left justification
Inputs	signal, signal array or constant
Outputs	signal or signal array

description

This connects signals or signal arrays.

If there is a single boolean parameter, a true value indicates sign extension will be used if the output is wider than the input. A false value or missing parameter indicates zero padding will be used if the output is wider than the input. If the output is narrower than the input then truncation will occur and the parameter is not relevant. The parameter may be omitted or null.

If there is a second parameter it is a right shift count. The sign extension indicated by the first parameter is observed. If the first parameter is present and true then the sign bit will be propagated during the right shift. The parameter may be omitted or null.

If there is a third parameter it indicates that the input is to be left justified with respect to the output, i.e. input bits wil be connected in order from MSB to LSB. There is no recognition of a sign bit. Where a size mismatch occurs truncation or zero padding will be used at the LSB end. The parameter may be omitted or null.

If there are no parameters then signal connection occurs in the following scenarios -

- signal = signal
- signal array = signal array
- signal array = signal
- signal array = constant

The first two cases connect signals or signal arrays together. The input (RHS) should be a source and the output (LHS) a sink. Signal arrays must be the same size, but the third case is allowed - a signal array connected to a single signal source (often GND or VCC).

In the last case a signal array is connected to a pattern of GND and VCC to implement a 64-bit constant. The constant (which may be signed) will be truncated to fit the signal array. This case may include a sign extension parameter, but the parameter is not relevant since the constant is guaranteed to fit the output width, including the sign.

dump format

```
1                       S11 = TS15
2                       E1[0..7] = glob.a[8..15]
3                       E2[0..2] = VCC
4                       E3[0..7] = 17
5                       E4[0..15] = glob.v[0..11] cast - pad
6                       E5[0..15] = glob.v[0..11] cast - sign_extend
```

logic description

No logic is generated by this element.

2.7.3 CRAM

purpose

Combinatorial-output RAM.

arguments

Parameters	Integer number of ports Integer data width in bits Integer address width in bits String[] optional initialisation String - "timingname" attribute key String - data for "timingname" attribute
Inputs	clock signal, port0 address array, port0 data_in array, or null, port0 write signal, or null, port0 clock signal, port1 address array, port1 data_in array, or null, port1 write signal, or null, port1 ..etc..
Outputs	data_out array, or null, port0 data_out array, or null, port1

description

This is a RAM with a clocked write but a combinatorial read. At all times the output for a port reflects the memory contents at the location indicated by the current address input.

The 2nd parameter gives the data width and the address width gives the memory depth (number of words). All port addresses must be the same width and all port data inputs and outputs must be the same width.

The initialisation string array has one entry for each initial value. Each entry is a hexadecimal string. The initialisation array may be smaller than the depth of the RAM but not larger. The initialisation parameter may be omitted in which case the RAM is explicitly initialised to all 0. Initialisation data is written to the NCF file.

If a timing name is supplied a TNM entry is added to the NCF file.

Ports which are not written have null arguments for the data in and the write enable. Ports which are not read have null arguments for the data out.

For Xilinx FPGAs Combinatorial-output RAM is CLB RAM. The address width is limited to a maximum of 9 bits (a depth of 512 words). The data width is not limited.

There is one optional attribute which is a timing group name.

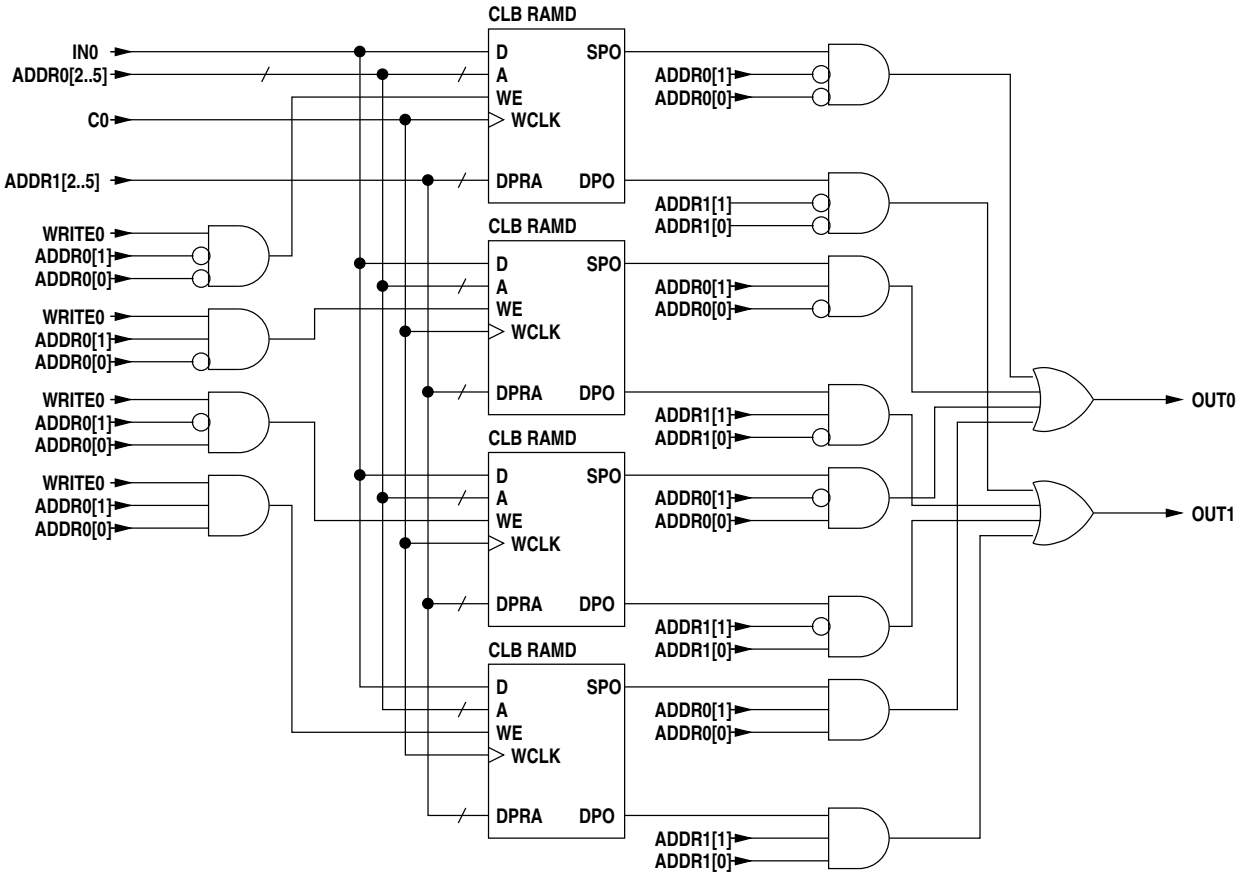
dump format

```
    CRAM - 2 ports
        OUT  null
        OUT  SAMPLEOUT168[0]
        <-
        CLK  glob.c100
        ADDR WADDR_160[0..1]
        DATA glob.frontend_0.sdc_in_0.reset_1[0]
        WE   VCC
        CLK  null
        ADDR RADDR_161[0..1]
    initialised{timingname=cram1}
```

The presence of an initialisation argument is flagged but the values are not listed.

logic description

For Xilinx combinatorial output RAM is CLB RAM. If two ports are used the second port is read-only. Single port CLB RAM has an address width of 5 and dual port CLB RAM an address width of 4. In both cases the data width is 1. For greater address widths CLB RAMs are multiplexed, for example a dual port 64 deep RAM -



This is only one data bit wide and is repeated for each additional data bit with the other signals in parallel.

2.7.4 DEL

purpose

To create a delay line to delay a signal one or more clock cycles. The delay may be variable.

arguments

Parameters	Integer number of clock cycles to delay if constant, else 0 String array initial words in line as array of hexadecimal strings
Inputs:	clock signal input signal array delay signal array number of clock cycles to delay if variable, else null enable signal delay steps if high - null for every cycle reset or null
Outputs	output signal

description

The input signal appears at the output one or more clock cycles later. The reset signal returns the logic to its initial state. It may be omitted or null.

If the reset is present then a fixed delay line will be created using flip-flops, in which case the variable length input must be null.

If the variable length input is not null the delay line will use CLB shift registers and the reset input must be null. The delay length is limited to 16 or 32 depending on the FPGA.

If the variable delay is used, this value selects the point in the delay line from which the output is taken. 0 selects the output of the 1st delay element and 15 the output of the 16th delay element. Note that changing this value will result in data being skipped (decreasing) or repeated (increasing).

If there is initialisation data the number of words will be equal to the delay line length. If there is no initialisation data these parameters will be omitted.

dump format

```
1    DEL E69[0..3] <- p.in[0..3] CLK glob.c166 delay 18 initialised
```

logic description

Xilinx implementation -

This element is implemented sharing the code for SPECIAL element (code 4) which provides a general delay line using Xilinx CLB shift registers.

A delay of three or fewer cycle is implemented using flip-flops. Delays of more than three cycles may use one or more CLB shift registers. If a reset signal is present flip-flops must be used, the reset signal being connected to the reset inputs of the flip-flops.

2.7.5 DFF

purpose

A D-type flip-flop.

arguments

Parameters	Boolean optional initial value Boolean false or omitted if set and reset are synchronous , true if asynchronous String optional timing group identifier String optional constraint String ... etc
Inputs	data_in signal, or null clock signal, or null clock enable signal, or null reset/clear signal, or null set/preset signal, or null
Outputs	data_out signal

A number of constraint strings may appear at the end of the parameter section. These strings will be applied to the FD prepended with "INST name " where 'name' is the generated netlist identifier for the FD.

description

This provides a basic D-type flip-flop. Unused arguments may be null and unused trailing arguments may be omitted.

dump format

```
1    DFF FDRS164 <- D=INV163, C=glob.sdclk0
```

2.7.6 DIVERGE

purpose

A controller for divergent queue signals. Where a queue is read within more than one module it is required that each module execute a read before the datum is popped. Until all destination modules have executed a read the queue must appear unavailable for reading to those modules that have already executed a read. This is accomplished by inserting this DIVERGE element between the queue .NE and .POP signals and the .NE and .POp signals seen by the queue reads in each destination module.

arguments

Parameters	none
Inputs	clock signal reset signal queue not empty signal module queue pop signal module queue pop signal . . .
Outputs	queue pop signal module queue not empty signal module queue not empty signal . . .

description

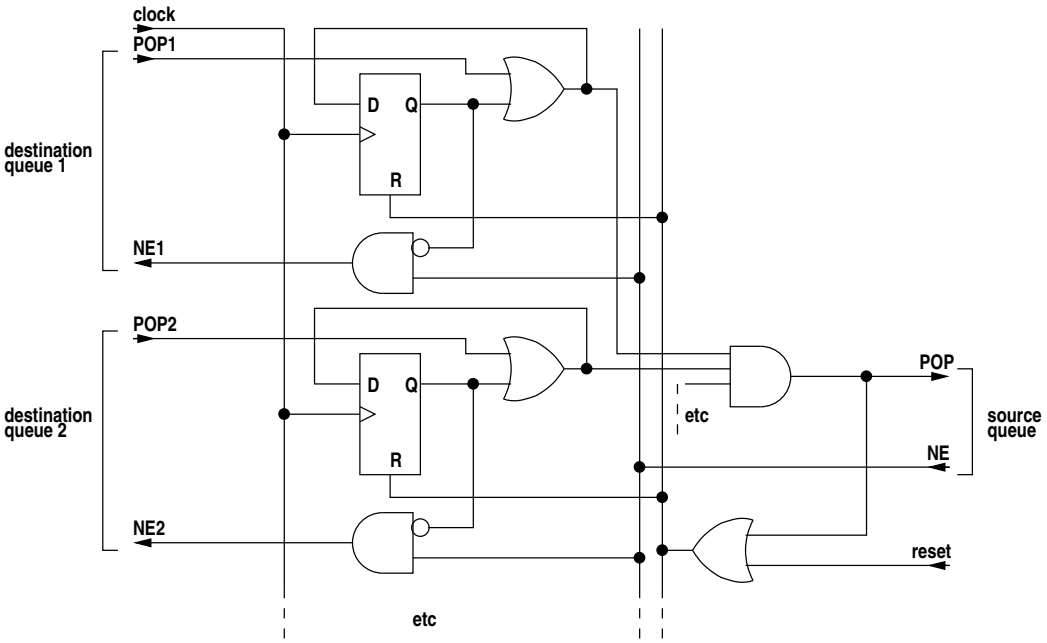
This splits the source queue .NE (read availability) and .POP (pop) signals into a pair per destination module. The 1st input is the output (read) clock signal for the queue.

The 2nd input is the .NE (read availability) signal from the queue. The 1st output signal is the .POP signal to the queue. The 2nd input is a reset which is asserted if the source queue is reset. The 3rd input is the .POP signal from the 1st module. The 2nd output is the .NE signal to the 1st module. Further input and output signals to other modules may follow. If there is only one module the .NE and .POP signals should be just connected through. If all destination pop signals go high simultaneously the pop signal to the source also goes high, otherwise it goes high when the last destination pop signal goes high. The destination pop signal is high for one clock cycle. When each destination pop signal goes high the matching destination availability signal will go low at the next clock cycle unless all destination modules have also signalled pop in which case destination availability signals will be the be the same as the source availability signal.

dump format

```
1    DIVERGE {
2        in:
3            c100
4            S11
5            glob.p.NE
6        out:
7            glob.p.POP
8            S15
9    }
```

logic description



2.7.7 DOWHILE

purpose

A *do while* statement.

arguments

Parameters	none
Inputs	start signal test signal block finish delayed signal clock signal optional reset signal or null
Outputs	start signal for the body DO WHILE finish signal

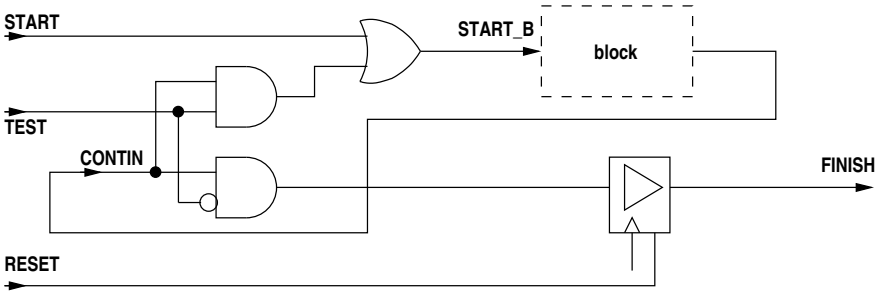
description

The start input signal starts the first iteration of the loop by propagating straight through to the body start output signal. The block finish delayed input signal comes from the block finish signal, possibly via an EXECP if the test signal comes from an expression containing queue reads. The block finish delayed input signal is then routed either through to the body start output signal (test succeeds) or via a one cycle delay to the finish output signal (test fails).

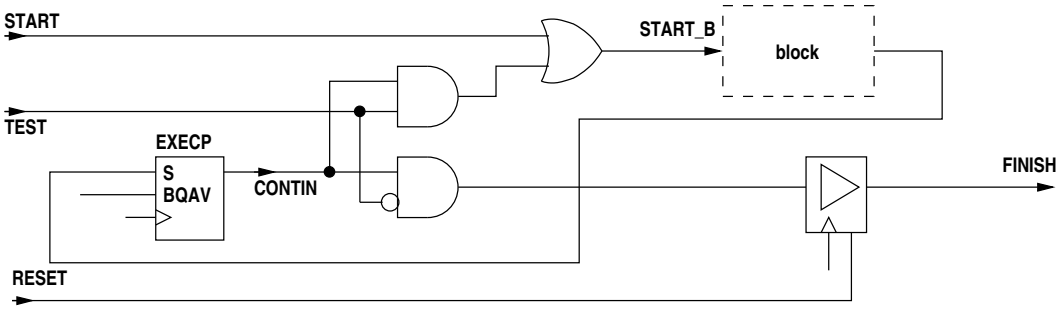
dump format

```
1    DO WHILE {
2        START_B     S23
3        FINISH      F35
4        <-
5        START       S24
6        TEST        E31[0]
7        CONTIN      F33
8        C           glob.c100
9        RESET       s34
10   }
```

logic description



The START signal is passed straight through to the block execution signal output START_B. The block finish signal FINISH_B loops back via conditional logic where the test expression is sampled. If the test succeeds FINISH_B is passed throught to START_B to begin the next iteration otherwise it is passed to FINISH via a DEL. A RESET clears the DEL. The DEL is required to avoid a race condition through to the following statement.



An EXECP is required if the test expression contains queue reads. The EXECP controls the FINISH_B signal prior to the conditional logic. A RESET clears the DEL and EXECPs. The DEL is required to avoid a race condition through to the following statement.

2.7.8 ELEMENT

purpose

Create an instance of a netlist element.

arguments

Parameters . — 2nd port .	String the element name String netlist block name or null String port name — 1st port integer port type — integer port width — integer port array format —
Inputs Inputs . .	input 1 input 2
Outputs Outputs . .	output 1 output 2

description

This is used to explicitly create a component element. It can be used to access library components which are not created implicitly by other TDEs. A TDE of this type can also be created from 3PL source code using inbuilt procedures **element()** and **elementconnect()**. This allows fpag-specific elements to be created by 3PL library code rather than being built into 3PL.

The first parameter is the element name. The optional second parameter is a netlist identifier for this instance of the element. If null an arbitrary identifier of the form "b123" will be generated.

The following parameters are quadruples giving the port name, port type (0 input, 1 output, 2 3-state output, 4 clock input), port width in bits and port array format, 0 for non-array ports, 1 for pins as A1, A2, A3 etc. and 2 for EDIF pin arrays. For each quadruple there will be an associated signal in the input or output list in the order of occurrence in the parameter list.

dump format

```
7                   ELEMENT DCM block dcm1
8                       PIN CLKIN I glob.ruclk
9                       PIN CLKFB I prog.cfb
10                      PIN CLK0  O prog.cfb_raw
11                      PIN CLKDV O prog.pclk_raw
```

2.7.9 EXECP

purpose

An execution wait for queue availability and/or priority grant.

arguments

Parameters	none
Inputs	clock signal start signal buffered queue availability signal unbuffered queue write availability signal unbuffered queue read availability signal priority output signal or null reset or null
Outputs	delayed start signal priority input signal or null execution write pending signal or null execution read pending signal or null

If the wait is for queue availability alone the priority input and output arguments are null. If the wait is for priority alone the queue availability arguments is null. The reset argument is null if there is no reset signal.

description

This element delays an execution signal if it is necessary to wait for queue availability, for priority or both. The buffered queue availability input is high if all associated buffered queues to be read or written are available. The unbuffered queue availability inputs are similar but the write and read availabilities are on separate inputs.

If the queue availability inputs are high and the priority encoder passes the signal, the execution signal is passed through without delay. If the queue availability inputs are not all high or priority has been pre-empted, the execution signal is stored until the queue availabilities are high and priority is granted at which time a single cycle output pulse is generated. An input execution signal must not occur when an output is pending however it may occur on the cycle following the output pulse. The queue availability inputs may be several queue availabilities ANDed together. The reset input, if not null, is used to clear any pending execution (waiting on queue availability or priority). The pending signals are used to notify unbuffered queues that a write or read is pending awaiting unbuffered queue availability. It is asserted when the start signal is asserted or the registered start signal is asserted and all buffered queues are available. The output need not have an argument (null) if unbuffered queues are not involved.

dump format

The dump format depends on whether the EXECP has priority inputs/outputs or unbuffered queues.

For normal buffered queues -

```
1    EXECP no priority, buffered queues only {
2        START_DEL SD423
3        <-
4        CLK       glob.c32
5        START_IN  F392
6        BQAV      tx.NE
7    }
```

With both buffered and unbuffered queues -

```
1    EXECP no priority, unbuffered (+ buffered) queues {
2        START_DEL SD23
3        WPENDING   WP18
4        RPENDING   RP17
5        <-
6        CLK       glob.c32
7        START_IN  S13
```

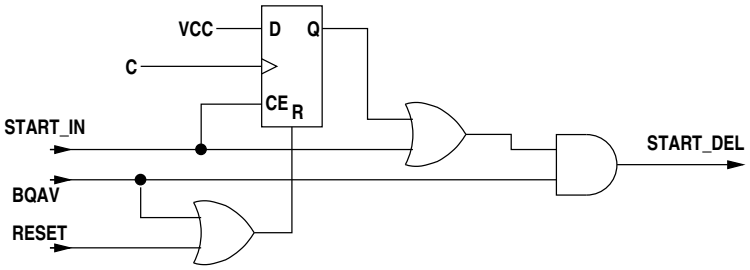
```
        BQAV      VCC
        UBQWAV    VCC
        UBQRAV    prog.ubq1.NE
    }
```

Buffered queues with priority -

```
    EXECP priority, buffered queues only {
        START_DEL SD9750
        PRI_IN    WPIN9742[0]
        <-
        CLK       glob.pclk
        START_IN  TS9737
        BQAV      kandinski.processing_0.mark.NF
        PRI_OUT   kandinski.processing_0.marker_0.pri[1]
```

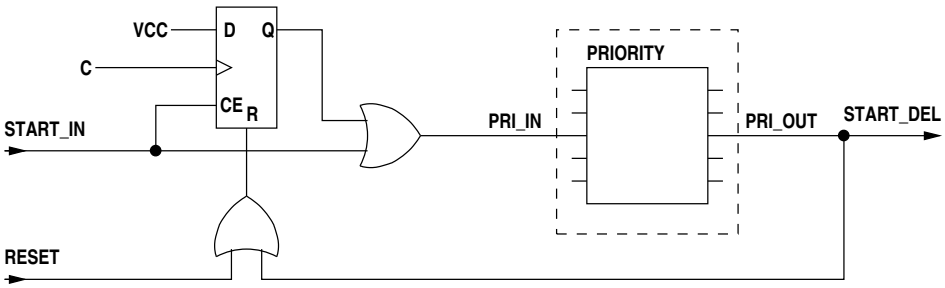
logic description

If there are no priority signals the delayed start output depends only on the start and buffered queue availability input signals -



BQAV is the AND of the read or write availability signals of all buffered queues upon which this EXECP execution depends. If BQAV is high START_IN is gated straight through to START_DEL and the flip-flop doesn't change, being held reset by BQAV. If BQAV is low when START_IN arrives the flip-flop is set and START_DEL remains low. When BQAV subsequently goes high START_DEL goes high because the flip-flop is set. On the next clock edge the flip-flop is reset and START_DEL goes low again.

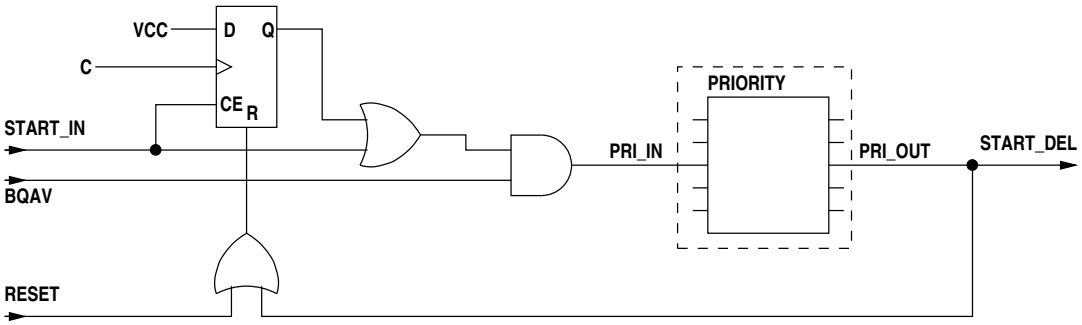
If there are priority input and output signals but no availability signal -



The delayed start now requires the priority encoder to pass the start, otherwise it is latched. The flip-flop is cleared when the delayed start finally occurs. The flip-flop will not latch the start signal if the priority encoder passes the signal.

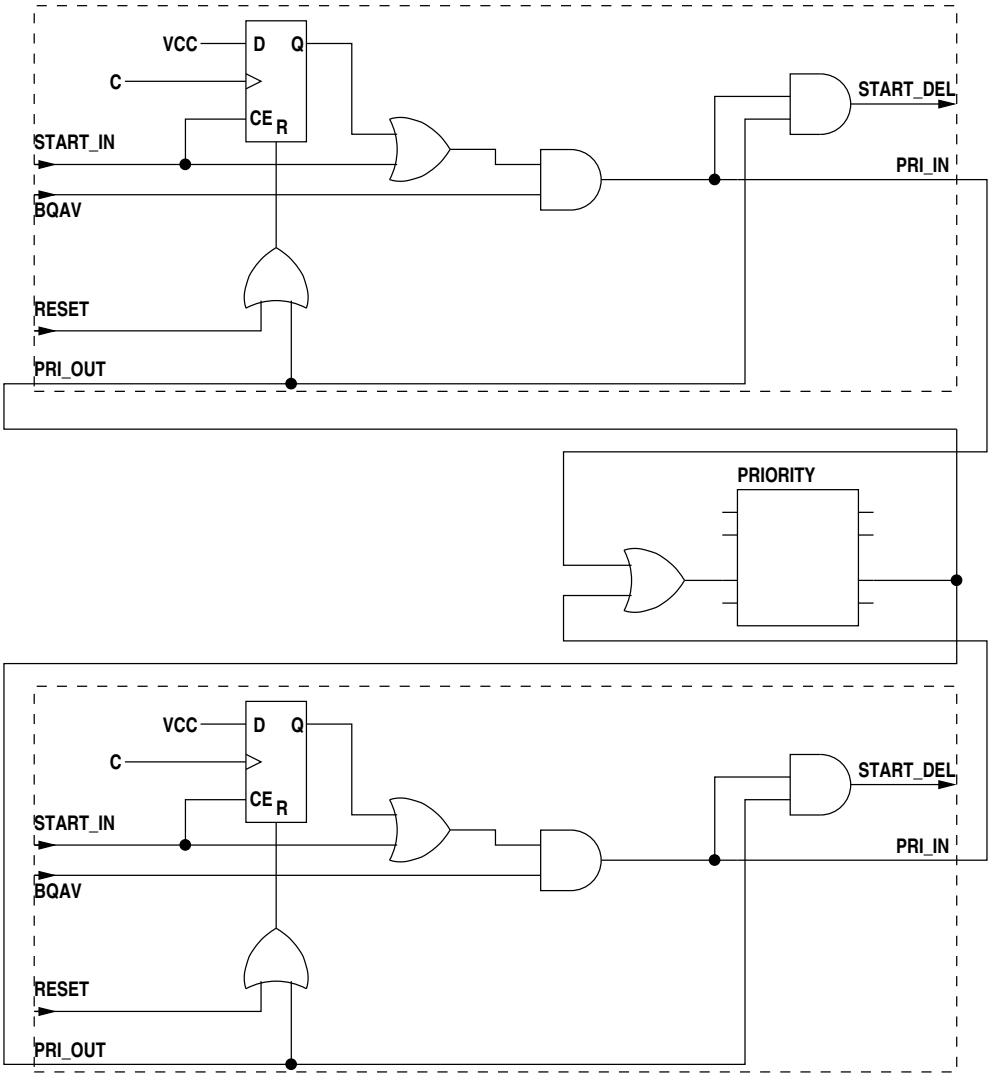
The priority encoder is required for **waitpri** and **waitprilock** statements. The priority encoder (priority mode variable) is shown above in a dashed rectangle as it exists outside the EXECP block, the input and output signals for a particular priority level being passed as arguments to the EXECP. The highest priority input, 0, is reserved, array members 1 and above being used for statement priority. Priority input 0 is used to lock out all other priority requests when **waitprilock** is used and is asserted by a flip-flop which is set by any executing waitprilock instances and cleared later by a **priunlock** statement.

If both priority and buffered queue availability signals are present -



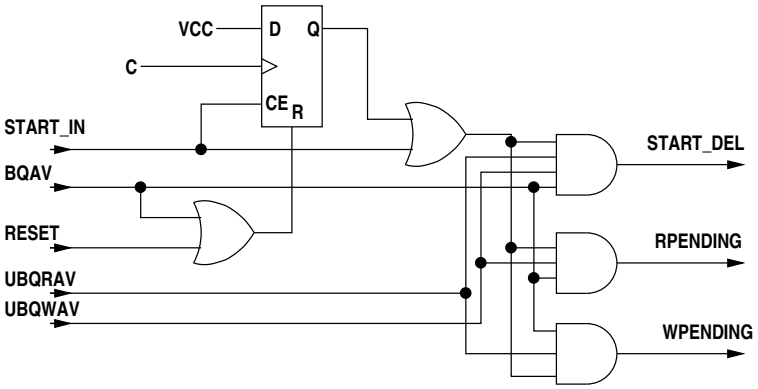
The delayed start now requires the priority encoder to pass the signal and the availability signal to be asserted. If this is true when the start signal is asserted the flip-flop will not be set as its reset will also be asserted.

Note that in the case where more than one waitpri uses the same priority level the START_DEL signal for each must be the AND of PRI_IN and PRI_OUT instead of just PRI_OUT, otherwise the START_DEL signals for the separate occurences would be connected! The multiple PRI_IN signals to the priority variables will be ORed.



Multiple use of the same priority level is determined by calling the isMultiple() method in class exec/Priority.java with the aray index as parameter. The Priority class instance and the index are found from the 4th input argument TDEVar.

If unbuffered queue availability signals are present -



BQAV, UBQRAV and UBQWAV are the ANDed availability signals for buffered queues, unbuffered queues being read and unbuffered queues being written respectively. Where any of these three have no queues associated with the EXECP the availability signal is null.

The delayed start now depends additionally on the unbuffered queues being available plus any buffered queues being available. When a start has been received and the buffered queue availability signal is high RPENDING is asserted if UBQAV is high and WPENDING is asserted if UBQWAV is high. These cross-dependencies for a chain of dependencies through related unbuffered queue assignments - see (2.8.17).

When all associated unbuffered queues are available START_DEL will be asserted in both the writing and reading EXECPs so that both threads can proceed. Since the QUEUEBUFFER simply has the data input connected to the output the written data will be transferred directly from writers to readers.

2.7.10 IBUF

purpose

A buffered external input signal.

arguments

Parameters	String - optional element identifier String - input package pin String - optional input package pin for differential input Boolean - ibufg flag Boolean - differential flag
Inputs	input signal from pad optional differential input signal from pad or null
Outputs	output signal optional differential output signal or null

description

This buffers an external input signal. The first parameter specifies the ID given to the IBUF element. The second parameter gives the primary FPGA pad designation. The third parameter is an optional second input pad for differential input or null. The fourth parameter is true for an IBUFG or IBUFGDS The last parameter is true if the input is differential (IBUFDS, IBUFGDS, IBUFDS_DIFF_OUT).

There is a primary input signal followed by a second input signal if there is differential input to the IBUF (IBUFDS, IBUFGDS, IBUFDS_DIFF_OUT) or null (IBUF, IBUFG).

There is a primary output signal followed by a second output signal if there is differential output from the IBUF (IBUFDS_DIFF_OUT) or null (IBUF, IBUFG, IBUFDS, IBUFGDS).

There must be a **loc** entry to specify the package pin. For differential inputs two consecutive pins are used, positive input followed by negative input.

dump format

```
1     IBUF    INPUT327[0] <- PORT_P88 loc=P88, IBUFG
2     IBUF    INPUT430[0] <- PORT_P91, PORT_P92 loc=P91,P92 IBUFDS
```

2.7.11 ILOOP

purpose

An infinite loop.

arguments

Parameters	none
Inputs	start signal block finish signal
Outputs	block start signal

description

The start input signal is the start signal for all infinite loops in this clock domain. The block finish signal is high when the last statement in the code block executes. This element simply ORs the two inputs.

Infinite loops containing a single statement that always executes in one clock cycle are replaced by a D-flip-flop which is set by the clock domain start signal and stays set thereafter since the contained statement is to execute on every clock cycle.

dump format

```
1     ILOOP   S12 <- glob.c100.start F14
```

logic description

The implementation is simply an OR of the two inputs.

2.7.12 INV

purpose

An single signal inverter.

arguments

Parameters	none
Inputs	input signal
Outputs	output signal

description

The input and output are single signals, the output being the input inverted.

dump format

```
1    E17[1] = inv(glob.a[1])
```

2.7.13 OBUF

purpose

A buffered external output signal.

arguments

Parameters	String - optional array of element identifiers String - output package pin String - optional output package pin for differential output boolean - differential flag
Inputs	input signal optional 3-state enable signal (+ve for high-Z) or null
Outputs	output signal to pad optional differential output signal to pad or null

description

This buffers an external output signal. The first parameter specifies the ID given to the OBUF element. The second parameter gives the primary FPGA pad designation. The third parameter is an optional second output pad for differential output or null. The fourth parameter flags a differential output.

The first input signal is the data input. The second input signal is an optional 3-state enable which must be low to drive data onto the output pad (OBUFT, OBUFTDS). If this argument is null output drive to the pad is continuous (OBUF, OBUFDS).

There is a primary output signal followed by a second output signal if there is differential output from the OBUF (OBUFDS, OBUFTDS) or null (OBUF, OBUFT).

There must be a **loc** entry to specify the package pin. For differential outputs two consecutive pins are used, positive output followed by negative output.

dump format

```
1    OBUF    PORT_P90 <- OUTPUTBIT213 loc=P90
2    OBUF    PORT_P92 <- OUTPUTBIT412 _EN=T31 loc=P92
```

2.7.14 OPERATOR

purpose

An arithmetic or logical operator.

arguments

The general form is

Parameters	enumerated type operator code 1st operand is signed 2nd operand is signed if binary or ternary 3rd operand is signed if ternary
Inputs	1st operand array 2nd operand array if binary or ternary 3rd operand array if ternary
Outputs	output array

The unused parameters and inputs are omitted, not null.

description

Operator codes are -

TDEOp.INV	unary logical inversion
TDEOp.NEG	unary arithmetic negation
TDEOp.ADD	binary integer addition
TDEOp.SUB	binary integer subtraction
TDEOp.ADDSUB	binary integer addition/subtraction
TDEOp.ALU	binary integer addition/subtraction/assign
TDEOp.MUL	binary integer multiplication
TDEOp.DIV	binary integer division
TDEOp.REM	binary integer remainder
TDEOp.DIVREM	binary integer division/remainder
TDEOp.EQ	binary equality test
TDEOp.NE	binary inequality test
TDEOp.LT	binary less-than test
TDEOp.LE	binary less-than-or-equal-to test
TDEOp.GT	binary greater-than test
TDEOp.GE	binary greater-than-or-equal-to test
TDEOp.AND	binary logical A AND B
TDEOp.NAND	binary logical A NAND B
TDEOp._AND	binary logical !A AND B
TDEOp.AND_	binary logical A AND !B
TDEOp.OR	binary logical A OR B
TDEOp.NOR	binary logical A NOR B
TDEOp._OR	binary logical !A OR B
TDEOp.OR_	binary logical A OR !B
TDEOp.XOR	binary logical A XOR B
TDEOp.XNOR	binary logical A XNOR B
TDEOp.LSH	binary left shift
TDEOp.RSH	binary right shift
TDEOp.COND	ternary conditional (`?:`)
TDEOp.ASSIGN	assignment (only used during optimisation)

Arithmetic operations are either Ptype.UINT (unsigned integer) or Ptype.INT (signed integer). These may be mixed. If either operand is signed the result will be signed. The width of the result is that necessary to retain all significant bits. Operands to equality and inequality relational operators may be Ptype.UINT, Ptype.INT, Ptype.BITS or Ptype.LOG. Operands to the other four relational operators are Ptype.UINT or Ptype.INT. The first operand to the shift operators may be Ptype.UINT, Ptype.INT or Ptype.BITS. The sceond operand (shift count) must be Ptype.UINT. The shift count may be constant or variable. The ternary conditional operator first operator must be Ptype.LOG and the other two operands must be matching types (may be mixed Ptype.UINT and Ptype.INT) and types Ptype.UINT, Ptype.INT, Ptype.BITS or Ptype.LOG.

The integer addition/subtraction operator provides conditional addition or subtraction by means of two additional input signals. The first selects addition (high) or subtraction (low). The second gates the second operand, high for add/subtract and low to inhibit add/subtract (output is first operand). This operation is available as an inbuilt function, not an operator.

The integer division/remainder operator produces the quotient and remainder in a single division. For the division and remainder operators the quotient and remainder are both produced and one of these is discarded. This operation is available as an inbuilt procedure, not an operator.

arguments

General form -

Parameters	operator code 1st operand is signed 2nd operand is signed
Inputs	1st operand array 2nd operand array
Outputs	output array

Adder subtracter form -

Parameters	TDEOp.ADDSUB 1st operand is signed 2nd operand is signed
Inputs	1st operand array 2nd operand array add/subtract control (high for add) pass 2nd operand control (2nd operand 0 if this is low) XOR to carry input
Outputs	output array

ALU form -

Parameters	TDEOp.ALU 1st operand is signed 2nd operand is signed flag indicating output connected to a static variable with a DSP attribute
Inputs	1st operand array 2nd operand array add/subtract control (high for add) A gate control - high for add/subtract, low for output = B input
Outputs	output array

Division remainder form -

Parameters	TDEOp.DIVREM 1st operand is signed 2nd operand is signed
Inputs	1st operand array 2nd operand array add/subtract select for ADDSUB gate 2nd operand for ADDSUB XOR carry in for ADDSUB
Outputs	1st output array remainder output array for DIVREM

dump format

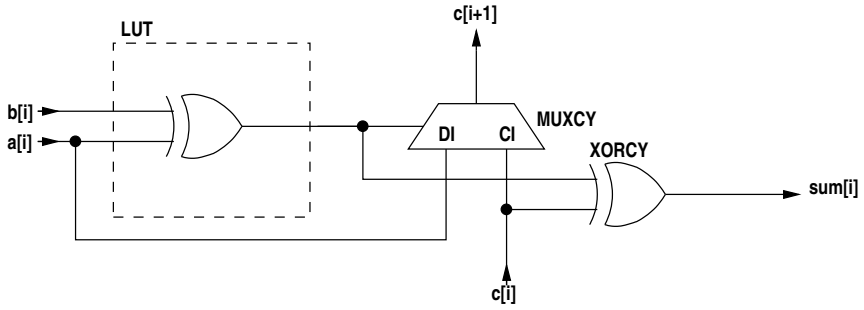
```
1    E3[0..8] = E2[0..7] + glob.a[8..15]
2    E15[0] = !E14[0]
3    E20[0..7] = E4[0] ? blog.a[0..7] : E3[0..7]
```

logic description

INV uses an inverter for each bit.

NEG is subtraction from zero - see **SUB** below. Note however that the "zero" (low) inputs will be eliminated during netlist optimisation.

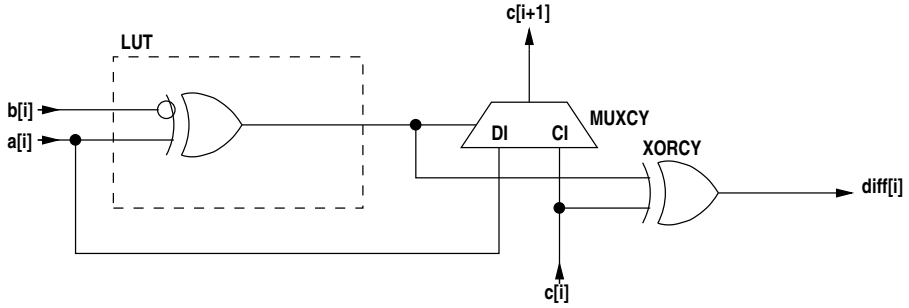
ADD is a 2's complement adder using the carry chain. A single stage of the adder is -



If one of the operands is smaller than the other it is padded with 0 if unsigned or the sign bit is extended if signed. The sum is one bit wider than the operands.

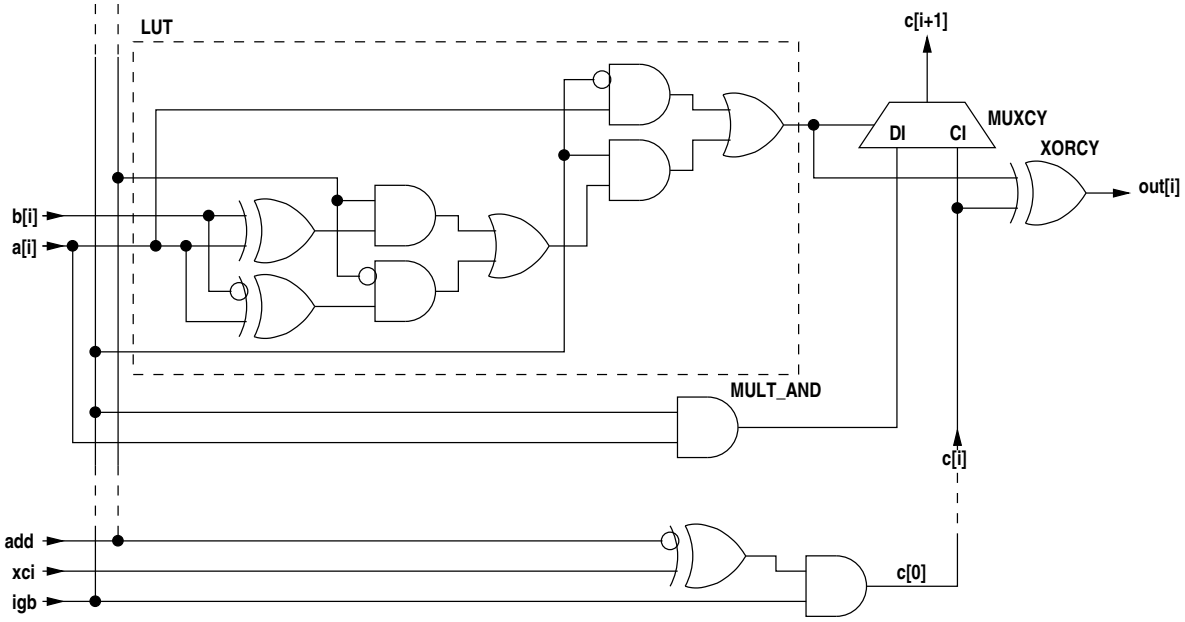
The carry in to the least significant bit is low. The carry out of the most significant bit is the most significant bit of the sum.

SUB is a 2's complement subtraction. The logic is the same as the adder except that the **b** input is inverted and the carry into the LSB is 1.



The **igb** input gates the **b** input as it does in the adder however if the **igb** input is used it should also drive the LSB carry in because if **igb** is low the LSB carry in should be also low.

ADDSUB is a combined adder/subtracter. The logic for one stage is -

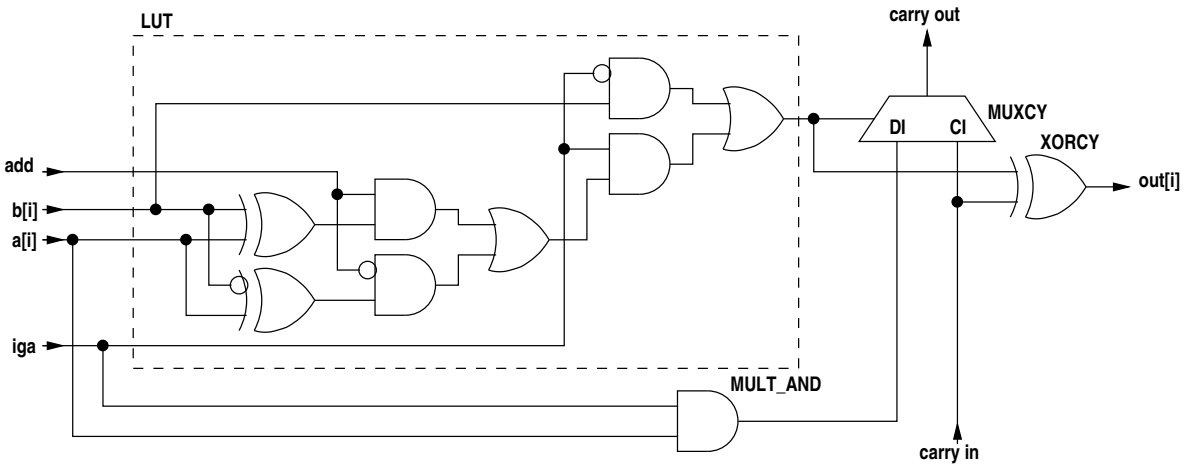


If the **igb** input is high and the **add** input is high the output is the sum of **a** and **b**. If the **add** input is low the output is the difference. If the **igb** input is low the output is **a**. The **xci** input is XORed with the carry input.

The LSB carry in is AND(XOR(NOT(add), xci), igb).

ALU is very similar to **ADDSUB** but this operation is intended as an ALU between a static variable and its input selector for implementing v++, v–, v += n, v -= n and v <- n. The A input is connected to the variable output. The gate control allows assignment of the B input for direct assignment, rather than A +- B.

The LSB carry in is AND(INV(add), iga).



If the 5^{th} parameter is true then this ALU is input to a static variable with the DSP attribute. If the FPGA is a Virtex5 then a DSP48E will be used for the ALU instead of the CLB logic previously illustrated. The A input is connected to the C input of the DSP48E and the B input is connected to the A input of the DSP48E. The ALU mode and OP mode inputs are -

operation	ALU mode	OP mode
add	0000	0110011
subtract	0011	0110011
assign	0000	0110000

MUL is a 2's complement multiplier. For XC3S, XC2V and XC2VP FPGAs the hardware multiplier is used. The operands are restricted to a maximum of 18 bits. For a Virtex5 the DSP48E multiplier is used and the operands are 18 and 25 bits.

For XC2S or XCV FPGAs the multiplier is implemented using successive adders. There are separate multipliers for unsigned and signed operands. This form of adder has a very long combinatorial path and because of this is little used since the clock rate is quite low (a few MHz). The logic is not documented here.

DIV, **REM** and**DIVREM** all use the same function which generates both the quotient and remainder in 2's complement arithmetic. There are separate signed and unsigned versions. This function uses successive adder/subtracters. It is even slower than the multiplier described above. The logic is not documented here.

EQ uses an XNOR of each pair of bits from the two operands and ANDs the outputs to get the equality output.

NE uses an XOR of each pair of bits from the two operands and ORs the outputs to get the inequality output.

LT, **LE**, **GT** and **GE** all use the **GE** comparison by changing the operand order and/or inverting the result. The **GE** relational operator uses the most significant bit of the difference after subtraction.

AND, **OR** and **XOR** use gates as one would expect, one per bit pair from the two operands.

LSH and **RSH** may have constant shift or variable shift.

Constant shift requires no logic and simply connects the output signal array to the input signal array with an offset. Left shifts connect least significant bits to zero. Right shifts connect most significant bits to zero if unsigned or to the input sign bit if signed.

Variable shifts use cascaded shifters, each shifting by a power of two. Each shifting stage is controlled by one bit of the shift count. The shifter is a 2-input multiplexer using one LUT.

COND implements the `?:` operator and is simply a 2-input multiplexer per output bit, each multiplexer using one LUT.

2.7.15 OR

purpose

An OR gate for control signals.

arguments

Parameters	none
Inputs	input signal input signal . .
Outputs	OR signal

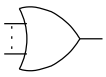
description

OR two or more input signals, the result appearing on the output.

dump format

```
1    S11 = A12 | A14 | S8
```

logic description



Xilinx implementation -

OR gates with 1 to 4 inputs will be implemented in a single LUT (single input gates will be eliminated during low-level code generation). OR gates with more than 4 inputs use the carry chain to OR together the outputs of 1 to 4 input LUT OR gates (actually NOR gates).

2.7.16 PORT

purpose

Flags a signal which is external to the current source code but is not external to the FPGA (i.e. not a pad connection). These signals are either connections to an imported core or are exported from the current source code which is itself a core.

arguments

Parameters	Integer type of signal string port name
Inputs	signal or signal array
Outputs	none

description

The first parameter is an integer type code for the port type which is 0 for input, 1 for output or 2 for 3-state output. The second parameter is the port name.

dump format

```
12                  PORT {
13                      par:
14                          Str     0
15                          Int     ADDRESS
16                      in:
17                          Var     glob.addr[0..18]
18                  }
```

2.7.17 PRIORITY

purpose

A priority encoder.

arguments

Parameters	none
Inputs	input signal array
Outputs	output signal array else signal or null

description

When one or more inputs are high the highest priority matching output will be high and all other outputs low. The inputs are ordered in decreasing priority (the 1st is highest).

The number of outputs is equal to the number of inputs or the number of inputs + 1. In the latter case the last output is the 'else' case, i.e. no inputs high.

dump format

```
1    PRIORITY {
2        in:
3            glob.a[0..2]
4        out:
5            prog.p[0..2]
6            prog.p.PE[0]
7    }
```

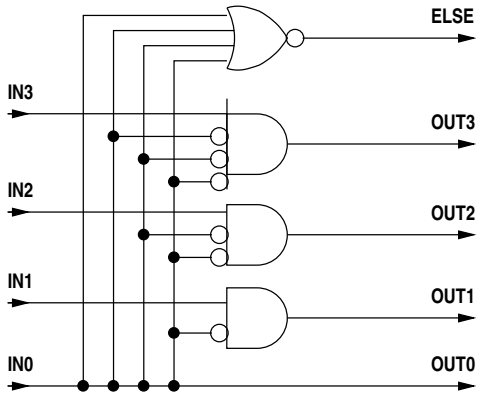
logic description

Xilinx implementation -

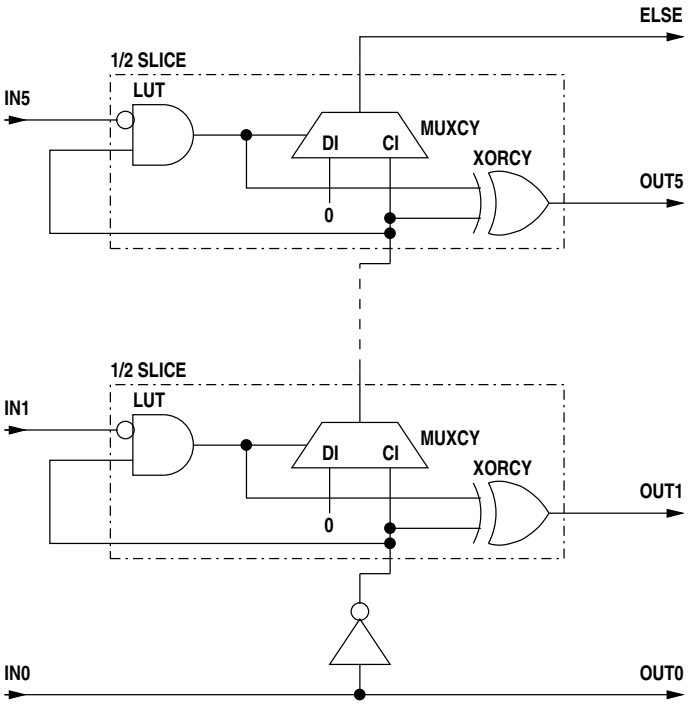
The highest priority output is connected directly to its input.

For one input the optional ELSE output is the inverted input.

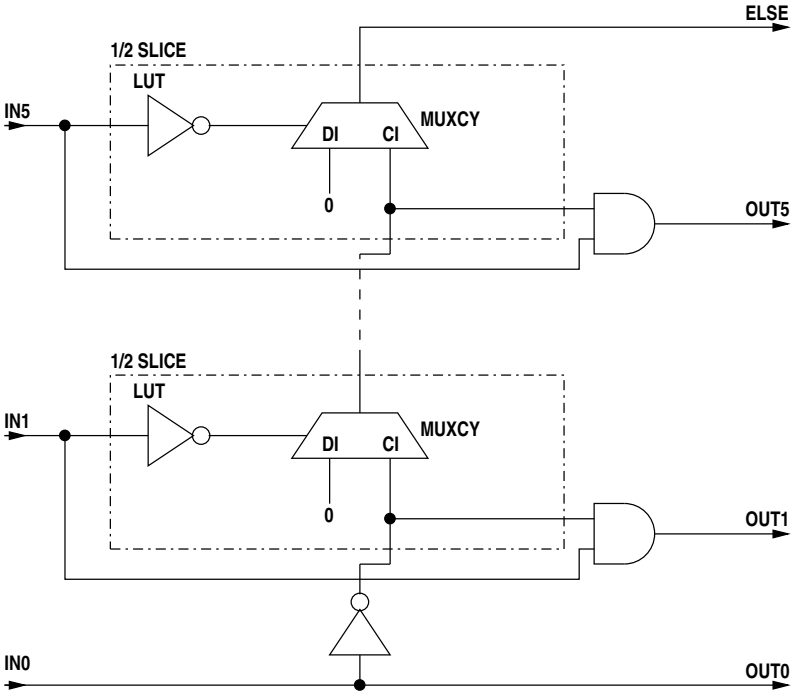
If the device is a CPLD the other outputs use AND gates. If the device is an FPGA and the number of inputs is not greater than the LUT width, LUTs are used.



For FPGAs with more inputs than the LUT width the carry chain is used. The implementation originally used the XORCY element effectively as an AND gate. This was achieved by ANDing the inverted input with the carry at each stage and XORing the LIUT output and carry in.



This implementation was functionally correct however the connection of each carry to the LUT input added a significant delay to each stage, a delay much larger than that of the carry chain itself. To eliminate this it was necessary to depart from this rather neat solution and instead add an external AND gate to each stage. These AND gates add an extra delay to each stage but in parallel, not cascaded. Hence the maximum path through the priority encoder is very much shorter.



2.7.18 QUEUEBUFFER

purpose

A queue buffer.

arguments

Parameters	Integer depth Boolean use hardware FIFO is possible, or null String initial value (hexadecimal string) or null
Inputs	input clock signal, or null if same as output clock output clock signal input data array push datum signal (.PUSH) pop datum signal (.POP) reset signal (optional)
Outputs	output data array read available signal (.NE - not empty) write available signal (.NF - not full) ready availability signal for unbuffered queue synchronous buffer occupancy count or null synchronous buffer space count or null asynchronous buffer write status asynchronous buffer read status reset signal (optional)

description

A normal queue buffer has a depth of 2. If the depth is > 2, an initial value is not allowed. If a separate input clock is supplied, depths start at 16 and are various powers of 2 depending on the size ranges of the block RAMs in the particular FPGA. If the input data array and output data array arguments are null then this is a null data queue buffer and the buffer depth is a power of 2 greater than 3.

The 2nd output signal is the read availability signal. This is high if the buffered queue is not empty. For an unbuffered queue it is high if the queue is waiting on a write.

The 3rd output signal is the write availability signal. This is high if the buffered queue is not full. For an unbuffered queue it is high if the queue is waiting on a read, i.e. it is connected to the POP input.

The 4th output is null for a buffered queue. For an unbuffered queue it is the AND of the push (write) and pop (read) inputs (which is also the AND of the read availability (NE) and write availability (NF) signals.

If the 5th output is not null this gives the number of words in a synchronous queue buffer synchronised to the queue clock (read and write).

If the 6th output is not null this gives the number of free spaces in a synchronous queue buffer synchronised to the queue clock (read and write).

If the 7th output is not null this gives the buffer status for an asynchronous queue buffer synchronised to the write (input) clock.

If the 8th output is not null this gives the buffer status for an asynchronous queue buffer synchronised to the read (output) clock.

If there is an 9th output this is a reset signal. This signal goes high for one clock cycle when the queuebuffer is reset via the reset() inbuilt procedure. For an asynchronous queuebuffer this signal is synchronised to the read (output) clock and is timed to follow the reset of the input (write) side of the queuebuffer.

The asynchronous queue buffer status is a 5-bit word whose bits are -

- bit 0 - empty to 1/4 full + 1
- bit 1 - 2 words to 1/2 full + 1
- bit 2 - 1/4 full + 1 to 3/4 full + 1
- bit 3 - 1/2 full + 1 to full

- bit 4 - 3/4 full + 1 to full

The bits are mutually exclusive and one of them is always set.

Synchronous data queue buffers have a default depth of 2. Synchronous null queue buffers have a default depth of 16. Asynchronous data queue buffers have a default depth of 16. Asynchronous null queue buffers have a default depth of 16. Null queue buffers may have a depth which is 4 or greater and which is a power of 2. Data queue buffers may have the following depths -

- Virtex - 16, 256, 512, 1024, 2048 or 4096
- Virtex-II - 16, 512, 1024, 2048 or 4096, 8192 or 16384

and for synchronous buffers depth 2 is also allowed and is the default.

dump format

```
1     QUEUEBUFFER (512)
2         OUT    glob.frontend_0.i1[0..7,8]
3         NE     glob.frontend_0.i1.NE[0]
4         NF     glob.frontend_0.i1.NF[0]
5         CNT    null
6         SPC    null
7         WSTAT  null
8         RSTAT  glob.frontend_0.i1.RSTAT[0,1,2,3,4]
9         <-
10        CLKIN  glob.sdclk0
11        CLKOUT glob.c100
12        DATA   glob.frontend_0.i1.D[0..7,8]
13        PUSH   glob.frontend_0.i1.PUSH
14        POP    glob.frontend_0.i1.POP
15        R      glob.frontend_0.i1.RES
```

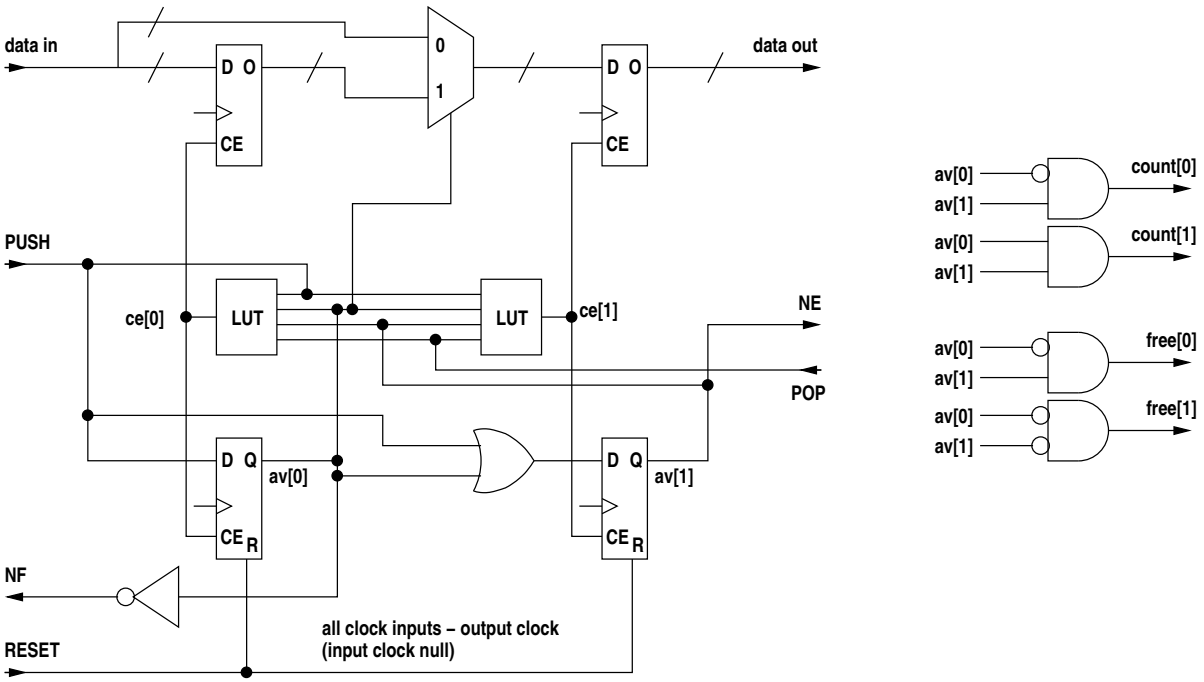
The minimum size of the buffer is two and this is implemented as two registers. **NF** stands for "Not Full", meaning that at least one free register is available for writing. **PUSH** stands for "push data onto queue". **NE** stands for "Not Empty", meaning one or more registers contains data which can be read. **POP** acknowledges a read, popping the word from the buffer (queue).

logic description

There are five separate implementations of queue buffers -

- a synchronous queue buffer of depth two (default)
- a synchronous queue buffer of depth greater than two
- an asynchronous queue buffer
- a asynchronous or asynchronous queue buffer with no data
- a asynchronous or asynchronous queue buffer of depth 0

The logic for the default synchronous queue buffer of depth two is -



The 2-input multiplexer shown uses a 3-input LUT per bit.

When PUSH is high input data is latched into one of two registers. If the buffer is empty the data is latched into the rightmost register to which the data output is directly connected. If another datum is written to the queue buffer it goes into the second (left) register. When a datum is popped the two registers are shifted right. This scheme avoids having combinatorial logic at the output thus minimising clock-to-data-out time. The tradeoff is a longer input setup time. The two flip-flops represent the occupancy status of the two registers. Each register/flip-flop pair has a common clock enable controlled by a 4-input LUT.

The truth table for the control logic is as follows -

t							t+1						
		av		ce			av		ce				
PUSH	POP	[0]	[1]	[0]	[1]	MUX	[0]	[1]	[0]	[1]	NE	NF	
0	0	0	0	0	0	X	0	0	0	0	0	1	empty
		0	1	0	0	X	0	1	0	1	1	1	1 entry
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	0	0	X	1	1	1	1	1	0	full
0	1	0	0	-	-	-	-	-	-	-	-	-	illegal state
		0	1	X	1	X	0	0	0	0	0	1	av[1] ↓ POP, now empty
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	1	1	1	0	1	0	1	1	1	av[0] ↓ POP, 1 remaining
1	0	0	0	0	1	0	0	1	0	1	1	1	av[1] ↑ PUSH, 1 entry
		0	1	1	0	X	1	1	1	1	1	0	av[0] ↑ PUSH, now full
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	-	-	-	-	-	-	-	-	-	illegal state
1	1	0	0	-	-	-	-	-	-	-	-	-	illegal state
		0	1	0	1	0	0	1	0	1	1	1	1 entry
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	-	-	-	-	-	-	-	-	-	illegal state

PUSH must not be raised when **NF** is low and **POP** must not be raised when **NE** is low.

The illegal states are described in the following table

illegal states

	PUSH	POP	av[0]	av[1]	
1	0	0	1	0	single entry must be at right
2	0	1	0	0	cannot pop when empty
3	0	1	1	0	single entry must be at right
4	1	0	1	0	single entry must be at right
5	1	0	1	1	cannot push when full
6	1	1	0	0	cannot pop when empty
7	1	1	1	0	single entry must be at right
8	1	1	1	1	cannot push when full

In entries 1, 3, 4 and 7 a single datum in the buffer must reside in the right register, so these states are not allowed.

In entry 2 the registers are both empty and so a **POP** is not allowed since there is no valid output (**NE** is low). The logic has been designed so that no state change occurs, i.e. the **POP** is ignored.

In entry 6 the registers are both empty and so a **POP** is not allowed since there is no valid output (**NE** is low), however **PUSH** is high. The logic has been designed so that the **PUSH** is handled but the **POP** is ignored.

In entries 5 and 8 both registers are full and hence no **PUSH** is allowed. Note that this is true even when **POP** is asserted since we cannot allow **NF** (not full) to depend on **POP** or we generate the very logic cascade this buffering scheme was designed to avoid. The logic has been designed to ignore **PUSH** in these cases.

The truth tables for the two LUTs are -

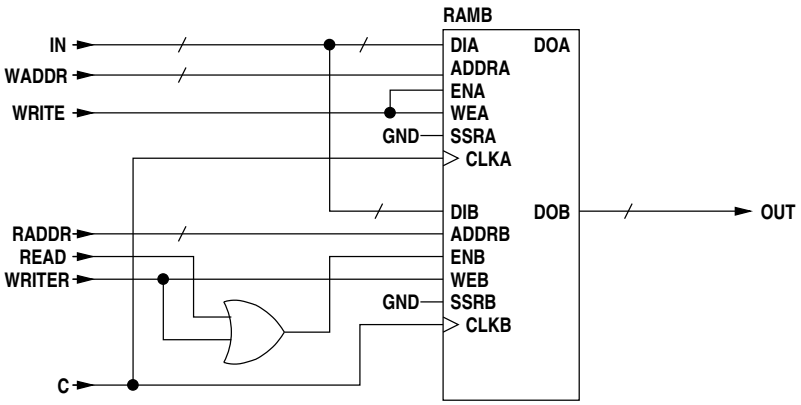
	PUSH	POP	av[0]	av[1]
ce[0]	0	1	1	1
	1	0	0	1
ce[1]	0	1	X	1
	1	X	0	0
	1	1	0	1

A synchronous queue buffer of depth greater than two is implemented using either CLB dual port RAM or dual port block RAM. The CLB RAM is used for a depth of 16 and block RAM for greater depths.

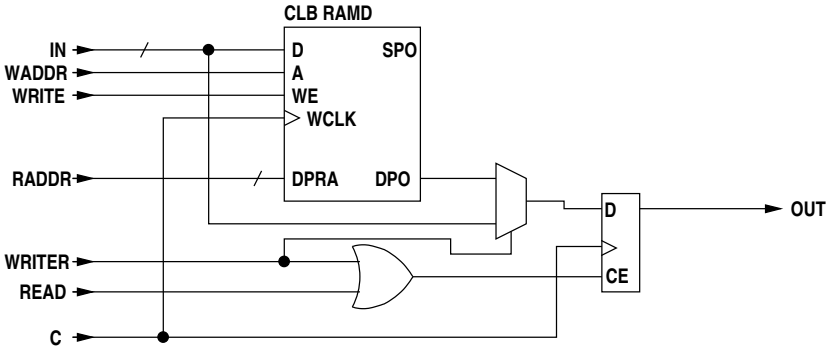
Because CLB RAM has combinatorial output and block RAM has registered output the CLB RAM is employed with a register appended to the output port so that the two forms of RAM can be used interchangably with the same control logic.

The queue buffer is a queue. A required characteristic of queue buffers is that on writing a datum to an empty queue buffer that datum must be available on the next clock cycle. This presents a problem with a dual-port FIFO using registered output RAM because a read must be issued once the queue becomes non-empty, taking an extra cycle. This can be avoided by writing the first datum into a separate register rather than into the empty RAM. For CLB RAM this is easily implemented however in the case of block RAM the control logic for this is somewhat complicated. An easier solution for block RAM is to write the first datum into an empty queue via the read port rather than the write port. Since the queue is empty the read port is guaranteed to be unused on that clock cycle. Since the normal write mode for block RAM is WRITE_FIRST and the CLB RAM with appended output register has been designed to behave the same way, the newly written datum will appear immediately in the output register upon writing. This avoids the extra cycle.

For a block RAM the memory logic is -



To make an equivalent functional unit using CLB RAM the logic is -



In both the above blocks **WRITER** writes directly to the output register whereas **WRITE** writes to the RAM. **READ** reads from the RAM to the output register.

The following terms are used in the logic description -

- empty - the RAM is empty
- full - the RAM is full
- NE - data is available at the output of the queue buffer
- NF - data can be written to the queue buffer
- read - read RAM
- write - write RAM
- writer - first write when queue buffer empty
- raddr - RAM read address
- waddr - RAM write address
- fcnt - RAM occupancy counter
- $\Uparrow$ - increment
- $\Downarrow$ - decrement

Note that a RAM occupancy counter, **fcnt**, is kept in addition to read and write address counters. **fcnt** is used to determine the RAM full and empty conditions and is potentially faster than doing comparisons between **raddr** and **waddr**. Additional counters are used for the optional word count and space count, again trading speed for logic and routing resources.

The following table shows the change of state and counter increment/decrement operations in controlling the queue buffer (**full** and **NF** are not shown) -

state		inputs		increment/decrement			memory control			new
empty	NE	PUSH	POP	waddr	raddr	fcnt	write	writer	read	NE
1	0	1	0				0	1	0	1
0	1	1	0	$\Uparrow$		$\Uparrow$	1	0	0	1
1	1	1	0	$\Uparrow$		$\Uparrow$	1	0	0	1
0	1	1	1	$\Uparrow$	$\Uparrow$		1	0	1	1
1	1	1	1				0	1	0	1
0	1	0	1		$\Uparrow$	$\Downarrow$	0	0	1	1
1	1	0	1				0	0	0	0

On first write to an empty queue buffer the datum is written directly to the read port of the RAM block and **NE** goes high, but **fcnt** stays at 0 as nothing is written to the RAM. The next write (if there is no read) will write to RAM and increment both**fcnt** and **waddr**. On acknowledging the output datum (i.e. popping a datum off the queue) if there is only the datum in the read port register **NE** will go low since **fcnt** is 0 (**empty** will be true). If there is a datum in the output register and another in the RAM then the last RAM read to the port register will occur and **fcnt** will decrement from 1 to 0 and empty will become true, but NE will not go low until the next **POP**.

Related equations -

$$\mathsf{NF} = \overline{\mathsf{full}}$$
$$\mathsf{read} = \mathsf{POP} \cdot \overline{\mathsf{empty}}$$
$$\mathsf{write} = \mathsf{PUSH} \cdot \mathsf{NE} \cdot \overline{(\mathsf{empty} \cdot \mathsf{POP})}$$
$$\mathsf{writer} = \mathsf{PUSH} \cdot \mathsf{empty} \cdot (\mathsf{NE} \cdot \overline{\mathsf{POP}})$$

State changes -

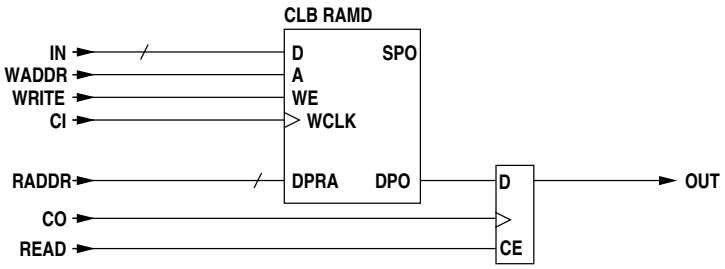
$$\mathsf{NE} \Leftarrow \overline{\mathsf{empty} \cdot \mathsf{NE} \cdot \overline{\mathsf{PUSH}} \cdot \mathsf{POP}}$$
$$\mathsf{empty} \Leftarrow \overline{\mathsf{write}} \cdot (\mathsf{fcnt} == 0 + \mathsf{fcnt} == 1 \cdot \mathsf{read})$$
$$\mathsf{full} \Leftarrow \overline{\mathsf{read}} \cdot (\mathsf{fcnt} == 11 \cdots 11 + \mathsf{fcnt} == 11 \cdots 10 \cdot \mathsf{write})$$

Increment/decrement counters -

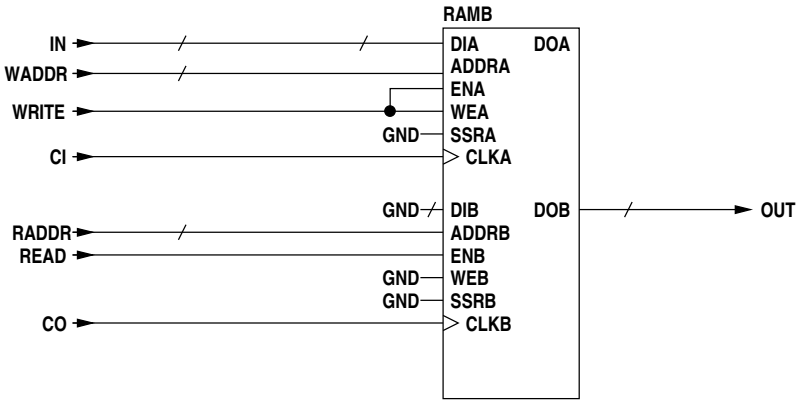
$$\mathsf{raddr} \Uparrow = \mathsf{read}$$
$$\mathsf{waddr} \Uparrow = \mathsf{write}$$
$$\mathsf{fcnt} \Uparrow = \mathsf{write} \cdot \overline{\mathsf{read}}$$
$$\mathsf{fcnt} \Downarrow = \mathsf{read} \cdot \overline{\mathsf{write}}$$

For an asynchronous queue buffer the input and output clocks differ and the scheme for avoiding an initial read on a write to an empty queue buffer cannot be used. The point at which a queue becomes available after a write to an empty buffer is now undefined since the clocks are separate and unrelated.

The CLB RAM block now simply has a register on the output of the read-only port.

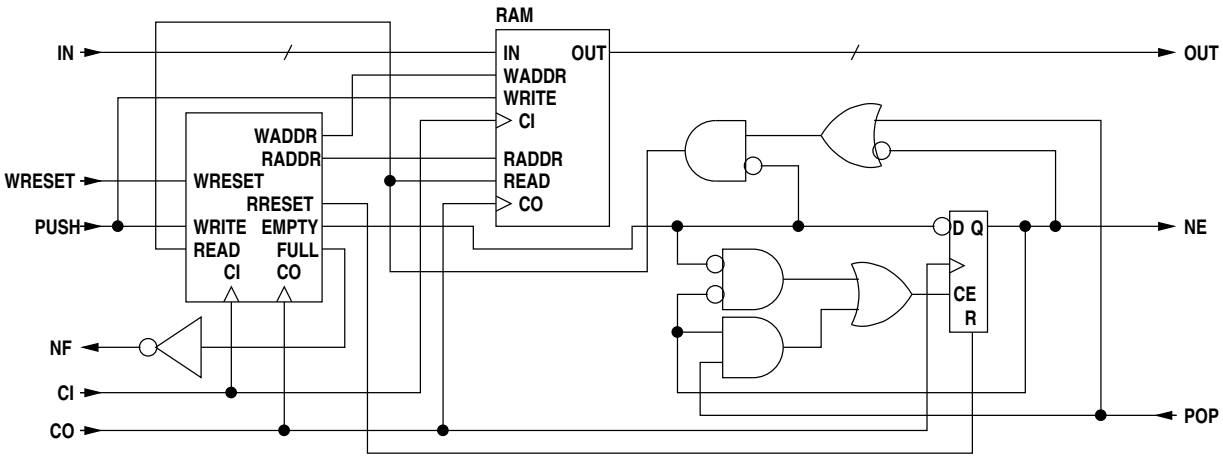


The block RAM has no additional logic.



The asynchronous queue buffer uses one of the above RAMs (the former for depth 16, the latter for greater depths) and an address generator block. The use of address comparison for full and empty determination means that the depth of the RAM FIFO is $2^n - 1$ however an additional datum is held in

the RAM output register giving a total buffer depth of 2^n. The additional logic shown provides the first read following a write to an empty buffer after which **NE** is raised.

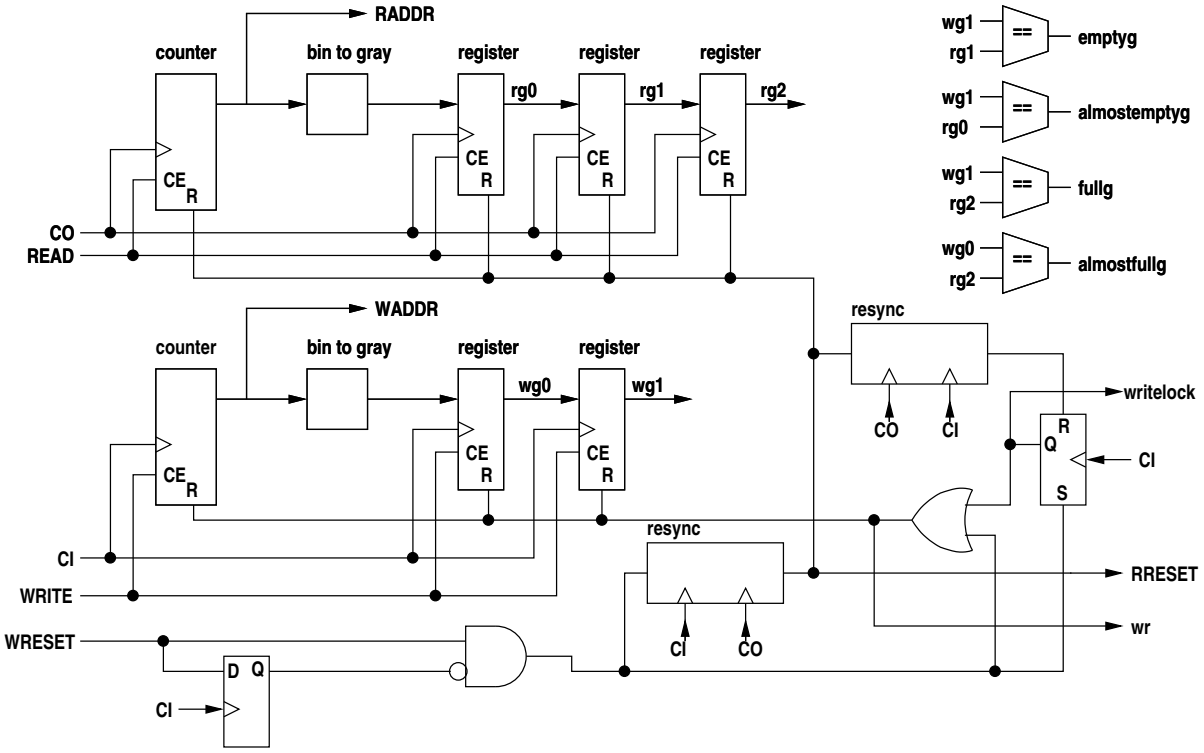


Resetting an asynchronous queue buffer must be carried out with care. A pending read must be allowed to complete after which **NE** will go low and no more reads will occur (since the queue buffer is now empty). A pending write must be ignored and **NF** forced low (the source will now remove **PUSH**, withdrawing the write request). **NF** must remain low during the reset process (which may be many cycles if the input clock is higher than the output clock) and then go high when the now empty queue buffer is again available for writing.

The reset input is synchronised to the write (input) clock. When asserted it resets the write address and clears the full flip-flop (and holds it clear during the reset process). At the same time it sets a blocking flip-flop to force **NF** low so that a write will not occur (the reset also immediately forces **NF** low to supress a possible write in the current cycle). The reset is resynchronised to the read (output) clock to later reset the read address, the empty flip-flop and the **NE** flag. A further pulse is then generated once again in the input clock domain to finally free the blocking flip-flop and raise **NF** after the output side has been reset. This mechanism is necessary to avoid attempts to write to the buffer before the output side is reset. On the other hand the output side reset must be synchronised to the output clock so that it can fit in with pending reads, lowering **NE** synchronously with the output to stop further reads.

If the queue buffer is never reset the reset logic is omitted.

The address generator follows the design given in Xilinx application note 131.



When **WRESET** is asserted it resets the write address counter to zero and the following grey code registers to the initial values -

wg0 100...000
wg1 100...001

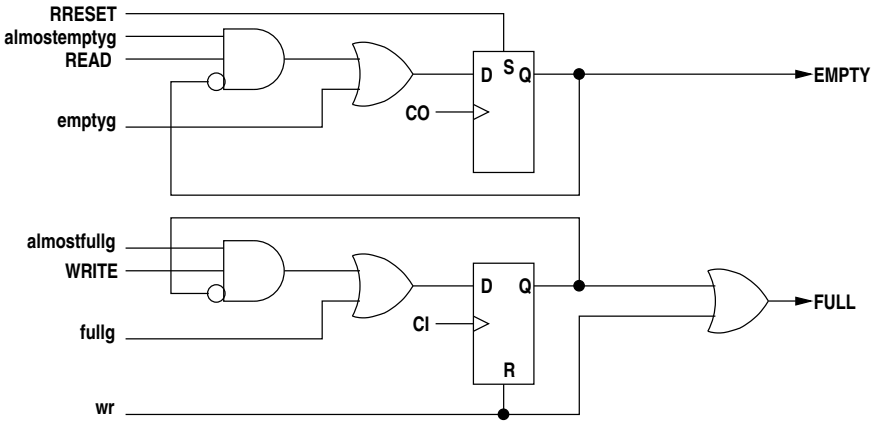
At the same time it is gated through to **FULL** and also latched in a flip-flop so that the queuebuffer appears full and will not be written on the current or subsequent clock cycles. The write reset is resynchronised to the read clock and the resulting signal is output to **RRESET** for possible daisy chaining to further queues in the pipeline. **RRESET** also resets the **EMPTY** flip-flop, resets the read address counter to zero and the following grey code registers to the initial values -

rg0 100...000
rg1 100...001
rg2 100...011

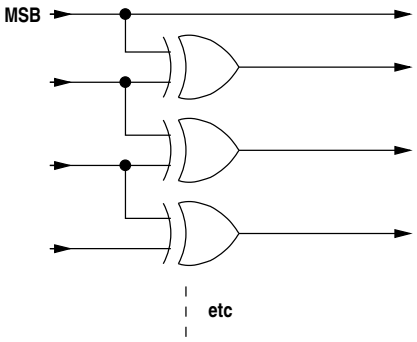
These initial values on the write and read sides ensure the queue buffer appears initially empty and empty after a reset. The values are the gray code sequence leading up to the current read and write addresses of 0 (gray code 0) and hence are gray code equivalents of -1, -2 and -3.

RRESET is then resynchronised back to the write clock and resets the flip-flop holding **FULL** asserted. The reset process thus takes a number of read and write clock cycles, the exact duration depending on the two clock frequencies. The reset inhibits any writes on the current clock cycle and from then on until the reset is complete. The queue buffer does not appear empty immediately the reset is asserted on the write side, however only one read can take place (if the queue buffer is not already empty) and that will read a correct datum. After that the queue buffer will be reset on the read side and will appear empty. The queue buffer can then only appear non-empty after a subsequent write following the reset.

The gray code address comparisons at various phases are compared to determine empty and full conditions -

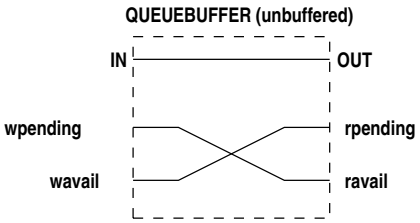


The binary to gray code converter logic is -



A (synchronous) unbuffered queue is created by connecting the **NE** (not empty) output net to the **PUSH** input net and the **NF** (not full) output net to the **POP** input net. Since the functionality of an unbuffered queue differs substantially from that of a buffered queue the inputs and outputs are better labelled **WRITE** for **PUSH**, **READ** for **POP**, **WAV** for **NF** and **RAV** for **NE**.

With these cross connections asserting the **WRITE** input asserts the **RAV** output and the queue now has read availability. In the same way asserting the **READ** input asserts the **WAV** output and the queue now exhibits write availability. The **WRITE** is driven by the **WPENDING** output of an EXECP and the **READ** is driven by the **RPENDING** output of an EXECP. This is described later (2.8.17). The data output is simply connected to the data input as the unbuffered queue does not store anything.



Where an unbuffered queue is written or read in more than one place in the source code the **WRITE** and **READ** inputs are the PENDING signals from the different EXECPs ORed.

2.7.19 REG

purpose

A register (static variable).

arguments

Parameters	String[] - block ids String - initial value boolean - IOB flag boolean - DSP flag
Inputs	clock signal (may be null if .D null) data signal array (.D - may be null) clock enable signal or signal array (.CE) set/reset signal or signal array (.RES)
Outputs	data signal array

description

When the clock enable is high data will be clocked in on the next clock edge. For a primitive type all clock enable inputs are tied together as are all reset inputs. For a compound type clock enables and resets are grouped in words corresponding to primitives. The resets return the state to the initial value (0 by default). If the clock is null, there are no inputs to this register so generate a constant instead.

Optional attributes are -

timingname	timing group name
loc	flip-flop location
iob	place flip-flop(s) in IOB(s)

If any **loc** attributes are provided there must be a **loc** entry per bit. These are ordered from the LSB.

dump format

```
1    REG
2        OUT  glob.v[0..7]
3        <-
4        CLK  glob.c100
5        DATA glob.v.D[0..7]
6        CE   glob.v.CE[0..7]
7        R    glob.v.RES[0..7]
8        {0x0}{timingname=reg1}
```

The last line shows the initial value folowed by any attributes.

logic description

If the clock input is null a constant is generated and the other inputs are ignored, otherwise each output bit has a D-type flip-flop. The data input must be the same width as the output. The clock enable and reset inputs may be arrays in which case they must be the same width as the input and output. Alternatively they may both be width 1 in which case all bits will share the same clock enable and reset. Either or both may be null.

Optional attributes are -

timingname	"str"	timing group name
loc	"str" or "[]str"	flip-flop location(s)
iob	"log"	place flip-flop(s) in IOB(s)

2.7.20 RESYNC

purpose

A pulse or level resynchroniser.

arguments

Parameters	none
Inputs	input signal input clock signal output clock signal
Outputs	output signal

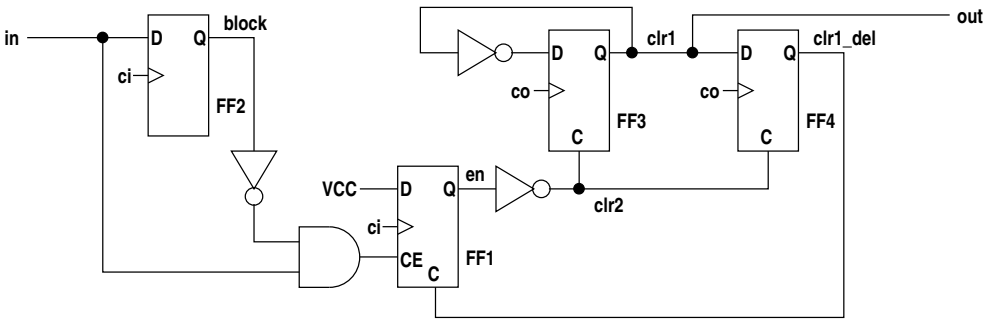
description

The input may be a pulse or a level. When the input rises the rise is captured by the next input clock edge and an output pulse of one period duration is generated on the next output clock edge.

dump format

```
1   RESYNC {
2       in:
3           glob.a[0]
4           glob.c100
5           glob.bus_clk
6       out:
7           E41[0]
8   }
```

logic description



Note that FF1, FF3 and FF4 have asynchronous clear inputs. Initially all flip-flop outputs are low, thus **block** and **clr1**/**out** are low. When **in** goes high both FF1 and FF2 outputs go high on the next edge of clock **ci**, the input clock. The **CE** input of FF1 goes low as **block** is now high. Since **en** is now high, **clr2** is low and this enables FF3 to toggle on the next edge of **co**, the output clock, raising the output **out**. On the next edge of **co** FF3 toggles low again and FF4 is set, raising **clr1_del**, which clears FF1. When **en** thus goes low **clr2** goes high clamping FF3 low and clearing FF4. FF1 is now free to again latch an input however if **in** is still high **block** will disable FF1. Thus **in** must be clocked low to lower **block** before FF1 can again start the pulse cycle. This circuit avoids race conditions between flip-flops in the two clock domains.

2.7.21 RRAM

purpose

Registered-output RAM.

arguments

Parameters	Integer number of ports Integer data width in bits Integer address width in bits String[] optional initialisation String - attribute key String - data for attribute key ... other attribute pairs
Inputs	clock signal, port0 address array, port0 data_in array, or null, port0 read signal, or null, port0 write signal, or null, port0 clock signal, port1 address array, port1 data_in array, or null, port1 read signal, or null, port1 write signal, or null, port1 ..etc..
Outputs	data_out array, or null, port0 data_out array, or null, port1

description

This is a RAM with clocked read and write. The output for a port reflects the memory contents following a read clock cycle.

The 2nd parameter gives the data width and the address width gives the memory depth (number of words). All port addresses must be the same width and all port data inputs and outputs must be the same width.

The initialisation string array has one entry for each initial value. Each entry is a hexadecimal string. The initialisation array may be smaller than the depth of the RAM but not larger. The initialisation parameter may be omitted in which case the RAM is explicitly initialised to all 0.

Other attributes (**writemode**, **writemodea**, **writemodeb**, **timingname**) if set result in entries written to the NCF file.

Ports which are not written have null arguments for the data in and the write enable. Ports which are not read have null arguments for the data out.

For Xilinx FPGAs registered-output RAM is block RAM. The address width is limited to a maximum of 12 bits (a depth of 4096 words) for Spartan 2, Virtex and Virtex E and 14 bits (a depth of 16384 words) for Spartan 3, Virtex 2 and Virtex 2 PRO. The data width is not limited.

Attributes are -

loc	block RAM location
timingname	timing group name
writemode	write mode
writemodea	port A write mode
writemodeb	port B write mode
continuous	continuous enable

There is one **loc** attribute per block. Block RAM formats are selected by address width, then the number of blocks is determined by the total data width divided by the block data width. Block locations are in order from the least significant data bits. The other attributes are be applied to each individual RAMB4_S, RAMB4_S_S, RAMB16_S or RAMB16_S_S prepended in each case with "INST name " where 'name' is the generated netlist identifier for the RAM component.

dump format

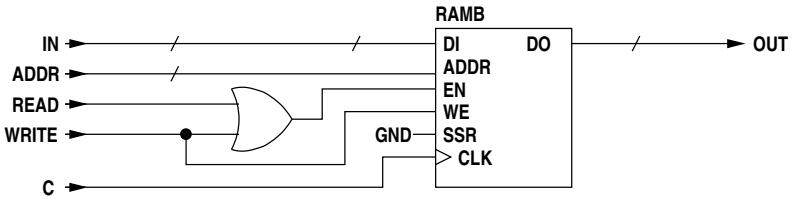
```
1     RRAM - 2 ports
2         OUT  glob.pileup_0.tmin_0[0..7]
3         OUT  glob.pileup_0.tmin_1[0..7]
4         <-
5         CLK  glob.c100
6         ADDR S5309[0..11]
7         DATA null
8         RE   S5310
9         WE   null
10        CLK  glob.c100
11        ADDR S5311[0..11]
12        DATA S5313[0..7]
13        RE   null
14        WE   S5312
15    initialised{continuous=true}
```

The presence of an initialisation argument is flagged but the values are not listed.

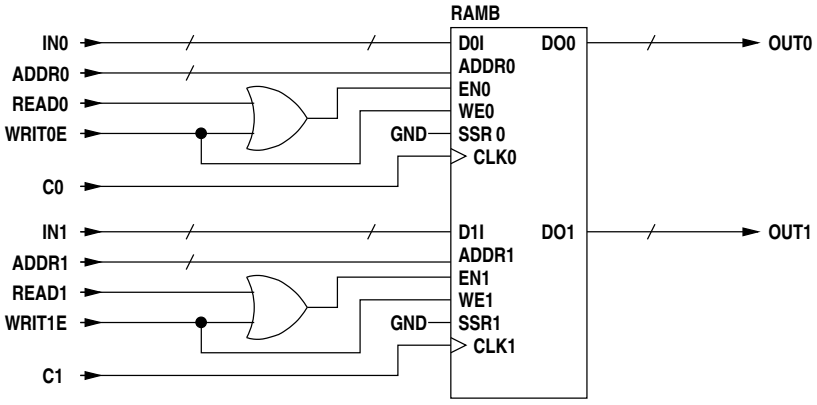
logic description

For Xilinx registered output RAM is block RAM. The register reset input is unused and the enable input is **READ** OR **WRITE** (gate omitted if only one of these inputs is used). BRAM ports are configured for the address width (if two ports they must be the same width) and the data width is matched by using as many blocks in parallel as required. WRITE_MODE is the default WRITE_FIRST.

For single port BRAM -



and for two port BRAM -



2.7.22 SAMPLE

purpose

Sample data from a different clock domain.

arguments

Parameters	none
Inputs	input data array input clock signal output clock signal
Outputs	output data array

description

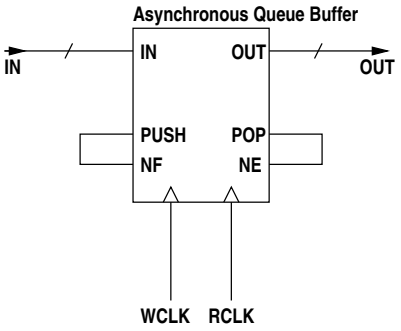
The input is sampled into the output clock domain. Sampled data will not be corrupted and will retain order, however depending on the relative clock rates data may be missed or multiply sampled (or both if the clock frequencies are close). The input may be a different size to the output, zero padding or truncation being applied as required.

dump format

```
1    SAMPLE {
2        params:
3        in:
4            glob.a[0..7]
5            glob.c100
6            glob.bus_clk
7        out:
8            E41[0..7]
9    }
```

logic description

Sampling is done by means of an asynchronous queue buffer of depth 4 (the minimum depth for an asynchronous queue buffer). The PUSH input is tied to the NF (not full) output and the POP input is tied to the NE (not empty) output. In the special case where the data is only one bit wide a single flip-flop clocked by the read clock is used instead since data skew is not relevant.



2.7.23 SELECT

purpose

Select one of several inputs using CLB logic.

arguments

Parameters	unselected out - optional use alu - optional dsp - optional
Inputs	select signal signal array select signal signal array . . .
Outputs	signal array

description

The inputs are in pairs. The first of each pair is an input signal which selects the second of the pair (a signal array) to be connected to the output signal array. The input arrays may be smaller than the output array in which case high-order input bits will padded or sign extended as appropriate. If no select signals are high the output is determined by hexadecimal string parameter **unselected out**, or is low if this parameter is not supplied. Parameters **use alu** and **dsp** are flags used by the platform-specific TDE list optimiser. Where an optional parameter any leading parameters must be supplied.

The selector may have duplicate inputs as these cannot always be identified as the TDEVars may be connected together later. Optimisation of duplicate inputs is resolved during later netlist optimisation.

dump format

```
SELECT {
    par: false
    in:
        S11
        glob.a[0..7]
        S15
        glob.b[24..31]
    out:
        glob.c[0..7]
}
```

logic description

Each output bit has an independent input selector. Where a data input is narrower than the output, high order data bits are connected to GND. These inputs will be optimised out during low-level code generation. If a data input is wider than the output the high order bits of the input will be ignored.

Xilinx target implementation -

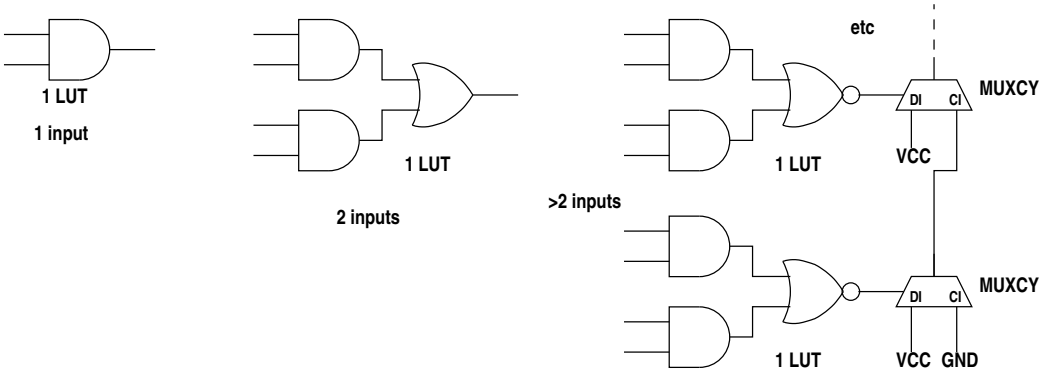
The logic depends on whether the default output is 0 or 1.

for default 0 output -

A 1-input bit selector is implemented as an AND gate with 2 inputs, the data bit and the selector bit. When the selector input is low, the output is low. When the selector input is high the output is the same as the data input.

A 2-input selector is implemented using a 4-input LUT. Each data input is ANDed with its selection input and the outputs from these two ANDed inputs are ORed.

Selectors with more than 2 inputs employ 4-input LUTs whose outputs are ORed using the carry chain MUCYs. In that case the LUT output is inverted.

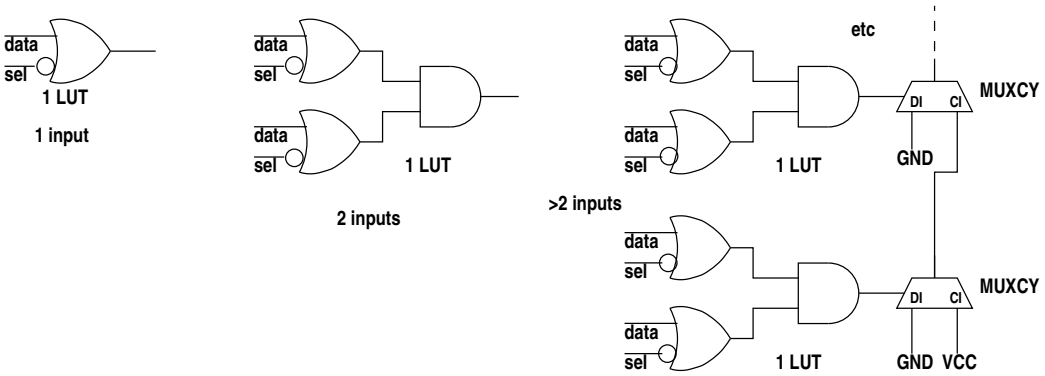


for default 1 output -

A 1-input bit selector is implemented as an OR gate with 2 inputs, the data bit and the inverted selector bit. When the selctor input is low the output is high. When the selector bit is high is the same as the data input.

A 2-input selector is implemented using a 4-input LUT. Each data input is ORed with its inverted selection input and the outputs from these two ORed inputs are ANDed.

Selectors with more than 2 inputs employ 4-input LUTs whose outputs are ANDed using the carry chain MUCYs.



2.7.24 SIG

purpose

Declare a signal or signal array.

arguments

Parameters	none
Inputs	signal or signal array signal or signal array . .
Outputs	none

description

There may be any number of inputs. Each input may be a single signal or an array. Declarations are created which will be dimensioned if arrays.

This element is only used by the simulator, not the code generator. It is not generated unless the -s or -T options are passed to the compiler.

dump format

```
1    SIG     glob.a
2    SIG     E1[1]
3    SIG     S11
4    SIG     mod1_1.a[8]
```

logic description

No logic is generated by this element.

2.7.25 SPECIAL

purpose

Provides special functions, perhaps vendor specific.

arguments

Parameters	Integer type of special function
Inputs	...
Outputs	...

description

The first parameter is an integer type code which selects the particular special function. At the moment there are only two special functions.

add line to the NCF or the XDC file

Parameters	Integer 0 or 1 String a format string
Inputs:	signal optional signal . .
Outputs:	none

The format string is written to the ISE NCF file or the Vivado XDC file. The format string may contain %n or %N conversions at which points netlist identifiers are to be inserted. The netlist identifiers are obtained from the input signals which are matched to the conversions in order. There will usually be only one of these though the format allows any number. %n is replaced by the identifier of the next input signal argument. %N is similar to the above but changes the identifier to upper case and removes any leading "GLOB_" string. This is used for generating clock timing group names from the clock signal identifier. A % followed by any other character is unchanged. The target mode signal arguments are matched to the %n and %N escape sequences left to right. The number of target mode arguments must must be suffient for the number of %n and %N escape sequences, any excess nets will being ignored.

2.7.26 START

purpose

Start an execution stream. There is one START for each clock domain.

arguments

Parameters	none
Inputs	start level signal clock signal
Outputs	start pulse signal

description

The start signal is a level. If there is no explicit start level then this signal will be VCC.

The clock signal is the clock for whose domain the start pulse is being generated.

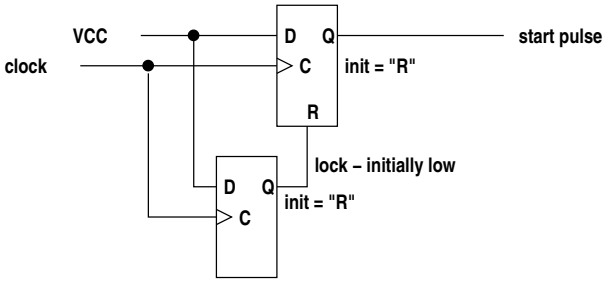
When the start signal rises the rise is captured by the next clock edge and an output pulse of one period duration is generated on the next clock edge. The logic is locked after this pulse so that any further start signal rises are ignored. The start level is a global signal for all execution threads to start. It must be re-synchronised to the clock domain to produce a single start pulse to start all execution threads in the domain. The start signal may be VCC if execution is to start immediately after configuration loading, otherwise it must be derived combinatorially from external inputs or from code generated by inbuilt procedure start() whose code is outside execution threads generated from 3PL source code.

dump format

```
1    START   glob.c100.start <- START=startsig, CLK=glob.c100
```

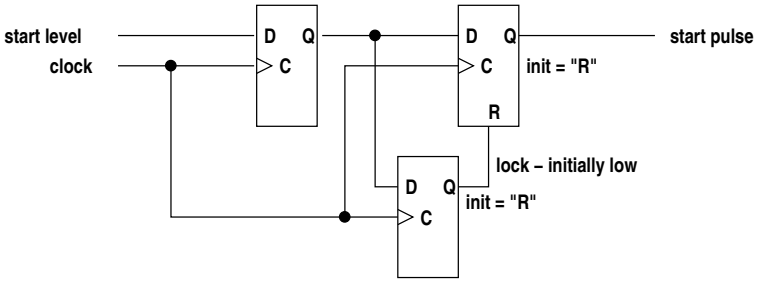
logic description

If no start signal is supplied then a start pulse is generated by a flip-flop whose D input is VCC. On the first clock edge the output will go high. At the same time the output of a second parallel flip-flop also goes high. This is a lock signal to the reset of the first flip-flop which on the next clock pulse will reset to low, the R input having precedence over the D input. The output of the first flip-flop will remain low as it is held reset.



If a start signal is supplied the start logic differs only in that the D inputs of two flip-flops are now driven by a the start level via an intervening flip-flop. This additional flip-flop is there to synchronise the asynchronous start level transition to a clock edge. Timing constrants will then ensure that paths to the following two flip-flops will meet set and hold requirements.

This form is not available for CPLDs.



The output pulse from this element is distributed to all ILOOP TDEs in the associated clock domain to start their execution. To avoid problems with a posibly large fanout an intervening single clock delay (DEL element, i.e. one DFF) is inserted in each start path, a one-cycle delay on startup not being significant.

2.7.27 TSELECT

purpose

Select one of several inputs using a 3-state driver.

arguments

Parameters	unselected out - optional
Inputs	select signal signal array select signal signal array . . .
Outputs	signal array

description

The inputs are in pairs. The first of each pair is an input signal which selects the second of the pair (a signal array) to be connected to the output signal array. If no select signals are high the output is determined by hexadecimal string parameter **unselected out**, or is low if this parameter is not supplied. The input arrays may be smaller than the output array in which case high-order input bits will be connected low.

dump format

```
1   TSELECT {
2       par:
3       in:
4           S11
5           glob.a[0..7]
6           S15
7           glob.b[24..31]
8       out:
9           glob.c[0..7]
10  }
```

logic description

Each output bit has a 3-state driver. Where a data input is narrower than the output, high order data bits are connected to GND. If a data input is wider than the output the high order bits of the input will be ignored. The control input is common to all outputs. Default output for each bit is determined by a pullup or pulldown.

Xilinx target implementation -

A BUFE is used for each output bit. Default output must be all 1s as pulldowns are not accepted. This TDE cannot be used for FPGAs which do not have internal 3-state buffers, e.g. Virtex-5 and later.

2.7.28 WAIT

purpose

A parallel block completion wait.

arguments

Parameters	none
Inputs	clock signal reset signal or null input signal . . .
Outputs	output signal

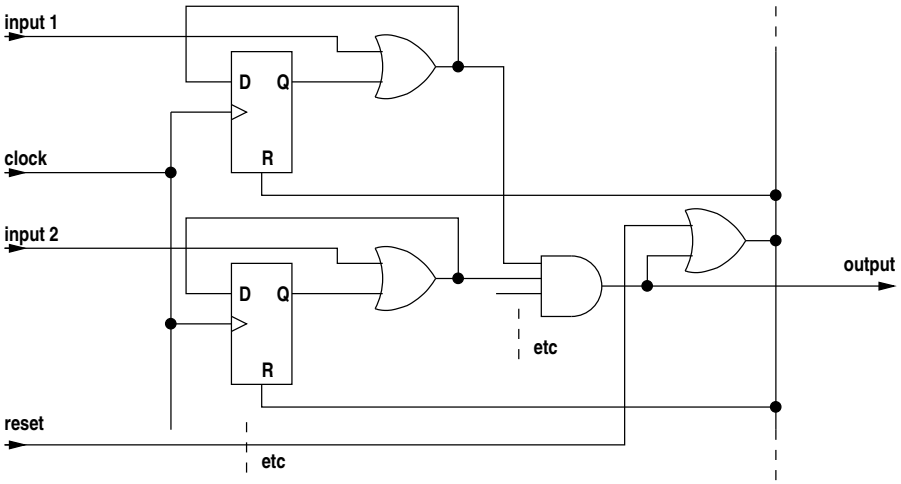
description

There are four or more inputs. The first input signal is the clock signal for the clock domain. The second input signal is a reset signal or is null if there is no reset. The third and following are finish signals. If all input signals go high simultaneously the output signal also goes high, otherwise the output signal goes high when the last input signal goes high. The output signal is high for one clock cycle. The reset input clears any pending latched completion signals.

dump format

```
1    WAIT {
2        in:
3            c100
4            null
5            S11
6            S13
7        out:
8            S15
9    }
```

logic description



If all WAIT inputs are high, all AND gate inputs are high (via the OR gates) and hence the WAIT output is high. None of the flip-flops are set because all resets are also driven high via the WAIT output. If one or more inputs is low when an input goes high the WAIT output will not go high and the flip-flop associated with that input will be set. When the last input finally goes high the previous latched inputs will be ANDed with it, the WAIT output will go high and all latched inputs will be reset. The reset input is ORed with the WAIT output to reset all flip-flops.

2.7.29 WHEN

purpose

A *when* statement.

arguments

Parameters	none
Inputs	start signal test signal true block finish signal or null false block finish signal or null
Outputs	true block start signal false block start signal WHEN finish signal or null

description

A start signal will be routed to either the true block start output or the false block start output according to whether the test input signal is high or low. The finish signals for the true and false blocks are returned as inputs and the OR of these emerges as the third output, signalling the end of the statement.

A missing true or false block will have the block start signal connected back to the associated block finish signal via a 1-cycle delay (TDEType.DEL). If both the true block and the false block require a single DEL for their finish signals these will be optimised to a separate common DEL from the start signal to the finish signal. In that case -

- true block finish signal

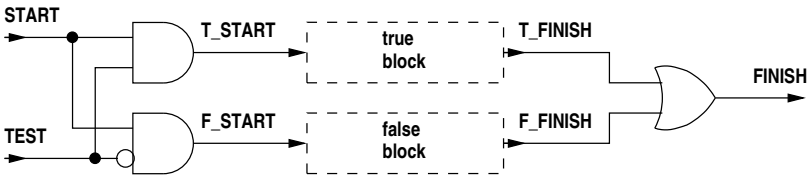
- false block finish signal
- WHEN finish signal

arguments to the WHEN will all be null.

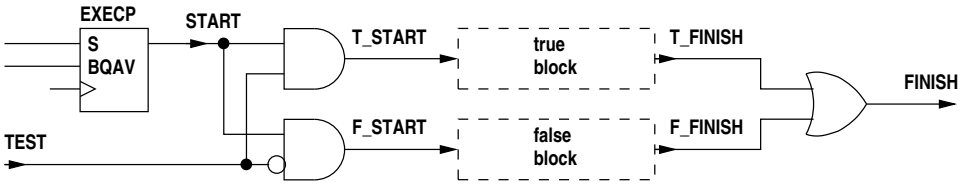
dump format

```
    WHEN {
        START           S22
        TEST            glob.cpu_write_static_0.write_1[0]
        T FINISH        TF24
        F FINISH        FF27
        T START         TS21
        F START         FS22
        FINISH          F30
    }
```

logic description



The START signal is directed either through output T_START or F_START depending on whether TEST is high or low. These outputs execute either the true code block or the false code block. The finish signals from the code blocks are ORed to get the finish signal for the WHEN. If one of the code blocks is missing a DEL will be inserted instead.



An EXECP is required at the START input if the test expression contains queue reads. The BQAV input to the EXECP is the AND of the .AV signals of all read queues in the test expression.

2.7.30 WHILE

purpose

A *while* statement.

arguments

Parameters	none
Inputs	start signal test signal continuation signal from the body clock signal optional reset signal or null
Outputs	start signal for the body WHILE finish signal

description

If the test input signal is high when start input signal goes high, the latter is routed to the body start output signal to execute the contained code block. If the test input signal is low when start input signal goes high, the latter is delayed by one clock cycle and then appears on the finish output signal (i.e. a *while* takes a minimum of one clock cycle). When the body code block finishes the last execution signal comes back to the continuation input signal. If the test input signal is high the continuation input signal is routed to the body start output signal to again execute the contained code block. If the test input signal is low the continuation input signal is routed through to the finish output signal, again with one clock cycle delay.

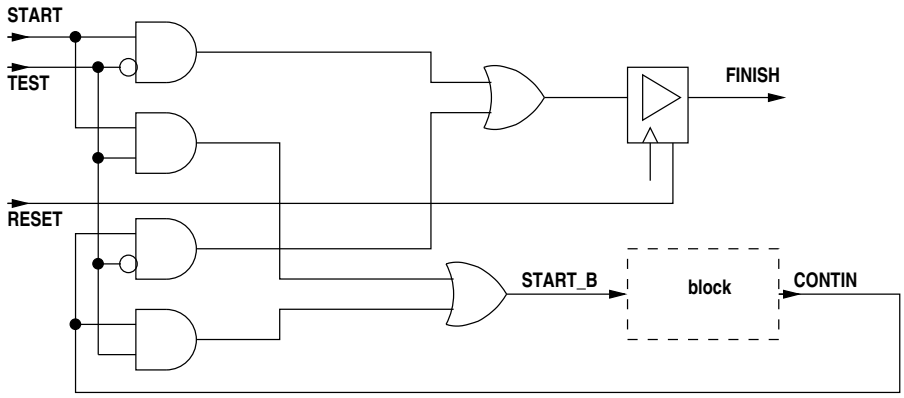
The start signal and the continuation signal may come from EXECP elements which wait on queue availability in the test value.

The optional reset signal clears internal state in the WHILE.

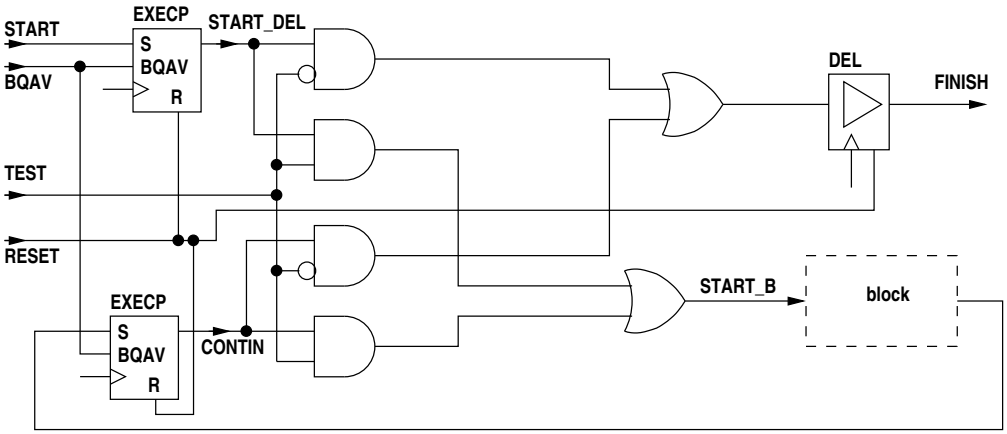
dump format

```
WHILE {
    START_B     S22
    FINISH      S30
    <-
    START       S24
    TEST        E31[0]
    CONTIN      S21
    C           glob.c100
    RESET       s34
}
```

logic description



The WHILE samples the test expression value on initial entry and on each subsequent iteration. If the test fails on initial entry the START signal is routed through a DEL to FINISH. If the test succeeds on initial entry START is passed on to the code block via START_B. When the code block finishes its finish signal re-enters the WHILE logic via CONTIN where the test value is again sampled. If the test succeeds the code block is executed again. If the test fails CONTIN is directed again through the DEL to FINISH. A RESET clears the DEL. The DEL is required to avoid a race condition through to the following statement.



Two EXECPs are required if the test expression contains queue reads. One controlling the START signal and the other controlling the CONTIN signal. The BQAV input to both EXECPs is the AND of the .AV signals of all read queues in the test expression. A RESET clears the DEL and EXECPs.

2.7.31 XOR

purpose

An XOR gate for control signals.

arguments

Parameters	none
Inputs	input signal input signal . .
Outputs	AND signal

description

Exclusive OR two or more input signals, the result appearing on the output.

dump format

```
1    S11 = A12 ^ A14 ^ S8
```

logic description

Xilinx implementation -

XOR gates with 1 to 4 inputs will be implemented in a single LUT (single input gates will be eliminated during low-level code generation). XOR gates with more than 4 inputs generate nested XOR gates.

2.8 Intermediate code examples

Note that in the following examples the intermediate code dumps and the associated diagrams only show sections of the logic that illustrate the points under discussion - they are not complete listings or diagrams.

In many cases clock inputs are not shown. The examples are constructed to use clock **c100** and this can be assumed unless shown otherwise.

In all the examples the signal **glob.c100.start** appears. This is a single cycle pulse, synchronised to the **c100** 100MHz clock, which is distributes to all execution threads to initiate them. In practice a recent change to the compiler has inserted a single cycle delay between this signal and every thread that it initiates but this is not shown in the examples. The purpose of the extra cycle is simply to avoid the path delays associated with distributing **glob.c100.start** which can have a very high fanout. The inserted flip-flop in the path to each thread will be placed close the the logic that it initiates resulting in a much shorter path into the thread. **glob.c100.start** will still have a high fanout but there is no combinatorial logic between its source and the myriad destination flip-flops.

2.8.1 top level sequential block

The code -

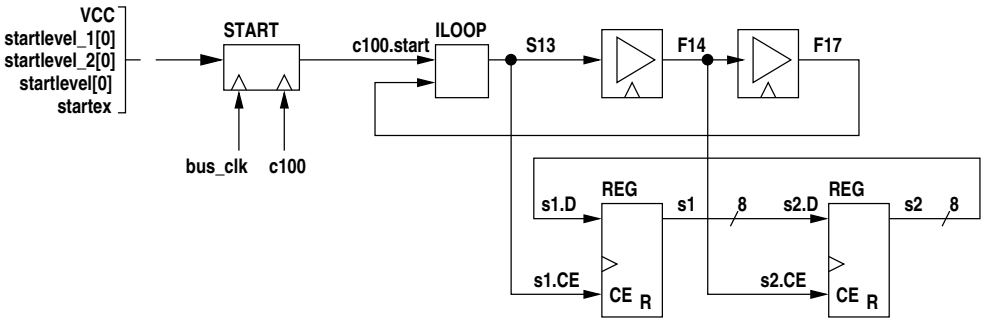
```
module "csiro/hymod/io1.3pl"

useclock(c100);

static({s1, s2}, "uint:8");

seq {
    s1 <- s2;
    s2 <- s1;
}
```

contains one execution thread. That thread will be an infinite loop containing two executable statements. The assignments do not wait on queues. The intermediate code would look like -

```
startex = glob.startlevel_1[0]
DEL F14 <- (1) glob.c100 S13
DEL F17 <- (1) glob.c100 F14
ILOOP       S13 <- glob.c100.start F17
glob.startlevel_1[0] = VCC
glob.startlevel_2[0] = VCC
glob.startlevel[0] = glob.startlevel_2[0]
START       glob.c100.start <- START=startex, CIN=glob.bus_clk, COUT=glob.c100
glob.s1.CE[0..7] = S13
glob.s1.D[0..7] = glob.s2[0..7]
glob.s1.RES[0..7] = GND
REG
    OUT  glob.s1[0..7]
    <-
    CLK  glob.c100
    DATA glob.s1.D[0..7]
    CE   glob.s1.CE[0..7]
    R    glob.s1.RES[0..7]
    {0x00}
glob.s2.CE[0..7] = F14
glob.s2.D[0..7] = glob.s1[0..7]
glob.s2.RES[0..7] = GND
REG
    OUT  glob.s2[0..7]
    <-
    CLK  glob.c100
    DATA glob.s2.D[0..7]
    CE   glob.s2.CE[0..7]
    R    glob.s2.RES[0..7]
    {0x00}
```

The multiple versions of value variable **startlevel** (created in header code csiro/hymod/io1.3pl) are a result of the internal intricacies of the compiler. The last value the variable contains is assigned to the unappended identifier **glob.startlevel** in an attempt to provide a useful value in simulation.

Diagrammatically this is -



The thread is started from signal **startex** which is connected to **VCC**. This is sampled by **bus_clk** and then resynchronised to generate a single pulse using clock **c100**. This is injected into **ILOOP** to start the infinite loop. The execution signal for the body of the infinite loop is **S13**. The loop body is a **seq** block containing two statements. The first is executed by **S13** and a finish signal is generated one clock cycle later (**F14**. This finish signal is the start signal for the next statement. A finish signal is generated one clock cycle later for the second statement (**F17**). This second finish signal marks the completion of the loop body code so it is fed back to the **ILOOP** to generate the next iteration. There is no delay in **ILOOP** (it is simply an OR gate) so the loop executes every two clock cycles.

The two execute signals, **S13** and **F14** control the assignments to the two static variables **s1** and **s2** via their CE inputs.

2.8.2 top level static assignment

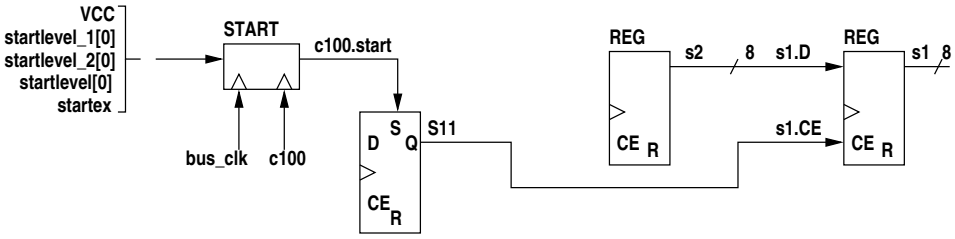
If we simplify the above code so that the infinite loop contains only a single assignment -

```
1    static({s1, s2}, "uint:8");
2
3    s1 <- s2;
```

then the intermediate code is -

```
1    startex = glob.startlevel_1[0]
2    glob.startlevel_1[0] = VCC
3    glob.startlevel_2[0] = VCC
4    glob.startlevel[0] = glob.startlevel_2[0]
5    START       glob.c100.start <- START=startex, CIN=glob.bus_clk, COUT=glob.c100
6    glob.s1.CE[0..7] = S11
7    glob.s1.D[0..7] = glob.s2[0..7]
8    glob.s1.RES[0..7] = GND
9    REG
10       OUT  glob.s1[0..7]
11       <-
12       CLK  glob.c100
13       DATA glob.s1.D[0..7]
14       CE   glob.s1.CE[0..7]
15       R    glob.s1.RES[0..7]
16       {0x00}
17   glob.s2.RES[0..7] = GND
18   REG
19       OUT  glob.s2[0..7]
20       <-
21       CLK  null
22       DATA null
23       CE   null
24       R    glob.s2.RES[0..7]
25       {0x00}  make const!
26   DFF S11 <- C=glob.c100, S=glob.c100.start
```

which diagrammatically is -



Since the **ILOOP** contains a single statement which unconditionally takes one clock cycle, i.e. it executes on every clock cycle, the loop can be eliminated and instead the destination register clock enable can be asserted when execution starts and remain asserted from then on. This is done by a D-flip-flop which is set by the start signal. The output of the flip-flop provides an execute signal for every clock cycle.

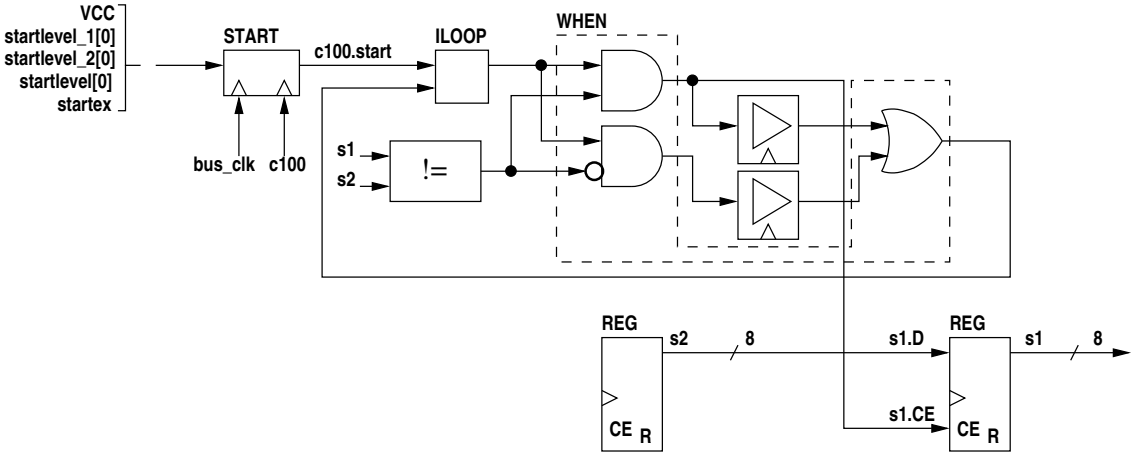
If the assignment is conditional -

```
1    static({s1, s2}, "uint:8");
2
3    when (s1 != s2)
4        s1 <- s2;
```

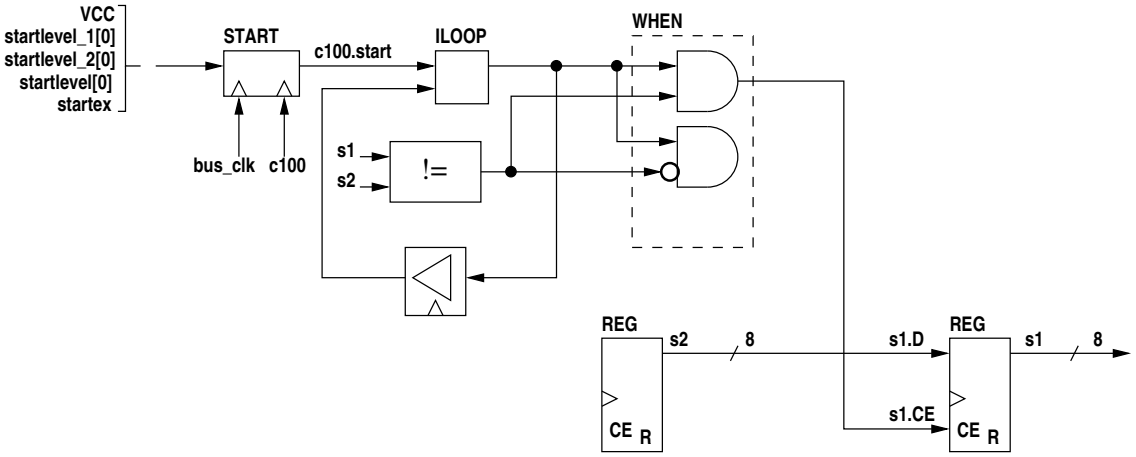
then the intermediate code becomes -

```
1    E8[0] = glob.s1[0..7] != glob.s2[0..7]      (uint, uint, log)
2    WHEN {
3            START    SD7
4            TEST     E8[0]
5            T_FINISH null
6            F_FINISH null
7            T_START  S17
8            F_START  FS6
9            FINISH   null
10   }
11   glob._sdreset_1[0] = VCC
12   glob._sdreset_2[0] = VCC
13   START  glob.c100.start <- START=VCC, CIN=glob.c100, COUT=glob.c100
14   glob._sdreset[0] = glob._sdreset_2[0]
15   glob.s1.CE[0..7] = S17
16   glob.s1.D[0..7] = glob.s2[0..7]
17   glob.s1.RES[0..7] = GND
18   REG
19       OUT  glob.s1[0..7]
20       <-
21       CLK  glob.c100
22       DATA glob.s1.D[0..7]
23       CE   glob.s1.CE[0..7]
24       R    glob.s1.RES[0..7]
25       {0x00}
26   glob.s2.RES[0..7] = GND
27   REG
28       OUT  glob.s2[0..7]
29       <-
30       CLK  null
31       DATA null
32       CE   null
33       R    glob.s2.RES[0..7]
34       {0x00}  make const!
35   DFF SD7 <- C=glob.c100, S=glob.c100.start
```

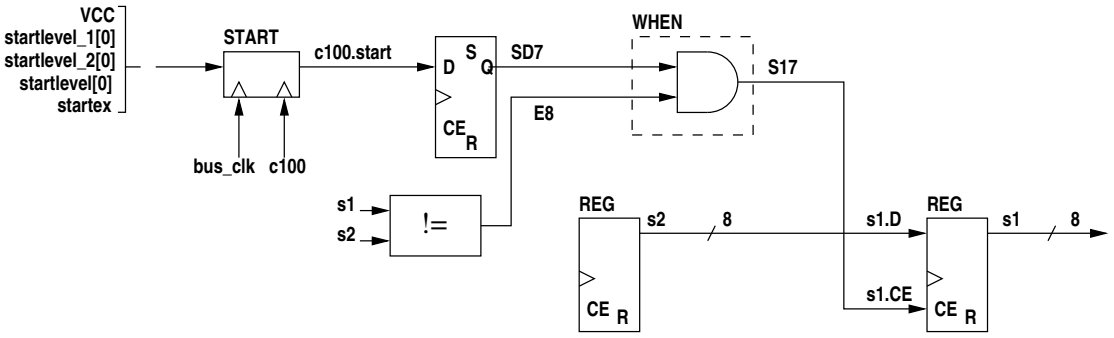
Now the unoptimised logic for the above code would be -



Since both the true and false bodies of the **WHEN** execute in a single cycle (the unused false statement still requires a single cycle delay) this will be optimised to a single shared delay -



The false side of the **WHEN** is now unused so its AND gate will be omitted. The **ILOOP** with a single delay feedback will be replaced with a set-once flip-flop -



Note that, like the simple assignment statement in the previous example, this logic has no execution thread any more. It is simply a register whose clock enable is asserted in clock cycles where the expression controlling the **WHEN** is true.

If the **WHEN** false code block contained a single cycle statement rather than being omitted then the second AND gate would be retained but otherwise the logic would be the same.

These examples show that top level assignments to static variables, even when conditional, have minimal logic overheads.

2.8.3 top level static assignment with queue reads

If the RHS of the assignment contains one or more queue reads then we must use an **EXECP**. For example -

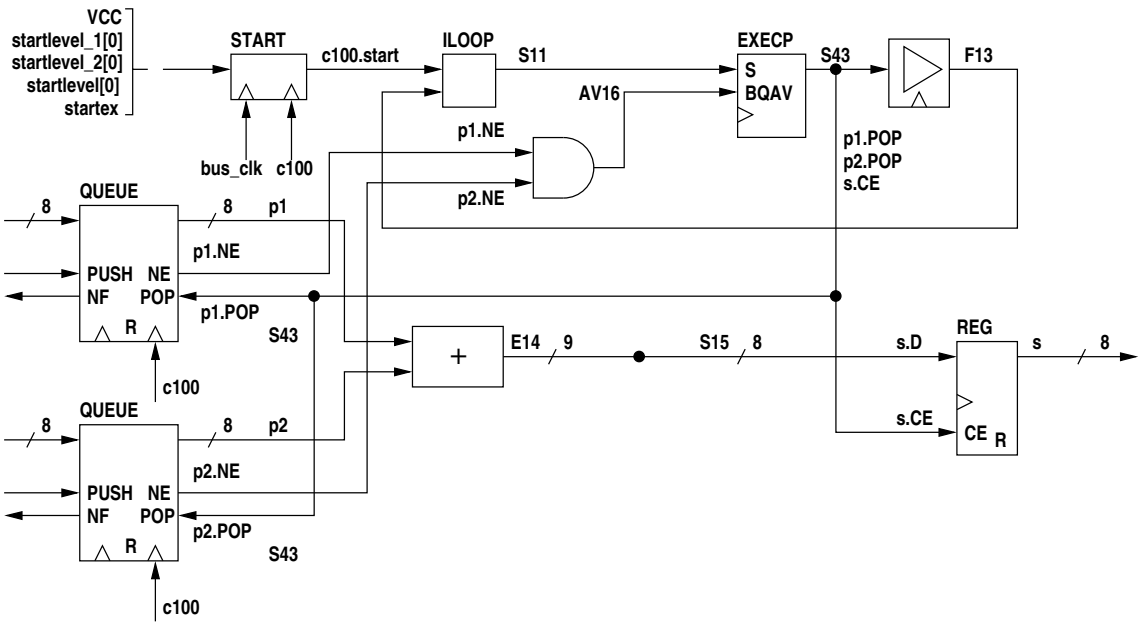
```
    static(s, "uint:8");
    queue({p1, p2}, "uint:8");

    s <- p1 + p2;
```

the intermediate code is -

```
    startex = glob.startlevel_1[0]
    glob.startlevel_1[0] = VCC
    glob.startlevel_2[0] = VCC
    glob.startlevel[0] = glob.startlevel_2[0]
    START       glob.c100.start <- START=startex, CIN=glob.bus_clk, COUT=glob.c100
    E14[0..8] = glob.p1[0..7] + glob.p2[0..7]   (uint, uint, uint)
    S15[0..7] = cast(false){E14[0..8]}
    AV16 = AV18[0] AND AV19[0]
    ACK20[0] = S43
    ACK22[0] = S43
    EXECP       S43 <- C=glob.c100, EXEC=S11, AV16
    DEL F13 <- (1) glob.c100 S43
    ILOOP       S11 <- glob.c100.start F13
    glob.s.CE[0..7] = S43
    glob.s.D[0..7] = S15[0..7]
    glob.s.RES[0..7] = GND
    REG
        OUT  glob.s[0..7]
        <-
        CLK  glob.c100
        DATA glob.s.D[0..7]
        CE   glob.s.CE[0..7]
        R    glob.s.RES[0..7]
        {0x00}
    glob.p1.RES = GND
    QUEUEBUFFER (2)
        OUT    glob.p1[0..7]
        NE     glob.p1.NE[0]
        NF     glob.p1.NF[0]
        CNT    null
        SPC    null
        WSTAT  null
        RSTAT  null
        <-
        CLKOUT glob.c100
        DATA   glob.p1.D[0..7]
        PUSH   glob.p1.PUSH
        POP    glob.p1.POP
        R      glob.p1.RES
    glob.p1.POP = ACK22[0]
    AV19[0] = glob.p1.NE[0]
    glob.p2.RES = GND
    QUEUEBUFFER (2)
        OUT    glob.p2[0..7]
        NE     glob.p2.NE[0]
        NF     glob.p2.NF[0]
        CNT    null
        SPC    null
        WSTAT  null
        RSTAT  null
        <-
        CLKOUT glob.c100
        DATA   glob.p2.D[0..7]
        PUSH   glob.p2.PUSH
        POP    glob.p2.POP
        R      glob.p2.RES
    glob.p2.POP = ACK20[0]
    AV18[0] = glob.p2.NE[0]
```

glob.p1.PUSH, **glob.p1.NF**, **glob.p2.PUSH** and **glob.p2.NF** are not connected here as they are used for assignments to the queues. Note that the code above would strictly give compiler errors since the queues have no sources (are not written).

The following shows the logic diagrammatically -



At the start **c100.start** is a single cycle pulse which will be passed through **ILOOP** to **S11**. If **AV16** is high (queues **p1** and **p2** are available for reading) then the pulse will in turn be passed as **S43**, which executes the assignment to static **s**. If **AV16** is low the input pulse **S11** is stored until **AV16** goes high when **S43** is finally driven by a single cycle pulse. The assignment executes in one cycle so a finish signal **F13** is generated by a one cycle delay from **S43**. The finish signal connects back to the **ILOOP** to cause the next iteration. If **AV16** remains high the assignment to **s** will occur on every clock cycle, **S11**, **S43**, and **F13** remaining high. **S43** executes the assignment by raising the clock enable to static register **s** (**s.CE**). At the same time it pops queues **p1** and **p2** since **p1.POP** and **p2.POP** are connected to **S43**. If either queue becomes unavailable for reading (empty) then **p1.NE** or **p2.NE** will go low and hence **AV16** will go low, causing the **EXECP** to wait.

Note that the input to static register **s** is 8 bits wide but the sum of the outputs of queues **p1** and **p2** is 9 bits wide. A cast() element from **E14** to **S15** will truncate the sum, ignoring the MSB.

2.8.4 top level queue assignment

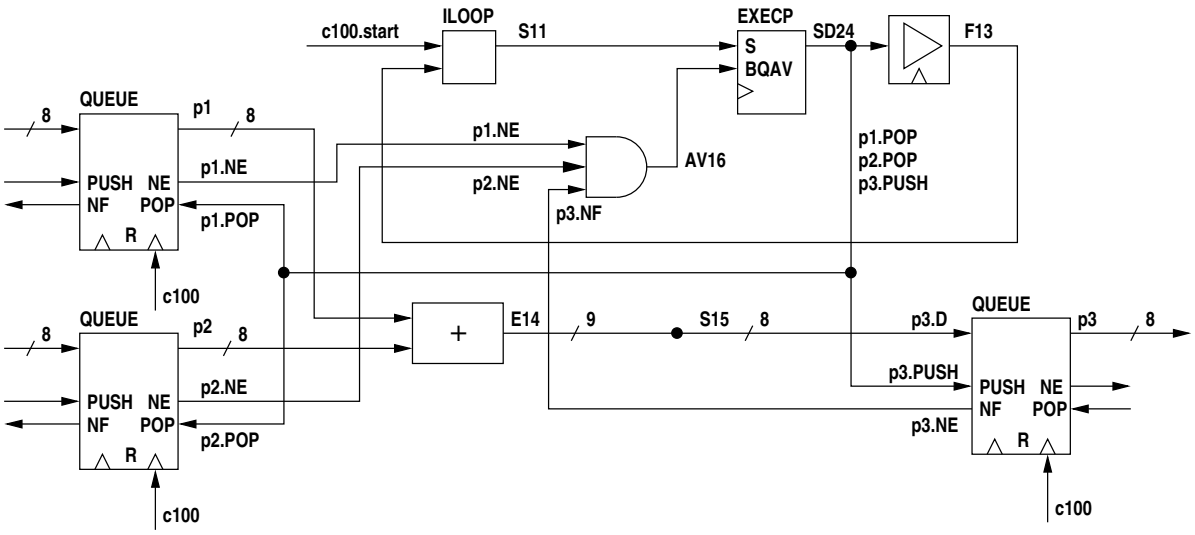
If we change the above code to a queue assignment -

```
1    static(s, "uint:8");
2    queue({p1, p2, p3}, "uint:8");
3
4    p3 <<- p1 + p2;
```

the intermediate code is -

```
1    E14[0..8] = glob.p1[0..7] + glob.p2[0..7]   (uint, uint, uint)
2    S15[0..7] = cast(false){E14[0..8]}
3    AV18[0] = glob.p2.NE[0]
4    AV19[0] = glob.p1.NE[0]
5    AV16 = glob.p3.NF[0] AND AV18[0] AND AV19[0]
6    EXECP       SD24 <- C=glob.c100, EXEC=S11, AV16
7    glob.p3.PUSH = SD24
8    ACK20[0] = SD24
9    glob.p2.POP = ACK20[0]
10   ACK22[0] = SD24
11   glob.p1.POP = ACK22[0]
```

```
    DEL F13 <- (1) glob.c100 SD24
    ILOOP       S11 <- glob.c100.start F13
    glob.p1.RES = GND
    QUEUEBUFFER (2)
        OUT    glob.p1[0..7]
        NE     glob.p1.NE[0]
        NF     glob.p1.NF[0]
        CNT    null
        SPC    null
        WSTAT  null
        RSTAT  null
        <-
        CLKOUT glob.c100
        DATA   glob.p1.D[0..7]
        PUSH   glob.p1.PUSH
        POP    glob.p1.POP
        R      glob.p1.RES

    glob.p2.RES = GND
    QUEUEBUFFER (2)
        OUT    glob.p2[0..7]
        NE     glob.p2.NE[0]
        NF     glob.p2.NF[0]
        CNT    null
        SPC    null
        WSTAT  null
        RSTAT  null
        <-
        CLKOUT glob.c100
        DATA   glob.p2.D[0..7]
        PUSH   glob.p2.PUSH
        POP    glob.p2.POP
        R      glob.p2.RES

    glob.p3.D[0..7] = S15[0..7]
    glob.p3.RES = GND
    QUEUEBUFFER (2)
        OUT    glob.p3[0..7]
        NE     glob.p3.NE[0]
        NF     glob.p3.NF[0]
        CNT    null
        SPC    null
        WSTAT  null
        RSTAT  null
        <-
        CLKOUT glob.c100
        DATA   glob.p3.D[0..7]
        PUSH   glob.p3.PUSH
        POP    glob.p3.POP
        R      glob.p3.RES
```



The assignment statement execution signal is now **SD24**. The **EXECP** now waits on **p3.NF**, **p1.NE** and **p2.NE**. The assignment statement execution signal now drives **p3.PUSH** to write to queue **p3**.

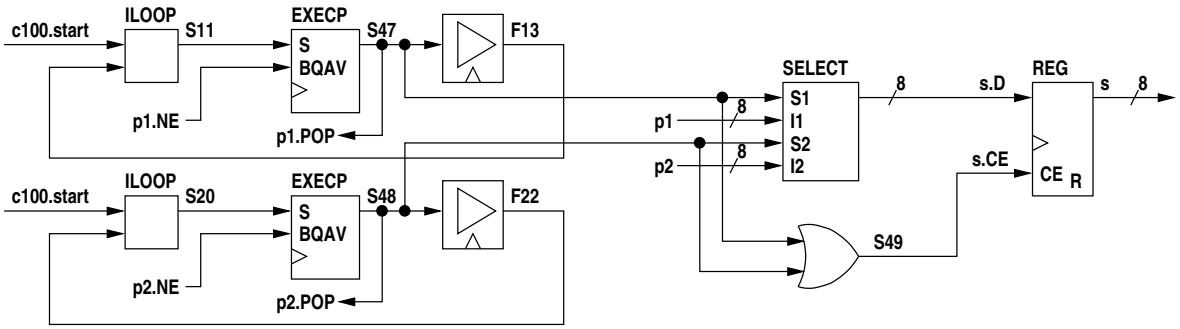
2.8.5 multiple assignments to a variable

The following code has two threads, each an assignment to the same static variable **s**. We assume here for the code to make sense that queues **p1** and **p2** are never available for reading simultaneously.

```
static(s, "uint:8");
queue({p1, p2}, "uint:8");

s <- p1;
s <- p2;
```

the intermediate code is -

```
AV16[0] = glob.p1.NE[0]
AV14 = AV16[0]
EXECP       S47 <- C=glob.c100, EXEC=S11, AV14
ACK17[0] = S47
glob.p1.POP = ACK17[0]
DEL F13 <- (1) glob.c100 S47
ILOOP       S11 <- glob.c100.start F13
AV25[0] = glob.p2.NE[0]
AV23 = AV25[0]
EXECP       S48 <- C=glob.c100, EXEC=S20, AV23
ACK26[0] = S48
glob.p2.POP = ACK26[0]
DEL F22 <- (1) glob.c100 S48
ILOOP       S20 <- glob.c100.start F22
S49 = S48 OR S47
glob.s.CE[0..7] = S49
SELECT {
    in:
        Var S47
        Var glob.p1[0..7]
        Var S48
        Var glob.p2[0..7]
    out:
        Var glob.s.D[0..7]
}
glob.s.RES[0..7] = GND
REG
    OUT  glob.s[0..7]
    <-
    CLK  glob.c100
    DATA glob.s.D[0..7]
    CE   glob.s.CE[0..7]
    R    glob.s.RES[0..7]
    {0x00}
```



Each thread is an **ILOOP** with an **EXECP** to wait for queue availability and a 1-cycle delay. If the first assignment statement executes **A18** goes high. If the second assignment statement executes **S21** goes high. These are ORed to drive the clock enable of static variable **s** to carry out the assignment.

The input data source is selected by the **SELECT** block with **A18** or **S21** selecting the appropriate data source (clearly it is a programming error if the assignments are not mutually exclusive).

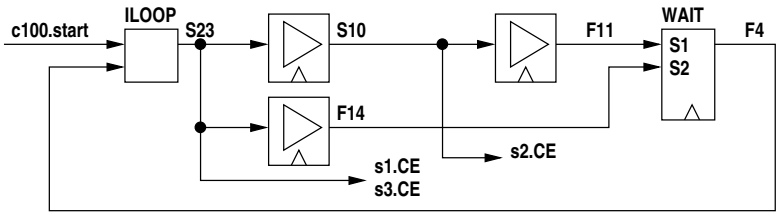
2.8.6 general parallel block

A PAR block is implemented by using the same start signal for all statements in the block and waiting for all their finish signals using a WAIT element. For example the code -

```
    static({s1, s2, s3}, "uint:8");

    par {
        seq  {
            s1 <- 3;
            s2 <- 4;
        }
        s3 <- 5;
    }
```

generates the intermediate code -

```
    DEL S10 <- (1) glob.c100 S23
    DEL F11 <- (1) glob.c100 S10
    DEL F14 <- (1) glob.c100 S23
    WAIT {
        in:
            Var glob.c100
            Var F11
            Var F14
        out:
            Var F4
    }
    ILOOP       S23 <- glob.c100.start F4
```



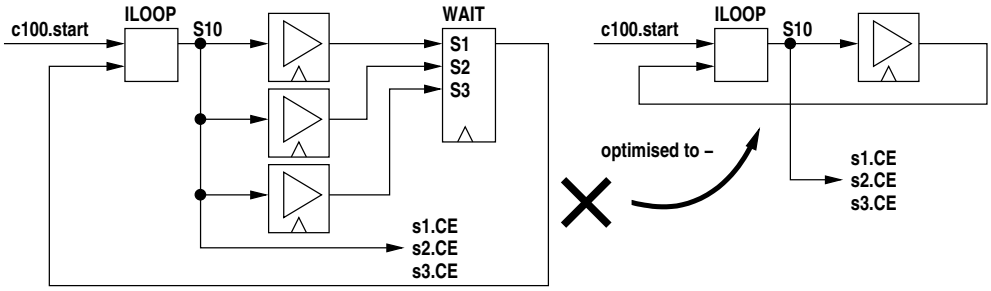
Execution signal **S23** executes the assignments to **s1** and **s2** and Execution signal **S10** executes the assignment to **s3**.

2.8.7 minimal parallel block

If the statements in a PAR block all take one clock cycle and have no queue reads the compiler will eliminate parallel **DEL** elements. For example the code -

```
    static({s1, s2, s3}, "uint:8");

    par {
        s1 <- 3;
        s2 <- 4;
        s3 <- 5;
    }
```

does not generate parallel delays, e.g. -



where **S10** executes the three assignments.

2.8.8 general when statement

A **WHEN** element directs an execution signal out through one of two outputs, the **TRUE** output or the **FALSE** output. These output signals execute the associated code block. The finish signals from the two code blocks must be ORed (they are mutually exclusive) to get the finish signal for the **WHEN**. This **OR** is contained in the **WHEN** element. A missing code block is replaced by a single cycle delay, hence the minimum time to execute a **WHEN** is one clock cycle. Note that if this minimum delay were not enforced we would have a combinatorial loop in this and other similar cases.

In the following example -

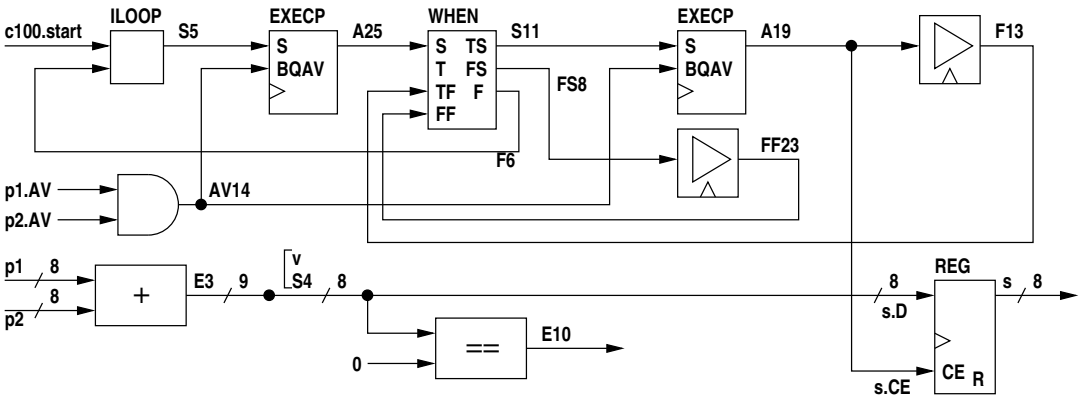
- the test contains queue reads
- the FALSE block is missing

```
1     static(s, "uint:8");
2     queue({p1, p2}, "uint:8");
3     value(v, "uint:8", p1 + p2);
4
5     when (v == 0)
6         s <- v;
```

the intermediate code is -

```
1     E3[0..8] = glob.p1[0..7] + glob.p2[0..7]    (uint, uint, uint)
2     S4[0..7] = cast(false){E3[0..8]}
3     E10[0] = S4[0..7] == 0      (uint, uint, log)
4     AV17[0] = glob.p1.NE[0]
5     AV16[0] = glob.p2.NE[0]
6     AV14 = AV16[0] AND AV17[0]
7     ACK18[0] = A19
8     ACK20[0] = A19
9     EXECP       A19 <- C=glob.c100, EXEC=S11, AV14
10    DEL F13 <- (1) glob.c100 A19
11    DEL FF23 <- (1) glob.c100 FS8
12    ACK24[0] = A25
13    ACK26[0] = A25
14    EXECP       A25 <- C=glob.c100, EXEC=S5, AV14
15    WHEN {
16            START               A25
17            TEST                E10[0]
18            T FINISH    F13
19            F FINISH    FF23
20            T START             S11
21            F START             FS8
22            FINISH              F6
23    }
24    ILOOP       S5 <- glob.c100.start F6
25    glob.s.CE[0..7] = A19
26    glob.s.D[0..7] = S4[0..7]
27    glob.s.RES[0..7] = GND
28    REG
29        OUT  glob.s[0..7]
30        <-
31        CLK  glob.c100
```

```
        DATA glob.s.D[0..7]
        CE   glob.s.CE[0..7]
        R    glob.s.RES[0..7]
        {0x00}
    glob.v[0..7] = S4[0..7]
```



Note that there are two **EXECP** elements, one to delay entry to the **WHEN** element until queues **p** and **p2** are available for reading and the second to delay execution of the assignment inside the true block of the **WHEN** until queues **p** and **p2** are available for reading. The inside **EXECP** is redundant since on entry to the true block the availability of the two queues has already been verified by the first **EXECP**. [1] This inefficiency can be eliminated by enclosing the **WHEN** by a SYNC element which will eliminate both **EXECP**s and replace them with a single **EXECP** before the **WHEN** (where the first one is now).

Note also that there is no false code block so a single cycle delay is used instead (**FS8** to **FF23**).

2.8.9 minimal when statement

The compiler recognises the special case where both the true and false code blocks execute in a single cycle, i.e. have a single **DEL** element. This also applies where a parallel block executes in one cycle and all the **DEL** elements are collapsed to a single **DEL** as described previously. Single **DEL** blocks for the true and false blocks can be replaced by a single **DEL** block from the start signal to the **WHEN**. As an example the following code -

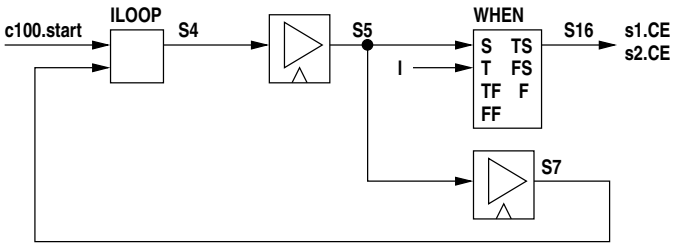
```
    static({s1, s2}, "uint:8");
    static(1, "log");

    seq {
        nop();
        when (1) par{
            s1 <- 3;
            s2 <- 4;
        }
}
```

results in the intermediate code -

```
    DEL S5 <- (1) glob.c100 S4
    DEL F7 <- (1) glob.c100 S5
    WHEN {
            START               S5
            TEST                glob.1[0]
            T_FINISH    null
            F_FINISH    null
            T_START             S16
            F START             FS9
            FINISH              null
```

[1] The compiler should omit queue availability testing inside the **WHEN** for queues already tested on entry to the **WHEN**, but when expressions containing queue reads are complicated this turns out to be surprisingly difficult.

```
    }
    ILOOP       S4 <- glob.c100.start F7
```



Notes -

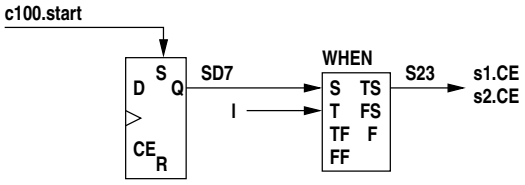
- the false start signal **FS9** goes nowhere
- the nop() generates the first **DEL** and has been included simply to stop the logic collapsing even further to a single D-flip-flop set by **c100.start** - see below
- **S16** is the execution signal for both assignments in the true code block of the **WHEN**

To expand on the comment about the nop(), if we simplify the code to -

```
    static({s1, s2}, "uint:8");
    static(l, "log");

    when (l) par{
        s1 <- 3;
        s2 <- 4;
    }
```

the intermediate code becomes -

```
    WHEN {
            START               SD7
            TEST                glob.l[0]
            T FINISH    null
            F FINISH    null
            T START             S23
            F START             FS6
            FINISH              null
    }
    DFF SD7 <- C=glob.c100, S=glob.c100.start
```



The **ILOOP** and any **DEL**s have disappeared. **SD7** goes high after **c100.start** and remains high thereafter. **s23** goes high whenever the **WHEN** test is true and causes the two assignments to occur.

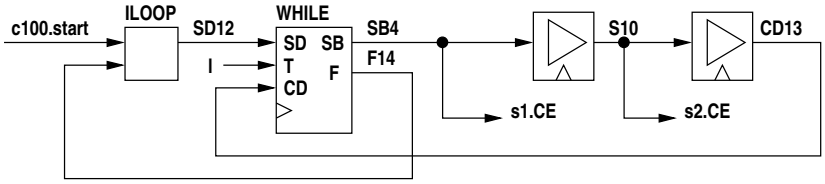
2.8.10 simple target while statement

The following example is a target while statement where the loop test does not contain queue reads -

```
    static({s1, s2}, "uint:8");
    static(l, "log");

    seq while (l) {
        s1 <- 5;
        s2 <- 6;
    }
```

the intermediate code is -

```
    DEL S10 <- (1) glob.c100 SB4
    DEL CD13 <- (1) glob.c100 S10
    WHILE {
            START_B         SB4
            FINISH          F14
            <-
            START       SD12
            TEST                glob.l[0]
            CONTIN      CD13
            C               glob.c100
    }
    ILOOP       SD12 <- glob.c100.start F14
```



The **WHILE** element has two entry points, **START_DEL** is the initial entry point and **CONTIN_DEL** is the entry point for further iterations. **START_B** is the start signal for the contained code block and **FINISH** is the finish signal from the **WHILE**.

The signal **SB4** executes the first assignment statement and **S10** executes the second. The loop takes two clock cycles per iteration with no extra overheads, but if on initial entry the test is false the **WHILE** has an internal delay of one clock cycle between the start input (**SD12** into F) and the finish signal (**F14** at F). This ensures that the **WHILE** takes a minimum of one clock cycle.

2.8.11 target while statement with queue waits

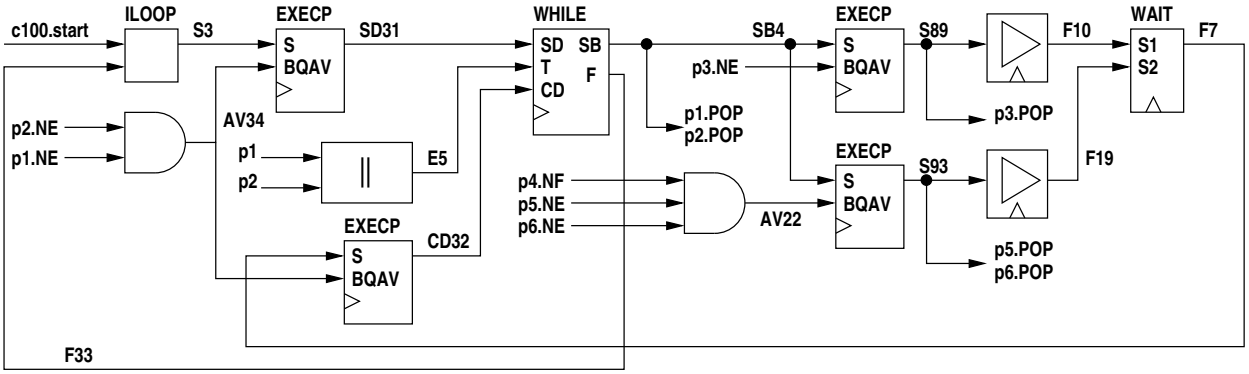
If the **WHILE** test expression contains queue reads an **EXECP** appears in both the START_DEL and CONTIN_DEL inputs. If the loop body contains queue reads then **EXECP** elements will appear there as well. The following example has both.

```
    static(s, "uint:8");
    queue({p1, p2, p3, p4, p5, p6}, "uint:8");
    static(l, "log");

    par while (p1 || p2) {
        s <- p3;
        p4 <<- p5 + p6;
    }
```

The intermediate code is -

```
    E5[0] = glob.p1[0] OR glob.p2[0]
    AV13[0] = glob.p3.NE[0]
    AV11 = AV13[0]
    EXECP       S89 <- C=glob.c100, EXEC=SB4, AV11
    ACK14[0] = S89
    glob.p3.POP = ACK14[0]
    DEL F10 <- (1) glob.c100 S89
    E20[0..8] = glob.p5[0..7] + glob.p6[0..7]   (uint, uint, uint)
    S21[0..7] = cast(false){E20[0..8]}
    AV24[0] = glob.p5.NE[0]
    AV25[0] = glob.p6.NE[0]
    AV22 = glob.p4.NF[0] AND AV24[0] AND AV25[0]
    EXECP       S93 <- C=glob.c100, EXEC=SB4, AV22
    ACK26[0] = S93
    ACK28[0] = S93
    glob.p5.POP = ACK26[0]
    glob.p6.POP = ACK28[0]
    DEL F19 <- (1) glob.c100 S93
    WAIT {
        in:
```

```
            Var glob.c100
            Var F10
            Var F19
        out:
            Var F7
    }
    WHILE {
            START_B             SB4
            FINISH              F33
            <-
            START       SD31
            TEST                E5[0]
            CONTIN      CD32
            C               glob.c100
    }
    ACK38[0] = SB4
    ACK40[0] = SB4
    glob.p2.POP = ACK38[0]
    glob.p1.POP = ACK40[0]
    AV37[0] = glob.p1.NE[0]
    AV36[0] = glob.p2.NE[0]
    AV34 = AV36[0] AND AV37[0]
    EXECP       SD31 <- C=glob.c100, EXEC=S3, AV34
    EXECP       CD32 <- C=glob.c100, EXEC=F7, AV34
    ILOOP       S3 <- glob.c100.start F33
```



The variables have not been shown. **S89** is the execution signal for the static assignment and **S93** is the execution signal for the queue assignment.

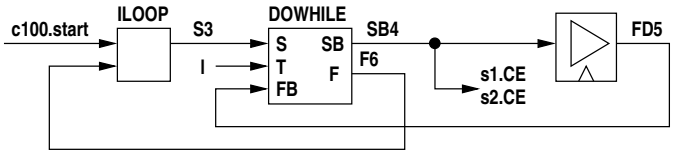
2.8.12 simple target do-while statement

The following code illustrates a target **DO_WHILE** loop with no queue dependencies -

```
    static({s1, s2}, "uint:8");
    static(l, "log");

    par do {
        s1 <- 4;
        s2 <- 5;
    } while (l);
```

The intermediate code is -

```
    DEL FD5 <- (1) glob.c100 SB4
    DO WHILE {
            START_B                 SB4
            FINISH                  F6
            <-
            START               S3
            TEST                glob.l[0]
            FINISH_B_DEL    FD5
    }
    ILOOP       S3 <- glob.c100.start F6
```



Note that since the two parallel assignment statements take a single cycle a single delay element is used to time them and single signal **SB4** executes both. No parallel **WAIT** is necessary.

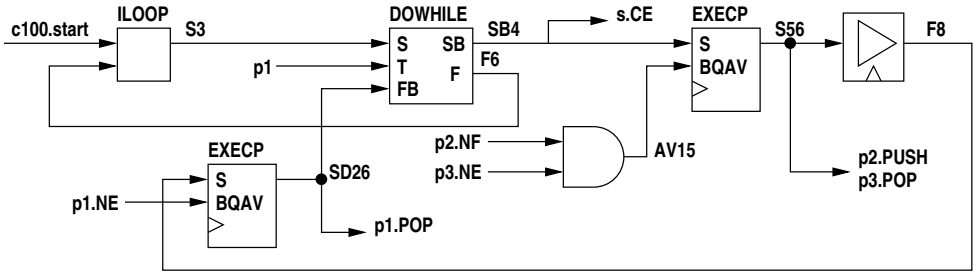
2.8.13 target do-while statement with queue waits

If we take the above example and introduce queue dependencies in both the test expression and one of the body statements -

```
1     static(s, "uint:8");
2     queue(p1, "log");
3     queue({p2, p3}, "uint:8");
4
5     par do {
6         s <- 4;
7         p2 <<- p3;
8     } while (p1);
```

the intermediate code is -

```
1         AV17[0] = glob.p3.NE[0]
2         AV15 = glob.p2.NF[0] AND AV17[0]
3         EXECP       S56 <- C=glob.c100, EXEC=SB4, AV15
4         ACK18[0] = S56
5         glob.p3.POP = ACK18[0]
6         glob.p2.PUSH = S56
7         DEL F8 <- (1) glob.c100 S56
8         AV23[0] = glob.p1.NE[0]
9         AV21 = AV23[0]
10        EXECP       SD26 <- C=glob.c100, EXEC=F8, AV21
11        DO WHILE {
12                START_B         SB4
13                FINISH          F6
14                <-
15                START                   S3
16                TEST                    glob.p1[0]
17                FINISH_B_DEL    SD26
18        }
19        ACK24[0] = SD26
20        glob.p1.POP = ACK24[0]
21        glob.s.CE[0..7] = SB4
22        ILOOP       S3 <- glob.c100.start F6
```



The parallel body of the loop has one assignment with queue dependencies and one without. The latter takes a fixed single cycle so it is executed using the block execute signal **SB4** whereas the queue assignment has an **EXECP** before being executed by **S56**. Since the queue assignment will take at least one cycle the delay for the static assignment and the parallel **WAIT** have been omitted by the compiler.

Note that the **FINISH_BLOCK** input to the **DO_WHILE** is via an **EXECP** since the block iteration test contains a queue read.

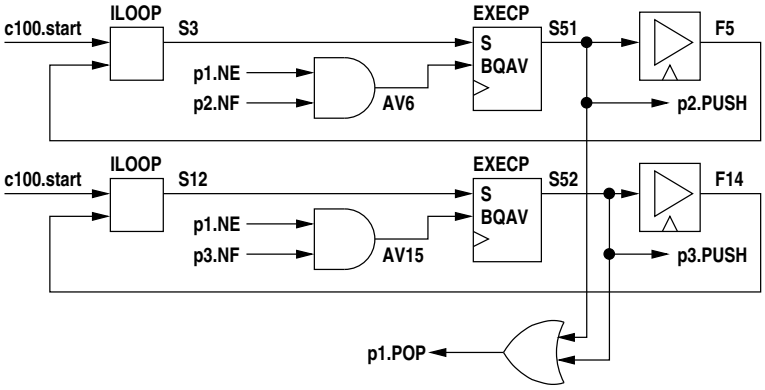
2.8.14 multiple queue reads within a module

Multiple queue reads read the same datum if simultaneous or separate data otherwise. The queue pops are ORed. The following code illustrates this -

```
    queue({p1, p2, p3}, "uint:8");

    p2 <<- p1;
    p3 <<- p1;
```

The intermediate code is -

```
    AV8[0] = glob.p1.NE[0]
    AV6 = glob.p2.NF[0] AND AV8[0]
    EXECP       S51 <- C=glob.c100, EXEC=S3, AV6
    ACK9[0] = S51
    DEL F5 <- (1) glob.c100 S51
    ILOOP       S3 <- glob.c100.start F5
    AV15 = glob.p3.NF[0] AND AV8[0]
    EXECP       S52 <- C=glob.c100, EXEC=S12, AV15
    ACK17[0] = S52
    DEL F14 <- (1) glob.c100 S52
    ILOOP       S12 <- glob.c100.start F14
    glob.p1.POP = ACK17[0] OR ACK9[0]
    glob.p2.D[0..7] = glob.p1[0..7]
    glob.p2.PUSH = S51
    glob.p3.D[0..7] = glob.p1[0..7]
    glob.p3.PUSH = S52
```



2.8.15 multiple queue reads in different modules

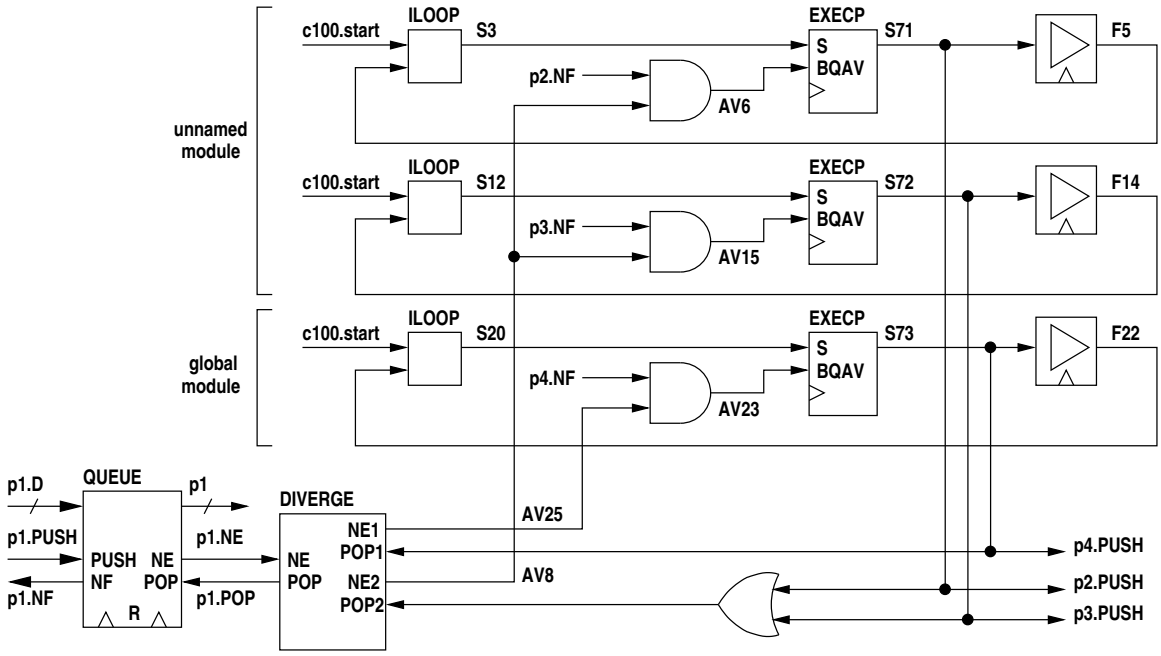
Where multiple queue reads are in different modules each module must execute a read before the queue is poped, this occurring simultaneously with the last read. Multiple reads within a module are again ORed.

```
    queue({p1, p2, p3, p4}, "uint:8");

    [
        p2 <<- p1;
        p3 <<- p1;
    ]
    p4 <<- p1;
```

Note that the first two assignments are in an inline module and the third assignment is in the file module. The intermediate code is -

```
    AV6 = glob.p2.NF[0] AND AV8[0]
    ACK9[0] = S71
    EXECP       S71 <- C=glob.c100, EXEC=S3, AV6
    DEL F5 <- (1) glob.c100 S71
```

```
    ILOOP       S3 <- glob.c100.start F5
    AV15 = glob.p3.NF[0] AND AV8[0]
    ACK17[0] = S72
    EXECP       S72 <- C=glob.c100, EXEC=S12, AV15
    DEL F14 <- (1) glob.c100 S72
    ILOOP       S12 <- glob.c100.start F14
    AV23 = glob.p4.NF[0] AND AV25[0]
    ACK26[0] = S73
    EXECP       S73 <- C=glob.c100, EXEC=S20, AV23
    DEL F22 <- (1) glob.c100 S73
    ILOOP       S20 <- glob.c100.start F22
    S69 = ACK26[0]
    S70 = ACK17[0] OR ACK9[0]
    DIVERGE {
        in:
            Var glob.c100
            Var glob.p1.RES
            Var glob.p1.NE[0]
            Var S69
            Var S70
        out:
            Var glob.p1.POP
            Var AV25[0]
            Var AV8[0]
    }
    glob.p1.RES = GND
    glob.p2.D[0..7] = glob.p1[0..7]
    glob.p2.PUSH = S71
    glob.p3.D[0..7] = glob.p1[0..7]
    glob.p3.PUSH = S72
    glob.p4.D[0..7] = glob.p1[0..7]
    glob.p4.PUSH = S73
```



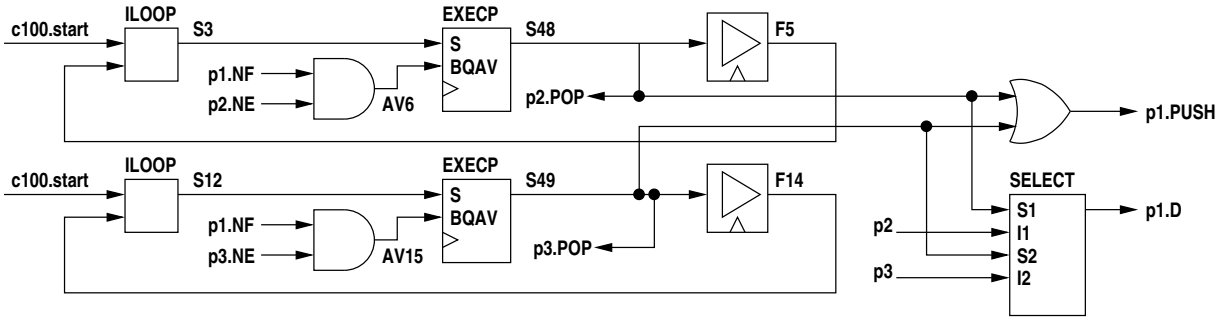
2.8.16 multiple queue writes

Multiple writes to queues are treated independently regardless of which module they are in. The writes are simply **OR**ed and the code must be constructed such that conflicts do not occur (there are no compile-time or run-time checks).

```
    queue({p1, p2, p3}, "uint:8");

    p1 <<- p2;
    p1 <<- p3;
```

The intermediate code is -

```
    AV8[0] = glob.p2.NE[0]
    AV6 = glob.p1.NF[0] AND AV8[0]
    EXECP       S48 <- C=glob.c100, EXEC=S3, AV6
    ACK9[0] = S48
    glob.p2.POP = ACK9[0]
    DEL F5 <- (1) glob.c100 S48
    ILOOP       S3 <- glob.c100.start F5
    AV17[0] = glob.p3.NE[0]
    AV15 = glob.p1.NF[0] AND AV17[0]
    EXECP       S49 <- C=glob.c100, EXEC=S12, AV15
    ACK18[0] = S49
    glob.p3.POP = ACK18[0]
    DEL F14 <- (1) glob.c100 S49
    ILOOP       S12 <- glob.c100.start F14
    S50 = S49 OR S48
    glob.p1.PUSH = S50
    SELECT {
        in:
            Var S48
            Var glob.p2[0..7]
            Var S49
            Var glob.p3[0..7]
        out:
            Var glob.p1.D[0..7]
    }
```



2.8.17 unbuffered queue reads and writes

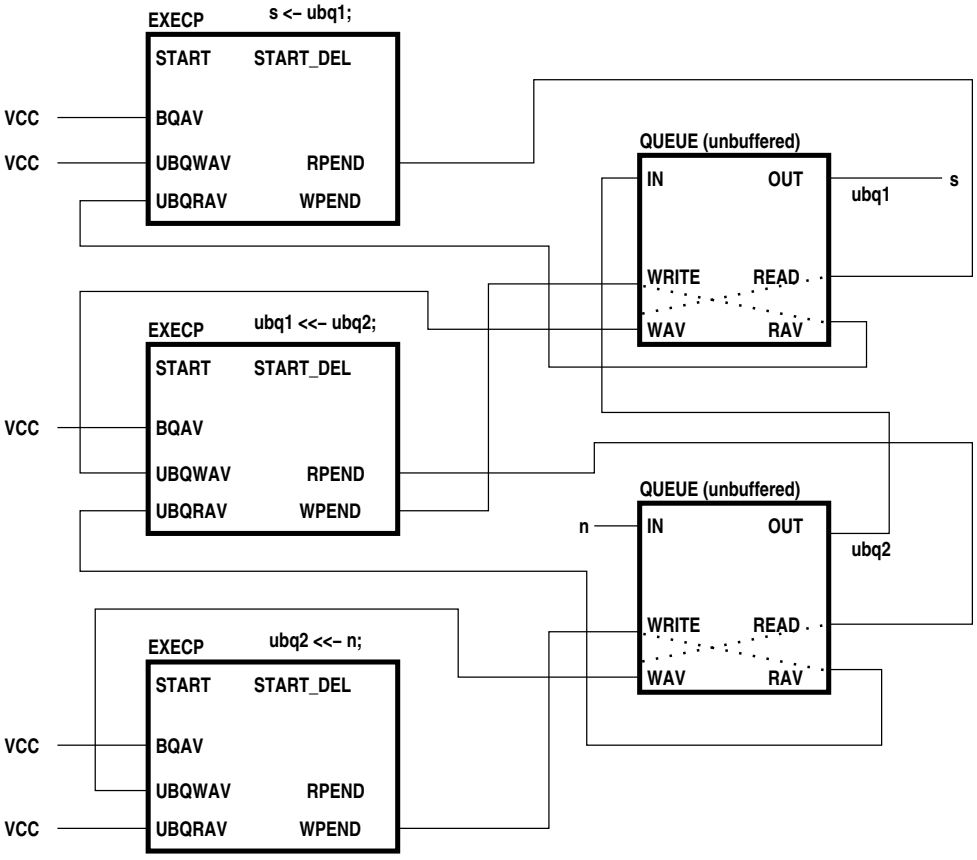
An unbuffered queue is used to synchronise two execution threads. In some texts this is referred to as a rendezvous. One thread reads the queue (uses its value) and the other thread writes to the queue (assigns to it). The first thread to reach the queue read/write will wait until the other thread also reaches the complementary write/read then they will both execute their respective statements and a datum will be transferred (if the type is not "null"). An unbuffered queue is only available for reading when a write is pending. Similarly it is only available for writing when a read is pending. Unbuffered queue read availability (**RAV** output) is high when the **WRITE** input is high. Similarly write availability(**WAV** output) is high when the **READ** input is high. Thus each thread accessing the queue must wait on availability and at the same time assert the **WRITE** or **READ** as required. A normal queue has **PUSH** or **POP** asserted by the **START_DEL** output of an EXECP when **BQAV** is high. An unbuffered queue **WRITE** or **READ** must assert the **WRITE** or **READ** ahead of execution when any/all buffered queues involved are available and the write or read is pending.

Thus for unbuffered queues reads and writes the **WPENDING** output of an EXECP is used to drive the **WRITE** and the **RPENDING** output the **READ** inputs to the queue buffer rather than using the **START_DEL** outputs (the label QUEUEBUFFER is a misnomer here as the queue is unbuffered, however for convenience it is regarded as a variant of the same element). The unbuffered queue contains no internal logic other than the cross conenctions from **WRITE** to **RAV**, **READ** to **WAV** and from data input to data output.

As an example consider the following three assignment statements executing in separate threads -

```
1     s <- ubq1;
2     .
3     .
4     ubq1 <<- ubq2;
5     .
6     .
7     ubq2 <<- n;
```

where **ubq1** and **ubq2** are unbuffered queues and **n** and **s** do not involve queues.



The three assignments do not contain buffered queue references so the **BQAV** inputs to the EXECPs are VCC¿

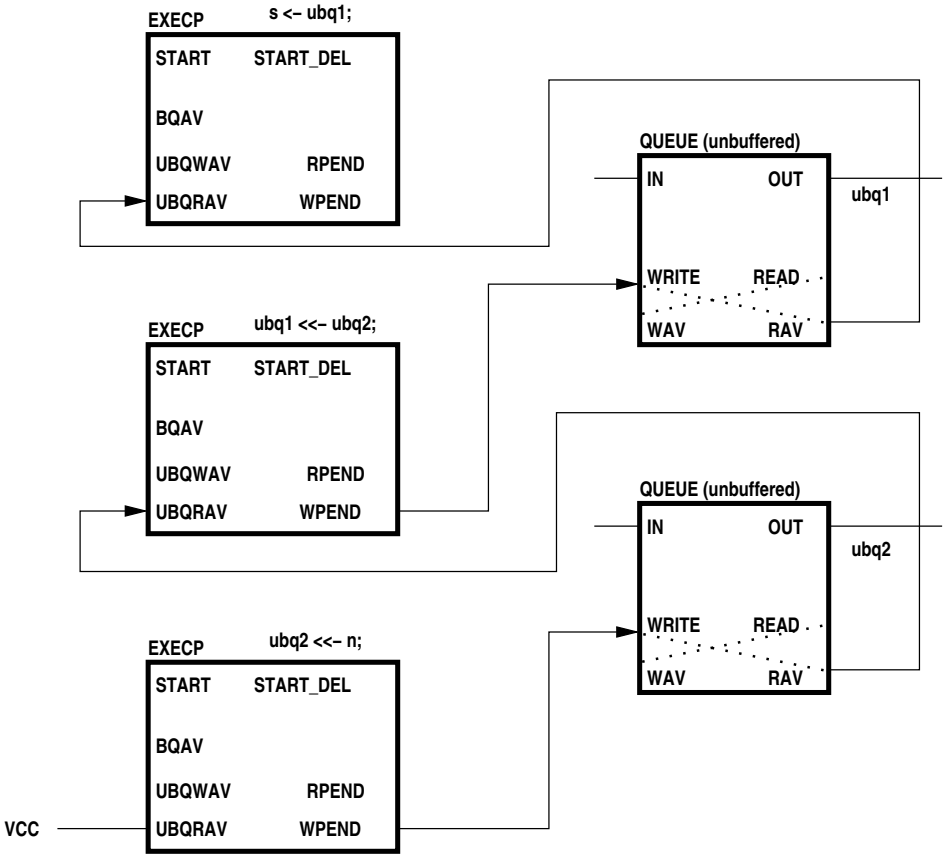
The first assignment does not assign to an unbuffered queue so the **UBQWAV** input is VCC. When execution reaches this statement the **RPEND** will be asserted since **UBQWAV** is high. This will flow into the **READ** input and out of the **WAV** output of **ubq1** to the **UBQWAV** input of the 2nd EXECP.

When execution reaches the second assignment the high **UBQWAV** will result in **RPEND** being asserted which will in turn flow through the **READ** and **WAV** ports of **ubq2** to the **UBQWAV** input of the last EXECP.

Similarly when execution reaches the third assignment the high **UBQRAV** will flow back through **ubq2**, the 2nd EXECP and **ubq1** to the **UBQRAV** input of the first EXECP.

Thus when all three assignment statements are ready to execute all three EXECPs will have **BQAV**, **UBQWAV** and **UBQRAV** high and can thus execute simultaneously (in one clock cycle), transfering the value of **n** through to **s** in the process.

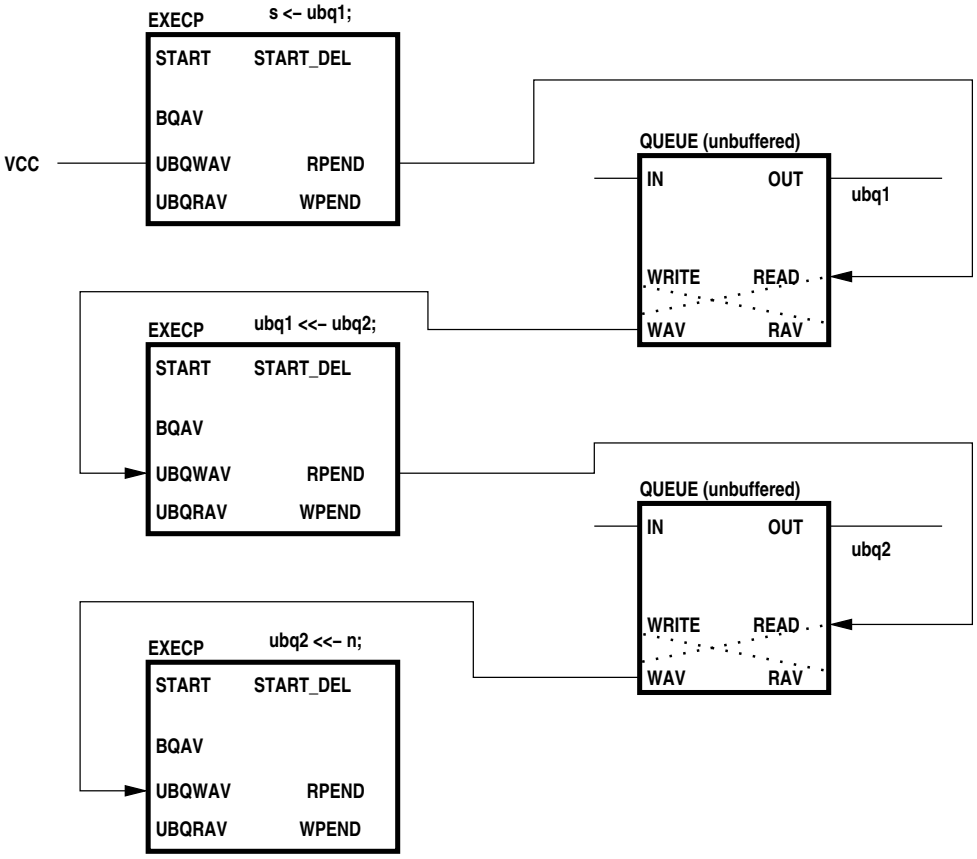
The following is a simplified diagram of the write chain from the above example -



The chain starts at the bottom left with VCC for UBQRAV since that assignment statement RHS does not contain an unbuffered queue.

Note that read availability anywhere along the chain will be true when all upstream assigment EXECPs have received a start signal in their respective execution streams and any associated buffered queues are availabile for reading or writing as appropriate.

The other half of the signalling, the read chain, is shown in the following diagram.



The chain starts at the top right with VCC for UBQWAV since that assignment statement LHS does not contain an unbuffered queue.

In a similar manner to the write chain write availability anywhere along the chain will be true when all downstream stream assigment EXECPs have received a start signal in their respective execution streams and any associated buffered queues are availabile for reading or writing as appropriate.

Since the pending outputs of EXECPs are only asserted if the **BQAV** input is high assignments containing both buffered queues and unbuffered queues will wait until any buffered queues are available to be read or written as appropriate.

Note that the availability of a buffered queue can be asserted and then de-asserted due to simultaneous queue operations by unrelated threads. This is not a problem as the availability of all buffered and unbuffered queues referenced by an assignment statement is assessed on a per-cycle basis.

Assignment statements containing unbuffered queues may contain mutiple queues of both types.

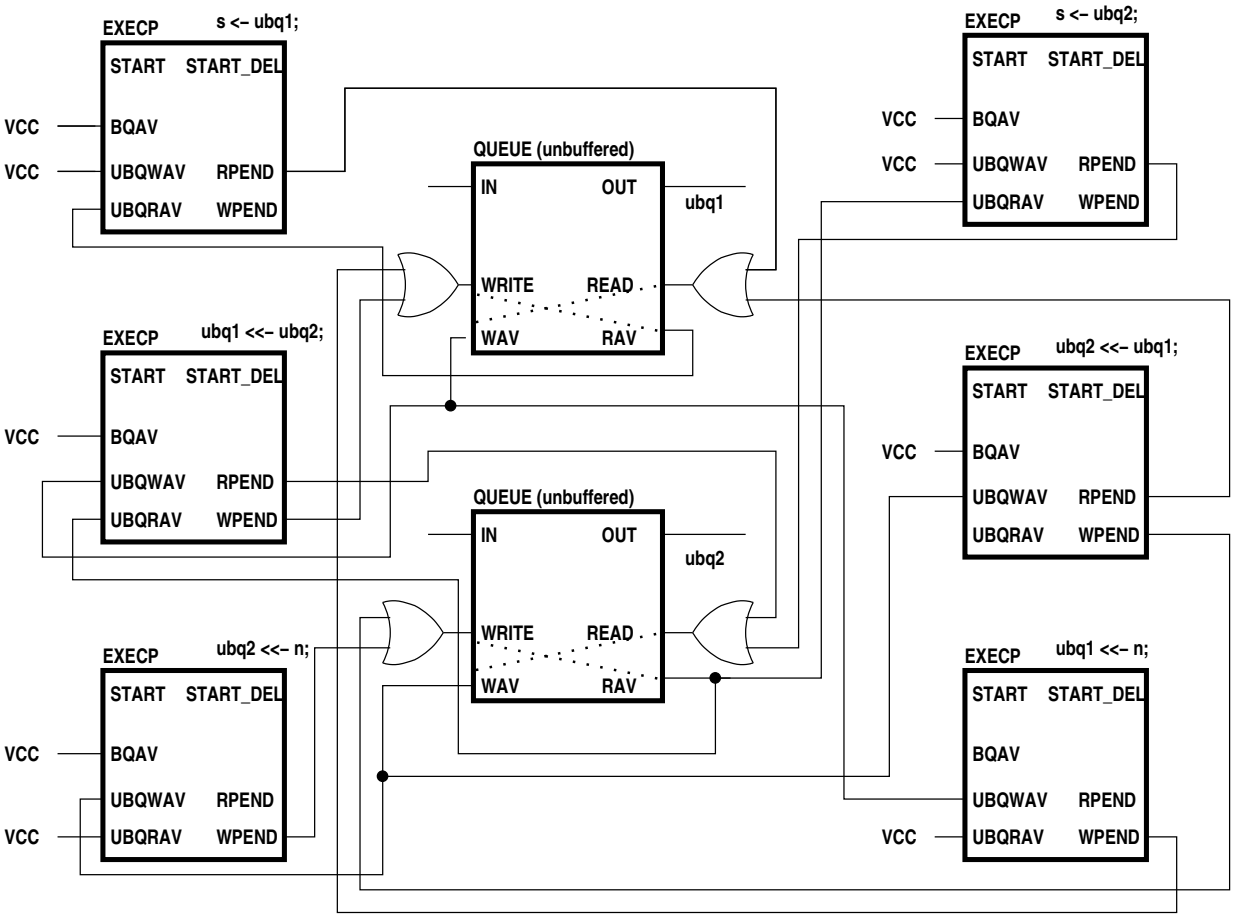
Buffered and unbuffered queues may be read or written in multiple places however simultaneous writes must be avoided, either by the inherent logic of the program or else explicitly by use of waitprilock statements. Simultaneous reads of unbuffered queues are valid, however unbuffered queue reads take no account of the modules in which they occur, all reads being independent.

Multiple **WRITE** and **READ** signals to an unbuffered queue are ORed to each input and responses are orchestrated by any EXECP elements involving those queues. The following example expands the previous example by including multiple unbuffered queue reads and writes -

```
1     s <- ubq1;
2     .
3     .
4     ubq1 <<- ubq2;
5     .
6     .
7     ubq2 <<- n;
8     .
9     .
10    s <- ubq2;
```

```
11      .
12      .
13      ubq2 <<- ubq1;
14      .
15      .
16      ubq1 <<- n;
```

Note that this example is intended to show logic generated for multiple unbuffered queue reads and writes and does not necessarily make any sense!



2.8.18 ”unless” constructs

The **unless** construct is implemented using ordinary intermediate code elements. The code follows one of two patterns -

- the exception expression contains no queue reads
- the exception expression does contain queue reads

If an exception code block is present then that is spliced into the execution logic. A flip-flop is added the block which is subject to the **unless** construct to register whether execution is taking place within the subject block. If the subject block does not have any executing statements within it then the exception is ignored.

If the exception expression contains no queue reads, a **WHEN** is used to detect the exception. The value of the exception expression is **AND**ed with the subject execution flip-flop to form the test input to the **WHEN**. The output of the **WHEN** resets all **DEL**, **WAIT**, **EXECP** and **WHILE** elements in the subject killing all execution. A second **WHEN** then loops through a **DEL** waiting for the exception experssion to go false. The false output of the second **WHEN** then drives the finish of an enclosing **ILOOP** driving the first **WHEN**.

If the exception expression contains queue reads, the first **WHEN** is preceded by an **EXECP** and the second **WHEN** is not needed as a queue read is a single event, not a duration.

In either case the finish signal is **OR**ed with the finish signal of the subject block so that it appears to complete.

An exception code block is spliced in after the **WHEN** and prior to the finish signal.

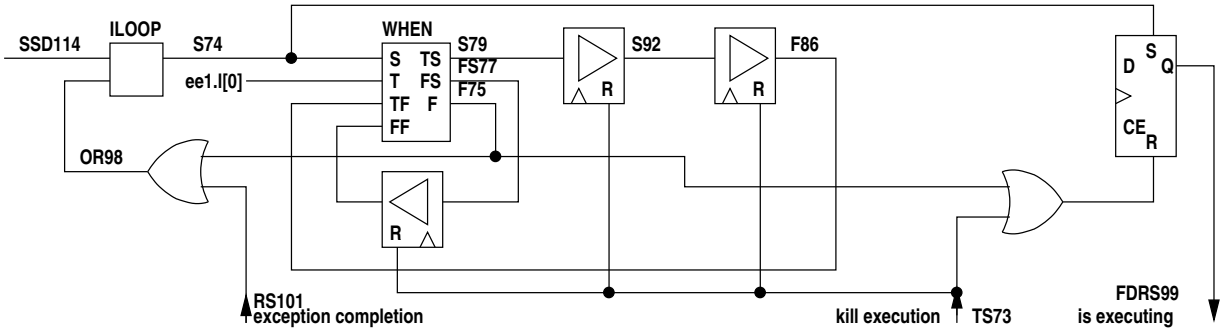
The following example, **ee1.3pl**, illustrates the case where the exception expression contains no queue reads -

```
module "csiro/hymod/io2.3pl"

useclock(c100);

static({e, l}, "log");
static({v1, v2, v3}, "uint:8");

when (l) {
    seq {
        v1 <- 3;
        par {
            v2 <- 5;
            v3 <- 7;
        }
    }
} unless (e) {
    seq {
        v1 <- 0;
        v2 <- 0;
    }
}
```

The intermediate code for the control logic for the **WHEN** is -

```
ILOOP  S74 <- SSD114 OR113
OR113 = F75 OR RS101
WHEN {
        T_START  S79
        F_START  FS77
        FINISH   F75
        <-
        START    S74
        TEST     ee1.l[0]
        T_FINISH F86
        F_FINISH FF95
}
DEL  S92 <- (1) glob.c166 S79 TS73
DEL  F86 <- (1) glob.c166 S92 TS73
DEL  FF95 <- (1) glob.c166 FS77 TS73
OR98 = F75 OR TS73
DFF {
        OUT      FDRS99
        <-
        D        -
        C        glob.c166
        CE       -
        R        OR98
        S        S74
}
.
.
ee1.v1.CE[0..7] = S79
.
.
ee1.v2.CE[0..7] = S92
.
.
ee1.v3.CE[0..7] = S92
```

In diagramatic form this is -



Note that there is no parallel wait - the optimiser has noted that the two assignments in parallel take exactly one clock cycle and has replaced two **DEL**s and a **WAIT** with one **DEL** (**S92** to **F86**). The first **DEL** in the **SEQ**, **S79** to **S92** is for the first assignment, hence v1.CE is driven by **S79**. The two parallel assignments, **v2.CE** and **v3.CE**, are driven by **S92**. **F86** completes the true block of the **WHEN** and feeds back to the true finish input to emerge as the finish output for the **WHEN**, **F75**.

If there were no **UNLESS** construct the flip-flop and three gates would be omitted, **F75** connecting back to the **ILOOP**.

The OR gate between **F75** and the **ILOOP** is to allow an execution pulse to be injected as an alternative finish signal for the **WHEN** after execution of the **WHEN** has been killed. **RS101** is the new execution completion pulse.

Execution of the **WHEN** is killed by resetting all control logic state within it, in this case just the three **DEL**s. The kill signal is **TS73**.

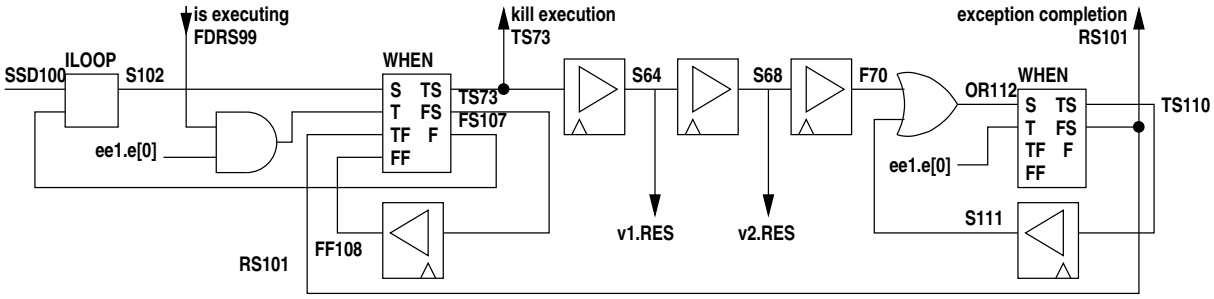
The exception logic needs to know if the **WHEN** is actually executing (it may be nested inside other constructs and not actually be executing at the moment - in this example it is always executing since it is at the top level, hence the **ILOOP**). The flip-flop is set when the **WHEN** begins execution and is reset when the **WHEN** finishes or execution is killed by **TS73**. If the false branch of the **WHEN** is taken there is a single **DEL** in the **ILOOP** and hence the **WHEN** executes on every cycle.

The intermediate code for the exception control logic is -

```
1     ILOOP  S102 <- SSD100 S103
2     AND106[0] = ee1.e[0] AND FDRS99
3     WHEN {
4         T_START  TS73
5         F_START  FS107
6         FINISH   S103
7         <-
8         START    S102
9         TEST     AND106[0]
10        T_FINISH RS101
11        F_FINISH FF108
12    }
13    DEL  S64 <- (1) glob.c166 TS73
14    DEL  S68 <- (1) glob.c166 S64
15    DEL  F70 <- (1) glob.c166 S68
16    DEL  FF108 <- (1) glob.c166 FS107
17    OR112 = F70 OR S111
18    WHEN {
19        T_START  TS110
20        F_START  RS101
21        FINISH   -
22        <-
23        START    OR112
24        TEST     ee1.e[0]
25        T_FINISH -
26        F_FINISH -
27    }
28    DEL  S111 <- (1) glob.c166 TS110
29    .
30    .
31    ee1.v1.RES[0..7] = S64
32    .
33    .
```

```
34      ee1.v2.RES[0..7] = S68
```

In diagramatic form this is -



This logic runs independently in its own **ILOOP**. The first **WHEN** waits for the exception expression (**ee1.l[0]**) to be asserted, but only if the associated subject construct is executing (**FDRS99**). When this is not true, the execution emerges on the false output, **FS107**, through a **DEL** to the false finish **FF108** and then emerges from the when finish output as **S103** and feeds back to the **ILOOP** for the next clock cycle. Thus the **WHEN** tests every clock cycle. When the exception occurs the true output of the **WHEN** is asserted. This is **TS73** which kills all execution in the associated subject code. Execution proceeds through three **DEL**s executing the **SEQ** in the exception code block. If there was no exception code block there would be a single **DEL** here. Progressive stages, **S64** and **S68**, reset variables **v1** and **v2** in the **SEQ** block. At the completion of the exception code block a second **WHEN** waits for the exception expression to go false, the true branch output looping back via a **DEL** and the OR gate. When ee1.e[0] goes false, the execution signal emerges on the true branch of the **WHEN**, **RS101**, which completes execution of the killed subject block. It also feeds back to the true finish input of the first when, emerging as the finish signal, **S103**, feeding back to the enclosing **ILOOP** to wait for the next exception.

If the exception expression contains queue reads a single **WHEN** combined with an **EXECP** is used, for example **ee2.3pl** -

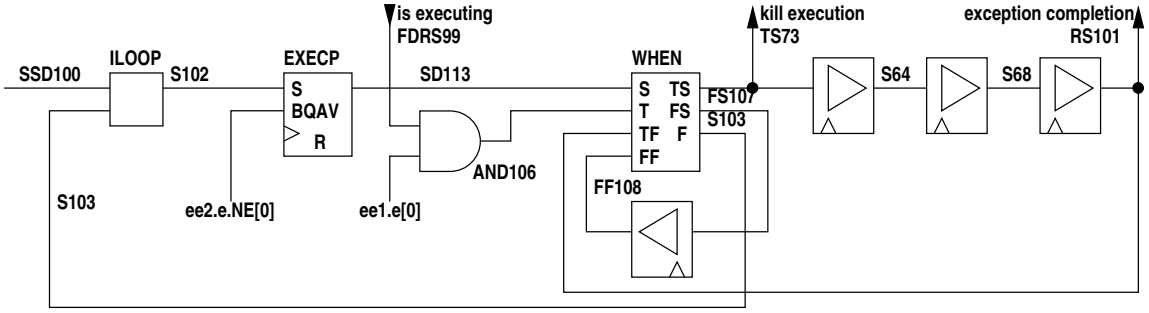
```
1   module "csiro/hymod/io1.3pl"
2
3   useclock(c166);
4
5   static(l, "log");
6   queue(e, "log");
7   static({v1, v2, v3}, "uint:8");
8
9   when (l) {
10      seq {
11          v1 <- 3;
12          par {
13              v2 <- 5;
14              v3 <- 7;
15          }
16      }
17  } unless (e) {
18      seq {
19          v1 <- 0;
20          v2 <- 0;
21      }
22  }
```

The intermediate code for the exception control logic containing a queue read is -

```
1   ILOOP  S102 <- SSD100 S103
2   EXECP {
3       START_DEL SD113
4       PRI_IN    -
5       PENDING   P112
6       <-
7       CLK       glob.c166
8       START_IN  S102
9       BQAV       ee2.e.NE
```

```
10              UBQAV     -
11          PRI_OUT   -
12          RESET     -
13      }
14      AND106[0] = ee2.e[0] AND FDRS99
15      WHEN {
16          T_START  TS73
17          F_START  FS107
18          FINISH   S103
19          <-
20          START    SD113
21          TEST     AND106[0]
22          T_FINISH RS101
23          F_FINISH FF108
24      }
25      DEL  S64 <- (1) glob.c166 TS73
26      DEL  S68 <- (1) glob.c166 S64
27      DEL  RS101 <- (1) glob.c166 S68
28      DEL  FF108 <- (1) glob.c166 FS107
29      .
30      .
31      ee2.v1.RES[0..7] = S64
32      .
33      .
34      ee2.v2.RES[0..7] = S68
```

In diagramatic form this is -



The exception logic now waits for queue **e** to be not empty. Execution is then passed to the **WHEN**. If the value is false, the **WHEN** loops back to the **ILOOP** via a **DEL** and **FF108**, emerging from the **WHEN** on the finish output **S103**.

If the exception value is true subject execution is killed and the execution signal goes through three **DEL**s as before. On emerging (**RS101**) it completes execution of the subject block and loops back to the true completion input of the **WHEN**, emerging as the **WHEN** finish signal **S103**, completing the **ILOOP**.

Chapter 3 - Writing inbuilt modules, procedures and functions

The following illustrates the class definition for an inbuilt function, in this case **sin()** -

```
1   package threepl.funcs;
2
3   import threepl.exceptions.ExEx;
4   import threepl.exec.Val;
5   import threepl.nodes.NodeList;
6   import threepl.parser.Constant;
7   import threepl.parser.SrcLoc;
8
9   /**
10   * An inbuilt function to get the sine.
11   */
12  public class SinFunc extends InbuiltFunc implements Constant {
13      .
14      .
15      .
16  }
```

For inbuilt modules and procedures the class definition is similar to that for functions -

```
1   package threepl.procs;
2
3   import threepl.exceptions.ExEx;
4   import threepl.exec.Ref;
5   import threepl.exec.Val;
6   import threepl.nodes.NodeList;
7   import threepl.parser.Constant;
8   import threepl.parser.SrcLoc;
9   import threepl.parser.Functions;
10
11
12  /**
13   * An inbuilt procedure to scan a string converting delimited fields to types specified
14   * by a format string.
15   */
16  public class SscanfProc extends InbuiltProc implements Constant {
17      .
18      .
19      .
20  }
```

For procedures there is an optional constructor, e.g. -

```
1   public SscanfProc () {
2       allowed_as_param = false;
3       check_null_input_args = true;
4       check_null_output_args = true;
5       target_inline = false;
6   }
```

These boolean variables are all declared and initialised in InbuiltProc.java and their functions are as follows -

- allowed_as_param - true if the procedure is allowed as a parameter-creation procedure, i.e. can appear as a parameter in a function module or procedure definition.
- check_null_input_args - if true null input arguments will throw an exception
- check_null_output_args - if true null output arguments will throw an exception
- target_inline - if true the procedure generates in-line target code

These booleans are set false by the super class but assigning them here improves readability. The null argument detection avoids the need in many cases for the module or procedure to explicitly check if an argument is null. Some procedures however accept null arguments and hence do not want this test enabled.

There are two additional member variables, applicable to both modules and procedures -

```
1    public TreeMap<String,Integer>  ipnames;
2    public TreeMap<String,Integer>  opnames;
```

which are null by default. These maps can contain identifiers to be associated with the module or procedure input and output parameters so that the module or procedure can be passed arguments either by position or by parameter=argument. If these maps are null then only the usual parameter/argument association by position is available. The map keys contain the parameter identifiers and the associated integers are the parameter position. The maps need not contain all parameters as it is often convenient to pass some initial mandatory arguments by position.

When a parameter name map has been provided the parameter=name arguments will be re-arranged in the argument list so that the module or procedure only sees arguments in the correct order. Note that for many modules and procedures null arguments are allowed in the call and in addition the argument re-arrangement for parameter=argument entries may also introduce null arguments.

Inbuilt modules, procedures and functions are listed in maps in InbuiltMods.java, InbuiltProcs.java and InbuiltFuncs.java e.g. -

```
1    public InbuiltProcs () {
2        procs = new TreeMap<String, InbuiltProc>();
3
4        procs.put("addmodeattribute", new AddModeAttributeProc());
5        .
6        .
7        .
8    }
```

Here the map key is the identifier by which the module, procedure or function is called and the associated entry is the actual module, procedure or function. The identifiers are lower case and the actual classes are camel case with "Mod", "Proc" or "Func" appended, but this is only a convention.

The calling procedure for inbuilt modules, procedures and functions is the same with the exception that functions do not have output parameters.

Functions have a getVal() method to return the function value. For example the **sin()** function -

```
1    public Val getVal (NodeList args) {
2        SrcLoc  loc = args.getCallLoc();
3
4        if (args.size() != 1)
5            throw new ExEx("sin() - must have one argument", loc);
6        .
7        .
8        .
```

class **NodeList** is a linked list of nodes in the compiled code tree. Variable **args** contains the list of arguments of the function call.

Class **SrcLoc** contains source code file name, line number and character column. **doc** is assigned the function call location.

The size of **args** is checked to ensure that the number of arguments provided is correct.

Modules and procedures have an execute() method. For example the procedure **sscanf()** -

```
    public void execute (
        NodeList    inargs,
        NodeList    outargs,
        boolean     toplevel
) {
    SrcLoc      loc = inargs.getCallLoc();
    if (inargs.size() != 2)
        throw new ExEx("sscanf() must have two input arguments", loc);
    if (outargs.size() == 0)
        throw new ExEx("sscanf() must have one or more output arguments", loc);
    .
    .
    .
```

The signature is similar for a module except that the **toplevel** parameter is not present.

Where a procedure is allowed to be called to create a parameter in a module, procedure or function definition the call to the procedure is more complicated, e.g. for inbuilt procedure **int()** -

```
    public IntProc () {
        allowed_as_param = true;
        check_null_input_args = true;
        check_null_output_args = false;
        target_inline = false;
        allowed_attributes = true;
    }

    /**
     * Execute the variable declaration procedure int().
     */
    public ArrayList<Var> execute (
        NodeList            inargs,
        NodeList            outargs,
        ArrayList<String>   names,
        RefOrVal            par_arg,
        int[]               dim_des,
        boolean             is_input,
        boolean             is_output
    ) {
        .
        .
        .
    }
```

The extra member variable **allowed_attributes** is true to indicate that the arguments to the procedure can contain attributes. Attributes are passed as attribute=value. Some must be processed by the module or procedure itself. In the case of procedures **clock()**, **input()** and **output()** attributes are processed by 3PL code whenever the procedure is called via a 3PL call-back procedure. If this variable is true then a mixture of parameter=argument and attribute=value arguments will be resolved. Arguments to the module or procedure will appear in the argument list as simple positional arguments as a result of the identifier resolution described above. Any that are not recognised are assumned to be attribute=value and these will be appended as additionak arguments following the positional arguments and will be class KeyMatchNode. These trailing attributes must be processed by the module or procedure.

The arguments to execute() are -

- inargs is a list of input argument tree nodes
- outargs is a list of output argument tree nodes
- names is a list of identifiers evaluated via method getIdents() prior to calling execute()
- par_arg is an argument passed to this variable as a parameter in a call
- dim_des is a dimensional description for a compound constant argument or is null - it must be null here!
- is_input is true if this variable is the input parameter for a module, procedure or function
- is_output is true if this variable is the output parameter for a module or procedure

The procedure must return an ArrayList¡Var¿ containing the variable(s).

The call arguments are supplied separately by **inargs** and **outargs**. The boolean argument **toplevel** is only present for procedures, not modules, and is true if the procedure call is not nested within a target

control structure wuch as a par, seq, sync, when etc. Its value is only used in the inbuilt procedure useclock() (UseCLockProc.java) to ensure that changes to the clock domain in use do not occur within target code structures.

As with function calls the location of the call is assigned to **loc**. The number of input and output arguments is checked.

A procedure may generate in-line code (a module cannot). This means that it must insert the generated code into the current execution thread. Such a procedure has member variable **target_inline** true. The call to such a procedure has a yet more complicated signature -

```
    public void execute (
        NodeList            inargs,
        NodeList            outargs,
        TDEVar              esig,
        ArrayList<TDEVar>   startl,
        ArrayList<TDEVar>   finishl,
        QueueRefs            queues,
        boolean             skipexecp,
        TDEVar              pri_in,
        TDEVar              pri_out
    ) {
        .
        .
        .
    }
```

- inargs is a list of input argument tree nodes
- outargs is a list of output argument tree nodes
- esig is an exception/restart signal, or null
- startl is a list of start signals for target statements below this node
- finishl is a list of finish signals for target statements below this node
- queues returns all the queue availability signals accumulated from code below
- skipexecp is true if a previous sync makes a queue availability wait unnecessary
- pri_in is an optional input signal to a priority encoder
- pri_out is an optional output signal from a priority encoder

A procedure may generate a single target executable statement. That statement may of course contain nested statements within control structures. Nested target statements are common in user-defined procedures but do not occur at present in inbuilt procedures.

3.1 functions with immediate mode arguments and return value

The function code fetches arguments by using the **getVal()** method, for example -

```
    Val val = args.getVal(0);
```

for the first argument (0). This is returned as class **Val** which is a universal class for both immediate and target expression values. The mode and type of the value can be tested by -

```
    if (val.getMode() != Mode.IMMEDIATE)
        throw new ExEx("sin() argument not immediate mode", loc);
    if (val.getPrimType() != Ptype.FLOAT)
        throw new ExEx("sin() argument not a float", loc);
```

For immediate mode arguments the value can be extracted. by one of the methods -

- getSingleEval(SrcLoc) - an enumerated type value as an EnumConst class
- getSingleFileVal(SrcLoc) - a file as a FileDesc class
- getSingleFval(SrcLoc) - a floating point value as a double
- getSingleIval(SrcLoc) - an integer as a long
- getSingleLISTval(SrcLoc) - a list as an ArrayList¡Val¿
- getSingleLval(SrcLoc) - a logical value as a boolean
- getSingleMAPval(SrcLoc) - a map as a TreeMap¡String,Val¿
- getSinglePval(SrcLoc) - a pointer value as a Ref class

- getSingleSval(SrcLoc) - a string
- getSingleTval(SrcLoc) - a type as class Type

For arrays and structs a member can be extracted by val.getVal(i) where i is the index into the array or struct. For an array **val** will contain a one-dimensional representation of the data and the offset must be determined from the subscripts and dimensions. For a struct the data types will usually be mixed. The data types for either will be those listed above for single values.

The function return value must be class Val, for example for sin() -

```
    double  f = val.getSingleFval(loc);
    return(new Val(Math.sin(f), loc));
```

Class **Val** has a number of constructors for immediate mode types -

- new Val(boolean, SrcLoc) - a boolean (type"log")
- new Val(long, SrcLoc) - an integer (type "uint" or "int")
- new Val(double, SrcLoc) - a floating point value (type "float")
- new Val(String, SrcLoc) - a string (type "str")
- new Val(Type, SrcLoc) - a type (stypw "type")
- new Val(Object[], String, SrcLoc) - an array
- new Val(Object[], Type[], WordSpec, Var, SubFieldList, SrcLoc) - struct

3.2 functions with target mode arguments or return value

An example where both the single input argument is target mode and the return value is also target mode is queueisempty() -

```
    package threepl.funcs;

    import threepl.codegen.TDEVar;
    import threepl.exceptions.ExEx;
    import threepl.exec.Queue;
    import threepl.exec.Val;
    import threepl.exec.Var;
    import threepl.exec.Var.IDtype;
    import threepl.nodes.NodeList;
    import threepl.parser.Constant;
    import threepl.parser.SrcLoc;

    /**
     * An inbuilt function to get a target value which is true if a
     * synchronous queue is empty.
     */
    public class QueueIsEmptyFunc extends InbuiltFunc implements Constant {

        /**
         * Calling interface for the function.
         * @param   args is a list of argument tree nodes
         * @return  the number of free spaces in the queue
         */
        public Val getVal (NodeList args) {
            SrcLoc  loc = args.getCallLoc();

            if (args.size() != 1)
                throw new ExEx("queueisempty() must have a single argument", loc);

            Val     val = args.getVal(0);
            Var     var = val.getVar();
            if (var == null)
                throw new ExEx("queueisempty(): argument is not a variable", loc);
            if (var.getMode() != Mode.QUEUE)
                throw new ExEx("queueisempty(): '" + var.getID(IDtype.CHAIN) + "' is not a queue variable", loc);

            TDEVar  t = ((Queue)var).getQueueEmpty(loc);

            Val oval = new Val(null, Mode.VALUE, t, loc);
            oval.addOVar(var);
            return(oval);
```

```
        }
    }
```

This follows the examples illustrated previously but the argument is now mode **queue**, not mode **immediate**. **val** is extracted from the first (only) input argument as before but now the argument is assumed to be a variable which is accessed by **Var var = val.getVar()**. If the argument is not a variable then this will be null and an exception is thrown. The mode is then checked and if not mode **queue** an exception is thrown.

The empty state of the queue is detected by method **getQueueEmpty(loc)**. Since this is a method specific to class Queue the variable **var** of class **Var** must be cast to class Queue. The result returned is a signal indicating if the queue is empty. The class for this is **TDEVar**. This class contains among other fields a signal identifier string and a wordspec of class **WordSpec** which contains both the type and a specification of bits within the signal. In this case the type is "log" and there is only one bit.

The return value, **oval**, needs to be Value mode contained within a **Val** class variable. This is created by a Val constructor -

```
    Val oval = new Val(null, Mode.VALUE, t, loc);
```

where the 1st argument is null because this Val does not represent an actual variable. The mode of the value is Mode.VALUE and the actual signal is the TDEVar t. loc is passed for possible error messages.

The statement oval.addOVar(var); adds the queue variable to a set of variables whose outputs contribute to oval. This is used to determine the clock domain of oval and check for consistency amongst multiple sources.

oval is then returned as the value of the function.

3.3 procedures with immediate mode arguments

There are few immediate procedures that have output arguments and those that do are too long to use as examples. Instead we could modify an existing inbuilt procedure to accept one output argument. As an example the append() procedure could assign the resulting list length to an output argument.

The example proceedure below appends an entry into a list. It is the same as the existing inbuilt procedure append() with the addition of a single output parameter which is assigned the resulting list length. it has one to three input arguments and one output argument. The first input argument is the list. The optional second input argument is the index which is type "int". If it is missing the entry will be appended to the end of the list. The third optional input argument is the value which can be an IMMEDIATE, VALUE or CLOCK value of any type. If it is "null" or missing then the value will be null. The output argument is the list length.

The class definition begins as -

```
package threepl.procs;

import java.util.ArrayList;

import threepl.exceptions.ExEx;
import threepl.exec.Type;
import threepl.exec.Val;
import threepl.nodes.NodeList;
import threepl.parser.Constant;
import threepl.parser.SrcLoc;

public class AppendProc extends InbuiltProc implements Constant {

    public AppendProc () {
        allowed_as_param = false;
        check_null_input_args = true;
        check_null_output_args = true;
        target_inline = false;
        ipnames.put("l", 0);
```

```
20          ipnames.put("index", 1);
21          ipnames.put("v", 2);
22      }
```

The imports are -

- java.util.ArrayList - lists in 3PL are implemented using ArrayList
- threepl.exceptions.ExEx - 3PL exceptions
- threepl.exec.Type - class for all types in 3PL
- threepl.exec.Val - class for all immediate or target expression values
- threepl.nodes.NodeList - class for nodes in the compiled code tree
- threepl.parser.Constant - various 3PL constants
- threepl.parser.SrcLoc - location in source file for error messages

The constructor initialises several class member variables -

- allowed_as_param = false - this procedure cannot be called to create a module, function or procedure parameter.
- check_null_input_args = true - input arguments are checked to ensure that there are no null arguments (but omitted arguments are OK). This avoids an explicit check for every argument in the procedure.
- check_null_output_args = true - likewise for output arguments.
- target_inline = false - this procedure does not generate any in-line target code so we dont need to handle a target execution thread.
- ipnames.put("l", 0) - the first parameter (0) can be passed as l=....
- ipnames.put("index", 1) - the second parameter (1) can be passed as index=....
- ipnames.put("v", 2) - the third parameter can be passed as v=...

Arguments can still be assigned to parameters by position in the call as well as by par=arg.

The method begins -

```
public void execute (
    NodeList    inargs,
    NodeList    outargs,
    boolean     toplevel
) {
    SrcLoc      loc = inargs.getCallLoc();
    if ((inargs.size() < 1) || (inargs.size() > 3))
        throw new ExEx("append() must have one to three input arguments", loc);
    if (outargs.size() > 1)
        throw new ExEx("append() must have 0 or 1 output arguments", loc);
```

inargs is a list of nodes in the compiled code tree that supply input arguments. Similarly **outargs** is a list of output arguments. **toplevel** is only true for procedures which produce target in-line code and which are called at the top level, i.e. are not nested within other target code structures such as **when**, **seq** etc. In this case it is not relevant.

The source file location is extracted from the input node list for use in error messages via thrown exceptions. The number of input and output arguments is checked.

Next the first input argument is evaluated, assigned to **lval** and checked that it is type "list" -

```
    Val     lval = inargs.getVal(0);
    if (lval == null)
        throw new ExEx("append() - 1st argument is missing", loc);
    if (lval.getPrimType() != Ptype.LIST)
        throw new ExEx("append() - 1st argument is not type \"list\"", loc);
```

The second argument is optional. If there are more input arguments the second input argument is evaluated, assigned to **ival** and checked for type. If OK the integer value is assigned to **index** (**index** is initialised to 0).

```
    Val     ival = null;
    int     index = 0;
    if (inargs.size() > 1) {
        ival = inargs.getVal(1);
        if ((ival.getPrimType() != Ptype.INT) && (ival.getPrimType() != Ptype.UINT))
            throw new ExEx("append() - 2nd argument is not integer type", loc);
        index = (int)ival.getSingleIval(loc);
    }
```

The third argument is also optional. If it is missing or null, null will be appended to the list. The argument can be any type but the allowed modes are restricted -

```
    Val     rval = null;
    if (inargs.size() > 2) {
        rval = inargs.getVal(2);
        if (rval != null) {
            switch (rval.getMode()) {
            case IMMEDIATE:
                if (rval.getValType(0) == Type.NULL)
                    rval = null;
                break;
            case CLOCK:
            case CMEMORY:
            case RMEMORY:
                break;
            default:
                throw new ExEx("append() - 3rd argument is not immediate, clock, cmemory or rmemory mode", loc);
            }
        }
    }
```

The list is now assigned to **al** and the index is checked for validity. If no index was supplied then the default value of 0 will pass this check. If **ival** is null then no index has been supplied so ArrayList.add() is called with just one argument and the item is appended to the list. If **ival** is not null then ArrayList(index, rval) is called.

```
    ArrayList<Object>   al = (ArrayList<Object>)lval.getVal(0);
    if ((index < 0) || (index > al.size()))
        throw new ExEx("append() - index is out of range (" + index + " > " + al.size() + ")", loc);
    if (rval != null)
        rval.setDummyVar(false, loc);
    if (ival == null)
        al.add(rval);
    else
        al.add(index, rval);
```

The line rval.setDummyVar(false, loc) inserts a dummy variable into **rval**. This ensures that a complex type such as an array or a struct that is present in a list can have a **Ref** generated if data internal to it is to be changed, for example -

```
var(ll, "list");
var(ii, "[10]uint");
append(ll,, ii);
ll[0][3] = 5;
```

Here the value of **ii**, not the variable itself, is appended to empty list **ll**. The last line accesses array member [3] of list entry [0] (the only entry) and changes it to 5. To assign to a member of a value in a list an instance of reference class **Ref** must be available and the added dummy variable provides this. This is a bit of a fiddle explicitly for lists and maps and so is rarely required or encountered.

Finally the list length must be assigned to the argument matching the output parameter, if present -

```
    if (outargs.size() > 0) {
        Ref ref = outargs.getRef(0, "append() - ");
        if (ref != null) {
            if (ref.getMode() != Mode.IMMEDIATE)
                throw new ExEx("append() - output argument not immediate", loc);
            if ((ref.getPrimType() != Ptype.INT) && (ref.getPrimType() != Ptype.UINT))
                throw new ExEx("append() - output argument not type \"int\" or \"uint\"", loc);
            ref.assignTo(AST.IMASS, new Val(al.size(), loc), loc);
        }
    }
```

If the size of the output arguments list is greater than 0 a reference to the argument variable is created by outargs.getRef(0, "append() - "), the first argument is 0 (the 1st argument) and the string 2nd argument provides a header string for any error messages should the getRef() fail. The check for **ref** should not be necessary here as we earlier (in the constructor) specified that null output arguments would be

trapped on calling. This check is needed if there are multiple arguments and an intermediate argument is omitted, in which case the **Ref** would return null. The mode and type of the output argument is checked and then finally the assignment to it is done. AST is the assignment type enum and IMASS indicates immediate assignment (=). There are other assignment types such as IMPLUSASS (+=), IMMINASS (-=), immulass (*=) etc. The value to be assigned is al.size(), the list size, and it must be wrapped in a **Val** class via call to the constructor Val(). Both the Val constructor and the assignTo() method have SrcLoc arguments (loc) in case exceptions are to be thrown.

3.4 procedures with target mode arguments - no inline target code

A procedure with target mode arguments that does not generate in-line target code is called with the same signature as procedures with immediate mode arguments, as in the example in the previous section. An example is clockfromexpr() which takes a value of type "log" and produces a value of mode **clock**.

The class isode starts in a similar manner to the previous exmple -

```
1  package threepl.procs;
2
3  import static threepl.ThreePL.tdelist;
4
5  import threepl.codegen.TDEConstants;
6  import threepl.codegen.TDEVar;
7  import threepl.exceptions.ExEx;
8  import threepl.exec.Clock;
9  import threepl.exec.Ref;
10 import threepl.exec.Val;
11 import threepl.nodes.NodeList;
12 import threepl.parser.Constant;
13 import threepl.parser.SrcLoc;
14
15
16 public class ClockFromExprProc extends InbuiltProc implements Constant, TDEConstants {
17
18     public ClockFromExprProc () {
19         allowed_as_param = false;
20         check_null_input_args = true;
21         check_null_output_args = true;
22         target_inline = false;
23     }
24
25     /**
26      * Execute the procedure clockfromexpr().
27      * @param   inargs is a list of input argument tree nodes
28      * @param   outargs is a list of output argument tree nodes
29      * @param   toplevel is true if we are at the module, procedure or function
30      *          level
31      */
32     public void execute (
33         NodeList    inargs,
34         NodeList    outargs,
35         boolean     toplevel
36     ) {
37         SrcLoc  loc = inargs.getCallLoc();
38         if (inargs.size() != 1)
39             throw new ExEx("clockfromexpr() must have 1 input argument", loc);
40         if (outargs.size() != 1)
41             throw new ExEx("clockfromexpr() must have 1 output argument", loc);
```

Again **target_inline** is assigned false.

The input argument is evaluated as before but it will now be target mode. The following checks are carried out -

- the number of words must be 1, otherwise it is a compound type (array, struct etc.).
- the mode must be INPUT, SELECTVALUE, VALUE, STATIC or PRIORITY mode.
- the argument must not contain queue reads.
- the type must be "log".

The signal **tdevi**, class TDEVar, is then extracted from the input argument.

```
    Val     vali = inargs.getVal(0);
    if (vali.numWords() != 1)
        throw new ExEx("clockfromexpr() input argument must be a single value", loc);
    switch (vali.getMode()) {
    case INPUT:
    case SELECTVALUE:
    case VALUE:
    case STATIC:
    case PRIORITY:
        break;
    default:
        throw new ExEx("clockfromexpr() input argument must be mode VALUE, SELECTVALUE, STATIC or PRIORITY", loc);
    }
    if ((vali.getQueues() != null) && !vali.getQueues().isEmpty())
        throw new ExEx("clockfromexpr() input argument cannot contain queues", loc);
    if (vali.getPrimType() != Ptype.LOG)
        throw new ExEx("clockfromexpr() input argument must be type log", loc);
    TDEVar  tdevi = vali.getTDEVar();
```

The output argument is now accessed a **Ref** class since it will have to be assigned to. It is checked to ensure it has mode **CLOCK**.

```
    Ref     refo = outargs.getRef(0, "clockfromexpr()");
    if (refo.getMode() != Mode.CLOCK)
        throw new ExEx("clockfromexpr() output argument must be mode CLOCK", loc);
    TDEVar  tdevo = refo.getTDEVar();
```

The output (sink) can now be connected to the input (source)

```
    tdelist.connect(tdevo, tdevi);
```

Finally the clock variable is fetched from the output argument **Ref** (oref) and some fields of the variable are set -

```
    Clock   cvar = (Clock)refo.getVar();
    cvar.setAssigned();
    cvar.setClkSourced();
```

setAssigned() labels the output argument clock variable as having been assigned. This method is defined in the super class **Var** and applies to all variables. For clock mode variables this is probably redundant. setClkSourced() sets a flag to indicate that the clock has a source. It also ensures that there are no other assignments to this variable.

3.5 procedures with target mode arguments - with inline target code

Procedures with in-line target code have a different signature for method execute(). As an example the inbuilt procedure assert() has the following initial code -

```
package threepl.procs;

import static threepl.ThreePL.getCurrentClock;
import static threepl.ThreePL.getCurrentClockVar;
import static threepl.ThreePL.tdelist;

import java.util.ArrayList;

import threepl.codegen.TDEConstants;
import threepl.codegen.TDEVar;
import threepl.exceptions.ExEx;
import threepl.exec.QueueRefs;
import threepl.exec.Ref;
import threepl.exec.Val;
import threepl.exec.Var;
```

```
16 import threepl.exec.WordSpec;
17 import threepl.nodes.NodeList;
18 import threepl.parser.Constant;
19 import threepl.parser.SrcLoc;
20
21 public class AssertProc extends InbuiltProc implements Constant, TDEConstants {
22
23     public AssertProc () {
24         allowed_as_param = false;
25         check_null_input_args = true;
26         check_null_output_args = true;
27         target_inline = true;
28     }
29
30     public void execute (
31         NodeList            inargs,
32         NodeList            outargs,
33         TDEVar              esig,
34         ArrayList<TDEVar>   startl,
35         ArrayList<TDEVar>   finishl,
36         QueueRefs           queues,
37         boolean             skipexecp,
38         TDEVar              pri_in,
39         TDEVar              pri_out
40     ) {
41         SrcLoc  loc = inargs.getCallLoc();
```

Note that **target_inline** has been assigned true.

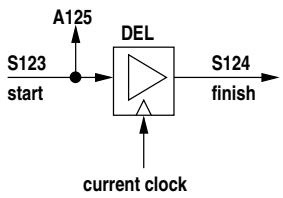
The arguments to execute() are -

- NodeList inargs - input argument node list.
- NodeList outargs - output argument node list.
- TDEVar esig - a signal that is asserted to break out of an enclosing control structure such as a loop. It is generated by an **unless** node to abort the block executing within the associated control structure.
- ArrayList¡TDEVar¿ startl - list of thread start TDEVars.
- ArrayList¡TDEVar¿ finishl - list of thread finish TDEVars.
- QueueRefs queues - a class containing queue references from procedures nested within this one. This is passed back to the calling context to generate queue availability waits controlling statement execution.
- boolean skipexecp - if true indicates that this procedure is guaranteed that any queued referenced will be available having already been tested.
- TDEVar pri_in - an optional input to a priority encoder for statements waiting on a priority variable.
- TDEVar pri_out - an optional output from a priority encoder.

In any particular procedure many of these arguments may be unused.

A procedure with target in-line code has a start signal and a finish signal. When the start signal is asserted for a clock cycle the procedure executes. When it completes it must assert the finish signal for one clock cycle.

The assert() procedure executes in one clock cycle. It has an optional input argument of type "log" which if true indicates that multiple calls to assert() can drive one output argument, the result being the OR of the asserts. We can omit this feature to simplify the procedure for the sake of the example code. In that case the logic might appear as in the following diagram -

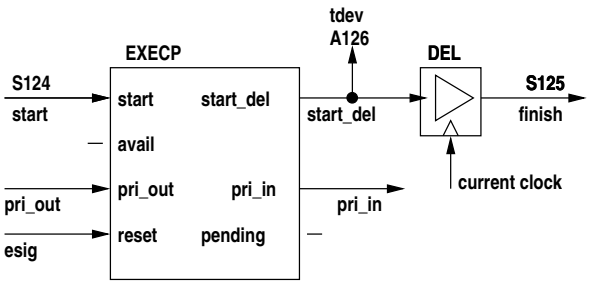


This simplified logic applies if there is no enclosing **priwait()** and hence **pri_in** and **pri_out** are null. In that case the method could be written as -

```
    .
    .  check input and output argument counts
    .
    Ref     oref = outargs.getRef(0, "assert() -");
    if (oref.getMode() != Mode.VALUE)
        throw new ExEx("assert() output argument must be value mode", loc);
    if (oref.getPrimType() != Ptype.LOG)
        throw new ExEx("assert() output argument must be type log", loc);

    TDEVar      start = tdelist.signal("S", loc);
    TDEVar      finish = tdelist.signal("S", loc);
    WordSpec    ws = new WordSpec(1, Ptype.LOG);
    TDEVar      tdev = tdelist.signal("A", ws, loc);
    Val         rval = new Val(null, Mode.STATIC, tdev, loc);

    rval.addOVar(getCurrentClockVar());
    oref.checkMatch(rval, false, "assert() ", loc);
    oref.assignTo(rval, loc);

    tdelist.connect(tdev, start);

    tdelist.del(finish, start, getCurrentClock(), esig);
    startl.add(start);
    finishl.add(finish);
```

oref is a reference to the output value mode variable.
start is a signal we create for the execution thread input.
finish is a signal we create which passes on the execution thread.
ws is a WordSpec for the assertion output which is type "log".
tdev is the assertion output TDEVar with a unique identifier "A.....".
rval is a **Val** class wrapper for **tdev**.

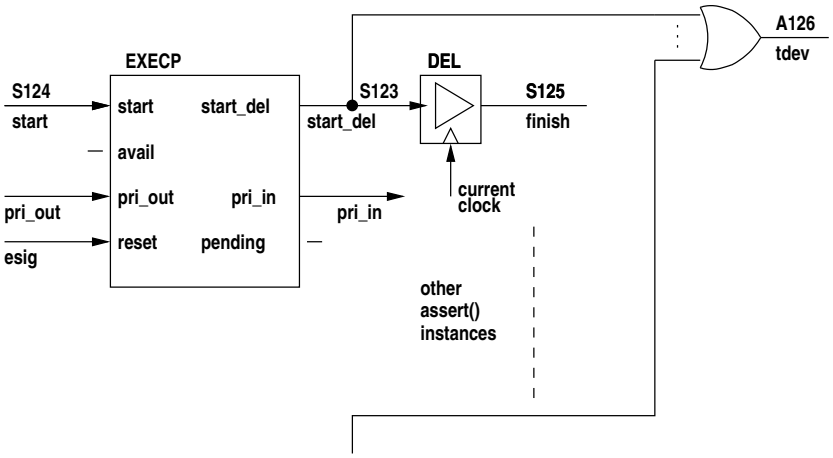
The assert() call could be enclosed within a priwait() in which case instead of simply connecting **tdev** to **start** the following would be required -



To handle the priwait() an EXECP TDE is required. This can be generated by calling TDEList.execp().

```
    if (pri_in == null)
        tdelist.connect(start_del, start);  // no waitpri
    else {
        TDEVar  v = tdelist.execp(null, start, pri_in, pri_out, esig, null, loc);
        tdelist.connect(start_del, v);
    }
```

If in addition we wish to implement the 2nd argument to assert() then an OR gate on the output is required -




```
    .
    .
    boolean or = false;
    if (inargs.size() == 1) {
        Val     or_val = inargs.getVal(0);
        if (or_val.getMode() != Mode.IMMEDIATE)
            throw new ExEx("assert() input argument must be immediate mode", loc);
        if (or_val.getPrimType() != Ptype.LOG)
            throw new ExEx("assert() input argument must be type log", loc);
        or = or_val.getSingleLval(loc);
    }
    .
    .
    if (or) {
        Var     ovar = oref.getVar();
        int     i = oref.getWordSpec().getWord(0);
        if (ovar.getVal(i) == null) {
            // No initial value so just assign pulse signal.
            oref.assignTo(rval, loc);
        } else {
            // Assign OR of initial value and new pulse signal.
            Val     oval = outargs.getVal(0);
            TDEVar  otdev = oval.getTDEVar();
            oref.assignTo(new Val(ovar, Mode.VALUE, tdelist.or(otdev, tdev, loc), loc), loc);
        }
    } else
        oref.assignTo(rval, loc);
    .
    .
```

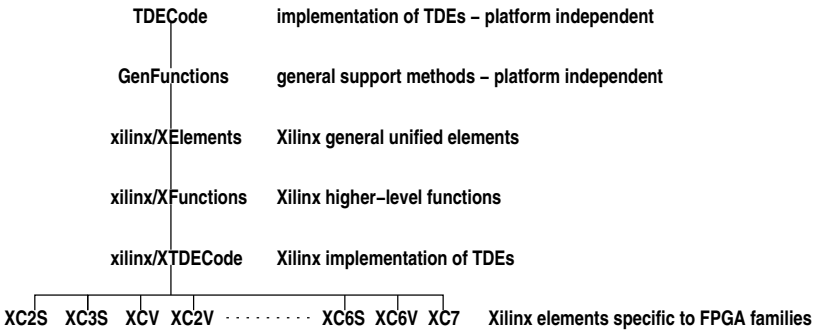
Note that the previous value of the output argument value mode variable is ORed with the assert signal `tdelist.or(otdev, tdev, loc)` Where several assert() statements use the same output argument the assert signals will be chained together by ORs. These will later be optimised into a single wide gate (TDEList.java optimiseTDEList() line 1199).

Chapter 4 - Netlist Code Generation

Netlist generation is done from the intermediate TDE list created during immediate execution.

Thus far only Xilinx FPGAs have been catered for. The class structure is intended to allow convenient porting to other vendor FPGAs.

The class structure is -



where each extends the class above. The FPGA object is one of the classes in the bottom row and it is assigned to public static variable -

```
1    public static TDECode family;
```

in source file **ThreePL.java**.

An attempt has been made to separate generic and platform-specific classes. The netlist code generation software is in directory **netlist** and the only platform code so far is in directory **netlist/xilinx**. The generic classes are -

TDECode - this is an abstract class which has three purposes -

1. to provide a platform-independent template for netlist generation from the TDE list. Abstract methods in TDECode, one for each TDE, are overridden by methods in Xilinx/XTDECode.
2. to provide common methods for accessing dimensional parameters associated with each specific FPGA type. Examples are LUT input width, block RAM sizes and availability of 3-state drivers.
3. to optimise the netlist in a general sense - more netlist optimisation is done in xilinx/XTDECode where it is specific to the FPGA family.

Element - represents an FPGA hardware element.

Net - represents a net.

GenFunctions - contains methods to extract subarrays of nets, construct lists of arguments, change the width of net arrays, generate strings, generate net arrays representing constants and connect nets and net arrays.

NetConstants - an interface supplying some enums used in netlist optimisation.

Ports - a description of the input, output and 3-state output pins of a particular element type (not of an instance of the element). This is used when generating each cell description in the netlist file.

Display, **NetListDraw**, **Properties**, **SigList** - these classes implement the schematic netlist display invoked by the -d option on the 3PL command line.

4.1 The Element class

This class represents an element, for example a flip-flop, a gate or a block of RAM.

String cellname - the element name to be used in the netlist, e.g. "FDRS" or "AND2".

String ident - a unique identifier string of the form "b123" (see **icount** below).

private static HashMap<String, Ports> celldecls - a map of cell types. The map key is the cell name and the value is class **Ports** describing the input and output ports. Ports contains 3 sets, inputs, outputs and 3-state outputs. Each of these sets contains Strings giving the port names.

HashSet<String> inportset - this is a set of the input pins for the element. It will be a common set for the element type stored in an instance of the **Ports** class kept in a static map **celldecls** (see below).

HashSet<String> outportset - like inportset above this is a set of output pins for the element.

HashSet<String> tsportset - like inportset above this is a set of 3-state output pins for the element.

The above sets are the same sets as those contained in the Ports class in the 'celldecls' map indexed by the cell name of this instance. When a port connection is added to an element the port name is checked against the appropriate set and if not present is added. Since the port sets are shared among elements with the same cell name the port sets for an element type are built up by usage and are not pre-defined. This can be a problem where nested gates are combined into a single gate during optimisation since that element prototype might not have been encountered during netlist building. For that reason AND, OR, XOR, XNOR gates and LUTs are initialised in map **celldecls** for number of inputs from 1 to the LUT width (XTDECode.initialise())

HashMap<String, Net> inports - a map of input ports where the map key is the port name and the value is an instance of class Net.

HashMap<String, Net> outports - a map of 2-state output ports where the map key is the port name and the value is an instance of class Net.

HashMap<String, Net> tsports - a map of 3-state output ports where the map key is the port name and the value is an instance of class Net.

Maps 'inports', 'outports' and 'tsports' represent the input and output connections to the element. The keys to these maps are the port name strings and the associated values are the connected nets (class 'Net').

Gtype element_type - a gate type enum used to identify elements that can be optimised. The gate type enums are defined in class **NetConstants**.

private static ArrayList<Element> instances = new ArrayList<Element>(20000) - a list of all elements in the netlist. Optimisable elements are added to the front of the list and a count of these is kept in static variable **ocount**. Elements whose type is NONE are added at the end of the list, all others being inserted at the front. Thus the elements that are labeled as being subject to optimisation are located at the head of list 'instances'. The number of such entries in the list is given by 'ocount'. This scheme avoids traversing the whole list of elements when performing an optimisation pass.

String expression - if a logical expression was used to generate a property, for example an expression for a LUT, then that expression is stored here. It is kept in this field for display purposes but is translated into a hexadecimal string and stored in field 'property_init' for later inclusion in the netlist file as an INIT property to initialise the element (LUT contents or flip-flop initial state).

String property_init - properties of the element. This might be initialisation for a LUT or a flip-flop.

public static Element gnd
public static Element vcc - publicly available GND and VCC elements. These are assigned in the constructor XElements() where also publicly available Nets **LO** and **HI** are created and connected to their output ports.

int count = 0 - this is a counter used for the schematic netlist display to identify elements shown more than once in the schematic in which case they are coloured to highlight the fact.

private static int ocount = 0 - count of optimisable elements - see above.

private static int icount = 0 - unique integer for generating cell identifiers (see **ident** above). It is incremented after each use.

NOTE: when 3PL is run repeatedly via the GUI, class TDECode and all subclasses down to the class specific to the FPGA family are constructed afresh. Thus all static variables are re-created. Classes Element and Net are NOT reconstructed and hence static variables in these classes must be explicitly re-initialised on each run! In the case of this class this is done by method init().

The following subsections describe constructors and methods. Many simple methods which only provide read or write access to class variables are omitted.

4.1.1 constructors

```
1    public Element (String name) { makeElement(name, null, Gtype.NONE); }
```

Construct a general element. These are given the gate type Gtype.NONE and are not subject to nelist optimisation. Parameter **name** is the element name, e.g. "IBUF". The resulting element is given an arbitrary unique netlist block name of the form "b123".

```
1    public Element (String name, String bname) { makeElement(name, bname, Gtype.NONE); }
```

Construct a general element. These are given the gate type Gtype.NONE and are not subject to nelist optimisation. The block name must be unique. Parameter **name** is the element name, e.g. "IBUF". Parameter **bname** is the netlist block name.

```
1    public Element (String name, Gtype element_type) { makeElement(name, null, element_type); }
```

Construct a gate element. The gate type is used to direct netlist optimisation. Parameter **name** is the element name, e.g. "IBUF". Parameter **element_type** is the element gate type, e.g. Gtype.MUXCY The resulting element is given an arbitrary unique netlist block name of the form "b123".

Each of these constructors calls the common private method **makeElement()**. Each element is added to the static list **instances**, optimisable elements at the front of the list to avoid traversing the whole list during optimisation.

4.1.2 method init()

```
1    public static void init () {
2        icount = 0; // unique integer for identifiers
3        ocount = 0; // count of optimisable elements
4        celldecls = new HashMap<String, Ports>();
5        instances = new ArrayList<Element>(20000);
6    }
```

This method initialises static variables. This must be done by a method, rather than during class initialisation, to allow code generation to be restarted when 3PL is repeatedly run interactively from the GUI.

4.1.3 method putCelldecl()

There are two signatures for this.

```
1    public static void putCelldecl (String name, int nports) {
2        celldecls.put(name, gatePorts(nports));
3    }
```

This method puts a logic gate cell declaration in the cell map. Normally the cell map is filled during calls to the constructors for class **Element**, but gate elements are entered explicitly to ensure that gate of all possible widths are present. This is because optimisation might reduce say an AND3 gate to an AND2 gate but the latter may not have been encountered in calls to the constructor.

```
1    public static void putCelldecl (String name, ArrayList inports, ArrayList outports) {
2        .
3        .
4        .
5    }
```

This method puts a general cell declaration in the cell map. This is used for elements specific to particular FPGAs, e.g. MULT18X18 or DSP48E and ensures that all inputs and outputs are included in the definition even if they are not included during a call to a constructor.

4.1.4 method gatePorts()

```
1    public static Ports gatePorts (int n) {
2        Ports   p = new Ports();
3        p.outports.add("O");
4        for (int i=0 ; i<n ; i++)
5            p.inports.add("I" + i);
6        return(p);
7    }
```

This method creates a port class for a gate element. The parameter indicates the number of input ports and there is one output port.

4.1.5 adding inputs

To add a single Net to a named port -

```
1    public void addInput (String port, Net n) {
2        switch (element_type) {
3        case AND:
4        case OR:
5        case XOR:
6            break;
7        default:
8            inportset.add(port);
9        }
10       if (n == null)
11           return;
12       inports.put(port, n);
13       n.connect(this, port, PortType.INPORT);
14   }
```

AND, OR and XOR gates have varying numbers of inputs and are subject to changes to the number of inputs during optimisation, so the input port set should not be touched. It can be assumed that the input port sets for all these gates that are within the accepted number of inputs for the FPGA family will have been already created. For all others the port string is added to the set of inports for this element type. The last statement creates a reciprocal link from the Net to this element.

To add an array of Nets -

```
    public void addInputArray (String port, Net[] n) {
        if (n == null)
            return;
        for (int i=0 ; i<n.length ; i++)
            addInput(port + i, n[i]);
    }
```

The port string has "0", "1" etc. appended.

To add an array of Nets with most significant bit padding -

```
    public void addInputArray (String port, Net[] n, int size, Net pad) {
        int portnum = 0;
        int asize = (n == null) ? 0 : n.length;
        for (int i=0 ; i<asize ; i++)
            addInput(port + portnum++, n[i]);
        for (int i=asize ; i<size ; i++)
            addInput(port + portnum++, pad);
    }
```

The port string has "0", "1" etc. appended. If the array is smaller than the designated size the remaining inputs are connected to Net **pad**. If the array is null all inputs are connected to Net **pad**.

To add an array of Nets with least significant bit padding -

```
    public void addInputArrayTop (String port, Net[] n, int size, int max, Net pad) {
        int npad = max - size;
        int asize = (n == null) ? 0 : n.length;
        int portnum = 0;
        for (int i=0 ; i<npad ; i++)
            addInput(port + portnum++, pad);
        for (int i=0 ; i<size ; i++)
            addInput(port + portnum++, (i < asize) ? n[i] : Net.LO);
    }
```

If the Net array is smaller than **size** then the most significant bits are padded as well.

To add an array of Nets across two different ports -

```
    public void addInputArray (
        String  port1,
        String  port2,
        Net[]   n,
        int     size1
    ) {
        int portnum1 = 0;
        int portnum2 = 0;
        int asize = (n == null) ? 0 : n.length;
        for (int i=0 ; i<asize ; i++)
            if (i < size1)
                addInput(port1 + portnum1++, n[i]);
            else
                addInput(port2 + portnum2++, n[i]);
    }
```

Parameter **port1** is the first input port identifying string head. Parameter **port2** is the second input port identifying string head. Parameter **n** is the net array. Parameter **size1** is the number of ports using the first head string.

To add an array of Nets across two different ports with most significant bit padding -

```
    public void addInputArray (
        String  port1,
        String  port2,
        Net[]   n,
        int     size1,
        int     size,
        Net     pad
    ) {
        int portnum1 = 0;
```

```
        int portnum2 = 0;
        int asize = (n == null) ? 0 : n.length;
        for (int i=0 ; i<asize ; i++)
            if (i < size1)
                addInput(port1 + portnum1++, n[i]);
            else
                addInput(port2 + portnum2++, n[i]);
        for (int i=asize ; i<size ; i++)
            if (i < size1)
                addInput(port1 + portnum1++, pad);
            else
                addInput(port2 + portnum2++, pad);
    }
```

4.1.6 adding outputs

For output Net and Net arrays these follow the same pattern as for inputs above -

```
    public void addOutput (String port, Net n) {
        outportset.add(port);
        if (n == null)
            return;
        if (n.getNumOutputs() != 0)
            throw new ExEx("net 2nd source");
        outports.put(port, n);
        n.connect(this, port, PortType.OUTPORT);
    }
```

There is a builtin check here to ensure that the Net is not connected to any other element outputs. The last statement creates a reciprocal link from the Net to this element.

And similarly -

```
    public void addOutputArray (String port, Net[] n)
    public void addOutputArray (String port1, String port2, int size1, Net[] n)
```

For 3-state outputs -

```
    public void addTS (String port, Net n) {
        tsportset.add(port);
        if (n == null)
            return;
        tsports.put(port, n);
        n.connect(this, port, PortType.TSPORT);
    }
```

4.1.7 method removeNet

This method removes a Net connected to an Element. The argument is the port to which the Net is connected.

```
    public void removeNet (String port) {
        if (inports.containsKey(port)) {
            inports.get(port).disconnect(this, port);
            inports.remove(port);
        } else if (outports.containsKey(port)) {
            outports.get(port).disconnect(this, port);
            outports.remove(port);
        } else if (tsports.containsKey(port)) {
            tsports.get(port).disconnect(this, port);
            tsports.remove(port);
        } else
            throw new ExEx("net not connected to element");
    }
```

The code checks the input, output and 3-state port maps in turn looking for the port name string as key. Inputs are searched first as they are the more numerous. The get() method returns the Net and the disconnect() method removes the Element from the Net. The remove() then removes the key and associated value ()Net from the particular Element port map.

4.1.8 method remapInputs

This method re-names the input ports of a gate Element. It is required when one or more inputs have been removed as a result of optimisation.

```
1     public void remapInputs () {
2         int         n = inports.size();
3         HashMap     ports_new = new HashMap();
4         Net         inet;
5         int         i = 0;
6         for (Map.Entry me : inports.entrySet()) {
7             String      newport = "I" + i++;
8             String      port = (String)me.getKey();
9             inet = (Net)me.getValue();
10            inet.changePort(this, port, newport);
11            ports_new.put(newport, inet);
12        }
13
14        inports = ports_new;
15        cellname = element_type.toString() + n;
16
17        Ports p;
18        if (celldecls.containsKey(cellname)) {
19            p = celldecls.get(cellname);
20            inportset = p.inports;
21            outportset = p.outports;
22            tsportset = p.tsports;
23        } else {
24            // Remapped wide gates may not have declared cell
25            // set entries - dont worry as wide gates will
26            // be converted so will never need these sets anyway.
27            inportset = null;
28            outportset = null;
29            tsportset = null;
30        }
31    }
```

The input port map is traversed and the Nets are placed in a new map where the keys (port names) are allocated sequentially as "I0", "I1" etc. That new map is then assigned as a replacement input port map. The Element cell name is changed to reflect the ne winput count, e.g. "AND5" may now be "AND3". The new cell name is now checked in the cell declaration map and if present the port sets for the Element are assigned from that map. If not present it means that the gate is a wide gate which has not been entered in the cell declaration map since it will be converted to use available library elements later.

4.1.9 method changeElement()

This method changes an element. This is used during optimisation to change an element into another. Previous connections are optionally stripped. Parameter **name** is the new element name, e.g. "INV". Parameter **element_type** is the element gate type, e.g. Gtype.INV. Parameter **strip** if true results in the element being stripped of input and output Nets. An example might be an XOR2 gate with one input connected to VCC being changed to an INV of the other input.

```
1     public void changeElement (String name, Gtype element_type, boolean strip) {
2         Ports   p;
3         if (strip)
4             strip();
5         cellname = name;
6         this.element_type = element_type;
7         if (celldecls.containsKey(name))
```

```
            p = celldecls.get(name);
        else {
            p = new Ports();
            celldecls.put(name, p);
        }
        if (strip) {
            inports = new HashMap<String, Net>();
            outports = new HashMap<String, Net>();
            tsports = new HashMap<String, Net>();
        }
        inportset = p.inports;
        outportset = p.outports;
        tsportset = p.tsports;
    }
```

The Element is first optionally stripped of all inputs and outputs, depending on boolean argument **strip**. The cell name string and element type are then overwritten from the supplied arguments. If the new element is contained in the cell declaration map then the ports sets are obtained from that map, otherwise new sets are created for the named cell. If the element has been stripped of connections then new port maps are created, otherwise the existing port maps remain. The port sets are then assigned from the sets obtained from the cell declarations map.

4.1.10 method strip()

This method disconnects the Element from all input and output nets and then clears all input and output port maps. This is used prior to deleting an Element or changing all its port connections during optimisation.

```
    public void strip () {
        for (Map.Entry me : inports.entrySet()) {
            String      pinname  = (String)me.getKey();
            Net         net = (Net)me.getValue();
            net.disconnect(this, pinname);
        }
        inports.clear();
        for (Map.Entry me : outports.entrySet()) {
            String      pinname  = (String)me.getKey();
            Net         net = (Net)me.getValue();
            net.disconnect(this, pinname);
        }
        outports.clear();
        for (Map.Entry me : tsports.entrySet()) {
            String      pinname  = (String)me.getKey();
            Net         net = (Net)me.getValue();
            net.disconnect(this, pinname);
        }
        tsports.clear();
    }
```

4.1.11 method addExpression()

This method adds an expression to a LUT. The expression string is kept for display purposes and is also translated into a property value for LUT initialisation. Parameter **s** is the expression string.

```
    public void addExpression (String s) {
        expression = s;
        property_init = Functions.bit_pattern(s, TDECode.lutwidth);
    }
```

4.1.12 method addProperty()

This method adds a property for LUT initialisation. Parameter **s** is the property string.

```
    public void addProperty (String s) {
        if (s == null)
            return;
        if (property_init != null)
            throw new ExEx("element INIT property multiply defined");
        property_init = s;
    }
```

4.1.13 methods for writing to the netlist file

Method

```
    static public void outputCells (PrintWriter pw, String view)
```

outputs the cell port definitions to the EDIF netlist. Only cells which have been instantiated are written out. Some cells may have been optimised out such that no instances remain in which case they are not written to the EDIF file cell definition section. Parameter **pw** is the output interface to the EDIF netlist file and Parameter **view** is the EDIF view parameter (the string "view_1" seems to be required and this originates in codegen.TDEList.ncode()).

Method

```
    static public void outputInstances (PrintWriter pw, String view)
```

outputs all Elements to the EDIF netlist.

Method

```
    static public void outputConstraints (String group, PrintWriter pw)
```

outputs all element constraints for a group. Parameter **pw** is the output interface to the constraint file for the group (name.ncf for group "n" and name.ucf for group "u"). Note that this method is called from netlist.xilinx.XTDECode() which also writes net constraints and general constraints to constraint files.

4.2 The Net class - a net label

The **Net** class is small, its main function being to hold a net identifier and associated NET class instance. The net connections to elements are instances of class **Net** (a net identifier wrapper), not class **NET** (the actual net). When two **Net**s are connected together it is not necessary to visit all elements to which the **Net**s are connected, instead one of the associated **NET** class instances is merged into the other and both **Net** instances now contain the same **NET** instance.

A **Net** constructor which is not passed an identifier will create one consisting of "n" with an appended unique integer.

The member variables for **Net** are -

private String id - the net identifier.

private NET net - the actual net.

There are six other member variables, input_elements, input_ports, output_elements, output_ports, input_index and output_index, which are just temporary variables used when traversing connected elements.

There are some class variables -

private static int icount - an integer used to form unique net identifiers of the form "n123". It is incremented after each use.

private static int hashcode_count - an integer used to form a crude unique hash code for use by methods **equals()** and **hashCode()** on instances of class **NET**. These methods are used by HashMap and HashSet classes to distinguish or equate NET class instances.

private static HashMap<String, Net> identmap = new HashMap<String, Net>(20000) - a map of nets. The key is a net identifier and the associated value is class Net.

private static HashSet<NET> instances = new HashSet<NET>(20000) - the set of NETs. This set is traversed when writing nets to the netlist file.

public static Net LO
public static Net HI - globally available GND and VCC nets. **LO** and **HI** are assigned in the constructor XElements() because they are then connected there to newly constructed Elements GND and VCC.

For an operation on a net the net identifier is used to extract the **Net** class instance from static map **instances**. The **Net** in turn contains variable **net**, class **NET**, which is the actual net. Many instances of net identifier class **Net** will contain the same actual **NET**, meaning that they are connected.

The main methods are described in the following subsections.

4.2.1 constructors and class creation methods

Constructor **Net()** creates an un-named net with a unique identifier of the form "n123".

A net with a constructed identifier can also be produced by constructor

```
1    public Net (String header)
```

which creates a net with a unique identifier by concatenating a unique integer to the header string supplied. This is used for signals which are not derived from source code variables but which are easier to trace in the netlist if given meaningfull identifier headers.

A named Net can be constructed using constructor

```
1    public Net (String ident, boolean large)
```

which creates a net using the supplied identifier. The second parameter **large**, if true, causes lists inside class NET to be created with a large initial size (10000) instead of the default size. This is used for Nets **GND** and **VCC** as they have a very large number of connections.

A named net can also be constructed using method

```
1    static public Net get (String ident)
```

which returns a named net if it already exists and otherwise constructs it.

An array of unnamed Nets can be created by method

```
1    static public Net[] netArray (int size)
```

where parameter **size** provides the dimension of the array.

4.2.2 method init()

This method re-initialises the class static variables prior to any run of 3PL.

4.2.3 method equals()

```
1    public boolean equals (Object o)
```

This method returns true if the argument is the same Net or is connected to the object Net (they share the same NET class instance). If the argument is null or not class Net then false is returned.

4.2.4 connection to an Element

```
1    public void connect (Element e, String port, PortType type)
```

This method connects a Net to an Element port but only modifies the Net structure, not that of the Element to which it connects. This is only called from methods **addInput()**, **addOutput()** or **addTS()** in class Element which handle the related changes in that class.

4.2.5 disconnection from an Element

```
1    public void disconnect (Element e, String port)
```

This method disconnects a Net to an Element port but only modifies the Net structure, not that of the Element to which it is connected. The method is only called in -

- Element.strip()
- Element.removeNet()
- TDECode.optimiseXOR()
- TDECode.inputEliminate()

In each case related changes to the Element are carried out elsewhere

4.2.6 connection to another Net

```
1    public void connect(Net n) {
2        if (n.equals(LO) || n.equals(HI)) {
3            n.connect(this);
4            return;
5        }
6
7        if (this.equals(n))
8            throw new ExEx("SYSTEM ERROR - netlist self-connect");
9
10       if ((getNumOutputs() + n.getNumOutputs()) > 1) {
11           msg("connection of nets will result in multiple sources");
12           .
13           .
14           .
15
16       }
17
18       // Add all element connections of the new Net to this Net.
19       net.nets.addAll(nnet.nets);
20       net.elements.addAll(nnet.elements);
21       net.ports.addAll(nnet.ports);
22       net.porttypes.addAll(nnet.porttypes);
23       net.inports += nnet.inports;
24       net.outports += nnet.outports;
25       net.tsoutports += nnet.tsoutports;
26
27       // Add in the new ident list.
28       net.idents.addAll(nnet.idents);
29
```

```
30          // Append UCF and NCF strings from Net n to this Net.
31          addConstraints("U", nnet.getConstraints("U"));
32          addConstraints("N", nnet.getConstraints("N"));
33
34          instances.remove(n.net);
35
36          // Change all Nets associated with n.net to reference net.
37          for (Net nn : nnet.nets)
38              nn.net = net;
39      }
```

This method connects two Nets. It does this by effectively merging the two NET subclasses so the the Nets share the same NET subclass; it adds the data from one NET to the other NET then deletes the former. It also traverses all Nets from the merged NET to change their nested NET class instance to the merged instance. If the argument Net is connected to GND or VCC then the method calls itself with the object and argument interchanged. This is to ensure that the Net identifier "GND" or "VCC" remains at the head of the list of connected identifiers and hence when **setPrefIdent()** is called from TDEList.ncode() the "GND" or "VCC" will be encountered first in the list and will be set as the final preferred identifier (otherwise an "OPAD..." identifier might be encountered and prevail and the GND net in the netlist file would be labelled thus).

4.2.7 method addProperty()

```
1       public void addProperty (String p, String v) {
2           net.properties.put(p,  v);
3       }
```

This method adds a property string to a net. P is the property identifier and v is the value string.

4.2.8 Output Net instance to the EDIF netlist file

```
1       static public void outputInstances (PrintWriter pw)
```

This method writes all Net instances to the EDIF file including all Element port connections.

4.3 The NET class - a net

The NET class represents a net. It will usually be linked from more than one instance of class Net. Class Net represents a particular net identifier. Class NET represents a net of multiple connected net identifiers.

Class NET has the following variables -

- **ArrayList<String>** idents - a list of all identifiers associated with this net, i.e. which are in the linked Net class instances
- **ArrayList<Net>** nets - a list of the linked Net class instances
- **String** prefident - the prefered identifier, selected from the associated identifiers
- **ArrayList<Element>** elements - an ordered list of Element class instances to which this net is connected
- **ArrayList<String>** ports - an ordered list of port names corresponding to the elements above
- **ArrayList<PortType>** porttypes - an ordered list of port types corresponding to the elements above (INPORT, OUTPORT, TSPORT)
- **int** inports - the number of input ports to which this net is connected
- **int** outports - the number of output ports to which this net is connected
- **int** tsoutports - the number of 3-state output ports to which this net is connected
- **public TreeMap<String,String>** properties = new TreeMap<String,String>() - a map of XDC properties for this net; actually only PULLUP and PULLDOWN

- **public TDEVPtype** extport = TDEVPtype.NONE - the port type if an EDIF external port - one of NONE, INPUT, OUTPUT or INOUT
- **public String** extportname - the port name if an external EDIF port

Methods for this class are -

public void setPrefIdent () - traverse the list of equivalent identifiers for this net and select the most "convenient" to appear in the EDIF netlist, copying it to field prefident. GND and VCC are obvious, and any starting with PORT or glob. are also desirable.

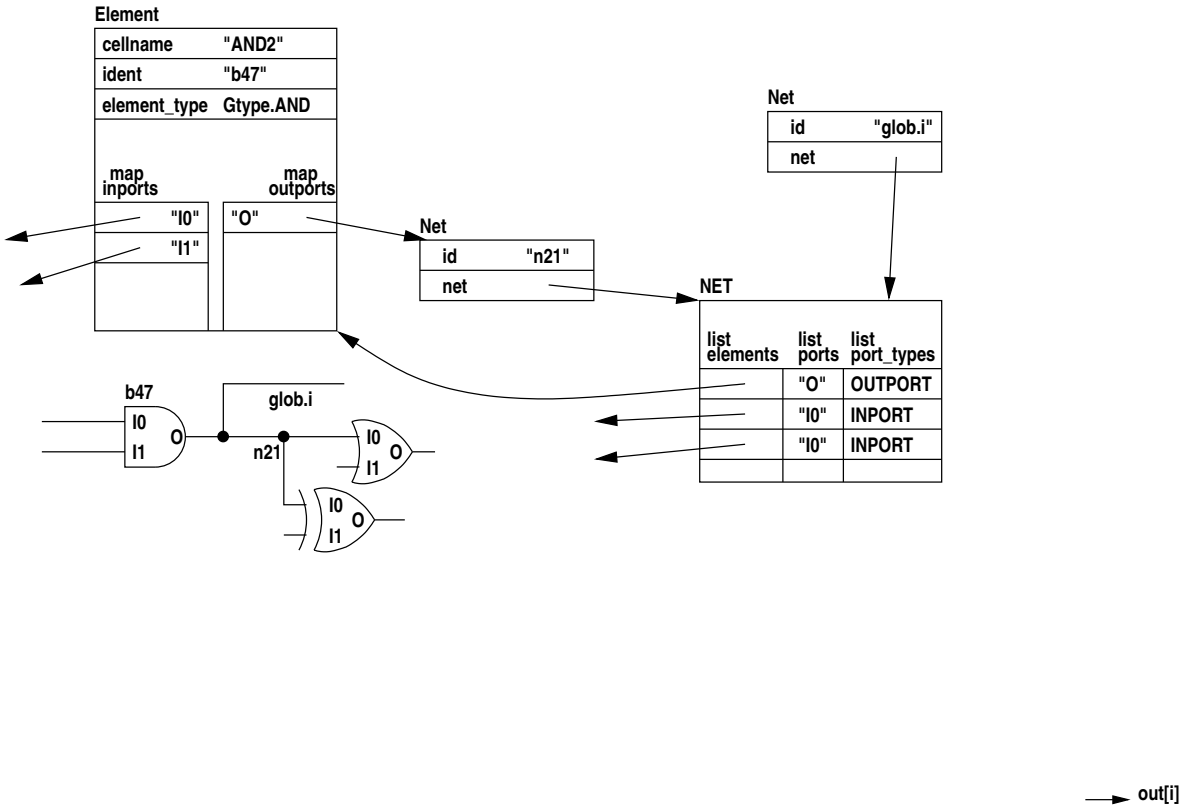
public String getEDIFIdent () - return the prefered identifier modified for EDIF syntax, removing a leading underscors and replacing '[' and ']' with an underscore.

public Net getNet () - get the first class Net from the list of linked Net class instances.

public boolean isExtOutPort () - return true if this is an EDIF output or 3-state output port.

4.4 The structure of the internal netlist

The Element class represents logic elements and the Net class the signals connecting element pins. The following figure illustrates this -



In this example nets "glob.i" and "n21" represent the same net, thus each is represented by a Net class instance but both contain the same NET subclass repreenting the actual net.

4.5 The TDECode class

The TDECode class has the following class variables -

static TDEVPtype[] ptypes = TDEVPtype.INPUT, TDEVPtype.OUTPUT, TDEVPtype.INOUT - an array to convert a numeric port type of 0, 1 or 2 from a TDEPORT TDE into a TDEVPtype enum used for entries in **ext_port_dirs** (described below).

static ArrayList<Object> ext_ports - a list of external ports, each member is a Net or else a port identifier string.

static ArrayList<TDEVPtype> ext_port_dirs - a list of port types, i.e. input, output or 3-state output.

static ArrayList<String> ext_port_pins - a list of port pins where these are FPGA pads.

static TreeMap<String,Object> portmap is a map of ports where the key is either the port identifier of else the port Net.

The following member variables contain information about the target FPGA.

- **String** fname - name of the FPGA family
- **int** lutwidth - number of inputs to CLB LUT
- **int** dspregwidth - effective width of multiplier/DSP block when used as a register
- **int** srladdrwidth - shift register address width
- **int** addsubinputwidth - maximum adder/subtracter input width
- **int** multinputwidth - maximum multiplier input width
- **int** divinputwidth - maximum divider input width
- **int** ramb_ports - maximum number of block RAM ports
- **boolean**[] ramb_readable - which block RAM ports are readable
- **boolean**[] ramb_writable - which block RAM ports are writable
- **int**[] ramb_dwidths = new int[20] - block RAM data widths for address widths
- **int** ramb_addrmin[] - block RAM smallest address width for # of ports
- **int** ramb_addrmax[] - block RAM largest address width for # of ports
- **int** ramc_ports - maximum number of CLB RAM ports
- **boolean**[] ramc_readable - which CLB RAM ports are readable
- **boolean**[] ramc_writable - which CLB RAM ports are writable
- **int**[] ramc_dwidths = new int[20] - CLB RAM data widths for address widths
- **int** ramc_addrmin[] - CLB RAM and ROM smallest address width for # of ports
- **int** ramc_addrmax[] - CLB RAM port max block address width for # of ports
- **int** romc_addrmin[] - CLB ROM port min block address width for # of ports
- **int** romc_addrmax[] - CLB ROM port max block address width for # of ports
- **int**[] queue_buffer_cram_depths - allowed depths for CLB RAM queue buffers
- **int** queue_buffer_min_cram_depth - smallest depth for CLB RAM queue buffers
- **int** queue_buffer_max_cram_depth - largest depth for CLB RAM queue buffers
- **int**[] queue_buffer_rram_depths - allowed depths for block RAM queue buffers
- **int** queue_buffer_min_rram_depth - smallest depth for block RAM queue buffers
- **int** queue_buffer_max_rram_depth - largest depth for block RAM queue buffers
- **boolean** defaultOutputDirect - is the output mode direct flag default?
- **boolean** defaultStaticContinuous - is the static mode continuous flag default?
- **boolean** deviceHasTBUF - does device have 3-state buffers?
- **boolean** deviceAllowsQueue - does device allow queue mode? queues require memory
- **boolean** deviceAllowsCMemory - does device allow cmemory mode? requires CLB RAM
- **boolean** deviceAllowsRMemory - does device allow rmemory mode? requires block RAM
- **int** ramc_mux_addr_bits_maxc - CLB RAM address bits beyond maximum the number of extra address bits for CLB RAM gained by MUXing
- **boolean** deviceFDRSEpatch - change FDRSE to FDCE + logic for CPLD
- **boolean** deviceCascadeGates - skip conversion of cascaded or wide gates
- **boolean** deviceCPLDarith - use CPLD addition/subtraction logic i.e. simple logic instead of carry chain

These variables are assigned in the constructor for the FPGA class, XC5V etc which extend class TDECode.

The following methods return FPGA parameters derived from the above class variables -

- LUTWidth() - return the LUT width for the FPGA
- AddSubWidth() - return the maximum adder/subtracter input width for the FPGA (9999 for no explicit limit)
- DivWidth() - return the maximum divider input width for the FPGA (9999 for no explicit limit)
- SRLAddrWidth() - return the maximum SRL (shift register) input width for the FPGA (0 if there is no SRL element)
- cramPorts() - return the number of memory ports for combinatorial-output RAM
- rramPorts() - return the number of memory ports for registered-output RAM
- cramAwidthMax() - return the maximum address width for a combinatorial-output RAM

- rramAwidthMax() - return the maximum address width for a registered-output RAM
- rramDwidth() - return the data width associated with an address width for a registered-output RAM
- cramReadable() - determine if a combinatorial-output RAM port is readable
- cramWritable() - determine if a combinatorial-output RAM port is writable
- rramReadable() - determine if a registered-output RAM port is readable
- rramWritable() - determine if a registered-output RAM port is writable
- queueDepthUsingCram() - determine if a requested queue buffer depth using combinatorial output memory is allowed
- queueDepthUsingRram() - determine if a requested queue buffer depth using registered output memory is allowed
- minQueueDepth() - return the minimum asynchronous queue depth
- maxQueueDepth() - return the maximum queue depth
- outputDirect() - determine if the output mode 'direct' attribute should be default for this device
- defaultContinuous() - determine if the selectvalue mode 'continuous' attribute should be default for this device
- hasTBUF() - determine if this device has internal 3-state buffers
- allowQueue() - determine if queue mode variables are allowed for this device
- allowCMemory() - determine if cmemory mode variables are allowed forthis device
- allowRMemory() - determine if rmemory mode variables are allowed forthis device
- needFDRSEpatch() - determine if this device requires gates to fully implement an FDRSE flip-flop
- needCascadeGates() - determine if this device must construct wide AND or OR gates by cascading, rather than using inbuit MUXes or a carry chain
- needCPLDarith() - determine if special arithmetic for CPLDs is needed; this means that addition and subtraction and magnitude comparisons will be done by gate logic rather than using carry chains

Abstract method outputExterns() writes all external ports to the EDIF netlist file. This method is overridden in class XTDECODE for Xilinx.

The following methods optimise the netlist code and are described elsewhere.

- optimiseGNDandVCC()
- optimiseINV()
- optimiseAND()
- optimiseOR()
- optimiseXOR()
- unNest()
- inputEliminate()
- optimiseLUT()
- optimiseFDRS()
- remapInputs()

The remainder of the methods are abstract and are overridden by methods in xilinx.XTDECode. All except the last seven are methods to convert TDEs to internal netlist constructs. The method names are self explanatory -

- TDESELECT()
- TDETSELECT()
- TDEREG()
- TDEINV()
- TDEAND()
- TDEOR()
- TDEXOR()
- TDEDFF()
- TDEIBUF()
- TDEOBUF()
- TDESTART()
- TDEILOOP()
- TDEWHEN()
- TDEWHILE()
- TDEDOWHILE()
- TDERESYNC()
- TDESAMPLE()
- TDEEXECP()
- TDECONNECT()
- TDEOPERATOR()

- TDEWAIT()
- TDEDEL()
- TDEDIVERGE()
- TDEQUEUEBUFFER()
- TDEPRIORITY()
- TDECRAM()
- TDERRAM()
- TDECLOCK()
- TDECLOCKINFO()
- TDEELEMENT()
- TDEELEMENTCONNECT()
- TDEPORT()
- TDESPECIAL()

The last seven abstract methods are miscellaneous. Again they are overridden by methods in xilinx.XTDECode.

TDEOptimise() applies family-specific optimisations to the TDE list.

optimise() carries out low-level optimisation of the internal netlist structure.

convert() converts generic gate elements to appropriate elements for the target device. Before this is called the netlist structure might contain an AND gate with 10 inputs. Following conversion that element would have been replaced perhaps by 3 4-input LUTs connected to carry chain MUXCYs.

verify() examines all elements to ensure there are no errors or inconsistencies. This method is not located in class netlist.Element as the checks are specific to target device elements rather than generic elements (there is however a method netlist.Net.verify() to check nets and this is generic since nets are not specific to the target device).

netFromPin() returns the identifier of the net attached to a pad given the pad identifier.

EDIFFileNameExtension() returns the EDIF file extension.

Abstract method outputConstraints() writes constraints to the approprite file(s) (for Xilinx this is the ISE NCF or Vivado XDC file). This method is overridden in class XTDECODE for Xilinx.

4.6 The GenFunctions class

GenFunctions extends TDECode. The methods in this class provide such general functions as -

- conversion between arrays and ArrayLists of Nets
- padding or sign extension of arrays of Nets
- extraction of sub-arrays of Nets
- padding or truncation of strings
- generation of hexadcimal strings from integers
- extraction of a bit from a hexadecimal string
- conversion of constants to arrays of Net
- connections between Nets and Net arrays

The methods are sufficiently simple that they are not listed or described here.

4.7 The xilinx/XC... FPGA family classes

Each of these classes extends XTDECode. One of these classes is instantiated as the object representing the FPGA or CPLD. These classes are specific to Xilinx FPGA families and are named appropriately -

- UNISIMS - Xilinx common library elements
- XC2C - Xilinx Coolrunner II CPLD
- XC2S - Xilinx Spartan 2C FPGA
- XC3S - Xilinx Spartan 3S FPGA
- XCV - Xilinx Virtex FPGA
- XC2V - Xilinx Virtex 2 FPGA

- XC2VP - Xilinx Virtex 2 PRO FPGA
- XC4V - Xilinx Virtex 4 FPGA
- XC5V - Xilinx Virtex 5 FPGA
- XC6S - Xilinx Spartan 6 FPGA
- XC6V - Xilinx Virtex 6 FPGA
- XC7 - Xilinx 7-series
- XC95 - Xilinx XC9500 CPLD

The UNISIMS class represents Xilinx common library elements but really these are already implemented in the super classes XFunctions and XElements so UNISIMS is really just a default object when no specific target device is specified.

XC2C and XC95 are CPLDs, not FPGAs. Much of the functionality of 3PL can be used with these devices but there are limitations.

These device-specific classes perform two functions -

- they provide parameters and flags to superclasses to tailor logic generation for the specific device, e.g. LUT inputs, CLB RAM sizes, block RAM sizes
- they provide methods for elements specific to the device, e.g. block RAM, DSP blocks

4.8 The xilinx/XElements class

XElements extends GenFunctions to vendor-specific elements. The methods generate basic elements such as gates and flip-flops.

The IBUF Element provides a simple example -

```
1     public Element IBUF (Net o, Net i) {
2         Element c = new Element("IBUF");
3         c.addInput("I", i);
4         c.addOutput("O", o);
5         return(c);
6     }
```

The new Element is created by a constructor, the element name string being passed as an argument to the constructor. The input and output Nets are added paired with the port name strings. The Element is returned.

Where an Element may be subject to later optimisation a gate type is added, e.g. the XORCY Element -

```
1     public Net XORCY (Net li, Net ci) {
2         Element c = new Element("XORCY", Gtype.XORCY);
3         Net     o = new Net();
4         c.addInput("LI", li);
5         c.addInput("CI", ci);
6         c.addOutput("O", o);
7         return(o);
8     }
```

In this case the output Net is returned.

The following basic Elements are constructed directly -

- LUT - a 1 to n input look-up table
- INV - inverter
- AND - 1 to n input AND gate
- OR - 1 to n input OR gate
- XOR - 1 to n input XOR gate
- XNOR - 1 to n input XNOR gate
- SRL16E - 16 bit CLB shift register
- SRL32E - 32 bit CLB shift register
- FDRS - reset/set flip-flop
- FDCP - clear/preset flip-flop
- MULT_AND - multiplier AND

- MUXCY - carry multiplexer
- MUXF5 - CLB multiplexer
- MUXF6 - CLB multiplexer
- XORCY - carry XOR
- BUFG - global clock buffer
- IPAD - input pad
- OPAD - output pad
- IOPAD - bi-directional pad
- BUFE - internal 3-state buffer, +ve enable
- BUFT - internal 3-state buffer, -ve enable
- IBUF - input buffer
- IBUFDS - differential input buffer
- IBUFG - input global clock buffer
- IBUFGDS - differential input global clock buffer
- OBUF - output buffer
- OBUFDS - differential output buffer
- PULLDOWN - internal longline pulldown
- PULLUP - internal longline pullup
- RAM_1 - single port CLB RAM
- ROM_1 - single port CLB ROM
- RAM_2 - dual port CLB RAM

Other Elements available in some Xilinx FPGA families but not others have methods which must be overridden in subclasses -

- MULT18X18 - 18 x 18 bit multiplier
- MULT18X18S - 18 x 18 bit multiplier with output register
- MULT25X18 - 25 x 18 bit multiplier
- MULT25X18S - 25 x 18 bit multiplier with output register
- MULREG - multiplier used as register only
- ramallocate - block RAM
- fifoallocate - block RAM as FIFO
- FIFO - FIFO using dual port block RAM
- Mult- multiplier
- dsp48_alu - an ALU using a DSP block

These are overridden in subclasses XC2V, XC2VP etc. where appropriate.

This subdivision is somewhat arbitrary; some Elements in the first group are not available in some FPGAs or CPLDs and conditional code ensures that these are not called is such cases. For consistency some of these should be moved to the second group and the methods moved into subclasses.

4.9 The xilinx/XFunctions class

XFunctions extends XElements.

This class is the heart of the code (netlist) generation. It has methods which use the Elements created in class XElements to provide higher level functions such as registers, multiplexers and queue buffers.

The following functional elements are implemented by methods in this class -

- registers - to implement static mode variables and to provide registers for other methods in this class
- resynchroniser - to transfer a pulse from one clock domain to another
- queue buffers - to implement queue mode variables
- delay line - to implement the TDEDEL intermediate code element
- shifters - to implement left and right constant and variable shift
- adder - to implement addition
- inverter - to implement bit inversion
- incrementer - to implement addition by 1
- subtracter - to implement subtraction
- decrementer - to implement subtraction by 1
- adder/subtracter - to implement selectable addition or subtraction
- ALU - to implement addition/subtraction/assignment in CLBs
- incrementer/decrementer - to implement selectable addition or subtraction of 1

- signed and unsigned multipliers - to implement multiplication using hardware if available and CLB logic otherwise
- signed and unsigned divider/remainder - to implement to implement division and remaindering using CLB logic
- numerical comparators - to implement equal, not equal, greater than, greater than or equal, less than and less than or equal numerical comparisons
- combinatorial RAM and ROM - to implement cmemory mode variables
- registered output RAM - to implement rmemory mode variables

4.9.1 registers

```
1   public Net[] reg (
2       Net[]               in,
3       Net                 c,
4       Net                 ce,
5       Net                 r,
6       String              sinit,
7       ArrayList<String>   constraints
8   ) {
9       int                 width = in.length;
10      Net[]               out = new Net[width];
11      ArrayList<String>   locs = new ArrayList<String>();
12
13      // Constraints here are single strings -
14      //  "TNM=..."
15      //  "LOC=..."
16      //  "DSP"
17      // having been modified in XTDECode.TDEREG() from the key=value pairs
18      // in the original TDE list.
19      // Location constraints should be one per bit.
20      // The DSP constraint is ignored.
21      if (constraints != null) {
22          Iterator<String>    it = constraints.iterator();
23          while (it.hasNext()) {
24              String s = it.next();
25              if (s.startsWith("LOC=")) {
26                  locs.add(s);
27                  it.remove();
28              }
29          }
30      }
31      if (sinit == null)
32          sinit = "0";
33      for (int bit=0 ; bit<width ; bit++) {
34          if (!locs.isEmpty())
35              constraints.add(0, locs.get(0));
36          if ((strhexbit(sinit, bit) & 1) == 0)
37              out[bit] = FDRS(in[bit], c, ce, r, null, "R", constraints);
38          else
39              out[bit] = FDRS(in[bit], c, ce, null, r, "S", constraints);
40          if (!locs.isEmpty()) {
41              locs.remove(0);
42              constraints.remove(0);
43          }
44      }
45      return(out);
46  }
```

This method creates a register which has a single clock enable and single reset common to all flip-flops in the register. It is called from XTDECode.TDEREG() to create a static mode variable that is one bit wide and from several methods in this class to create registers. It is only in the former case that the constraints argument will be non-null.

The code first searches the constraints list for location constraints (they start with "LOC="). It removes these, copying them to a new list. As each bit is created a location constraint is copied from the list (if there are any) and is put back at the front of the constraints list. At the end of the loop that location constraint is removed from both lists.

If there is no initialisation string the string "0" is assigned.

For each register bit the appropriate bit is extracted from the initialisation string using method strhexbit() from class genFunctions. Method FDRS() from class XElements is then called with either an "R" or "S" property argument and the reset signal is connected to either the R or S input as appropriate.

```
    public Net[] reg (
        Net[]       in,
        Net         c,
        Net[]       ce,
        Net[]       r,
        String      sinit,
        ArrayList   constraints
    ) {
        int         width = in.length;
        Net[]       out = netArray(width);
        ArrayList   locs = new ArrayList();
        boolean     dsp = false;

        .
        . code for use of multiplier/DSP register
        .

        if (sinit == null)
            sinit = "0";
        out = new Net[width];
        for (int bit=0 ; bit<width ; bit++) {
            if (!locs.isEmpty())
                constraints.add(0, (String)locs.get(0));
            if ((strhexbit(sinit, bit) & 1) == 0)
                out[bit] = FDRS(in[bit], c, ce[bit], r[bit], null, "R", constraints);
            else
                out[bit] = FDRS(in[bit], c, ce[bit], null, r[bit], "S", constraints);
            if (!locs.isEmpty()) {
                locs.remove(0);
                constraints.remove(0);
            }
        }
        return(out);
    }
```

This method differs from the previous in two respects -

- if there is no DSP constraint it is very similar to the previous method except that the clock enable and reset signals are now arrays, one for each flip-flop, rather than common single signals
- if there is a DSP constraint, the register is not initialised and the FPGA has a hardware multiplier or DSP block then the register is implemented using the output register of a multiplier or DSP block

A map, **hm**, and two classes, **Cer** and **Group**, were defined in the method for use when a DSP constraint is acted upon. These are used to group flip-flops within the register which share the same clock enable and reset signals. Each such group is implemented in a separate multiplier or DSP block since the clock enables and resets are common to all bits in those hardware registers. If a group entry is larger than the width of the hardware register then multiple units will be used. The **Cer** class contains a Net pair, a clock enable and a reset. These are used as the key to map **hm** which thus has an entry for each unique pair. The associated map value is class **Group** which has an input list and an output list. These contain the input and output signals of flip-flops in each CE/R group in LSB to MSB bit order.

The declarations are -

```
    HashMap     hm = null;
    class Cer {
        Net ce;
        Net r;

        Cer (Net ce, Net r) {
            this.ce = ce;
            this.r = r;
        }

        public boolean equals (Object o) {
```

```
12              return(ce.equals(((Cer)o).ce) && r.equals(((Cer)o).r));
13          }
14          public int hashCode () {
15              return((ce.hashCode() << 16) + r.hashCode());
16          }
17      }
18      class Group {
19          ArrayList   inputs = new ArrayList();
20          ArrayList   outputs = new ArrayList();
21      }
```

The equals() and hashCode() methods are implemented to enable correct functioning of HashMap keys.

The code to create the group data structure is -

```
1       hm = new HashMap();
2       for (int i=0 ; i<width ; i++) {
3           Cer x = new Cer(ce[i], r[i]);
4           if (!hm.containsKey(x))
5               hm.put(x, new Group());
6           Group g = (Group)hm.get(x);
7           g.inputs.add(in[i]);
8           g.outputs.add(out[i]);
9       }
```

The code to create the register is -

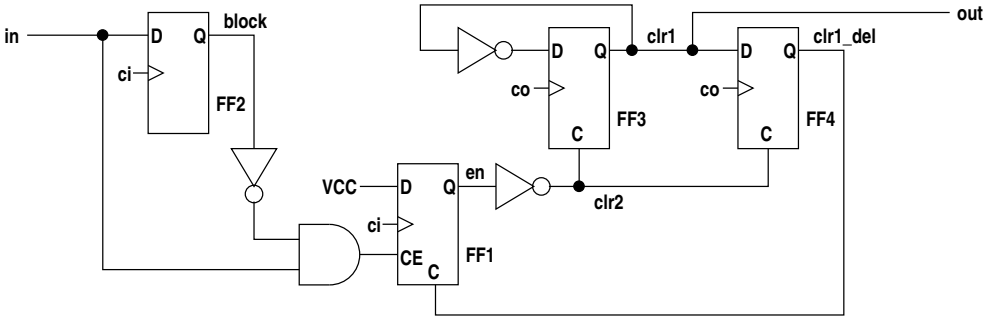
```
1       Iterator    mit = hm.entrySet().iterator();
2       while (mit.hasNext()) {
3           Map.Entry   me = (Map.Entry)mit.next();
4           Cer         cer = (Cer)me.getKey();
5           Group       g = (Group)me.getValue();
6           Net[]       gout = new Net[1];
7           Net[]       gin = new Net[1];
8           int         gwidth = gout.length;
9           int         mwidth = DSPRegWidth();
10          int         k;
11          int         l = gwidth;
12          gout = (Net[])g.outputs.toArray(gout);
13          gin = (Net[])g.inputs.toArray(gin);
14
15          for (int j=0 ; j<gwidth ; j+=mwidth, l-=mwidth) {
16              k = (l >= mwidth) ? j + mwidth - 1 : j + l - 1;
17              MULREG(subarray(gin, j, k), subarray(gout, j, k), c, cer.ce, cer.r);
18          }
19      }
20      return(out);
```

This iterates through the entries in the map converting each group into a register using one or more multipliers or DSP blocks. The cell type of Element(s) produced is MULREG which is later converted to MULT18X18 or DSP48E depending on the FPGA type (method **convertElements()** in classes **XC2VP**, **XC5V** etc.).

4.9.2 resynchroniser

A resynchroniser generates a pulse in another clock domain from a rising edge. When the input rises the rise is captured by the next input clock edge and an output pulse of one period duration is generated on the next output clock edge. This method is used by intermediate code TDE **TDERESYNC** and also indirectly by methods **async_buffer()** and **triggerbuffer**() (via **asynch_fifo_control()**) in this class.

The logic circuit used is -



Note that FF1, FF3 and FF4 have asynchronous clear inputs. Initially all flip-flop outputs are low, thus **block** and **clr1**/**out** are low. When **in** goes high both FF1 and FF2 outputs go high on the next edge of clock **ci**, the input clock. The **CE** input of FF1 goes low as **block** is now high. Since **en** is now high, **clr2** is low and this enables FF3 to toggle on the next edge of **co**, the output clock, raising the output **out**. On the next edge of **co** FF3 toggles low again and FF4 is set, raising **clr1_del**, which clears FF1. When **en** thus goes low **clr2** goes high clamping FF3 low and clearing FF4. FF1 is now free to again latch an input however if **in** is still high **block** will disable FF1. Thus **in** must be clocked low to lower **block** before FF1 can again start the pulse cycle. This circuit avoids race conditions between flip-flops in the two clock domains.

The code to generate this is -

```
1     public Net resync (Net in, Net ci, Net co) {
2         Net en;
3         Net block;
4         Net clr1 = new Net();
5         Net clr1_del = new Net();
6         Net clr2;
7
8         block = FDRS(in, ci, null, null, null, null, null);
9         en = FDCP(Net.HI, ci, AND(in, INV(block)), clr1_del, null, null, null);
10        clr2 = INV(en);
11        clr1.connect(FDCP(INV(clr1), co, null, clr2, null, null, null));
12        clr1_del.connect(FDCP(clr1, co, null, clr2, null, null, null));
13        return(clr1);
14    }
```

Note the use of the connect() method. For example in the statement

```
1     clr1.connect(FDCP(INV(clr1), co, null, clr2, null, null, null));
```

the output Net is also used as an input argument, i.e. there is feedback. Thus the Net **clr1** is declared and assigned a new instance, then passed as an input argument. The result returned from the call is then connected to Net **clr1**. It cannot be assigned as that would overwrite the previous Net instance. A similar though more indirect argument applies to Net **clr1_del**.

4.9.3 queue buffers

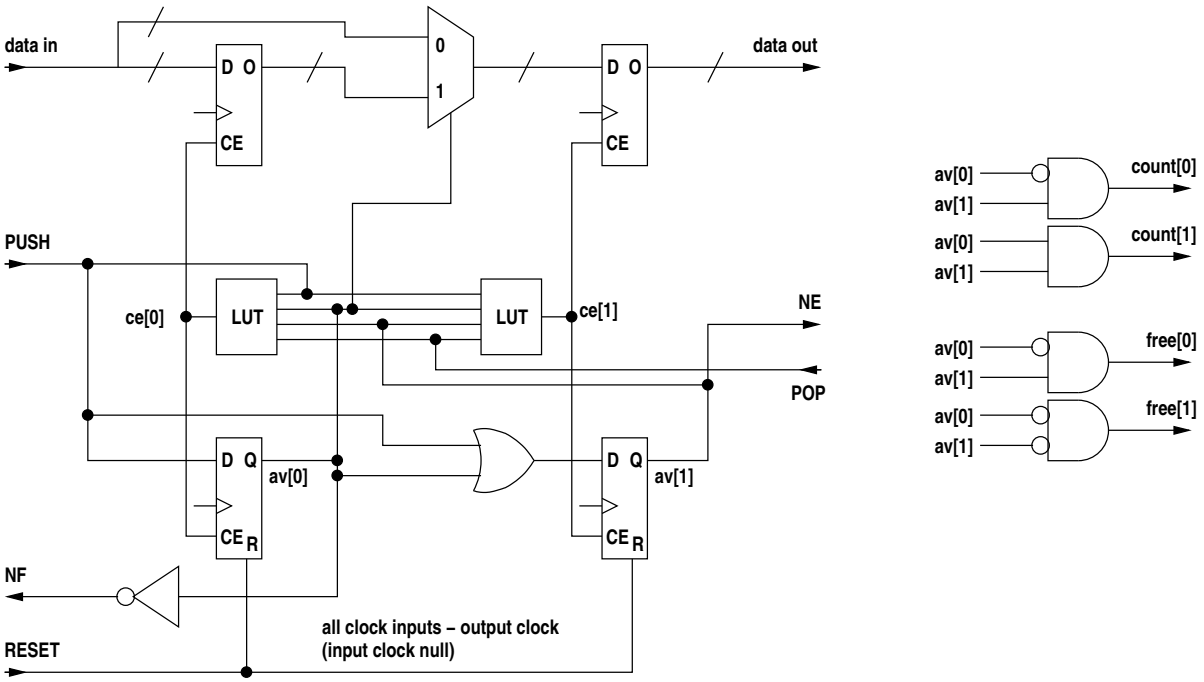
These were described in some detail in the section on intermediate code TDEs. Some of that description is repeated here.

There are five methods for generating queue buffers -

- queuebuffer() - synchronous 2-deep default queue buffer
- sync_buffer() - synchronous n-deep queue buffer
- async_buffer() - asynchronous n-deep queue buffer
- triggerbuffer() - synchronous or asynchronous n-deep queue buffer with no data
- queueunbuffered() - a synchronous 0-deep queue buffer (no buffer)

queuebuffer()

This method creates a synchronous queue buffer of depth 2. The logic for the default synchronous queue buffer of depth two is -



The 2-input multiplexer shown uses a 3-input LUT per bit.

When PUSH is high input data is latched into one of two registers. If the buffer is empty the data is latched into the rightmost register to which the data output is directly connected. If another datum is written to the queue buffer it goes into the second (left) register. When a datum is popped the two registers are shifted right. This scheme avoids having combinatorial logic at the output thus minimising clock-to-data-out time. The tradeoff is a longer input setup time. The two flip-flops represent the occupancy status of the two registers. Each register/flip-flop pair has a common clock enable controlled by a 4-input LUT.

The truth table for the control logic is as follows -

t							t+1						
		av		ce			av		ce				
PUSH	POP	[0]	[1]	[0]	[1]	MUX	[0]	[1]	[0]	[1]	NE	NF	
0	0	0	0	0	0	X	0	0	0	0	0	1	empty
		0	1	0	0	X	0	1	0	1	1	1	1 entry
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	0	0	X	1	1	1	1	1	0	full
0	1	0	0	-	-	-	-	-	-	-	-	-	illegal state
		0	1	X	1	X	0	0	0	0	0	1	av[1] ↓ POP, now empty
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	1	1	1	0	1	0	1	1	1	av[0] ↓ POP, 1 remaining
1	0	0	0	0	1	0	0	1	0	1	1	1	av[1] ↑ PUSH, 1 entry
		0	1	1	0	X	1	1	1	1	1	0	av[0] ↑ PUSH, now full
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	-	-	-	-	-	-	-	-	-	illegal state
1	1	0	0	-	-	-	-	-	-	-	-	-	illegal state
		0	1	0	1	0	0	1	0	1	1	1	1 entry
		1	0	-	-	-	-	-	-	-	-	-	illegal state
		1	1	-	-	-	-	-	-	-	-	-	illegal state

PUSH must not be raised when **NF** is low and **POP** must not be raised when **NE** is low.

The illegal states are described in the following table

	PUSH	POP	av[0]	av[1]	
1	0	0	1	0	single entry must be at righXTDECode extends XFunctions. Its main function is to implement
2	0	1	0	0	cannot pop when empty
3	0	1	1	0	single entry must be at right
4	1	0	1	0	single entry must be at right
5	1	0	1	1	cannot push when full
6	1	1	0	0	cannot pop when empty
7	1	1	1	0	single entry must be at right
8	1	1	1	1	cannot push when full

In entries 1, 3, 4 and 7 a single datum in the buffer must reside in the right register, so these states are not allowed.

In entry 2 the registers are both empty and so a **POP** is not allowed since there is no valid output (**NE** is low). The logic has been designed so that no state change occurs, i.e. the **POP** is ignored.

In entry 6 the registers are both empty and so a **POP** is not allowed since there is no valid output (**NE** is low), however **PUSH** is high. The logic has been designed so that the **PUSH** is handled but the **POP** is ignored.

In entries 5 and 8 both registers are full and hence no **PUSH** is allowed. Note that this is true even when **POP** is asserted since we cannot allow **NF** (not full) to depend on **POP** or we generate the very logic cascade this buffering scheme was designed to avoid. The logic has been designed to ignore **PUSH** in these cases.

The truth tables for the two LUTs are -

	PUSH	POP	av[0]	av[1]
ce[0]	0	1	1	1
	1	0	0	1
ce[1]	0	1	X	1
	1	X	0	0
	1	1	0	1

The code for this is -

```
    public Net[] queuebuffer (
        Net[]   in,     // data input
        Net     push,   // push-to-queue input
        Net     pop,    // pop-from-queue input
        Net     c,      // clock
        Net     ne,     // returned not empty signal
        Net     nf,     // returned not full signal
        Net     r,      // reset input
        Net     rr,     // reset output for chaining resets
        Net[]   count,  // returned buffer data count or null
        Net[]   free,   // returned free space count or null
        String  sinit   // initialisation hex string or null
    ) {
        int     width = in.length;
        String  id = (sinit != null) ? sinit : "";
        String  io =  (sinit != null) ? "S" : "R";

        Net[][] data = new Net[2][width];
        for (int i=0 ; i<2 ; i++)
            for (int j=0 ; j<width ; j++)
                data[i][j] = new Net();
        Net[]   av = new Net[2];
        Net[]   ce = new Net[2];

        av[0] = new Net("av0");
        av[1] = new Net("av1");
        ce[0] = LUT("(!I0*I1*I2*I3)+(I0*!I1*!I2*I3)", push, pop , av[0] , av[1]);
        ce[1] = LUT("(!I0*I1*I3)+(I0*!I2*!I3)+(I0*I1*!I2*I3)",
                                                push, pop , av[0] , av[1]);
        if (in != null)
            data[0] = reg(in, c, ce[0], null, null, null);
        av[0].connect(FDRS(push, c, ce[0], r, null, null, null));
        if (in != null)
```

```
34              data[1] = reg(mux2(av[0], in, data[0]), c, ce[1], r, id, null);
35          av[1].connect(FDRS(OR(push, av[0]), c, ce[1], r, null, io, null));
36
37          ne.connect(av[1]);
38          nf.connect(INV(av[0]));
39
40          if (count != null) {
41              count[0].connect(AND(av[1], INV(av[0])));
42              count[1].connect(AND(av[1], av[0]));
43          }
44          if (free != null) {
45              if (count != null)
46                  free[0] = count[0];
47              else
48                  free[0].connect(AND(av[1], INV(av[0])));
49              free[1].connect(AND(INV(av[1]), INV(av[0])));
50          }
51
52          if (rr != null) {
53              if (r != null)
54                  rr.connect(r);
55              else
56                  rr.connect(Net.LO);
57          }
58
59          if (in != null)
60              return(data[1]);
61          else
62              return(null);
63      }
```

Signals **push** and **pop** are control inputs with obvious functions. Signals **ne** and **nf** are argument Nets which are connected in the method to the logic signalling "not empty" and "not full". These are used to determine write and read availability respectively. Net **r** is an optional reset input. Net **rr** is a supplied Net which if not null is connected to the **r** input if present, otherwise to GND. This output signal is available so that resets can be chained down a pipeline of queue variables.

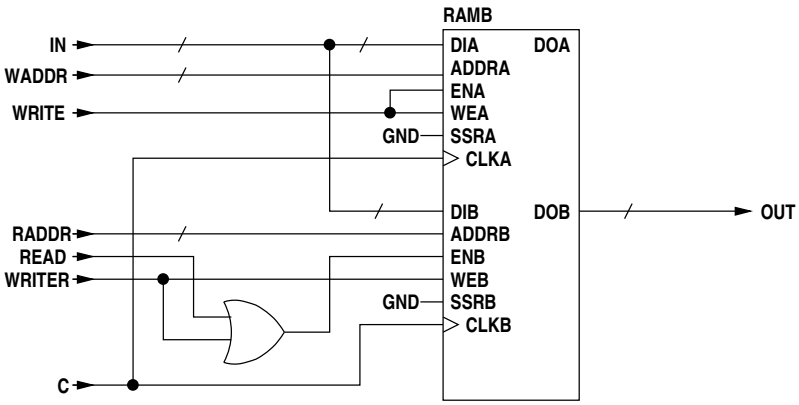
sync_buffer()

This method creates a synchronous queue buffer of depth greater than 2. A synchronous queue buffer of depth greater than two is implemented using either CLB dual port RAM or dual port block RAM. The CLB RAM is used for a depth of 16 and block RAM for greater depths.

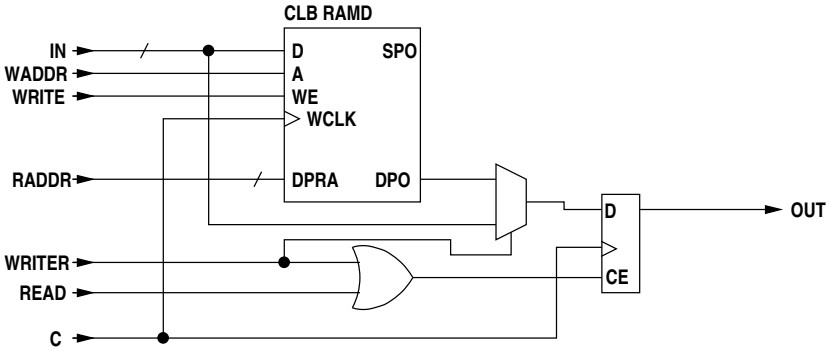
Because CLB RAM has combinatorial output and block RAM has registered output the CLB RAM is employed with a register appended to the output port so that the two forms of RAM can be used interchangably with the same control logic.

The queue buffer is a queue. A required characteristic of queue buffers is that on writing a datum to an empty queue buffer that datum must be available on the next clock cycle. This presents a problem with a dual-port FIFO using registered output RAM because a read must be issued once the queue becomes non-empty, taking an extra cycle. This can be avoided by writing the first datum into a separate register rather than into the empty RAM. For CLB RAM this is easily implemented however in the case of block RAM the control logic for this is somewhat complicated. An easier solution for block RAM is to write the first datum into an empty queue via the read port rather than the write port. Since the queue is empty the read port is guaranteed to be unused on that clock cycle. Since the normal write mode for block RAM is WRITE_FIRST and the CLB RAM with appended output register has been designed to behave the same way, the newly written datum will appear immediately in the output register upon writing. This avoids the extra cycle.

For a block RAM the memory logic is -



To make an equivalent functional unit using CLB RAM the logic is -



In both the above blocks **WRITER** writes directly to the output register whereas **WRITE** writes to the RAM. **READ** reads from the RAM to the output register.

The code for the memory logic is contained in method queuemem() -

```
    private Net[] queuemem (
        int     depth,  // buffer depth
        int     dwidth, // data width
        int     awidth, // address width
        Net[]   in,     // data input
        Net[]   raddr,  // read address
        Net[]   waddr,  // write address
        Net     read,   // read enable
        Net     write,  // write enable
        Net     writer, // write enable for the read port or null
        Net     ci,     // write clock
        Net     co      // read clock
    ) {
        Net[]   out;
        boolean is_sync = (ci == co);

        if (depth <= queue_buffer_max_cram_depth) {
            // use CLB RAM
            Net[] o = new Net[dwidth];
            for (int i=0 ; i<dwidth ; i++)
                o[i] = new Net("o" + i + "_");
            sram2(  dwidth,
                      ci, write, waddr,   in, null,
                    null,  null, raddr, null,    o,
                    null, null );
            if (is_sync)
                out = reg(mux2(writer, o, in), co, read, null, null, null);
            else
                out = reg(o, co, read, null, null, null);
            return(out);
        }

        // use block RAM
        out = new Net[dwidth];
        for (int i=0 ; i<dwidth ; i++)
```

```
36              out[i] = new Net("out" + i + "_");
37          Net[]   rin = is_sync ? in : null;
38          rram2 ( dwidth, awidth,
39                  write,  write, ci, waddr,  in, null,
40                   read, writer, co, raddr, rin,  out,
41                  null, new ArrayList() );
42          return(out);
43      }
```

If the buffer depth is less than or equal to the maximum CLB RAM depth (TDECode.queue_buffer_max_cram_depth) then CLB RAM is used (method XFunctions.sram2()), otherwise block RAM is used (method XFunctions.rram2()). In the case of a synchronous buffer the first write to an empty FIFO asserts **writer** which casues the output register to be directly loaded with the datum.

The following terms are used in the logic description -

- empty - the RAM is empty
- full - the RAM is full
- NE - data is available at the output of the queue buffer
- NF - data can be written to the queue buffer
- read - read RAM
- write - write RAM
- writer - first write when queue buffer empty
- raddr - RAM read address
- waddr - RAM write address
- fcnt - RAM occupancy counter
- $\Uparrow$ - increment
- $\Downarrow$ - decrement

Note that a RAM occupancy counter, **fcnt**, is kept in addition to read and write address counters. **fcnt** is used to determine the RAM full and empty conditions and is potentially faster than doing comparisons between **raddr** and **waddr**. Additional counters are used for the optional word count and space count, again trading speed for logic and routing resources.

The following table shows the change of state and counter increment/decrement operations in controlling the queue buffer (**full** and **NF** are not shown) -

state		inputs		increment/decrement			memory control			new
empty	NE	PUSH	POP	waddr	raddr	fcnt	write	writer	read	NE
1	0	1	0				0	1	0	1
0	1	1	0	$\Uparrow$		$\Uparrow$	1	0	0	1
1	1	1	0	$\Uparrow$		$\Uparrow$	1	0	0	1
0	1	1	1	$\Uparrow$	$\Uparrow$		1	0	1	1
1	1	1	1				0	1	0	1
0	1	0	1		$\Uparrow$	$\Downarrow$	0	0	1	1
1	1	0	1				0	0	0	0

On first write to an empty queue buffer the datum is written directly to the read port of the RAM block and **NE** goes high, but **fcnt** stays at 0 as nothing is written to the RAM. The next write (if there is no read) will write to RAM and increment both**fcnt** and **waddr**. On acknowledging the output datum (i.e. popping a datum off the queue) if there is only the datum in the read port register **NE** will go low since **fcnt** is 0 (**empty** will be true). If there is a datum in the output register and another in the RAM then the last RAM read to the port register will occur and **fcnt** will decrement from 1 to 0 and empty will become true, but NE will not go low until the next **POP**.

Related equations -

$$\mathsf{NF} = \overline{\mathsf{full}}$$
$$\mathsf{read} = \mathsf{POP} \cdot \overline{\mathsf{empty}}$$
$$\mathsf{write} = \mathsf{PUSH} \cdot \mathsf{NE} \cdot \overline{(\mathsf{empty} \cdot \mathsf{POP})}$$
$$\mathsf{writer} = \mathsf{PUSH} \cdot \mathsf{empty} \cdot \overline{(\mathsf{NE} \cdot \overline{\mathsf{POP}})}$$

State changes -

$$\mathsf{NE} \Leftarrow \overline{\mathsf{empty} \cdot \mathsf{NE} \cdot \overline{\mathsf{PUSH}} \cdot \mathsf{POP}}$$
$$\mathsf{empty} \Leftarrow \overline{\mathsf{write}} \cdot (\mathsf{fcnt} == 0 + \mathsf{fcnt} == 1 \cdot \mathsf{read})$$
$$\mathsf{full} \Leftarrow \overline{\mathsf{read}} \cdot (\mathsf{fcnt} == 11\cdots11 + \mathsf{fcnt} == 11\cdots10 \cdot \mathsf{write})$$

Increment/decrement counters -

raddr $\Uparrow=$ read
waddr $\Uparrow=$ write
fcnt $\Uparrow=$ write $\cdot \overline{\text{read}}$
fcnt $\Downarrow=$ read $\cdot \overline{\text{write}}$

The code is -

```
    public Net[] sync_buffer (
        Net[]   in,     // data input
        Net     push,   // write-to-queue input
        Net     pop,    // pop-from-queue input
        Net     c,      // clock
        Net     ne,     // returned not empty
        Net     nf,     // returned not full
        Net[]   count,  // returned buffer data count or null
        Net[]   free,   // returned free space count or null
        Net     r,      // buffer reset
        Net     rr,     // buffer reset output
        boolean fifo,   // use FIFO hardware if present
        int     depth,  // buffer depth - a power of 2 constrained by FPGA family
        Var     wcvar,  // write clock variable
        Var     rcvar   // read clock variable
    ) {
        int     dwidth = in.length;
        Net     empty = new Net("EMPTY");
        Net     full = new Net("FULL");

        if (((this instanceof XC5V) || (this instanceof XC6V)) && (depth >= 512) && (depth <= 8192) && fifo) {
            // For Virtex5 or Virtex6 block RAM FIFO can use inbuilt FIFO logic.
            if (rr != null) {
                if (r != null)
                    rr.connect(r);
                else
                    rr.connect(Net.LO);
            }
            ne.connect(INV(empty));
            nf.connect(INV(full));
            Net[]   out = new Net[dwidth];
            for (int i=0 ; i<dwidth ; i++)
                out[i] = new Net("out" + i + "_");
            queuefifo(depth, in, push, pop, out, empty, full, count, free, r, c, c, wcvar, rcvar);
            return(out);
        }

        int     awidth = bits(depth-1, false);
        Net     read = new Net("READ");
        Net     write = new Net("WRITE");
        Net     writer;
        Net[]   raddr, waddr, out;

        read.connect(AND(pop, INV(empty)));
        write.connect(AND(push, ne, INV(AND(empty, pop))));
        writer = AND(push, empty, INV(AND(ne, INV(pop))));
        ne.connect(FDRS(INV(AND(empty, ne, INV(push), pop)), c, OR(push, pop), r, null, null, null));
        nf.connect(INV(full));

        raddr = cb(awidth, false, c,  read, r, null, null);
        waddr = cb(awidth, false, c, write, r, null, null);

        int     cwidth = awidth + 1;
        Net[]   fcnt;
        Net     ed;
        Net     fd;

        fcnt = cb(awidth, false, c, XOR(read, write), r, read, null);

        ed = AND(INV(write), eqzeros(fcnt, 1, awidth-1), OR(INV(fcnt[0]), read));
        empty.connect(FDRS(ed, c, null, null, r, "S", null));
        fd = AND(INV(read), eqones(fcnt, 1, awidth-1), OR(fcnt[0], write));
        full.connect(FDRS(fd, c, null, r, null, null, null));

```

```
        if (count != null) {
            Net[]   fcntr;
            fcntr = cb(cwidth, false, c, XOR(pop, push), r, pop, null);
            connect(count, fcntr);
        }
        if (free != null) {
            Net[]   ecntr;
            String  sinit = Integer.toHexString(1 << awidth);
            ecntr = cb(cwidth, false, c, XOR(pop, push), r, push, sinit);
            connect(free, ecntr);
        }

        Net mread = OR(read, writer);
        out = queuemem(depth, dwidth, awidth, in, raddr, waddr, mread, write, writer, c, c);
        if (rr != null) {
            if (r != null)
                rr.connect(r);
            else
                rr.connect(Net.LO);
        }
        return(out);
    }
```

Calls to constructor Net(), e.g. Net(”EMPTY”), create nets which have unique identifiers constructed by appending an integer to the argument string. The head strings are chosen to label nets with meaningful identifiers in the netlist but have no other significance.

If the FPGA is Virtex5 or Virtex6 and the buffer depth is between 512 and 8192 inclusive then method queuefifo() is called to use the inbuilt block RAM FIFO logic. This can be used for both synchronous and asynchronous buffers. The code for this method is -

```
    private void queuefifo(
        int     depth,  // buffer depth
        Net[]   in,     // data input
        Net     push,   // true to push a datum
        Net     pop,    // true to pop a datum
        Net[]   out,    // data output
        Net     empty,  // true if the FIFO is empty
        Net     full,   // true if the FIFO is full
        Net[]   count,  // number of contained data
        Net[]   free,   // number of free spaces
        Net     r,      // reset signal or null
        Net     cw,     // write clock
        Net     cr,     // read clock
        Var     wcvar,  // write clock variable
        Var     rcvar   // read clock variable
    ) {
        int     dwidth = in.length;
        int     awidth = bits(depth-1, false);
        Net[]   rdcount = null;
        Net[]   wrcount = null;
        Net         r3 = new Net();
        double      minfreq = Math.min(wcvar.getClkMinFreq(), rcvar.getClkMinFreq());
        double      wcmaxfreq = wcvar.getClkFreq();
        int         rcycles = (int)(Math.ceil(wcmaxfreq / minfreq)) * 4;
        Net[]       srl16in = new Net[1];
        Net[]       srl16out = null;
        ArrayList   srl16init = new ArrayList();
        srl16in[0] = r3;

        for (int i=0 ; i<rcycles ; i++)
            srl16init.add("1");
        srl16init.add("0");

        if (r != null)
            srl16out = del(rcycles+1, srl16in, null, cw, OR(r, r3), null, srl16init);
        else
            srl16out = del(rcycles+1, srl16in, null, cw, r, null, srl16init);
        srl16out[0].connect(r3);

        if (awidth < 9) // should have a variable for this
            awidth = 9; // minimum block RAM address width (depth 512)
```

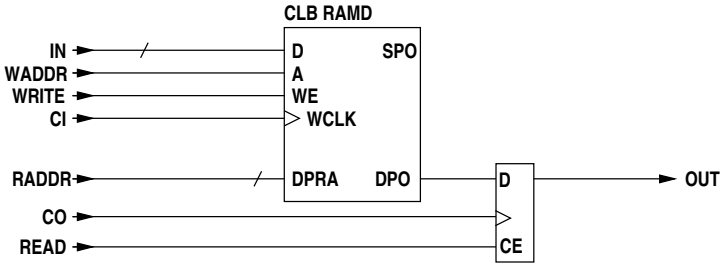
```
42
43          int bdwidth = ramb_dwidths[awidth];
44          int nb = (dwidth - 1) / bdwidth + 1;
45          int totwidth = nb * bdwidth;
46
47          if ((count != null) || (free != null)) {
48              rdcount = netArray(awidth);
49              wrcount = netArray(awidth);
50          }
51
52          for (int i=0 ; i<nb ; i++) {
53              int l = i * bdwidth;
54              int u = l + bdwidth - 1;
55              if (u >= dwidth)
56                  u = dwidth - 1;
57              Net[]   id = (in != null) ? subarray(in, l, u) : null;
58              Net[]   od = (out != null) ? subarray(out, l, u) : null;
59
60              if (i == 0)
61                  FIFO(   depth, id, push, pop, od, empty, full, rdcount, wrcount,
62                          r3, cw, cr  );
63              else
64                  FIFO(   depth, id, push, pop, od, null, null, null, null,
65                          r3, cw, cr  );
66          }
67
68          // count and free only used for synchronous queues
69          if (count != null) {
70              Net[] xx = sub(wrcount, rdcount, null, false, false);
71              for (int i=0 ; i<(count.length-1) ; i++)
72                  count[i].connect(xx[i]);
73              count[count.length-1].connect(full);
74          }
75          if (free != null) {
76              Net[] yy = sub(rdcount, wrcount, null, false, false);
77              for (int i=0 ; i<(free.length-1) ; i++)
78                  free[i].connect(yy[i]);
79              free[free.length-1].connect(full);
80          }
81      }
```

Net **r3** is the FIFO reset. This must be high for at least 3 clock cycles of both the read and the write clock. A reset must be done after power-up so is done after configuration. **rcycles** is the number of cycles of the write clock **cw** necessary for at least 3 cycles of both clocks. The reset pulse is generated using one or more CLB shift registers. The initial condition is 01...1, so the output is initially 1. The output is connected to the CE and the data input so the contents will shift until the 0 is at the output at which point it will stop. The external reset **r** is ORed in to the CE to start the shift sequence of **rcycles** 1s and a 0 each time.

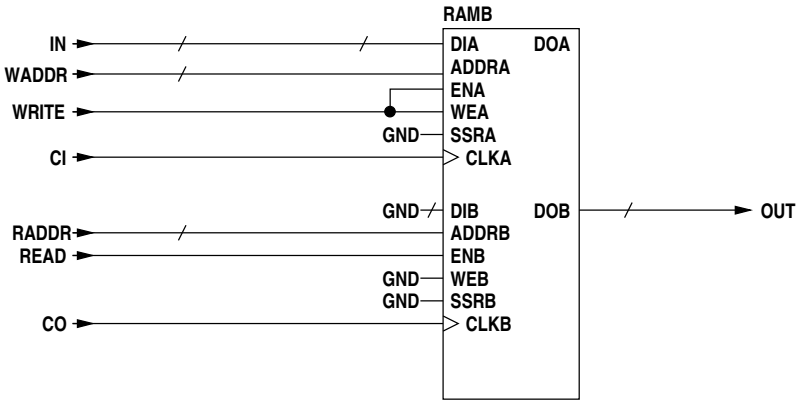
async_buffer()

For an asynchronous queue buffer the input and output clocks differ and the scheme for avoiding an initial read on a write to an empty queue buffer cannot be used. The point at which a queue becomes available after a write to an empty buffer is now undefined since the clocks are separate and unrelated.

The CLB RAM block now simply has a register on the output of the read-only port.

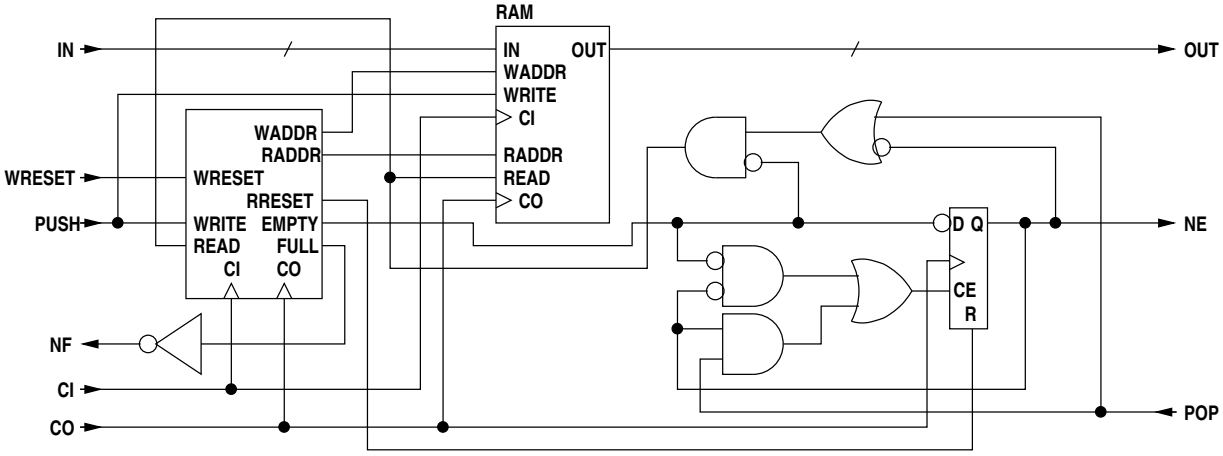


The block RAM has no additional logic.



In method $\mathbf{queuemem()}$ described previously the boolean $\mathbf{is\_sync}$ selects whether this extra logic is inserted or not.

The asynchronous queue buffer uses one of the above RAMs (the former for depth 16, the latter for greater depths) and an address generator block. The use of address comparison for full and empty determination means that the depth of the RAM FIFO is $2^n - 1$ however an additional datum is held in the RAM output register giving a total buffer depth of 2^n. The additional logic shown provides the first read following a write to an empty buffer after which $\mathbf{NE}$ is raised.



The code for this is -

```
    public Net[] async_buffer (
        Net[]   in,     // the data input
        Net     push,   // push-to-queue input
        Net     pop,    // pop-from-queue input
        Net     ci,     // input clock
        Net     co,     // output clock
        Net     ne,     // returned not empty
        Net     nf,     // returned not full
        Net[]   wstat,  // returned status synchronised to the read clock
        Net[]   rstat,  // returned status synchronised to the write clock
        Net     r,      // reset input synchronised to the write clock or null
        Net     rr,     // reset output synchronised to the read clock or null
        boolean fifo,   // use FIFO hardware if present
        int     depth,  // buffer depth
        Var     wcvar,  // write clock variable
        Var     rcvar   // read clock variable
    ) {
        Net     full = new Net("FULL");
        Net     empty = new Net("EMPTY");
        int     dwidth = in.length;

        if (((this instanceof XC5V) || (this instanceof XC6V)) && (depth >= 512) && (depth <= 8192) &&
            (wstat == null) && (rstat == null) && fifo) {
            // For Virtex5 or Virtex6 block RAM FIFO can use inbuilt FIFO logic.
```

```
25              if (rr != null) {
26                  if (r != null)
27                      rr.connect(r);
28                  else
29                      rr.connect(Net.LO);
30              }
31              ne.connect(INV(empty));
32              nf.connect(INV(full));
33              Net[]   out = new Net[dwidth];
34              for (int i=0 ; i<dwidth ; i++)
35                  out[i] = new Net("out" + i + "_");
36              queuefifo(depth, in, push, pop, out, empty, full, null, null, r, ci, co, wcvar, rcvar);
37              return(out);
38          }
39
40          int     awidth = bits(depth-1, false);
41          Net[]   waddr = new Net[awidth];
42          Net[]   raddr = new Net[awidth];
43          Net     read;
44          Net     write = push;
45          Net[]   out;
46          Net     oack;
47
48          if ((rr == null) && (r != null))
49              rr = new Net("rr");
50
51          nf.connect(INV(full));
52
53          for (int i=0 ; i<awidth ; i++) {
54              waddr[i] = new Net("WADDR" + i + "_");
55              raddr[i] = new Net("RADDR" + i + "_");
56          }
57
58          oack = OR(AND(ne, pop), AND(INV(ne), INV(empty)));
59
60          asynch_fifo_control (
61              write, ne, pop, ci, co, r,                      // input nets
62              waddr, raddr, rstat, wstat, full, empty, rr,    // output nets
63              awidth, depth
64          );
65
66          ne.connect(FDRS(INV(empty), co, oack, rr, null, null, null));
67
68          read = AND(OR(INV(ne), pop), INV(empty));
69          out = queuemem(depth, dwidth, awidth, in, raddr, waddr, read, write, null, ci, co);
70          return(out);
71      }
```

If the FPGA is Virtex5 or Virtex6 and the buffer depth is between 512 and 8192 inclusive and the write status (**wstat**) and read status arguments (**rstat**) are null and parameter **fifo** is true then method **queuefifo()** is called to use the FIFO logic associated with block RAMs (the read and write status cannot be derived from that logic).

As in the schematic diagram above the two main blocks are memory (method **queuemem()**) and asynchronous control logic (method **asynch_fifo_control()**).

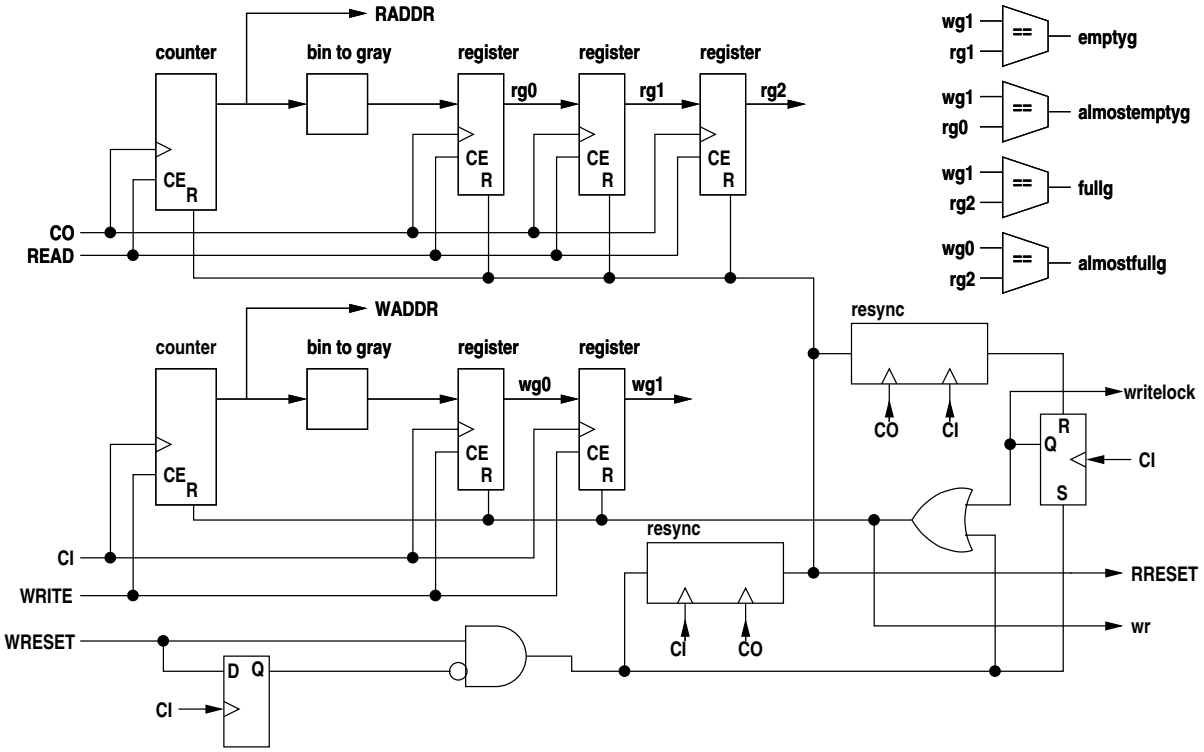
Resetting an asynchronous queue buffer must be carried out with care. A pending read must be allowed to complete after which **NE** will go low and no more reads will occur (since the queue buffer is now empty). A pending write must be ignored and **NF** forced low (the source will now remove **PUSH**, withdrawing the write request). **NF** must remain low during the reset process (which may be many cycles if the input clock is higher than the output clock) and then go high when the now empty queue buffer is again available for writing.

The reset input is synchronised to the write (input) clock. When asserted it resets the write address and clears the full flip-flop (and holds it clear during the reset process). At the same time it sets a blocking flip-flop to force **NF** low so that a write will not occur (the reset also immediately forces **NF** low to supress a possible write in the current cycle). The reset is resynchronised to the read (output) clock to later reset the read address, the empty flip-flop and the **NE** flag. A further pulse is then generated once again in the input clock domain to finally free the blocking flip-flop and raise **NF** after the output side

has been reset. This mechanism is necessary to avoid attempts to write to the buffer before the output side is reset. On the other hand the output side reset must be synchronised to the output clock so that it can fit in with pending reads, lowering **NE** synchronously with the output to stop further reads.

If the queue buffer is never reset the reset logic is omitted.

The address generator follows the design given in Xilinx application note 131.



When **WRESET** is asserted it resets the write address counter to zero and the following grey code registers to the initial values -

wg0 100...000
wg1 100...001

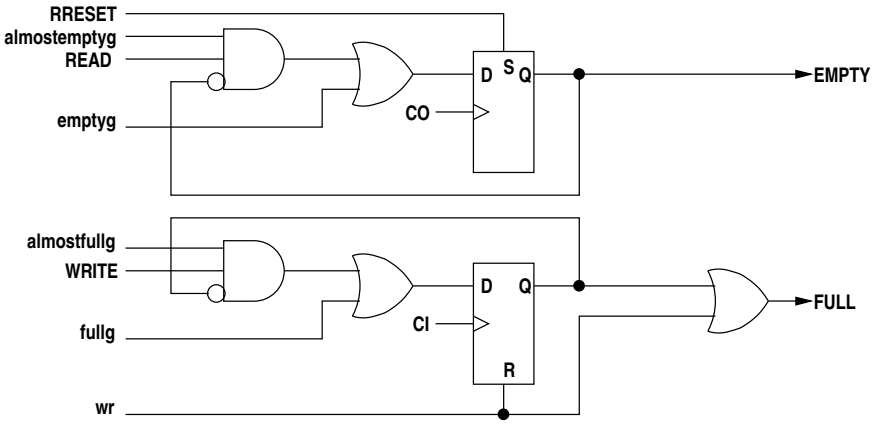
At the same time it is gated through to **FULL** and also latched in a flip-flop so that the queuebuffer appears full and will not be written on the current or subsequent clock cycles. The write reset is resynchronised to the read clock and the resulting signal is output to **RRESET** for possible daisy chaining to further queues in the pipeline. **RRESET** also resets the **EMPTY** flip-flop, resets the read address counter to zero and the following grey code registers to the initial values -

rg0 100...000
rg1 100...001
rg2 100...011

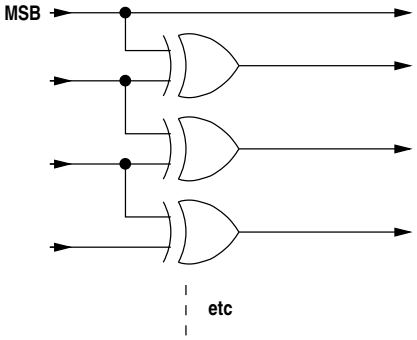
These initial values on the write and read sides ensure the queue buffer appears initially empty and empty after a reset. The values are the gray code sequence leading up to the current read and write addresses of 0 (gray code 0) and hence are gray code equivalents of -1, -2 and -3.

RRESET is then resynchronised back to the write clock and resets the flip-flop holding **FULL** asserted. The reset process thus takes a number of read and write clock cycles, the exact duration depending on the two clock frequencies. The reset inhibits any writes on the current clock cycle and from then on until the reset is complete. The queue buffer does not appear empty immediately the reset is asserted on the write side, however only one read can take place (if the queue buffer is not already empty) and that will read a correct datum. After that the queue buffer will be reset on the read side and will appear empty. The queue buffer can then only appear non-empty after a subsequent write following the reset.

The gray code address comparisons at various phases are compared to determine empty and full conditions -



The binary to gray code converter logic is -



which is coded as -

```
    private Net[] bin_to_gray (Net[] in) {
        int     size = in.length;
        Net[]   out = new Net[size];

        out[size-1] = in[size-1];
        for (int i=size-2 ; i>=0 ; i--)
            out[i] = XOR(in[i+1], in[i]);
        return(out);
}
```

The asynchronous control logic in method asynch_fifo_control() is coded as -

```
    private void asynch_fifo_control (
        Net write, Net ne, Net pop, Net wc, Net rc, Net r,      // input nets
        Net[] waddr, Net[] raddr,                               // output nets
        Net[] rstat, Net[] wstat, Net full, Net empty, Net rr,  // output nets
        int awidth, int depth
        // write is the write enable
        // ne is true if the asynchronous FIFO is not empty
        // pop is true to pop a datum
        // wc is the write clock
        // rc is the read clock
        // r is a reset synchronised to the write clock or null
        // waddr is the write address
        // raddr is the read address
        // rstat is the read buffer status
        // wstat is the write buffer status
        // full is the net to return the full condition
        // empty is the net to return the empty condition
        // rr is a reset synchronised to the read clock
        // awidth is the address width
        // depth is the FIFO depth
    ) {
        Net[]   wg0, wg1, rg0, rg1, rg2;
        Net     emptyg, almostemptyg, fullg, almostfullg;
        Net     wr = null;  // extended write reset
        Net     writelock = null;
```

```
        Net     read;
        // Initial gray code values of pipelined read and write address registers.
        // These are address sequences leading up to the initial read and
        // write addresses of 0, hence they are gray code values of -1, -2 and -3.
        String  igm1 = intToGrayString(-1, awidth);
        String  igm2 = intToGrayString(-2, awidth);
        String  igm3 = intToGrayString(-3, awidth);

        if ((r != null) && r.isConnected(Net.LO))
            r = null;
        if (r != null) {
            r = AND(r, INV(FDRS(r, wc, null, null, null, null, null)));
            rr.connect(resync(r, wc, rc));
            writelock = FDRS(null, wc, null, resync(rr, rc, wc), r, "R", null);
            wr = OR(r, writelock);
        } else if (rr != null)
            rr.connect(Net.LO);

        read = AND(OR(INV(ne), pop), INV(empty));

        connect(raddr, cb(awidth, false, rc, read, rr, null, null));
        rg0 = reg(bin_to_gray(raddr), rc, read, rr, igm1, null);
        rg1 = reg(rg0, rc, read, rr, igm2, null);
        rg2 = reg(rg1, rc, read, rr, igm3, null);

        connect(waddr, cb(awidth, false, wc, write, wr, null, null));
        wg0 = reg(bin_to_gray(waddr), wc, write, wr, igm1, null);
        wg1 = reg(wg0, wc, write, wr, igm2, null);

        emptyg       = eq(wg1, rg1, false, false);
        almostemptyg = eq(wg1, rg0, false, false);
        fullg        = eq(wg1, rg2, false, false);
        almostfullg  = eq(wg0, rg2, false, false);
        Net empty_d;
        if (lutwidth >= 5)
            empty_d = OR(AND(OR(INV(ne), pop), INV(empty), almostemptyg), emptyg);
        else {
            Net n = AND(OR(INV(ne), pop), INV(empty), almostemptyg);
            empty_d = MUXF5(emptyg, n, Net.HI);
        }
        empty.connect(FDRS(empty_d, rc,  null, null, rr, "S", null));
        if (r != null) {
            Net full_;
            full_ = FDRS(OR(fullg, AND(almostfullg, write, INV(full))), wc, null, wr, null, "R", null);
            full.connect(OR(full_, wr));
        } else
            full.connect(FDRS(OR(fullg, AND(almostfullg, write, INV(full))), wc, null, null, null, "R", null));

        if ((rstat != null) || (wstat != null)) {
            Net[]   rgtop, wgtop;
            rgtop = new Net[2];
            wgtop = new Net[2];
            rgtop[0] = rg1[awidth-2];
            rgtop[1] = rg1[awidth-1];
            wgtop[0] = wg1[awidth-2];
            wgtop[1] = wg1[awidth-1];
            if (rstat != null)
                asynch_fifo_control_status(rstat, rgtop, wgtop, rc, rr);
            if (wstat != null)
                asynch_fifo_control_status(wstat, rgtop, wgtop, wc, wr);
        }
    }
```

Method **asynch_fifo_control_status()**, which is called if the read status (**rstat**) or write status (**wstat**) arguments are non-null, generates logic to return the buffer occupancy status. This status gives an approximate view, the asynchronous nature of the queue buffer precluding actual occupancy counts.

```
    private void asynch_fifo_control_status (Net[] stat, Net[] rgtop, Net[] wgtop, Net c, Net r) {
        Net n, n1, n2, n3, n4;
        // Empty to 1/4 Full - quads equal
        n = AND(eq(rgtop, wgtop, false, false), INV(OR(stat[3], stat[4])));
        stat[0].connect(FDRS(n, c,  null, null, r, "S", null));
        // 1 word to 1/2 Full - quad 1 ahead
```

```
7         n1 = AND(decode(rgtop, 0), decode(wgtop, 1));
8         n2 = AND(decode(rgtop, 1), decode(wgtop, 3));
9         n3 = AND(decode(rgtop, 3), decode(wgtop, 2));
10        n4 = AND(decode(rgtop, 2), decode(wgtop, 0));
11        n = OR(n1, n2, n3, n4);
12        stat[1].connect(FDRS(n, c, null, r, null, "R", null));
13        // 1/4 full to 3/4 Full - quad 2 ahead
14        n1 = AND(decode(rgtop, 0), decode(wgtop, 3));
15        n2 = AND(decode(rgtop, 1), decode(wgtop, 2));
16        n3 = AND(decode(rgtop, 3), decode(wgtop, 0));
17        n4 = AND(decode(rgtop, 2), decode(wgtop, 1));
18        n = OR(n1, n2, n3, n4);
19        stat[2].connect(FDRS(n, c, null, r, null, "R", null));
20        // 1/2 Full to Full - quad 3 ahead
21        n1 = AND(decode(rgtop, 0), decode(wgtop, 2));
22        n2 = AND(decode(rgtop, 1), decode(wgtop, 0));
23        n3 = AND(decode(rgtop, 3), decode(wgtop, 1));
24        n4 = AND(decode(rgtop, 2), decode(wgtop, 3));
25        n = OR(n1, n2, n3, n4);
26        stat[3].connect(FDRS(n, c, null, r, null, "R", null));
27        // 3/4 Full to Full - quad 4 ahead/equal
28        n = AND(eq(rgtop, wgtop, false, false), OR(stat[3], stat[4]));
29        stat[4].connect(FDRS(n, c, null, r, null, "R", null));
30    }
```

triggerbuffer()

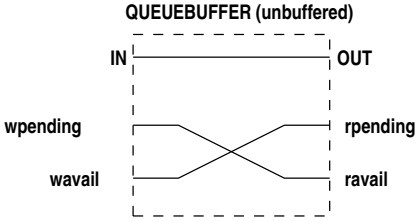
This method creates a synchronous or asynchronous queue buffer that has only the control logic and no actual data buffer. The code is just the control logic taken from methods **sync_buffer()** and **async_buffer()**.

queueunbuffered()

A synchronous unbuffered queue is not really a queue and hence the control inputs and outputs are labelled differently to a buffered queue -

buffered	unbuffered
NE	ravail
NF	wavail
PUSH	wpending
POP	rpending

A synchronous unbuffered queue is created by connecting the **ravail** output net to the **wpending** input net and the **wavail** output net to the **rpending** input net. With these cross connections asserting the **wpending** input asserts the **ravail** output and the queue now has read availability. In the same way asserting the **rpending** input asserts the **wavail** output and the queue now exhibits write availability. These cross connections are valid because wpending implies a write is ready to proceed and that a read can now be made. Similarly rpending implies that read is ready to proceed and hence a write can now be executed. The rpending and wpending logic cannot be driven from the **START_DEL** output of two EXECPs as for a buffered queue as these are not asserted until the availability outputs are asserted. This circular logic will deadlock. Instead the **wpending** and **rpending** inputs are driven by the **PENDING** output of EXECPs. This has been described previously (2.8.17). The data output is simply connected to the data input as the unbuffered queue does not store anything.



```
    ravail.connect(wpending);
    wavail.connect(rpending);
    return(in);
```

Asynchronous unbuffered queues are not implemented.

queuefifo()

For synchronous queues created by **sync_buffer()** or asynchronous queues created by **async_buffer()**, if the FPGA is Virtex 5, Virtex 6 or system7 and the depth is less than 8192 and the fifo attribute has been set on the queue, then methid **queuefifo()** will be called. This will in turn call **fifallocate()** in the FPGA class. This in turn will construct a list of **FifoBlock** instances to the required width and then call method FIFO() in class **FifoBlock** on each to construct the block RAM FIFOs.

4.9.4 delay line

```
    public Net[] del (
        int         n,      // line length
        Net[]       in,     // input net array
        Net[]       len,    // optional variable length selection input
        Net         c,      // clock
        Net         ce,     // optional clock enable
        Net         r,      // optional reset
        ArrayList   init    // initial delay line contents as a list of
                            // hexadecimal strings
    ) {
        if ((n == 0) && (len == null))
            return(in);

        int             blklen = 1 << srladdrwidth;
        int             width = in.length;
        Net[]           out = new Net[width];
        boolean         flip_flops = (n == 1) || ((r != null) && (!r.isConnected(Net.LO)));
        boolean         varlen = (len != null);
        StringBuffer    binit = null;
        int             del_len;

        if (varlen)
            n = blklen;

        for (int k=0 ; k<width ; k++) {
            // if delay of only 1 use flip-flop
            // if reset input is used, must use flip-flops
            if (flip_flops) {
                Net[]   s = new Net[n+1];
                s[0] = in[k];
                for (int i=0 ; i<n ; i++)
                    if ((init == null) || strhexbit((String)init.get(i), k) == 0)
                        s[i+1] = FDRS(s[i], c, ce, r, null, "R", null);
                    else
                        s[i+1] = FDRS(s[i], c, ce, null, r, "S", null);
                out[k] = s[n];
                continue;
            }

            // use delay lines
            String sinit;
            if (init != null) {
                binit = new StringBuffer();
                for (int i=0 ; i<n ; i++) {
                    int b = strhexbit((String)init.get(i), k);
                    binit.insert(0, (b == 0) ? "0" : "1");
                }
                sinit = binToHex(binit);
            } else
                sinit = "0";
```

```
51              if (varlen) {
52                  // variable length
53                  switch (srladdrwidth) {
54                  case 4:
55                      out[k] = SRL16E(in[k], c, ce, len, sinit, null);
56                      break;
57                  case 5:
58                      out[k] = SRL32E(in[k], c, ce, len, sinit, false, null);
59                  }
60              } else {
61                  // Constant length.
62                  // Insert a flip-flop at the output to improve timing,
63                  // so use n-1 as length.
64                  int     blocks = (n - 2) / blklen + 1;
65                  Net[]   s = new Net[blocks+1];
66                  boolean q31;
67                  int     ffinit = 0;
68                  if ((binit != null) && (binit.charAt(binit.length() - n) == '1')) {
69                       // remove 1 bit - used with flip-flop
70                      ffinit = 1;
71                      binit.deleteCharAt(binit.length() - n);
72                  }
73                  len = new Net[srladdrwidth];
74                  s[0] = in[k];
75                  for (int i=0, count=n-1 ; i<blocks ; i++, count-=blklen) {
76                      int m = (count < blklen) ? count-1 : blklen-1;
77                      for (int j=0 ; j<srladdrwidth ; j++)
78                          len[j] = (((m >> j) & 1) == 1) ? Net.HI : Net.LO;
79                      if (binit != null)
80                          sinit = binToHex(binit);
81                      switch (srladdrwidth) {
82                      case 4:
83                          s[i+1] = SRL16E(s[i], c, ce, len, stringsize(sinit, srladdrwidth), null);
84                          break;
85                      case 5:
86                          q31 = (i != (blocks - 1));
87                          s[i+1] = SRL32E(s[i], c, ce, len, stringsize(sinit, srladdrwidth), q31, null);
88                      }
89                      del_len = 1 << srladdrwidth;
90                      if (binit != null) {
91                          if (binit.length() >= del_len)
92                              binit.delete(binit.length() - del_len, binit.length());
93                          else
94                              binit.delete(0, binit.length());
95                      }
96                  }
97
98                  if ((init == null) || ffinit == 0)
99                      out[k] = FDRS(s[blocks], c, ce, r, null, "R", null);
100                 else
101                     out[k] = FDRS(s[blocks], c, ce, null, r, "S", null);
102             }
103         }
104         return(out);
105     }
```

A delay line can be fixed or variable length and may be initialised. An initialised delay line with feedback can be used to generate a sequence. The parameter **init** is a list of hexadecimal strings, one per delay line stage, i.e. each string is a value across the width of the delay line.

Variable **blklen** is the length of an SRL shift register (16 or 32 depending on FPGA). If the delay line is to have variable length (**varlen** is true) it is limited to a maximum of blklen (line 21).

Variable **flip_flops** is true if the delay line length is 1 or if the reset **r** is non-null and is not tied false. If true, flip-flops will be used to construct the delay line rather than CLB shift registers (shift registers cannot be reset).

The outer loop starting at line 22 takes each bit of the data width in turn.

If **flip_flops** is true (line 25) flip-flops are used for the delay line. The inner loop starting at line 28 iterates along the length of the delay line creating an FDRS for each bit. The FDRS is initialised by

extracting the matching bit from the initialisation string and either the **R** or **S** input is connected to net **r** as appropriate. On completion of the inner loop the continue statement skips to line 98 to continue the outer loop.

Where shift CLB registers are used (**flip_flops** is false) the initialisation data must be transposed as the SRL registers run lengthwise. This is done into variable **sinit**. If the shift register is variable length then a single SRL16 or SRL32 is created. The net array **len** is the address input to select the length of the line dynamically.

If the length of the line is fixed then SRL16 or SRL32 elements are concatenated in the loop starting at line 71 to get the required line length. The data inputs, data outputs, line length and initialisation data are all segmented to match the concatenated elements. Note that the final unit of the delay line is implemented using flip-flops. This is done because the output-to-clock delay of an SRL16 or SRL32 is quite long and this can cause problems with timing constraints. The addition of a trailing flip-flop alleviates this problem. Note that a variable length line cannot have trailing flip-flops (unless a minimum delay of 2 were acceptable) and hence may experience timing constraint problems.

4.9.5 shifters

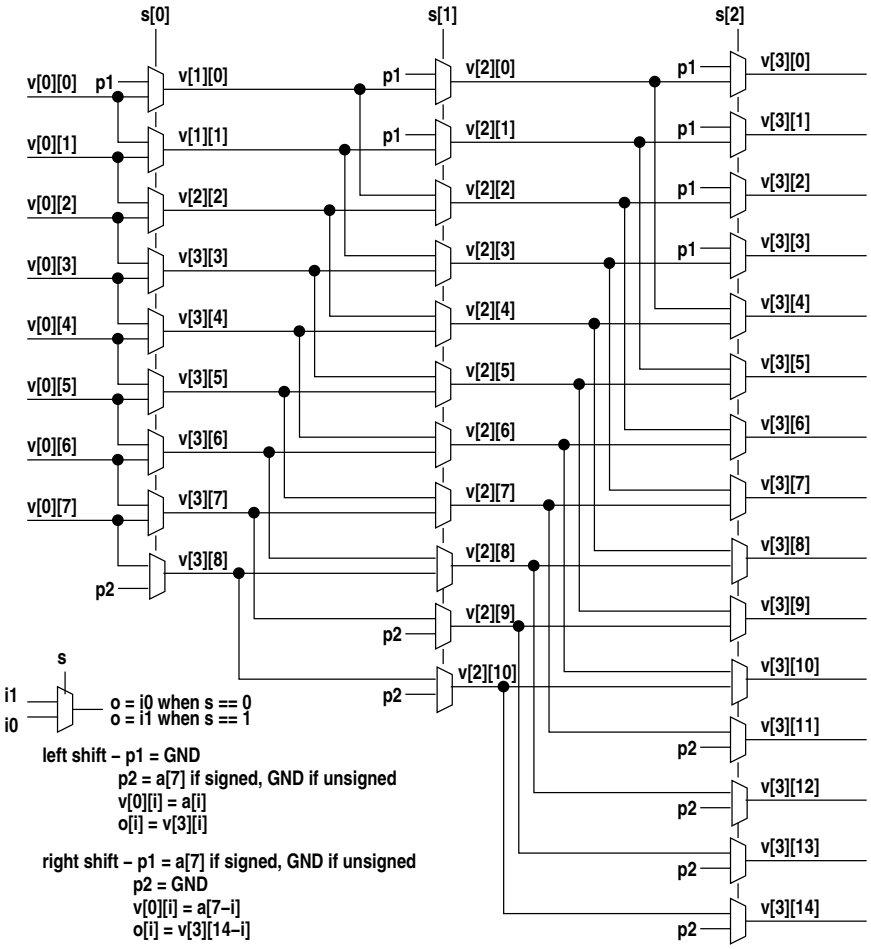
Left and right fixed shift is performed by methods **lshift()** and **rshift()**. These generate no logic and simply connect the output net array to the input net array with a displacement. Padding to GND is done to outputs shifted off-end except in the case of signed most significant bits which are connected to the input MSB (sign bit).

Variable shift is performed by method **var_shift3pl()**.

```
1     public Net[] var_shift3pl (
2         Net[]   a,      // input argument - signed or unsigned
3         Net[]   s,      // shift argument - unsigned
4         int     owidth, // output bit width
5         boolean right,  // true for right shift, false for left shift
6         boolean signeda // true if the input is signed
7     ) {
8         int     iwidth = a.length;
9         int     shiftsize = s.length;
10        int     shiftbits = (1 << shiftsize) - 1;
11        long    ssize = Math.min(shiftbits, owidth);
12        int     shiftdim = bits(ssize, false);
13        Net[]   shift = new Net[shiftdim];
14        int     stages = shiftdim;
15        Net[][] v = new Net[stages+1][owidth];
16        Net     p1, p2;
17        Net[]   o = new Net[owidth];
18        int     i, j, k, l, l1, l2;
19
20        if (right) {
21            if (signeda)
22                p1 = a[iwidth-1];
23            else
24                p1 = Net.LO;
25            p2 = Net.LO;
26
27        } else {
28            if (signeda)
29                p2 = a[iwidth-1];
30            else
31                p2 = Net.LO;
32            p1 = Net.LO;
33        }
34
35        // prune the shift array to match the output width if too big
36        shift = subarray(s, 0, shiftdim-1);
37
38        for (i=0 ; i<iwidth ; i++)
39            if (right)
40                // reverse input bit order for right shift
41                v[0][i] = a[iwidth-i-1];
```

```
42              else
43                  v[0][i] = a[i];
44
45          k = iwidth;
46          for (j=1 ; j<=stages ; j++) {
47              k = Math.min(owidth, iwidth + (1 << j) - 1);
48              l = iwidth + (1 << (j-1)) - 1;
49              for (i=0 ; i<k ; i++) {
50                  l1 = i - (1 << ( j - 1));
51                  l2 = i;
52                  if (l1 < 0)
53                      v[j][i] = mux2(shift[j-1], v[j-1][l2], p1);
54                  else if (l2 >= l)
55                      v[j][i] = mux2(shift[j-1], p2, v[j-1][l1]);
56                  else
57                      v[j][i] = mux2(shift[j-1], v[j-1][l2], v[j-1][l1]);
58              }
59          }
60
61          for (i=0 ; i<owidth ; i++)
62              if (right)
63                  // reverse output bit order for right shift
64                  o[owidth-i-1] = v[stages][i];
65              else
66                  o[i] = v[stages][i];
67          return(o);
68      }
```

This constructs a variable left shift and simply reverses both input and output arrays for a right shift. The shift is unsigned so it cannot be negative. The shift is accomplished by successive stages of multiplexers which select either unshifted bits or bits shifted by a power of 2. Each stage is controlled by one bit of the shift value. Note that the extra width of the output array (over that of the input array) is handled differently depending on whether the shift is left or right. The output width is determined in constructor nodes.ArgParams() called from nodes.ExprNode.getVal() and depends on the input type. For a signed or unsigned integer the width is expanded for left shift but not for right shift. Thus for right shift the bits shifted off end are lost. For fixed point types right shift bits are retained within the fractional part of the word but are lost beyond that. Where the output array is smaller than the output array of the shifter, $\mathbf{v[stages][]}$, the output array is assigned at the lower subscript end and the higher subscript bits are not connected. The resulting logic is illustrated by the following diagram for the case of 8-bit input and a 3-bit shift value -



4.9.6 adder

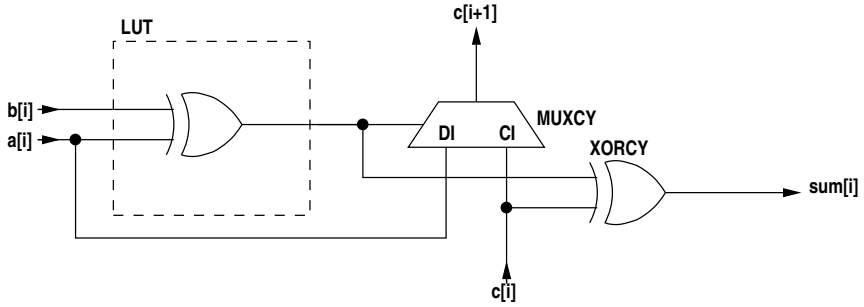
```
1     public Net[] add (Net[] a, Net[] b, boolean asigned, boolean bsigned, int osize) {
2         int     awid = a.length + ((!asigned && bsigned) ? 1 : 0);
3         int     bwid = b.length + ((asigned && !bsigned) ? 1 : 0);
4         int     isize = Math.max(awid, bwid);
5
6         if (osize == 0)
7             osize = isize + 1;
8
9         Net[]   ps = new Net[osize];    // partial sum
10        Net[]   s = new Net[osize];     // sum
11        Net[]   c = new Net[osize];     // carries
12
13        a = sig_expand(a, osize, asigned);
14        b = sig_expand(b, osize, bsigned);
15
16        int i = 0;
17
18        // Starting at the LSB, while either operand is GND simply copy
19        // the other operand to the output. When neither is GND, exit
20        // the loop and proceed to normal addition logic for the rest of the
21        // bits.
22        for ( ; i<osize ; i++) {
23            if (a[i].isConnected(Net.LO))
24                s[i] = b[i];
25            else if (b[i].isConnected(Net.LO))
26                s[i] = a[i];
27            else
28                break;
29        }
30        if (i < osize)
```

```
31              c[i] = Net.LO;  // start carry chain if adder still needed
32
33          // Adder stages (if required; i < size).
34          int ilast = osize - 1;
35          if (!needCPLDarith()) {
36              // FPGAs
37              for ( ; i<osize ; i++) {
38                  if (add_sub_use_lut)
39                      ps[i] = LUT("I0^I1", a[i], b[i]);
40                  else
41                      ps[i] = XOR(a[i], b[i]);
42                  s[i] = XORCY(ps[i], c[i]);
43                  if (i != ilast)
44                      c[i+1] = MUXCY(ps[i], a[i], c[i]);
45              }
46          } else {
47              // CPLDs
48              for ( ; i<osize ; i++) {
49                  if (cpld_use_lut)
50                      s[i] = LUT("I0^I1^I2", a[i], b[i], c[i]);
51                  else
52                      s[i] = OR(  AND(    a[i],  INV(b[i]), INV(c[i])),
53                                  AND(INV(a[i]),     b[i],  INV(c[i])),
54                                  AND(INV(a[i]), INV(b[i]),     c[i]),
55                                  AND(    a[i],      b[i],      c[i])   );
56                  if (i != ilast) {
57                      if (cpld_use_lut)
58                          c[i+1] = LUT("I0*I1*!I2+I0*!I1*I2+!I0*I1*I2+I0*I1*I2",
59                                                          a[i], b[i], c[i]);
60                      else
61                          c[i+1] = OR(    AND(    a[i],      b[i], INV(c[i])),
62                                          AND(    a[i],  INV(b[i]),    c[i]),
63                                          AND(INV(a[i]),     b[i],     c[i]),
64                                          AND(    a[i],      b[i],     c[i])  );
65                  }
66              }
67          }
68
69          return(s);
70      }
```

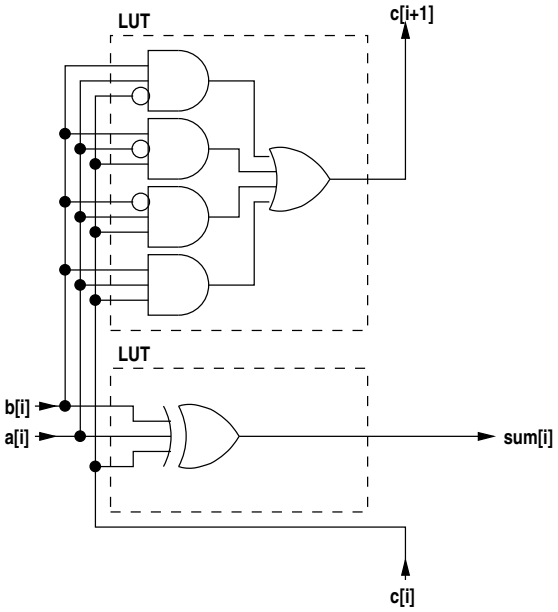
This creates an adder. The input nets may be different sizes. The output net is one larger than the largest input net. The MSB of the result is the final carry bit.

If the bits of one operand starting from the least significant end are a sequence of zeros (GND) then those adder stages are omitted and corresponding bits from the other operand become the sum bits. The adder stages are constructed from this point onwards (note that the for loops do not initialise loop variable **i** which remains at the point the input constant search finished). If the two operands do not have overlapping variable bits then the extra top bit is set appropriately (0 or one of the signs) and no adder stages are constructed (i == size so the following for loops do not execute).

For FPGAs the carry chain is used.



If **needCPLDarith()** returns true then the adder is constructed only from LUT logic because there is no carry chain available.



The variables **add_sub_use_lut** and **cpld_use_lut** are public static variables in ThreePL.jj and select whether the combinatorial logic is expressed as a LUT or a combination of AND, OR, XOR and INV elements. While the boolean logic is identical the ability of the place-and-route tools to optimise the logic may be affected by the choice. By default these variables are true to select encoding as LUTs. The **-G** option to 3PL renders these variables (and many similar ones in other methods) false.

4.9.7 inverter

The code to create a 1's complement inverter is trivial.

4.9.8 incrementer

The incrementer method is only used for signed division (method **part_sdiv()**) and for an up/down counter (**method cb()**). It is derived from the code for method **add()** with the second input **b** omitted and the carry input connected to an **enable** input. Thus when **enable** is low the output is the same as the input **a** and when **enable** is high the output is **a** + 1. The result is the same width as the input, not one greater.

4.9.9 subtracter

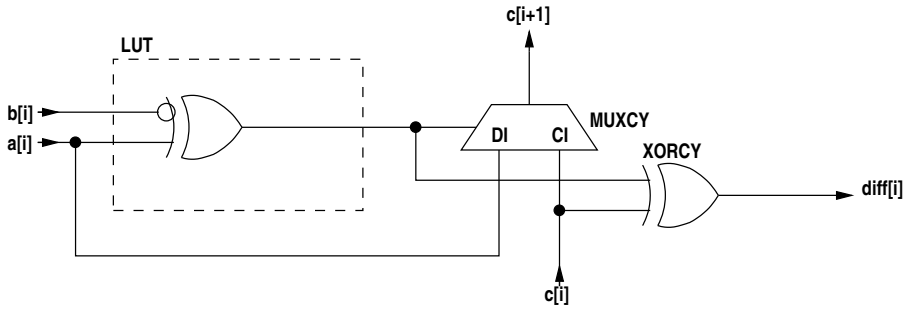
```
1     public Net[] sub (Net[] a, Net[] b, boolean asigned, boolean bsigned, int osize) {
2         int     awid = a.length + ((!asigned && bsigned) ? 1 : 0);
3         int     bwid = b.length + ((asigned && !bsigned) ? 1 : 0);
4         int     isize = Math.max(awid, bwid);
5
6         if (osize == 0)
7             osize = isize + 1;
8
9         Net[]   pd = new Net[osize]; // partial difference
10        Net[]   d = new Net[osize];  // difference
11        Net[]   c = new Net[osize];  // carries
12
13        a = sig_expand(a, osize, asigned);
14        b = sig_expand(b, osize, bsigned);
15
16        int i = 0;
17
```

```
18          // Starting at the LSB, while 2nd operand is GND simply copy
19          // the 1st operand to the output. When 2nd operand is no
20          // longer GND, exit the loop and proceed to normal subtraction
21          // logic for the rest of the bits. 2nd operand should never be
22          // all zeros as the subtraction would have been optimised out
23          // prior to this.
24          for ( ; i<osize ; i++) {
25              if (b[i].isConnected(Net.LO))
26                  d[i] = a[i];
27              else
28                  break;
29          }
30          c[i] = Net.HI;
31
32          int ilast = osize - 1;
33          if (!needCPLDarith()) {
34              // FPGAs
35              for ( ; i<osize ; i++) {
36                  if (add_sub_use_lut)
37                      pd[i] = LUT("I0^!I1", a[i], b[i]);
38                  else
39                      pd[i] = XOR(a[i], INV(b[i]));
40                  d[i] = XORCY(pd[i], c[i]);
41                  if (i != ilast)
42                      c[i+1] = MUXCY(pd[i], a[i], c[i]);
43              }
44          } else {
45              // CPLDs
46              for ( ; i<osize ; i++) {
47                  if (cpld_use_lut)
48                      d[i] = LUT("I0^!I1^I2", a[i], b[i], c[i]);
49                  else
50                      d[i] = OR(  AND(    a[i],      b[i],   INV(c[i])),
51                                  AND(INV(a[i]), INV(b[i]),  INV(c[i])),
52                                  AND(INV(a[i]),     b[i],       c[i]),
53                                  AND(    a[i],  INV(b[i]),      c[i])   );
54                  if (i != ilast)
55                      if (cpld_use_lut)
56                          c[i+1] = LUT("I0*!I1*!I2+!I0*!I1*I2+I0*I1*I2+I0*!I1*I2",
57                                                          a[i], b[i], c[i]);
58                      else
59                          c[i+1] = OR(    AND(    a[i],  INV(b[i]), INV(c[i])),
60                                          AND(    a[i],      b[i],      c[i]),
61                                          AND(INV(a[i]), INV(b[i]),     c[i]),
62                                          AND(    a[i],  INV(b[i]),     c[i])  );
63              }
64          }
65          return(d);
66      }
```

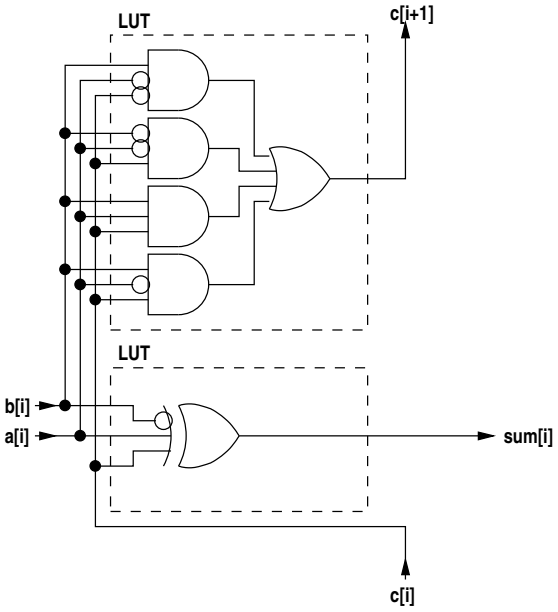
The input operands are expanded to the size of the largest of the two plus one to match the required size of the difference.

If the bits of the second operand starting from the least significant end are a sequence of zeros (GND) then those subtracter stages are omitted and corresponding bits from the other operand become the difference bits. The subtracter stages are constructed from this point onwards (note that the for loops do not initialise loop variable **i** which remains at the point the input constant search finished).

For FPGAs the carry chain is used.



If **needCPLDarith()** returns true then the adder is constructed only from LUT logic because there is no carry chain available.



As in the case of **add()** the variables **add_sub_use_lut** and **cpld_use_lut**, which are public static variables in ThreePL.jj, select whether the combinatorial logic is expressed as a LUT or a combination of AND, OR, XOR and INV elements.

4.9.10 decrementer

The decrementer method is only used for an up/down counter (**method cb()**). It is derived from the code for method **sub()** with the second input **b** omitted and the carry input connected to the inverted **enable** input. Thus when **enable** is low the output is the same as the input **a** and when **enable** is high the output is **a** - 1. The result is the same width as the input, not one greater.

Warning - method **cb()** only calls **decr()** if its sixth parameter **down** is not null and its second parameter **dwn** is true. This does not occur in the code as it currently is and hence **decr()** has never been called and its validity has not been tested!

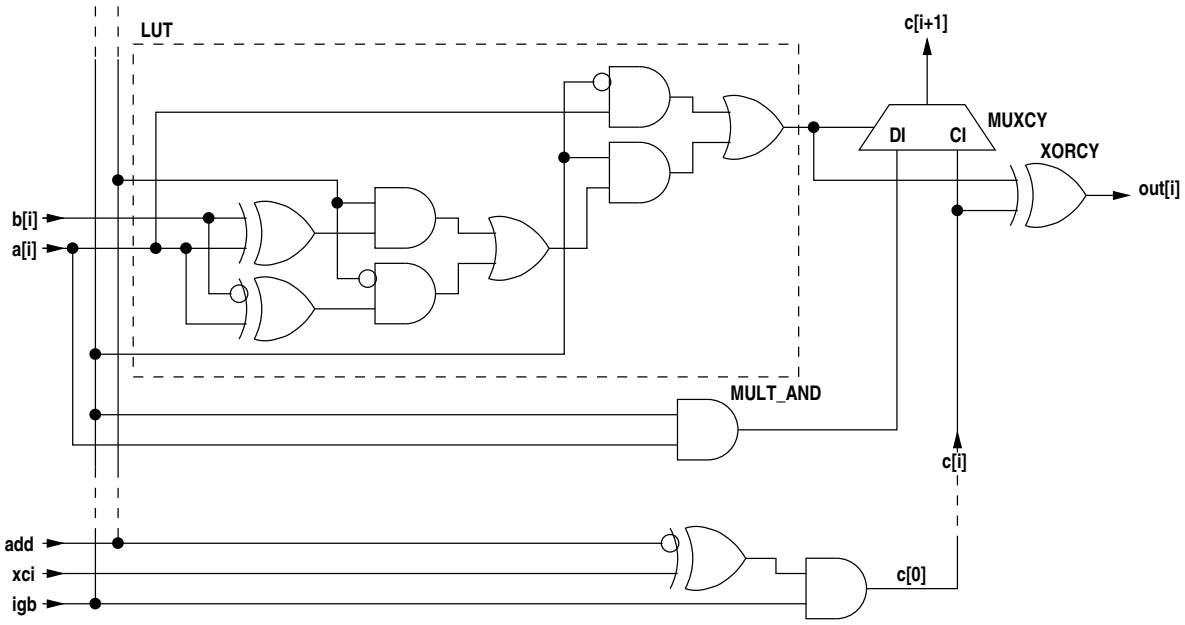
4.9.11 adder/subtracter

This adder/subtracter is used internally in XFunctions.java for the implementation of division. It is also used for the inbuilt function **addsub()** (called in XTDECode.java.TDEOPERATOR()). It provides the ability to gate the second operand so that the add/subtract is supressed and it XORs the carry input with an input Net so that the carry input can be inverted. For addition this will increment the sum with no extra logic.

```
1     public Net[] addsub (Net[] a, Net[] b, Net add, Net igb, Net xci, boolean asigned, boolean bsigned) {
2         if (needCPLDarith())
3                 throw new ExEx("CPLD CODE ERROR - addsub() used");
4
5         int     awid = a.length + ((!asigned && bsigned) ? 1 : 0);
6         int     bwid = b.length + ((asigned && !bsigned) ? 1 : 0);
7         int     size = Math.max(awid, bwid) + 1;
8         Net[]   di = new Net[size];
9         Net[]   psd = new Net[size]; // partial sum/difference
10        Net[]   sd = new Net[size];  // sum/difference
11        Net[]   c = new Net[size];   // carries
12
13        a = sig_expand(a, size, asigned);
```

```
14          b = sig_expand(b, size, bsigned);
15          if (xci.isConnected(Net.LO))
16              c[0] = AND(INV(add), igb);
17          else
18              c[0] = AND(XOR(INV(add), xci), igb);
19          for (int i=0 ; i<size ; i++) {
20              if (igb.isConnected(Net.LO))
21                  psd[i] = a[i];
22              else if (igb.isConnected(Net.HI)) {
23                  if (add.isConnected(Net.LO))
24                      psd[i] = XOR(a[i], INV(b[i]));
25                  else if (add.isConnected(Net.HI))
26                      psd[i] = XOR(a[i], b[i]);
27                  else
28                      psd[i] = OR(AND(add, XOR(a[i], b[i])), AND(INV(add), XOR(a[i], INV(b[i]))));
29              } else {
30                  if (add_sub_use_lut)
31                      psd[i] = LUT("(!I0*I1)+(I0*(I3*(I1^I2)+!I3*(I1^!I2)))", igb, a[i], b[i], add);
32                  else
33                      psd[i] = OR(    AND(INV(igb), a[i]),
34                                      AND(igb, OR(    AND(add, XOR(a[i], b[i])),
35                                                      AND(INV(add), XOR(a[i], INV(b[i]))))));
36              }
37              if (igb.isConnected(Net.LO))
38                  di[i] = Net.LO;
39              else if (igb.isConnected(Net.HI))
40                  di[i] = a[i];
41              else
42                  di[i] = MULT_AND(a[i], igb);
43              if (i != (size-1))
44                  c[i+1] = MUXCY(psd[i], di[i], c[i]);
45              sd[i] = XORCY(psd[i], c[i]);
46          }
47          return(sd);
48      }
```

The implementation does not provide a non-carry-chain version for CPLDs, hence the check on entering the method.



Note that if **igb** is low **a[i]** is passed through the LUT and the XORCY unchanged. The LUT function passes a[i] through to its output and the XORCY has a low from the carry on its other input. The carry chain is low along its length as **c[0]** is forced low and the **DI** inputs to the MUXCYs are forced low.

The **add** input inverts the **b[i]** inputs and also controls the carry input, **c[0]** (low for add, high for subtract). **xci** inverts that **c[0]** input.

The expression represented in lines 37 and 39 and in the logic diagram could be written more concisely

but it does not matter as it would be converted to the same function of four variables in the LUT in either case.

4.9.12 ALU

The **alu()** code is very similar to the **addsub()** with the following exceptions -

- the **xci** input and function is omitted
- it has an **iga** input instead of **igb**, i.e. it gates the **a** input to give output = b when asserted
- there is a **dsp** parameter which if true and the device is a Virtex5 or later will implement the ALU in a DSP block instead of using CLB logic.

The output Net array is one larger than the largest operand.

4.9.13 incrementer/decrementer

This method is only called in one place, method cb() in class XFunctions. It increments or decrements its operand according to the second argument. It is derived from method **addsub()**. The output Net array is the same size as the operand (first argument).

4.9.14 signed and unsigned multipliers

The multiplier method, **mul()**, is divided into four sections depending on FPGA type -

- XC3S, XC2V or XC2VP
- XC4V
- XC5V or XC6V
- others

This method must be called such that if the operands are of different sizes the first must be the larger except in the case where one operand is a constant in which case the constant operand is the second one.

For Spartan 3, Virtex 2 or Virtex 2 Pro the MULT18X18 is used. Up to four multipliers can be combined to give a maximum of 35 by 35 bits signed (See Xilinx application note XAPP467 and Xilinx white paper WP277). Since the MULT18X18 performs signed multiplication, unsigned operands cannot use the top bit, which must be 0 to give a positive sign. For this reason operand size checks are different for signed and unsigned operands.

For the Virtex 4 a MULT18X18 is created but this will later be converted to a DSP48 by the place-and-route software.

For Virtex 5 and Virtex 6 a single MULT25X18 is used. This size was deemed sufficient and larger multipliers using combined MULT25X18 elements have not been coded.

For other types of FPGA (Spartan 2, Virtex, Virtex E) a multiplier is constructed using cascaded adders. This usually results in considerable signal path lengths through combinatorial logic and hence this form of multiplier will usually only work at relatively low clock frequencies. All current FPGA platforms have hardware multipliers and this combinatorial logic is only included for obsolete FPGAs still in use. There is a library, **multlib.3pl**, which implements pipelined multiplies which will operate at high clock frequencies (which of course have multi-cycle latency).

If the operands are mixed signed and unsigned, the unsigned operands has a (zero) sign bit added. If either operand is signed method **muls()** is called, otherwise **mulu()** is called. These methods both call method **mul_add()** for each bit of the second operand. If the second operand is a constant then adders are only created for the 1 bits and the partial product is passed on unchanged for 0 bits. For a constant multiplier with only a few 1 bits the resulting multiplier will have a significantly smaller delay than for the case of a variable multiplier.

4.9.15 signed and unsigned divider/remainder

Division method **div()** and remainder method **rem()** call the combined quotient/remainder method **quotrem()**. **quotrem()** in turn calls **quotrems()** for signed operands or **quotremu()** for usigned operands. **quotrems()** calls recursive method **part_sdiv()** and **quotremu()** calls recursive method **part_udiv()**. These recursive methods construct staged adders and subtracters to carry out the division. These stages are not pipelined and hence the use of division or remainder operations is likely to result in high path delays. These are probably not practical and have been included for completeness.

4.9.16 numerical comparators

Methods **eqones()** and **eqzeros()** test for all ones and all zeros respectively and are implemented using AND/OR gates. For wide inputs these will be optimised using MUXes or carry chains. Methods **eq()** and **ne()** compare operands and are similarly implemented to the methods above.

Magnitude comparisons gt, ge, lt and le are all implemented using only the method **ge()** with suitable operand order and output inversion. This is implemented as subtraction using the carry chain if necessary, the final carry bit giving the result.

4.9.17 combinatorial RAM and ROM

Methods **sram1()**, **sram2()** and **srom1()** build combinatorial RAMS (CLB RAM) of various widths and depths calling methods **RAM_1()**, **RAM_2()** or **ROM_1()** in class XElements().

4.9.18 registered output RAM

Methods **rram1()** and **rram2()** call method **ramallocate()** in the FPGA class. This method creates a list of class RamBlock to map the memory across the required number of blocks and then traverses that list calling RamBlock.RAMB() in class RamBlock to generate the blocks elements.

4.10 The xilinx/XTDECode class

XTDECode extends XFunctions. Its main function is to implement the abstract methods in class TDECode which generate netlist code from TDEs. These methods call methods in class xilinx/XFunctions which in turn call methods in class xilinx/XElements.

Also included in this file are methods to handle NCF (ISE) and XDC (Vivado) constraints.

4.11 The xilinx/RamBlock class

This class provides methods to generate RAM blocks. Since the RAM blocks come in only a few sizes across the FPGA families the code is concentrated here rather than being duplicated in the various FPGA family classes (described earlier).

4.12 The xilinx/FifoBlock class

Thsi class provides methods to generate FIFOs based on RAM blocks (described earlier).

Chapter 5 - Optimisation

Optimisation is carried out in the following places -

- When a TDE is added to the TDE list checks are made on gates and selectors for ground, VCC or duplicated inputs and these are optimised where appropriate. The result may be the elimination of the TDE.
- After completion of immediate execution the complete TDE list is repeatedly traversed optimising both logic functions and control structures.
- The TDE list is then traversed to perform optimisation specific to the target device.
- The TDE list is again repeatedly traversed optimising both logic functions and control structures to remove any redundancies introduced by the target_specific optimisation pass.
- After conversion of the TDE list to an internal netlist structure that structure is repeatedly traversed to optimise the low-level logic. Some of the methods called are generic (in netlist/TDECode.java) and others are overridden by methods in subclasses (in netlist/xilinx/XTDECode.java).

5.1 Optimisation on TDE addition

A TDE is added to the TDE list by calling method codegen.TDEList.addTDE(). This method initially applies method codegen.TDE.redundant() to the TDE and if the return value is true it returns without adding the TDE to the list. Method redundant() checks three types of TDE -

- a SELECT
- an AND, OR or XOR gate TDE
- a parallel WAIT TDE

5.1.1 SELECT TDE optimisation

For a SELECT any GND data inputs, and associated select inputs, are removed if the no_delete flag is not set[1]. If an input is removed the deletedinput class boolean is set true.

Inputs are then scanned to find duplicated data sources and these are eliminated and their select signals ORed.

If the selector now has no inputs the output is connected to GND and the method returns true so that the SELECT TDE is not added to the TDE list.

If the selector now has one data input and -

- no GND inputs have been removed (the deletedinput boolean false)
- the no_delete flag is false

[1] The no_delete flag is set for a queue input where individual components are assigned rather than the whole variable. A queue by default guarantees that partial assignment will result in unassigned components being zeroed. The selector for such a queue cannot be eliminated as at the very least it must be able to gate the queue input to zero when not selected.

- the ALU flag is false[2]

the output is connected to that input and the method returns true so that the SELECT TDE is not added to the TDE list.

The above three conditions reflect cases where the selector must be retained even though it has only one input as it performs a gating function on that single input.

Note that codegen.TDE.redundant() is also called again during TDE list optimisation since inputs may have become connected or grounded in the meantime due to other optimisation. The deletedinput boolean will remain set if any ground inputs are removed during any of these calls so that the selector is not eliminated during later calls regardless of other optimisations.

5.1.2 Gate TDE optimisation

For a gate with only one input the output signal is connected to the input signal and the method returns true.

5.1.3 Parallel WAIT TDE optimisation

For a parallel WAIT with only one input the output signal is connected to the input signal and the method returns true.

5.2 TDE list general optimisation

Method codegen.TDEList.optimise() is called from parser.ThreePL to optimise the completed TDE list.

The intermediate code list is optimised by repeated applications of the following -

1. eliminate suitable CONNECT TDEs by merging their TDEVars. This is only done for -
   - control signals, which are single bit and have no WordSpec.
   - clock signals, again single bit.
   - complete TDEVars, i.e. the WordSpec of each is of a complete variable.

   This is performed by calling TDEVar.merge(). These merges are performed solely to enhance the readability of the optional TDE list file and they do not reduce logic resources or enhance performance.
2. Combine any **EXECP**s with a common start and identical avail signal.
3. Combine any parallel **DEL**s with a common start driving a common **WAIT**.
4. Eliminate any **DEL** in parallel with one or more EXECPs with a common start driving a common WAIT.
5. Eliminate any **WAIT** that has a single input.
6. Eliminate any duplicate **SELECT** inputs.
7. Find any AND or OR gates which have duplicated inputs and remove the duplicates. If this results in a single input remaining, eliminate the gate and connect the output to the input.
8. Find any **INV**s with GND or VCC input and eliminate them, connecting the output to the inverse of the constant input.
9. Find any nested **AND**, **OR** or **XOR** gates and consolidate them.
10. Find any control (not source code) **AND** or **OR** gates whose outputs are not connected and eliminate them.

[2] The ALU flag is true if the selector is an input to a static variable which has the ALU attribute true.

11. Find any **DEL**s whose outputs are not connected and eliminate them.
12. Find any WHENs which have a single **DEL** in both the true block and the false block. Remove these and instead connect a single DEL from the start input to both the true block finish input and the false block finish input. This will result in an OR with identical inputs which will be eliminated by another optimisation iteration.
13. Eliminate any **ILOOP**s which have the same output start signal and input finish signal. These infinite loops are empty, probably because they contain only value equivalence statements.
14. Find any **ILOOP**s which have a single **DEL** from the output start signal to the input finish signal. Replace the pair with a DFF whose SET input is driven by the start signal.
15. Find multiple **DFF**s driven by a common **START** and eliminate all but one.
16. Find any branch statements in which all branches, including the default, have a single DEL. Remove all the DELs and the final associated OR and replace with a single DEL from the start signal.

These optimisations are repeatedly applied to the list until a pass makes no change.

5.3 TDE list device-specific optimisation

This optimisation is done in netlist.xilinx.XTDECode.TDEOptimise() which is called within codegen.TDEList.ncode() which has been called from parser.ThreePL.mainContinue().

At the moment this performs three optimisations -

- It finds any SELECT TDEs which have an ALU flag and optimises them by moving any input ALU operations into a shared single ALU between the selector and its static, queue or selectvalue variable.
- It finds any SELECT TDEs which have an ALU flag, have a single input and have not been optimised out and removes them, connecting the single input to the output.
- For devices which have DSP blocks the ALU logic is implemented in the DSP block instead of using CLB logic where appropriate.

5.4 Netlist optimisation

This optimisation is done in netlist.xilinx.XTDECode.optimise() called from codegen.TDEList.ncode() and consists of five phases -

- optimise gates - calling netlist.xilinx.XTDECode.optimisegates()
- optimise output buffers - calling netlist.xilinx.XTDECode.optimiseOBUF()
- optimise 3-state output buffers - calling netlist.xilinx.XTDECode.optimiseOBUFT()
- remove resulting unconnected FD or INV - calling netlist.xilinx.XTDECode.optimisegates()
- optimise GND and VCC - calling netlist.TDECode.optimiseGNDandVCC()

netlist.xilinx.XTDECode.optimisegates() iterates through all elements selecting those with a gate type, Gtype, which is not Gtype.NONE (defined in interface netlist.NetConstants). According to gate type it selectively calls -

- netlist.TDECode.optimiseINV() - If the output is not connected, delete the inverter. Check for a low or high input. If high delete the gate, connecting the output net to Net.LO. If low delete the gate, connecting the output net to Net.HI. Check for an output which is also an INV (and the output does not go anywhere else), in which case delete both, connecting the input of the 1st to the output of the 2nd. If the intermediate output does go elsewhere, delete the 2nd inverter and connect the signals driven by its output to the input of the 1st inverter.
- netlist.TDECode.optimiseAND() - If the output is not connected, delete the gate. Check for any low or high inputs or duplicated inputs. For high inputs, disconnect their nets. For any low inputs delete the gate connecting the output net to Net.LO. For any duplicated inputs delete all except one. If no inputs remain, delete the gate, connecting the output net to Net.LO. If one input remains, delete the gate, connecting the output net to the remaining input net. Check if the output goes only to another AND gate in which case eliminate this gate and add its inputs to the following gate.

- netlist.TDECode.optimiseOR() - If the output is not connected, delete the gate. Check for any low or high inputs or duplicated inputs. For low inputs, disconnect their nets. For any high inputs delete the gate, connecting the output net to Net.HI. For any duplicated inputs delete all except one. If no inputs remain, delete the gate, connecting the output net to Net.LO. If one input remains, delete the gate, connecting the output net to the remaining input net.
- netlist.TDECode.optimiseXOR() - If the output is not connected, delete the gate. Check for any low inputs. If found delete the inputs, disconnecting their nets. If no inputs remain, delete the gate, connecting the output net to Net.LO. Check for any high inputs. If found delete the inputs, disconnecting their nets, counting them in the process. If there were an odd number of deleted high inputs, convert the gate to XNOR. If one input remains, delete the gate, connecting the output net to the remaining input net.
- netlist.TDECode.optimiseLUT() - Check a LUT for an unconnected output and remove if so.
- netlist.xilinx.XTDECode.optimiseMUXCY() - Check a MUXCY for output not connected, in which case remove it. Check for constant S input, in which case delete the MUXCY and connect the output net to the appropriate input net. Check for inputs CI and DI both constant, in which case output is Net.HI, Net.LO, S or !S. Check for input CI low and DI connected to S, in which case output is Net.LO.
- netlist.xilinx.XTDECode.optimiseMUXF() - Check a MUXF5, MUXF6 etc for constant S input. If so delete the MUXF and connect the output net to the appropriate input net. Check for inputs I0 and I1 both constant, in which case output is Net.HI, Net.LO, S or !S.
- netlist.xilinx.XTDECode.optimiseXORCY() - Check an XORCY for an unconnected output and remove if so. If the CI input is Net.LO, eliminate the XORCY and connect the output net to the LI input. If the CI input is Net.HI, strip the XORCY, change it to an INV and connect the LI input as input I and reconnect the output.
- netlist.xilinx.XTDECode.optimiseSRL() - Check an SRL for an unconnected output and remove if so.
- netlist.xilinx.XTDECode.optimiseMULT_AND() - Check a MULT_AND for an unconnected output and remove if so. If any inputs are Net.LO, remove and connect output to Net.LO. If any inputs are Net.HI, remove the input.
- netlist.xilinx.XTDECode.optimiseCLB_RAM() - Check a dual-port CLB RAM for an unconnected output and remove if so. If data input is Net.LO and RAM is not initialised, remove and connect used outputs to Net.LO.
- netlist.xilinx.XTDECode.optimiseBLK_RAM() - If a data input is Net.LO or unconnected on both ports and the RAM is not initialised, disconnect both associated outputs if connected.
- netlist.xilinx.XTDECode.optimiseMULT() - Optimise a MULT18X18, MULT18X18S, MULT25X18 or MULT25X18S. If no outputs are connected, eliminate the element. If all A inputs are Net.LO or all B inputs are Net.LO, eliminate and connect outputs to Net.LO. If the outputs are connected only to D inputs of FDRSs which all share the same C, CE and R, do not use S and are not initialised, eliminate the FDRSs and convert the element to a MULT18X18S. If the element is a MULT18X18S or MULT25X18S and has A inputs connected to a MULT18X18 or MULT25X18 and B inputs are Net.LO then combine the two into a single MULT18X18S or MULT25X18S.
- netlist.xilinx.XTDECode.optimiseDSP48E() - Optimise a DSP48E which does not use the output register P.
  1. If the outputs are connected only to D inputs of FDRSs which all share the same C, CE and R, do not use S and are not initialised, or
  2. If the outputs are connected only to a MULREG
  eliminate the FDRSs or MULREG and change the DSP48E to use the output register.
- netlist.TDECode.optimiseFDRS() - If the output is not connected, delete the DFF. Check for R or S wired high and initial state to match. If so delete the DFF and connect the output net high or low. Check for D input GND or null, SET input GND or null and initial state low, in which case delete and connect output to GND.

Note that the methods in TDECode are generic while those in xilinx.XTDECode are vendor-specific (and internally may be family-specific).

netlist.xilinx.XTDECode.optimiseOBUF() checks for an OBUF whose I input comes from an FD whose output is also used elsewhere. The FD is replicated and the link to the old FD is removed.

netlist.xilinx.XTDECode.optimiseOBUFT() checks for an OBUFT whose T input comes from an FD, usually via an inverter. The FD replicated and the link to the old FD is removed. If there is an inverter, one is placed in the input to the new FD and its initial state is inverted and the S and R inputs are

swapped. It also checks for an OBUFT whose I input comes from an FD whose output is also used elsewhere. The FD is replicated and the link to the old FD is removed.

netlist.TDECode.optimiseGNDandVCC() checks that Net.LO (GND) and Net.HI (VCC) are used. If not used the element is removed.

Appendix A - Array dimensionality

```
a type [10]uint:8 -

    a       ->  [10]uint:8
    a[]     ->  [10]uint:8
    a[2..4] ->  [3]uint:8
    a[3]    ->  uint:8

a type (f1, log, f2, uint:4) -

    a       ->  (f1, log, f2, uint:4)
    a.f1    ->  log
    a.f2    ->  uint:4

a type [10][5]uint:8 -

    a       ->  [10][5]uint:8
    a[2..4] ->  [3][5]uint:8
    a[2]    ->  [5]uint:8
    a[][2]  ->  [10]uint:8

a type [10](f1, log, f2, uint:4) -

    a[2..4]     -> [3](f1, log, f2, uint:4)
    a[2]        -> (f1, log, f2, uint:4)
    a[2..4].f1  -> [3]log
    a[2].f2     -> uint:4

a type [10][5](f1, log, f2, uint:4) -
```

```
    a[2..4]          ->  [3][5](f1, log, f2, uint:4)
    a[][3]           ->  [10][5](f1, log, f2, uint:4)
    a[2][2..4]       ->  [3](f1, log, f2, uint:4)
    a[2..4][3]       ->  [3](f1, log, f2, uint:4)
    a[2][3]          ->  (f1, log, f2, uint:4)
    a[2..4].f1       ->  [3][5]log
    a[][3].f2        ->  [10][5]uint:4
    a[2][2..4].f1    ->  [3]log
    a[2..4][3].f2    ->  [3]uint:4
    a[2][3].f1       ->  log


a type [10][5](f1, log, f2, [4]uint:4) -


    a[2..4]          ->  [3][5](f1, log, f2, [4]uint:4)
    a[][3]           ->  [10][5](f1, log, f2, [4]uint:4)
    a[2][2..4]       ->  [3](f1, log, f2, [4]uint:4)
    a[2..4][3]       ->  [3](f1, log, f2, [4]uint:4)
    a[2][3]          ->  (f1, log, f2, [4]uint:4)
    a[2..4].f1       ->  [3][5]log
    a[][3].f2        ->  [10][5][4]uint:4
    a[2][2..4].f1    ->  [3]log
    a[2..4][3].f2    ->  [3][4]uint:4
    a[2][3].f1       ->  log

    a[][].f2         ->  [10][5][4]uint:4
    a[][2..3].f2     ->  [10][3][4]uint:4
    a[1..3][].f2     ->  [3][5][4]uint:4
    a[2][2..3].f2    ->  [3][4]uint:4
    a[1..3][0].f2    ->  [3][4]uint:4
    a[1][1].f2       ->  [4]uint:4

    a[][].f2[1..2]       ->  [10][5][2]uint:4
    a[][2..3].f2[1..2]   ->  [10][3][2]uint:4
    a[1..3][].f2[1..2]   ->  [3][5][2]uint:4
    a[2][2..3].f2[1..2]  ->  [2][2]uint:4
    a[1..3][0].f2[1..2]  ->  [2][2]uint:4
    a[1][1].f2[1..2]     ->  [2]uint:4

    a[][].f2[1]          ->  [10][5]uint:4
    a[][2..3].f2[1]      ->  [10][2]uint:4
    a[1..3][].f2[1]      ->  [3][5]uint:4
    a[2][2..3].f2[1]     ->  [2]uint:4
    a[1..3][0].f2[1]     ->  [3]uint:4
    a[1][1].f2[1]        ->  uint:4
```

Notice that in every case the sub-type consists of a 0 to n dimensional array of an unchanged component of the original type. For any reference to a compound type dimensional adjustments always appear in the leading dimensions. Arrays that are nested within a structure cannot be changed from their declared dimensions without selecting out intervening fields, e.g. for

```
    [10][5](f1, log, f2, [4]uint:4)
```

to select the array in field f2 or a subarray of that array we might have

```
a[][].f2 -> [10][5][4]uint:4
```

or

```
a[][].f2[1..2] -> [10][5][2]uint:4
```

Thus

1. Any sub-type is a list of dimensions followed by an unchanged component of the original type.
2. That component will be a primitive or a structure. If a structure it will be a structure from the original type, i.e. no dimensions will be changed.

A WordSpec contains both a dimensional description and a check type. Two WordSpec class instances are compatible for assignment if the dimensional descriptions are identical and the types match (there is a strict type match and a weak match - the former requires target primitive widths to be identical where the latter does not) then the WordSpecs are equivalent. A relaxation of word width allows for assignment where words widths are not identical and padding or truncation occurs (the onus is on the programmer to ensure that the range of truncated results is such that they remain valid).